Programming I · Object-Oriented Programming

C++ и Borland C++ Builder от нуля до экзамена

Полный конспект: 15 лекций по силлабусу, задачи с решениями, все тесты курса и итоговое повторение

1. Читай лекцию

Объясняю простыми словами: суть → пример → вывод программы → нюансы. Красные блоки — частые ошибки, фиолетовые — лайфхаки.

2. Реши задачи

В каждой лекции задачи от лёгких к сложным. Сначала реши сам, потом смотри решение.

3. Пройди тест

Вопросы преподавателя (на английском, как на экзамене), задачи «на 20 секунд» и мои новые вопросы, включая сложные с кодом.

4. Итог в конце

Каждая лекция на одной карточке, все задачи и большой тест. Повторяй ночью перед экзаменом.

Каждый пример кода с блоком «Вывод» скомпилирован и запущен в Borland C++ 6 (bcc32), поэтому вывод в конспекте взят из настоящего запуска. Где в ключе преподавателя ошибка, это отмечено **красным** с объяснением.

Содержание

1 Парадигмы, алгоритмы, синтаксис C++, структура программы, типы данных, указатели

алгоритм и блок-схемы · алфавит и лексемы · идентификаторы · константы · escape-последовательности · типы · enum · массивы · указатели

2 Операции и выражения

арифметика · / и % · ++ и -- · присваивания · сравнения · логика · битовые · запятая · ?: · sizeof · :: · . и -> · приоритеты · преобразование типов · math.h · random · printf

3 Ветвления if, switch, goto и цикл for

if / if-else / вложенные · else-if лесенка · switch, case, break, default · goto и метки · for: инициализация, условие, шаг · вложенные циклы

4 Циклы while и do...while, break и continue, одномерные и двумерные массивы

цикл с предусловием · с постусловием · отличия · break · continue · массивы: объявление, инициализация, индексы · поиск, сумма, максимум, сортировка · матрицы и диагонали

5 Функции: объявление, параметры по значению и по ссылке, const, значения по умолчанию, inline, массивы, рекурсия

прототип и определение · return · формальные и фактические параметры · по значению / по ссылке / через указатель · const · параметры по умолчанию · inline · массивы в функции · локальные, глобальные, static · перегрузка · рекурсия

6 Тип char и строки: массивы символов, string.h и класс string

char и коды ASCII · символьные константы · '\0' · объявление и ввод строк · strlen, strcpy, strcmp, strcat и другие · ошибки · класс string · файлы (fopen, fgets, fprintf)

7 Ключевые понятия и принципы объектно-ориентированного программирования

процедурный подход и ООП · объект, класс, атрибут, метод · состояние и поведение · инкапсуляция · наследование · полиморфизм · абстракция · лабораторная Dog / Poodle / Cat

8 Классы и объекты в C++

структура класса · поля и методы · public / private / protected · class и struct · методы вне класса через :: · создание объектов, new/delete, массивы объектов · . и -> · указатель this · static-члены · объекты как параметры · const-методы

9 Интегрированная среда разработки Borland C++ Builder

что такое IDE · главное меню и панели · палитра компонентов · дизайнер форм · инспектор объектов · редактор кода · менеджер проекта · файлы проекта · компиляция и запуск · виды ошибок · отладка

10 Создание проекта. Работа с формами. Дизайнер форм

проект оконного приложения · структура и файлы · форма как основа GUI · компоненты на форме · размещение и выравнивание · связь формы и кода · первое приложение · две формы: Show и ShowModal · компоненты во время работы

11 Работа с формой: свойства, методы и события

свойство и метод · Object Inspector · главные свойства формы · изменение свойств в коде · свойства только для

чтения · методы формы · событие и обработчик · Sender · общий обработчик · события формы · интерактивное приложение

12 Иерархия классов: наследование, переопределение, виртуальные функции

базовый и производный класс · синтаксис · одиночное и множественное · public / protected / private наследование · порядок конструкторов и деструкторов · переопределение · virtual · абстрактный класс · полиморфизм · иерархия VCL: TObject → … → TButton · Owner и Parent

13 Пользовательский класс. Конструктор и деструктор

проектирование класса · объявление и определение · конструктор по умолчанию · с параметрами · список инициализации · копирования · перегрузка конструкторов · деструктор · освобождение ресурсов · время жизни объекта · лабораторная Book

14 Пользовательский класс: свойства

что такое свойство · инкапсуляция данных · private-поля · get и set · проверка значений · вычисляемые свойства и только для чтения · интерфейс и реализация (.h / .cpp) · __property в C++ Builder · свойства в Object Inspector

15 Создание пользовательского компонента

компонент, объект, класс · стандартные и пользовательские компоненты · выбор базового · мастер New Component · свойства, методы, события · регистрация Register · установка в палитру (пакет) · использование и тестирование · этапы разработки

★ Итог: всё повторение в одном месте

план подготовки · главные ловушки · ошибки в ключах · каждая лекция на карточке · все задачи · комплексные задачи · большой тест

ЛЕКЦИЯ 1 ИЗ 15

Парадигмы, алгоритмы, синтаксис C++, структура программы, типы данных, указатели

Понять, что такое программа и алгоритм, научиться читать и писать простейшую программу на C++ в Borland и знать базовые типы данных.

1.1 Программирование и парадигмы

Программирование — это запись алгоритма на языке, который понимает компьютер. Компьютер умеет только очень простые действия: сложить, сравнить, перейти. Программа собирает из них нужное поведение.

Парадигма — это стиль, «философия» написания программ. Самые важные:

Парадигма	Суть одним предложением	Пример языка
Императивная (процедурная)	Пишем пошаговые команды и делим программу на функции	C, Pascal
Структурная	Только три конструкции: следование, ветвление, цикл (без `goto`)	Pascal, C
Объектно-ориентированная (ООП)	Программа — набор объектов, у каждого есть данные и поведение	C++, Java, C#
Функциональная	Программа — композиция функций без изменения данных	Haskell, Lisp
Логическая	Описываем факты и правила, решение ищет машина	Prolog

Запомни

C++ — **мультипарадигменный** язык: на нём можно писать и процедурно (как на C), и объектно (классы). Первые 6 лекций — процедурный стиль, с 7-й начинается ООП.

1.2 Классификация языков программирования

- **Машинный язык** — нули и единицы, понятен процессору напрямую.
- **Низкоуровневые** (ассемблер) — команды процессора словами: `MOV AX, 5`. Быстро, но тяжело писать.
- **Высокоуровневые** — C, C++, Pascal, Python, Java: понятны человеку, не зависят от процессора.
- По способу выполнения: **компилируемые** (C, C++ — весь текст переводится в `.exe` заранее) и **интерпретируемые** (Python, JavaScript — выполняются построчно).

Как спрашивают на экзамене

«C++ — язык высокого уровня, компилируемый, мультипарадигменный, создан Бьёрном Страуструпом (1983) на основе языка C. Язык C создал **Деннис Ритчи** (1972), на нём была написана система **UNIX**». Эти факты встречаются в тестах.

1.3 Алгоритм и его свойства

Алгоритм — конечная последовательность точных действий, которая за конечное число шагов приводит к решению задачи.

Свойства алгоритма (их часто спрашивают устно):

1. **Дискретность** — разбит на отдельные шаги.
2. **Понятность** — каждая команда понятна исполнителю.
3. **Определённость (детерминированность)** — нет двусмысленностей, одинаковые входные данные дают одинаковый результат.
4. **Конечность (результативность)** — завершается за конечное число шагов и даёт результат.
5. **Массовость** — работает для целого класса задач, а не для одного набора чисел.

Способы записи алгоритма

- **Словесный** — обычным текстом: «1. Ввести a и b. 2. Сложить…».
- **Псевдокод** — смесь текста и программы: `если a > b то max := a иначе max := b`.
- **Блок-схема** — графически (ниже).
- **Программа** — на языке программирования.

Блок-схемы: основные фигуры

Фигура	Что означает
Овал (скруглённый прямоугольник)	Начало / конец
Параллелограмм	Ввод / вывод данных
Прямоугольник	Действие, вычисление (`s = a + b`)
Ромб	Условие (проверка), из него два выхода: «да» и «нет»
Прямоугольник с двойными боковыми линиями	Вызов подпрограммы (функции)
Шестиугольник	Цикл с параметром (`for`)
Стрелки	Порядок выполнения

Три базовые структуры

- **Следование (линейный алгоритм)** — действия выполняются друг за другом.
- **Ветвление** — выбор пути по условию (ромб).
- **Цикл** — повтор действий, пока выполняется условие.

Как запомнить

Любую программу можно построить только из этих трёх «кирпичиков». Это **теорема о структурном программировании** (Бём–Якопини).

```
Линейный алгоритм: площадь прямоугольника
  ( Начало )
      ↓
  / Ввод a, b /
      ↓
  [ S = a * b ]
      ↓
  / Вывод S /
      ↓
  ( Конец )
```

1.4 Алфавит и лексемы C++

Алфавит C++:

- латинские буквы `A–Z`, `a–z` и знак подчёркивания `_` (считается буквой);
- арабские цифры `0–9`;
- специальные символы: `+ - * / % = < > ! & | ^ ~ ? : ; , . ( ) [ ] { } # " ' \`;
- пробельные символы: пробел, табуляция, перевод строки.

Русские буквы в алфавит НЕ входят (их можно писать только внутри комментариев и строк).

Лексема — минимальная смысловая единица языка, «слово» программы. Компилятор сначала разбивает текст на лексемы. Виды лексем: идентификаторы, ключевые слова, константы (литералы), знаки операций, разделители.

```
int sum = a + 15;   →   int | sum | = | a | + | 15 | ;   (7 лексем)
```

1.5 Идентификаторы и ключевые слова

Идентификатор — имя, которое программист даёт переменной, константе, функции, типу, классу.

Правила:

1. Состоит из латинских букв, цифр и `_`.
2. **Начинается с буквы или `_`**, не с цифры.
3. Не совпадает с ключевым словом.
4. **Регистр важен**: `Sum`, `sum` и `SUM` — три разных имени.
5. Без пробелов, дефисов и русских букв.

Имя	Можно?	Почему
`count`, `_tmp`, `x1`, `maxValue`	да	всё по правилам
`1x`	нет	начинается с цифры
`my-var`	нет	дефис — это минус
`float`	нет	ключевое слово
`сумма`	нет	русские буквы
`my var`	нет	пробел

Ключевые (зарезервированные) слова — слова с заранее определённым смыслом, их нельзя использовать как имена: `int, float, double, char, bool, void, if, else, for, while, do, switch, case, break, continue, return, const, class, public, private, struct, enum, sizeof, new, delete, true, false…`

Хороший стиль

Давай осмысленные имена: `sum`, `count`, `maxAge`, а не `a1`, `qq`. Для констант часто пишут БОЛЬШИМИ буквами: `MAX_SIZE`.

1.6 Константы (литералы)

Вид	Примеры	Тип
Целые	`25`, `-7`, `0`	`int`
Восьмеричные	`017` (начинается с 0) = 15	`int`
Шестнадцатеричные	`0x1F` = 31	`int`
Вещественные	`3.14`, `2.`, `.5`, `1.5e3` (=1500)	`double`
Символьные	`'A'`, `'7'`, `'\n'`	`char`
Строковые	`"Hello"`	массив `char`
Логические	`true`, `false`	`bool`

Именованная константа объявляется через `const` (или по-старому через `#define`):

```C++
const double PI = 3.14159;   // типизированная константа — лучше
#define N 10                 // макрос препроцессора — просто подстановка текста
//---
cout << PI * 2 << " " << N * 2;
```

Вывод
```
6.28318 20
```

Ошибка: изменить константу

`const int a = 5; a++;` — **ошибка компиляции**: константу менять нельзя. Этот вопрос есть в банке тестов.

Ловушка с нулём впереди

`int x = 010;` — это **8**, а не 10! Число с ведущим нулём записано в восьмеричной системе.

```C++
int x = 010, y = 0x10;
cout << x << " " << y;
```

Вывод
```
8 16
```

1.7 Escape-последовательности и комментарии

Escape-последовательность — это обратный слэш `\` и символ. Вместе они обозначают **один** управляющий символ.

Запись	Смысл
`\n`	новая строка
`\t`	горизонтальная табуляция
`\\`	сам обратный слэш
`\"`	двойная кавычка внутри строки
`\'`	одинарная кавычка
`\a`	звуковой сигнал
`\0`	нулевой символ (конец строки)
`\r`	возврат каретки в начало строки

```cpp
cout << "A\tB\n";
cout << "Path: C:\\Borland\n";
cout << "He said \"Hi\"";
```

Вывод

```
A       B
Path: C:\Borland
He said "Hi"
```

Комментарии компилятор пропускает:

```cpp
// однострочный комментарий — до конца строки
/* многострочный
   комментарий */
//---
cout << 1; // работает
```

Вывод

```
1
```

> **Осторожно**
>
> Вложить `/* … */` в другой `/* … */` нельзя: первый же `*/` закроет комментарий.

1.8 Структура программы в Borland C++

Минимальная консольная программа:

```cpp
#include <iostream.h>   // 1. директивы препроцессора: подключаем библиотеки
#include <conio.h>

int main()              // 2. главная функция — отсюда начинается выполнение
{                       // 3. тело функции в фигурных скобках
    int a = 5, b = 7;   //    объявления переменных
    int s = a + b;      //    операторы, каждый заканчивается ;
    cout << "Sum = " << s << endl;
    return 0;           // 4. код завершения: 0 = всё хорошо
}
```

Вывод
```
Sum = 12
```

Разберём по частям:

1. `#include <…>` — подключает **заголовочный файл** (`.h`) с описаниями функций: `iostream.h` — `cin` / `cout`, `conio.h` — `getch()`, `clrscr()`, `math.h` — математика, `stdlib.h` — `rand()`, `random()`, `string.h` — строковые функции.
2. `main()` — **с неё начинается выполнение любой программы**. Она одна на программу.
3. `{ }` — тело функции. Каждый оператор заканчивается `;`.
4. `cout << …` — вывод на экран, `cin >> …` — ввод с клавиатуры.

Особенности Borland C++ Builder

- В Borland пишут `#include <iostream.h>` (старый стиль, `using namespace std` не нужен). В современном C++: `#include <iostream>` + `using namespace std;`.
- Чтобы консольное окно не закрывалось сразу, в конце ставят `getch();` из `conio.h`: программа ждёт нажатия клавиши.
- В Builder консольный проект создаётся через **File → New → Other → Console Wizard**.
- Часто встречается `void main()` — в Borland это работает, но по стандарту правильно `int main()`.

Самые частые ошибки новичка

- Забыть `;` в конце оператора → `Statement missing ;`.
- Написать `Cout` или `Main` с большой буквы → компилятор не найдёт (регистр важен).
- Забыть `#include <iostream.h>` → `Undefined symbol 'cout'`.
- Перепутать `<<` (вывод) и `>>` (ввод): `cout >> x` — ошибка.

1.9 Линейная структура (следование)

Линейная программа: ввод → вычисление → вывод, без условий и циклов.

```cpp
// Пример: перевод секунд в часы, минуты, секунды
int t = 3725;
int h = t / 3600;
int m = t % 3600 / 60;
int s = t % 60;
cout << h << ":" << m << ":" << s;
```

Вывод
```
1:2:5
```

1.10 Типы данных

Тип определяет: **сколько памяти** занимает переменная, **какие значения** она хранит и **какие операции** с ней можно делать.

Тип	Размер (Borland, 32 бит)	Диапазон / смысл
`char`	1 байт	−128…127 (символ = его код)
`unsigned char`	1 байт	0…255
`short`	2 байта	−32 768…32 767
`int`	4 байта	≈ −2,1 млрд…2,1 млрд
`unsigned int`	4 байта	0…≈4,29 млрд
`long`	4 байта	как `int` в Borland
`float`	4 байта	≈ 7 значащих цифр (**одинарная** точность)
`double`	8 байт	≈ 15 значащих цифр (**двойная** точность)
`long double`	10 байт	ещё точнее
`bool`	1 байт	`true` (1) / `false` (0)
`void`	—	«нет значения» (для функций)

Модификаторы: `signed` / `unsigned` (со знаком / без знака), `short` / `long` (короче / длиннее).

Узнать размер можно операцией `sizeof`:

```cpp
cout << sizeof(char) << " " << sizeof(short) << " " << sizeof(int) << " "
     << sizeof(float) << " " << sizeof(double) << " " << sizeof(bool);
```

Вывод
```
1 2 4 4 8 1
```

Объявление переменной: `тип имя;` или сразу с начальным значением.

```cpp
int a;              // объявили (значение пока «мусор»!)
int b = 10, c = 20; // объявили и инициализировали
float x = 2.5;
char ch = 'Q';
bool ok = true;
//---
cout << b + c << " " << x * 2 << " " << ch << " " << ok;
```

Вывод
```
30 5 Q 1
```

Переполнение

Если положить в переменную число больше её диапазона, оно «перескочит» на другой конец диапазона:

```cpp
short s = 32767;
s = s + 1;
cout << s;
```

Вывод

```
-32768
```

Неинициализированная переменная

`int a; cout << a;` выведет случайный «мусор» из памяти. Всегда задавай начальное значение счётчикам и суммам: `int sum = 0;`.

1.11 Перечисляемый тип enum

`enum` — набор именованных целых констант. Удобен для дней недели, цветов, состояний.

```cpp
enum Color { RED, GREEN, BLUE };          // RED=0, GREEN=1, BLUE=2
enum Day { MON = 1, TUE, WED, SUN = 7 };  // TUE=2, WED=3
//---
Color c = BLUE;
Day d = WED;
cout << c << " " << d << " " << SUN;
```

Вывод

```
2 3 7
```

Фишка / совет

Если значение не указано, оно на 1 больше предыдущего. Первое по умолчанию равно 0.

1.12 «Множественность» данных: массивы (кратко)

Когда данных много (оценки 30 студентов), заводить 30 переменных неудобно. **Массив** — набор элементов **одного типа** под **одним именем**, к каждому обращаемся по **индексу**. Индексы начинаются **с 0**.

```cpp
int a[5] = {3, 8, 1, 9, 4};
cout << a[0] << " " << a[4] << " " << sizeof(a) / sizeof(a[0]);
```

Вывод

```
3 4 5
```

Подробно массивы — в лекции 4.

1.13 Указатели и адреса памяти

Каждая переменная лежит в памяти по какому-то **адресу** (номер ячейки).

- `&x` — **взять адрес** переменной x.
- `int *p;` — **указатель** на int: переменная, которая хранит адрес.
- `*p` — **разыменование**: «то, что лежит по адресу p».

```C++
int x = 10;
int *p = &x;      // p хранит адрес x
*p = 25;          // меняем x через указатель
cout << x << " " << *p << " " << (p == &x);
```

Вывод
```
25 25 1
```

Как читать

`int *p = &x;` → «p — указатель на int, в него кладём адрес x». После этого `*p` и `x` — одна и та же ячейка памяти.

Ошибки с указателями

- `int *p; *p = 5;` — указатель никуда не указывает (мусорный адрес) → программа может упасть.
- Используй `NULL` (или `0`) для «пустого» указателя и проверяй: `if (p != NULL) …`.
- `*` в объявлении (`int *p`) и `*` в выражении (`*p = 5`) означают разное: в первом случае «это указатель», во втором «значение по адресу».

Указатель и массив тесно связаны: имя массива — это адрес его первого элемента.

```C++
int a[3] = {10, 20, 30};
int *p = a;           // то же, что &a[0]
cout << *p << " " << *(p + 1) << " " << p[2];
```

Вывод
```
10 20 30
```

Адресная арифметика

`p + 1` сдвигает указатель на **один элемент** (для int — на 4 байта), а не на 1 байт. Поэтому `*(a + i)` — то же самое, что `a[i]`.

Разбор задач

Задача 1.1. Перевод температуры лёгкая

Дана температура в градусах Цельсия `c = 36.6`. Выведи её в Фаренгейтах по формуле F = C·9/5 + 32.

Решение

```cpp
double c = 36.6;
double f = c * 9 / 5 + 32;
cout << f;
```

Вывод
```
97.88
```

Обрати внимание: если бы `c` была `int` и мы написали `9/5`, получили бы 1 (целое деление!). Поэтому делим вещественное `c * 9` на 5.

Задача 1.2. Цифры трёхзначного числа лёгкая

Дано трёхзначное число `n = 472`. Выведи его цифры через пробел и их сумму.

Решение

```cpp
int n = 472;
int a = n / 100;        // сотни
int b = n / 10 % 10;    // десятки
int c = n % 10;         // единицы
cout << a << " " << b << " " << c << " sum=" << a + b + c;
```

Вывод
```
4 7 2 sum=13
```

Приём: `n % 10` — последняя цифра, `n / 10` — «отрезать» последнюю цифру.

Задача 1.3. Обмен значений средняя

Поменяй местами значения `a = 3` и `b = 8`: (1) через третью переменную; (2) без неё.

Решение

```cpp
int a = 3, b = 8, t;
t = a; a = b; b = t;            // способ 1
cout << a << " " << b << endl;
a = a + b; b = a - b; a = a - b; // способ 2 (арифметика)
cout << a << " " << b;
```

Вывод
```
8 3
3 8
```

Задача 1.4. Указатели средняя

Объяви `int x = 7, y = 3;` и указатель `p`. Через указатель: сначала увеличь x в 2 раза, потом переставь p на y и прибавь к y 10. Выведи x и y.

Решение

```cpp
int x = 7, y = 3;
int *p = &x;
*p = *p * 2;
p = &y;
*p += 10;
cout << x << " " << y;
```

> **Вывод**
> 14 13

Задача 1.5. Размер типов и переполнение сложная

Что выведет программа? Объясни каждое число.

```cpp
unsigned char u = 255;
u++;
char c = 'A' + 2;
int big = 2147483647;
big++;
cout << (int)u << " " << c << " " << big;
```

Решение

> **Вывод**
> 0 C -2147483648

`unsigned char` хранит 0...255, после 255 идёт 0. `'A'` имеет код 65, 65+2 = 67 — это `'C'`. Максимальный `int` плюс 1 переполняется в минимальный.

Контрольные вопросы из силлабуса (устно)

1. **Что такое алгоритм и его свойства?** — Конечная последовательность точных шагов, решающая задачу. Свойства: дискретность, понятность, определённость, конечность (результативность), массовость.
2. **Способы записи алгоритма?** — Словесный, псевдокод, блок-схема, программа.
3. **Что такое блок-схема и её фигуры?** — Графическая запись алгоритма: овал (начало/конец), параллелограмм (ввод/вывод), прямоугольник (действие), ромб (условие), шестиугольник (цикл for), стрелки.
4. **Базовые типы алгоритмов?** — Линейный (следование), разветвляющийся, циклический.
5. **Чем они отличаются?** — Линейный выполняет всё по порядку; разветвляющийся выбирает путь по условию; циклический повторяет действия.
6. **Общая структура программы на C++?** — Директивы `#include`, (глобальные объявления), функция `main()` с телом в `{ }`, внутри — объявления и операторы с `;`, `return 0`.

7. **Элементы синтаксиса C++?** — Алфавит, лексемы: идентификаторы, ключевые слова, константы, знаки операций, разделители; комментарии.
8. **Идентификаторы, ключевые слова, константы, переменные?** — Имя объекта; зарезервированное слово; неизменяемое значение; именованная область памяти, значение которой меняется.
9. **Комментарии и escape-последовательности?** — `//…`, `/*…*/` компилятор игнорирует; `\n`, `\t`, `\\`, `\"` — управляющие символы в строках.
10. **Базовые типы данных?** — `char`, `int` (`short`, `long`, `unsigned`), `float`, `double`, `bool`, `void`.

Тест по лекции 1

1. What is an identifier? банк №1

a) dynamic memory
b) an array made up of a set of elements
c) the name of an object in a program
d) the way a program is passed
e) words intended for the compiler

2. In C, which function does the execution of any program start from? банк №3

a) clrscr()
b) random
c) It is different in each program
d) main()
e) randomize()

3. Where are library functions located? банк №4

a) in library files
b) on the Internet
c) on a CD
d) in the computer's RAM
e) No correct answer

4. A sequence of characters made up of Latin letters and digits, which must begin with a letter, is called ... банк №5

a) constant
b) function
c) comment
d) identifier
e) No correct answer

5. Which keyword is used to declare constants in C? банк №6

a) int
b) const
c) procedure
d) var
e) All of the above are correct

6. What is a variable? банк №13

a) dividing memory classes into several parts

b) an expression made of an operand, an operation sign and brackets, expressing a value of a defined type

c) a memory area with a certain name, in which data of a defined type is stored

d) storing data of an undefined type in memory

e) No correct answer

7. What is the #define directive used for? банк №14

a) to set the value of a constant

b) for macros

c) for a symbol that controls conditional compilation

d) an expression made of an operand, an operation sign and brackets, expressing a value of a defined type

e) No correct answer

8. What is the extension of source (header) files that contain functions or defined data, and what are these files called? банк №15

a) ".c", ".c files"

b) ".f", ".f files"

c) ".htp", ".htp files"

d) ".h", ".h files"

e) No correct answer

9. What is a variable? банк №17

a) a quantity that takes different values while the program is running

b) the fulfillment of a certain condition of an algorithm

c) dividing a memory class into several areas

d) a sequence of programs

e) No correct answer

10. The correct declaration of an integer variable a: банк №21

a) float a;

b) a int;

c) int a;

d) integer a;

e) char a;

11. What characters is the body of a function enclosed in? банк №25

a) * *

b) / /

c) { }

d) ( )

e) /* */

12. What characters is a comment written inside in C? банк №26

a) * *
b) { }
c) (* *)
d) /* */
e) < >

13. What is a lexeme? банк №31

a) the smallest meaningful unit of a language
b) phrases
c) sentences
d) a page change
e) the body of a program

14. What does the statement "int n;" mean? банк №32

a) variable n takes real values
b) the body of a program
c) that the specifier n is given
d) variable n takes integer values
e) No correct answer

15. In C, uppercase and lowercase letters denoted by the same character ... банк №33

a) change depending on the language version
b) are considered different
c) change depending on the compiler
d) are considered the same
e) All of the above are correct

16. Names given to variables, constants, data types and functions are called ... банк №34

a) identifiers
b) pointers
c) keywords
d) directives
e) No correct answer is given

17. How is the single-precision real data type declared? банк №36

a) int
b) char
c) double
d) float
e) No correct answer

18. How is the double-precision real data type declared? банк №37

a) int
b) char
c) double
d) float
e) No correct answer

19. How is the character data type declared? банк №38

a) float
b) char
c) double
d) int
e) No correct answer

20. What are keywords? банк №39

a) operations
b) ordinary identifiers
c) functions
d) words entered into a special reserved set
e) No correct answer

21. The first system software product written in C? банк №40

a) UNIX
b) NC
c) Windows XP
d) MS-DOS
e) No correct answer

22. Which fragment declaring variables in C is correct? банк №42

a) main () int x, y; {float a, b; char c;}
b) main () { int x, y; float a, b; char c;}
c) {main (); int x, y; float, a, b; char c;}
d) main () int x, y; float a, b; char x;
e) No correct answer

23. How is the logical type defined in C? банк №43

a) char
b) The logical type is not defined in C
c) string
d) bool
e) No correct answer

24. What values can a logical quantity take in C? банк №44

a) true, false
b) + and -
c) 0, 255
d) 0 and 1
e) No correct answer

25. Who founded the C language? банк №93

a) Bill Gates
b) Dennis Ritchie
c) J. Neumann
d) Bjarne Stroustrup
e) No correct answer

26. In C. The control character “\n” банк №94

a) move to the next page
b) write the \ symbol n times
c) a blank space
d) move to a new line
e) No correct answer

27. In C. The control character “\t” банк №95

a) horizontal tab
b) write the \ symbol t times
c) move to the next page
d) move to a new line
e) No correct answer

28. Which of these is NOT part of the C alphabet? банк №97

a) Special symbols
b) Uppercase letters of the Russian alphabet
c) Lowercase and uppercase letters of the Latin alphabet
d) Arabic digits 0-9
e) No correct answer

29. In C. The range of values of the char type банк №98

a) -128...127
b) -65365...65365
c) -256...255
d) 0...65365
e) No correct answer

30. In C, which function does the execution of any program start from? банк №123

a) random
b) main()
c) clrscr()
d) randomize()
e) No correct answer

31. What characters is the body of a function enclosed in? банк №130

a) {}
b) <>
c) ()
d) []
e) No correct answer

32. In C, in which part of the program are variables declared? банк №134

a) anywhere, only before the variable is used
b) only at the beginning
c) at the end
d) nowhere
e) No correct answer

33. Which of the following data types does NOT belong to C++? банк №171

a) double
b) int
c) real
d) float
e) No correct answer

34. What is a finite sequence of steps used to solve a problem? 20 сек · B1·1

Ответь одним словом или символом.

35. Which function is the starting point of a C++ program? 20 сек · B2·1

Ответь одним словом или символом.

36. Which symbol ends a C++ statement? 20 сек · B3·1

Ответь одним словом или символом.

37. Which operator is used for assignment? 20 сек · B4·1

Ответь одним словом или символом.

38. Which keyword is used to define an integer variable? 20 сек · B5·1

Ответь одним словом или символом.

39. Which header is commonly used for cin and cout? 20 сек · B6·1

Ответь одним словом или символом.

40. Which operator is used for equality comparison? 20 сек · B7·1

Ответь одним словом или символом.

41. Which operator is used for input with cin? 20 сек · B8·1

Ответь одним словом или символом.

42. Which keyword is used to return a value from a function? 20 сек · B9·1

Ответь одним словом или символом.

43. Which keyword is used to declare a constant? 20 сек · B10·1

Ответь одним словом или символом.

44. Which data type is commonly used for whole numbers? 20 сек · B11·1

Ответь одним словом или символом.

45. Which function is used to display data on the screen? 20 сек · B12·1

Ответь одним словом или символом.

46. Which operator is used for remainder after integer division? 20 сек · B13·1

Ответь одним словом или символом.

47. Which operator is used for output with cout? 20 сек · B14·1

Ответь одним словом или символом.

48. Which keyword is used to create a condition in C++? 20 сек · B15·1

Ответь одним словом или символом.

49. Which of the following is a valid C++ identifier? новый

a) 2ndValue
b) _total2
c) my-sum
d) float

50. What is the value of `x` after `int x = 012;`? новый

a) 12
b) 10
c) 2
d) Compilation error

51. Which flowchart symbol is used for a condition? новый

a) rectangle
b) parallelogram
c) rhombus (diamond)
d) oval

52. Which property of an algorithm means "it must finish after a finite number of steps"? новый

a) discreteness
b) finiteness
c) mass character
d) determinism

53. What is the output? сложный

```
enum Level { LOW = 2, MID, HIGH = MID * 3 };
//---
cout << LOW + MID + HIGH;
```

a) 6
b) 11
c) 14
d) Compilation error

54. What is the output? сложный

```
int a[4] = {5, 10, 15, 20};
int *p = a + 1;
cout << *p + *(p + 2) << " " << p[1];
```

a) 25 15
b) 30 15
c) 30 10
d) 15 20

55. What is the output? сложный

```
int x = 5;
int *p = &x;
int *q = p;
*q = *p + 3;
x = x * 2;
cout << *p;
```

a) 8
b) 10
c) 16
d) 5

56. What is the output? сложный

```
short s = -32768;
s--;
cout << s;
```

a) -32769
b) 32767
c) 0
d) Compilation error

57. How many lexemes (tokens) are in `y = x * 25;`? новый

a) 4
b) 5
c) 6
d) 7

Ключи

1 — c	**2** — d	**3** — a	**4** — d	**5** — b	**6** — c
7 — a	**8** — d	**9** — a	**10** — c	**11** — c	**12** — d
13 — a	**14** — d	**15** — b	**16** — a	**17** — d	**18** — c
19 — b	**20** — d	**21** — a	**22** — b	**23** — d	**24** — a
25 — b	**26** — d	**27** — a	**28** — b	**29** — a	**30** — b
31 — a	**32** — a	**33** — c	**34** — Algorithm	**35** — main	**36** — ;
37 — =	**38** — int	**39** — iostream	**40** — ==	**41** — >>	**42** — return
43 — const	**44** — int	**45** — cout	**46** — %	**47** — <<	**48** — if
49 — b	**50** — b	**51** — c	**52** — b	**53** — c	**54** — b
55 — c	**56** — b	**57** — c			

23. В чистом C раньше логического типа не было (только `_Bool` с 1999 г.), а в C++ есть `bool`. Тест про C++, поэтому `bool`.

49. Идентификатор может начинаться с `_` или буквы; цифра в начале, дефис и ключевые слова запрещены.

50. Ведущий ноль — восьмеричная запись: 1·8 + 2 = 10.

53. LOW = 2, MID = 3, HIGH = 3·3 = 9, сумма 14.

54. p указывает на a[1] = 10; *(p+2) = a[3] = 20; 10 + 20 = 30; p[1] = a[2] = 15.

55. p и q указывают на x. *q = 5 + 3 → x = 8, потом x = 16;* p читает x.

56. Переполнение `short` вниз: после минимального значения идёт максимальное.

57. `y`, `=`, `x`, `*`, `25`, `;` — шесть лексем.

ЛЕКЦИЯ 2 ИЗ 15

Операции и выражения

Уверенно вычислять любое выражение «в уме» так же, как это сделает компилятор, и знать стандартные функции библиотек.

2.1 Операнд, операция, выражение

- **Операнд** — то, над чем выполняется действие: переменная, константа, результат функции.
- **Операция (оператор)** — само действие: `+`, `*`, `=`, `&&`...
- **Выражение** — комбинация операндов, знаков операций и скобок, которая даёт значение определённого типа: `(a + b) * 2`.

По числу операндов операции бывают **унарные** (`-x`, `!x`, `++x`), **бинарные** (`a + b`) и одна **тернарная** (`a ? b : c`).

2.2 Арифметические операции

Операция	Смысл	Пример	Результат
`+ - *`	сложение, вычитание, умножение	`7 * 3`	21
`/`	деление	`7 / 2` и `7.0 / 2`	**3** и 3.5
`%`	остаток от деления (только целые!)	`7 % 2`	1

Главное правило деления

Если **оба** операнда целые, `/` выполняет **целочисленное деление**: дробная часть просто отбрасывается. `5/2 = 2`, `7/3 = 2`, `1/2 = 0`. Чтобы получить дробный результат, хотя бы один операнд должен быть вещественным: `5.0/2 = 2.5`, `(double)5/2 = 2.5`.

```cpp
int a = 17, b = 5;
cout << a / b << " " << a % b << " " << (double)a / b << " " << a / b * b + a % b;
```

Вывод

```
3 2 3.4 17
```

Формула, которая решает половину задач

`a == (a / b) * b + a % b` — всегда. Поэтому `n % 10` — последняя цифра, `n / 10` — число без последней цифры, `n % 2 == 0` — n чётное.

Частые ошибки

- `double x = 1 / 3;` → x = **0**, потому что `1/3` посчиталось в целых ДО присваивания. Правильно: `1.0 / 3`.
- `%` с вещественными: `7.5 % 2` — **ошибка компиляции**. Для вещественных есть функция `fmod(7.5, 2)` из `math.h`.
- Деление на ноль целых (`5 / 0`) — аварийное завершение программы.

С отрицательными числами деление округляет **к нулю**, а знак остатка совпадает со знаком делимого:

```cpp
cout << -7 / 2 << " " << -7 % 2 << " " << 7 % -2;
```

Вывод

```
-3 -1 1
```

2.3 Инкремент и декремент

`++` увеличивает переменную на 1, `--` уменьшает. Есть две формы:

Форма	Название	Что делает
`++x`	префиксная	**сначала** увеличивает, **потом** отдаёт новое значение
`x++`	постфиксная	**сначала** отдаёт старое значение, **потом** увеличивает

```cpp
int x = 5, y;
y = x++;   // y = 5, x = 6
cout << x << " " << y << endl;
y = ++x;   // x = 7, y = 7
cout << x << " " << y;
```

Вывод

```
6 5
7 7
```

Как не путаться

Читай слева направо: в `x++` сначала стоит **x** (берём его), потом **++** (увеличиваем). В `++x` сначала **++**, потом x.

Нельзя менять переменную дважды в одном выражении

`x++ + ++x`, `cout << x++ << ++x`, `i = i++` — это **неопределённое поведение**: разные компиляторы дают разный результат. Borland 6 для `x = 5; x++ + ++x` выдаёт 12, а для `cout << x << " " << x++` считает аргументы справа налево. На экзамене такие вопросы бывают, ответ берётся «как в Borland», но в своих программах так не пиши.

`++` применяется **только к переменным**: `5++` или `(a+b)++` — ошибка компиляции.

2.4 Присваивание и составные присваивания

`=` — **присвоить** (а не «равно»!). Справа вычисляется значение, оно кладётся в переменную слева.

Составные операции — короткая запись:

Запись	Эквивалент
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x %= 5`	`x = x % 5`

Правая часть вычисляется целиком

`x *= y + 1` — это `x = x * (y + 1)`, а не `x = x * y + 1`. Например, при x = 2: `x *= (x + 1)` → 2·3 = 6.

Присваивание само является выражением, поэтому можно писать цепочкой: `a = b = c = 0;` (выполняется справа налево).

```C++
int a, b, c;
a = b = c = 4;
a += b *= c -= 1;   // c = 3, b = 12, a = 16
cout << a << " " << b << " " << c;
```

Вывод
```
16 12 3
```

2.5 Операции сравнения

`==` равно, `!=` не равно, `<`, `>`, `<=`, `>=`. Результат — `true` (1) или `false` (0).

```C++
int a = 5, b = 7;
cout << (a == b) << (a != b) << (a < b) << (a >= 5);
```

Вывод
```
0111
```

Ошибка №1 во всех тестах: `=` вместо `==`

`if (x = 5)` — это не проверка, а **присваивание**! x станет 5, а значение выражения 5 ≠ 0 считается истиной, поэтому условие выполнится всегда. Правильно: `if (x == 5)`.

Защита от ошибки

Пиши константу слева: `if (5 == x)`. Если случайно напишешь `5 = x`, компилятор сразу выдаст ошибку.

Осторожно

`=` — не операция сравнения. Вопрос «what is NOT a comparison sign?» → ответ `=`.

2.6 Логические операции

Операция	Название	Истина, когда
`&&`	И (логическое умножение)	**оба** операнда истинны
`||`	ИЛИ (логическое сложение)	**хотя бы один** истинен
`!`	НЕ (отрицание)	операнд ложен

В C/C++ **0 — ложь, всё остальное — истина**. Результат логической операции — 1 или 0.

```cpp
int x = 7;
cout << (x > 0 && x < 10) << (x < 0 || x == 7) << !x << !!x;
```

Вывод
```
1101
```

Сокращённое вычисление

Если по левому операнду ответ уже ясен, правый **не вычисляется**: в `a && b` при ложном `a`, в `a || b` при истинном `a`. Это важно, если справа есть `++` или деление:

```cpp
int a = 0, b = 5;
if (a != 0 && b / a > 1) cout << "big";   // деления на 0 не будет!
int k = 0;
if (a == 0 || ++k) { }
cout << "k=" << k;
```

Вывод
```
k=0
```

Частая ошибка

Проверка «x между 1 и 10» пишется `1 <= x && x <= 10`. Запись `1 <= x <= 10` компилируется, но работает неверно: сначала `1 <= x` даёт 0 или 1, и это всегда ≤ 10, поэтому условие всегда истинно.

2.7 Битовые (поразрядные) операции

Работают с каждым битом числа отдельно (только целые типы):

Операция	Смысл	Пример ($5 = 101_2$, $3 = 011_2$)
`&`	побитовое И	`5 & 3` = 001 = **1**
`|`	побитовое ИЛИ	`5 | 3` = 111 = **7**
`^`	исключающее ИЛИ (XOR)	`5 ^ 3` = 110 = **6**
`~`	инверсия всех битов	`~5` = **−6**
`<<`	сдвиг влево (×2 за каждый сдвиг)	`5 << 2` = **20**
`>>`	сдвиг вправо (÷2 за каждый сдвиг)	`20 >> 3` = **2**

```cpp
cout << (5 & 3) << " " << (5 | 3) << " " << (5 ^ 3) << " " << ~5 << " " << (5 << 2) << " " << (20 >> 3);
```

Вывод
```
1 7 6 -6 20 2
```

`&` ≠ `&&`, `|` ≠ `||`

`&&` — логическое И (ответ 0/1, сокращённое вычисление). `&` — побитовое (работает с битами, вычисляет обе части). Логическое И в тестах — **&&**.

Лайфхак

`x & 1` — проверка нечётности (последний бит), `x << 1` — умножение на 2, `~x` = `-x - 1`.

2.8 Операция «запятая»

Вычисляет выражения слева направо, результат — **последнее** значение. Чаще всего встречается в `for`: `for (i = 0, j = 10; i < j; i++, j--)`.

```cpp
int a, b;
a = (b = 3, b * 4);   // b = 3, потом a = 12
cout << a << " " << b;
```

Вывод
```
12 3
```

2.9 Условная (тернарная) операция `? :`

`условие ? значение_если_да : значение_если_нет` — короткая замена `if…else` для **выражений**.

```cpp
int a = 8, b = 13;
int mx = (a > b) ? a : b;
int x = -4;
cout << mx << " " << (x >= 0 ? x : -x) << " " << (a % 2 == 0 ? "even" : "odd");
```

Вывод
```
13 4 even
```

Вложенный тернарный оператор: максимум из трёх или знак числа.

```cpp
int a = 4, b = 9, c = 6;
int m = a > b ? (a > c ? a : c) : (b > c ? b : c);
int n = -5;
int sign = n > 0 ? 1 : n < 0 ? -1 : 0;
cout << m << " " << sign;
```

Вывод
```
9 -1
```

2.10 sizeof, `::`, `.` и `->`

- `sizeof(тип)` или `sizeof(переменная)` — размер в байтах. Правильная запись в тестах: `sizeof(type)`.
- `::` — **разрешение области видимости**. `::x` — глобальная переменная x, даже если есть локальная с тем же именем. Ещё: `Класс::метод`.
- `.` — доступ к полю объекта/структуры: `p.x`.
- `->` — доступ к полю через **указатель**: `ptr->x` ≡ `(*ptr).x`.

```C++
int x = 100;            // глобальная
struct Point { int x, y; };
//---
int x = 5;              // локальная закрывает глобальную
Point p = {3, 4};
Point *ptr = &p;
ptr->y = 10;
cout << x << " " << ::x << " " << p.x << " " << p.y << " " << sizeof(p);
```

> **Вывод**
> `5 100 3 10 8`

2.11 Приоритет операций

От высшего к низшему (что выполняется раньше):

Уровень	Операции
1	`::`
2	`()` `[]` `.` `->` постфиксные `x++ x--`
3	унарные: `++x --x ! ~ -x * &` `sizeof` приведение `(type)`
4	`* / %`
5	`+ -`
6	`<< >>`
7	`< <= > >=`
8	`== !=`
9	`&` → 10 `^` → 11 `|`
12	`&&`
13	`||`
14	`?:`
15	`= += -= *= …` (справа налево)
16	`,`

Короткое правило

Арифметика → сравнения → логика (`&&` раньше `||`) → присваивание. Сомневаешься — **ставь скобки**, это никогда не ошибка.

```cpp
cout << 2 + 3 * 4 << " " << (2 + 3) * 4 << " " << 10 - 4 - 3 << " " << (1 || 0 && 0) << " " << 17 % 5 * 2;
```

Вывод

```
14 20 3 1 4
```

Ловушка с `cout` и `<<`

`cout << a & b;` не скомпилируется, а `cout << a == b;` тоже ошибка: `<<` сильнее `&` и `==`. Выражения с битовыми и сравнениями внутри `cout` всегда бери в скобки.

2.12 Преобразование типов

Неявное (автоматическое) в выражении — к «более широкому» типу:

`char, short → int → unsigned → long → float → double → long double`

Смешали	Результат
`short` + `int`	`int`
`int` + `float`	`float`
`long` + `float`	`float`
`float` + `double`	`double`
`char` + `int`	`int` (символ превращается в код)

При присваивании значение приводится к типу **левой** части: `int k = 3.99;` → k = 3 (дробь отбрасывается, не округляется!).

Явное приведение: `(double)a`, `(int)x`, `double(a)`.

```cpp
int a = 7, b = 2;
double r1 = a / b;
double r2 = (double)a / b;
int k = 3.99;
char c = 'a' + 1;
cout << r1 << " " << r2 << " " << k << " " << c << " " << 'a' + 1;
```

Вывод

```
3 3.5 3 b 98
```

Частая ошибка

`(double)(a / b)` — бесполезно: сначала посчитается целое `a/b` = 3, а потом 3 превратится в 3.0. Приводить нужно **операнд**, а не готовый результат.

2.13 Стандартные библиотеки и функции

Заголовок	Что внутри
`iostream.h`	`cin`, `cout`, `endl` (ввод-вывод потоками)
`stdio.h`	`printf`, `scanf`, работа с файлами `fopen`...
`math.h`	математические функции
`stdlib.h`	`rand`, `random`, `randomize`, `abs`, `atoi`, `exit`
`conio.h`	`getch`, `clrscr` (консоль Borland)
`string.h`	`strlen`, `strcpy`, `strcmp`, `strcat`...

math.h

Функция	Смысл	Пример
`sqrt(x)`	квадратный корень	`sqrt(16)` = 4
`pow(x, y)`	x в степени y	`pow(2, 10)` = 1024
`fabs(x)`	модуль вещественного	`fabs(-2.5)` = 2.5
`abs(x)`	модуль целого (stdlib.h)	`abs(-7)` = 7
`ceil(x)`	округление **вверх**	`ceil(3.2)` = 4
`floor(x)`	округление **вниз**	`floor(3.8)` = 3
`log(x)`	натуральный логарифм ln	`log(M_E)` = 1
`log10(x)`	десятичный логарифм lg	`log10(1000)` = 3
`exp(x)`	e^x	`exp(1)` ≈ 2.718
`sin cos tan`	тригонометрия (**аргумент в радианах!**)	`sin(30 * M_PI / 180)` = 0.5
`asin acos atan`	обратные: arcsin, arccos, arctg	`acos(1)` = 0

```cpp
cout << sqrt(pow(5, 4)) << " " << ceil(3.226) << " " << floor(3.63) << " "
     << fabs(-2.5) << " " << log10(1000) << " " << pow(32, 1.0 / 5);
```

Вывод

```
25 4 3 2.5 3 2
```

Корень n-й степени

`pow(x, 1/5)` — **ошибка**: `1/5` в целых = 0, получится $x^0 = 1$. Правильно: `pow(x, 1.0/5)` или `pow(x, 1/5.)`.

Градусы и радианы

`sin(30)` — синус 30 **радиан**, а не градусов. Перевод: `x * 3.14159 / 180`.

Случайные числа (Borland)

- `randomize();` — запустить генератор так, чтобы при каждом запуске числа были разные (подключает время).
- `random(n)` — случайное целое от **0 до n−1**.
- `rand()` — от 0 до `RAND_MAX` (32767); стандартный способ: `rand() % n`.
- Число в диапазоне [a; b]: `random(b - a + 1) + a` или `rand() % (b - a + 1) + a`.

> **Как спрашивают на экзамене**
>
> В банке тестов: «[−20; 50]» → ключ `random(70)-20`; «x = random(20)−5» → ключ [−5; 15]. Строго говоря, правые границы на единицу меньше (49 и 14), см. пояснения к тесту. Отвечай по ключу.

conio.h

`getch()` ждёт нажатия любой клавиши и **не показывает** её на экране (удобно в конце программы). `clrscr()` очищает экран.

printf: форматированный вывод (stdio.h)

Формат	Что выводит
`%d`, `%i`	целое со знаком
`%u`	беззнаковое десятичное
`%o`	беззнаковое восьмеричное
`%x`	беззнаковое шестнадцатеричное
`%f`	вещественное (`%.2f` — 2 знака после точки)
`%e`	вещественное в экспоненциальной форме
`%c`	символ
`%s`	строка

```C++
printf("%d %o %x %.2f %c %s\n", 255, 8, 255, 3.14159, 'Z', "ok");
printf("%e", 1500.0);
```

Вывод

```
255 10 ff 3.14 Z ok
1.500000e+03
```

Разбор задач

Задача 2.1. Гипотенуза лёгкая

Даны катеты a = 3, b = 4. Найди гипотенузу и площадь треугольника.

Решение

```C++
double a = 3, b = 4;
double c = sqrt(a * a + b * b);
double s = a * b / 2;
cout << "c=" << c << " S=" << s;
```

Вывод
```
c=5 S=6
```

Задача 2.2. Запись формулы в C++ лёгкая

Запиши в линейном виде и посчитай при x = 2: $y = (x^2 + 3x) / (2\sqrt{x} + 1) + |x - 5|$.

Решение

```C++
double x = 2;
double y = (x * x + 3 * x) / (2 * sqrt(x) + 1) + fabs(x - 5);
printf("%.4f", y);
```

Вывод
```
5.6120
```

Числитель и знаменатель — **в скобках**! Без скобок `x*x + 3*x / 2*sqrt(x) + 1` посчитается совсем другое.

Задача 2.3. Чётность и знак без if средняя

Для n = −7 выведи: «even»/«odd» и «positive»/«negative»/«zero», используя только тернарный оператор.

Решение

```C++
int n = -7;
cout << (n % 2 == 0 ? "even" : "odd") << " "
     << (n > 0 ? "positive" : n < 0 ? "negative" : "zero");
```

Вывод
```
odd negative
```

Внимание: `-7 % 2` = −1, а не 1. Поэтому проверять нечётность через `n % 2 == 1` для отрицательных **нельзя**, а через `n % 2 != 0` или `n % 2 == 0` можно.

Задача 2.4. Пошаговое вычисление средняя

Что выведет программа? Пройди по шагам.

```cpp
int a = 10, b = 3;
a %= b + 1;
b *= a++ + 2;
int c = a > b ? a - b : b - a;
cout << a << " " << b << " " << c;
```

Решение

> **Вывод**
> `3 12 9`

`a %= 4` → a = 2. `b *= a++ + 2` → b = 3·(2 + 2) = 12, затем a = 3. `3 > 12` ложно → c = 12 − 3 = 9.

Задача 2.5. Деньги без потери копеек сложная

Сумма в тенге с копейками хранится как `double sum = 1234.56;`. Раздели её на купюры по 1000 и 100 и выведи остаток в копейках, без ошибок округления.

Решение

```cpp
double sum = 1234.56;
int kop = (int)(sum * 100 + 0.5);   // всё в целые копейки, +0.5 для округления
int k1000 = kop / 100000;
kop %= 100000;
int k100 = kop / 10000;
kop %= 10000;
cout << k1000 << " x1000, " << k100 << " x100, rest " << kop << " kop";
```

> **Вывод**
> `1 x1000, 2 x100, rest 3456 kop`

Фишка: вещественные числа хранятся неточно (1234.56·100 может оказаться 123455.9999…), поэтому переходим к целым с округлением `+0.5`.

Задача 2.6. Биты сложная

Число n = 45. С помощью битовых операций: проверь, нечётное ли оно; умножь на 8; найди остаток от деления на 4; установи бит №1 (`| 2`).

Решение

```cpp
int n = 45;   // 101101
cout << (n & 1) << " " << (n << 3) << " " << (n & 3) << " " << (n | 2);
```

Вывод

```
1 360 1 47
```

`n & (2^k - 1)` — это остаток от деления на 2^k.

Контрольные вопросы из силлабуса (устно)

1. **Что такое выражение?** — Комбинация операндов, знаков операций и скобок, дающая значение определённого типа.
2. **Операнды и операции?** — Операнд — над чем действуем; операция — само действие (+, *, = ...).
3. **Арифметические операции?** — `+ - * / %`, плюс унарные `-`, `++`, `--`.
4. **Операции сравнения и логические?** — `== != < > <= >=`; `&& || !`.
5. **Присваивание и составное присваивание?** — `=`; `+= -= *= /= %=` (например, `x += 3` ≡ `x = x + 3`).
6. **Приоритет и ассоциативность?** — Порядок выполнения операций; ассоциативность — направление при равном приоритете (присваивание справа налево, арифметика слева направо).
7. **Зачем дополнительные библиотеки?** — Готовые функции: `math.h` (sqrt, pow), `stdlib.h` (random), `string.h`, `conio.h`.
8. **Директива препроцессора, #include?** — Команда, выполняемая до компиляции; `#include` вставляет содержимое заголовочного файла.
9. **Как хранятся данные в памяти?** — В ячейках (байтах) с адресами; переменная занимает `sizeof(тип)` байт подряд.
10. **Что такое указатель?** — Переменная, хранящая адрес другой переменной.
11. **& и *?** — `&x` берёт адрес; `*p` даёт значение по адресу.
12. **Доступ через указатель?** — `int *p = &x; *p = 10;` (меняет x).

Тест по лекции 2

1. Which header file contains the prototype of the rand() function? банк №8

a) #include <conio.h>
b) #include <stdio.h>
c) #include <iostream.h>
d) #include <math.h>
e) #include <stdlib.h>

2. If float and double types are mixed, what is the result? банк №10

a) int;
b) double
c) float;
d) short;
e) long;

3. If long and float types are mixed, what is the result? банк №19

a) int
b) double
c) char
d) float
e) No correct answer

4. How is the remainder (modulo) operation written? банк №20

a) /
b) <<
c) &
d) %
e) \

5. In which option is sizeof written correctly? банк №22

a) sizeof [type]
b) sizeof: type
c) sizeof type
d) sizeof_ type
e) sizeof (type)

6. Which option correctly shows logical negation? банк №23

a) & (ampersand)
b) ! (exclamation mark)
c) ? (question mark)
d) \ (backslash)
e) = (equals sign)

7. If short and int types are mixed, the result is ... банк №27

a) int
b) short
c) float
d) double
e) char

8. What does the *= operation mean? банк №30

a) comment
b) multiply and assign
c) multiply each element
d) assignment
e) multiplication

9. What is the printf() format for printing a real number? банк №35

a) %f
b) %u
c) %o
d) %x
e) No correct answer

10. In C, what is the value of the expression 5/2? банк №45

a) 1
b) 2.5
c) 2
d) 5
e) No correct answer

11. In C, what is the value of the expression 7/2? банк №46

a) 5
b) 2
c) 3.5
d) 3
e) No correct answer

12. In C, what is the value of the expression 7/3? банк №47

a) 3
b) 2
c) 3.3
d) 2.5
e) No correct answer

13. In C, what is the value of the expression 5%2? банк №48

a) 1
b) 2
c) 3
d) 2.5
e) No correct answer

14. In C, what is the value of the expression 7%2? банк №49

a) 1
b) 3.5
c) 3
d) 4
e) No correct answer

15. In C, which operation finds the quotient of dividing an integer by an integer (like the div function in Pascal)? банк №50

a) /
b) &
c) \
d) %
e) No correct answer

16. In C, which expression does NOT increase a variable's value by 1? банк №51

a) x+=1
b) x*=1
c) x++
d) x=x+1
e) No correct answer

17. In C, how is the statement that produces a random integer in the range [-20, 50] written? банк №52

a) random(50)-20;
b) random(30)+20;
c) random(70)-20;
d) random(50);
e) No correct answer

18. In C, as a result of x= random(20)-5, what range is the value of x in? банк №53

a) [5,15]
b) [-5,20]
c) [5,20]
d) [-5,15]
e) No correct answer

19. In Borland C++ Builder, which statement starts the random number generator so that different random numbers are produced on every run of the program? банк №54

a) getch();
b) main();
c) randomize();
d) time();
e) No correct answer

20. In C, after `x=0; x++; x=x+1; x=x*x;` what is the value of x? банк №55

a) 3
b) 4
c) 2
d) 0
e) No correct answer

21. In C, after `x=0; x++; x=x*1; x=x*x;` what is the value of x? банк №56

a) 2
b) 0
c) 4
d) 1
e) No correct answer

22. In C, which statement is n%=3 equivalent to? банк №57

a) n=n*3
b) n=n%3
c) n=n\3
d) n=n/3
e) No correct answer

23. In C, show the correct program for calculating the volume of a cube with a given side: банк №58

a) v=a*a;
b) v=pow (a,4);
c) v=pow (a,3);
d) v=pow (a,2);
e) No correct answer

24. In C, what does the program `{ float x=60, y; y=atan(x*3.14/180); cout<< y;}` calculate? банк №60

a) arccotangent
b) tangent
c) cotangent
d) arctangent
e) No correct answer

25. In C, which file defines the standard function `pow(x,y);`? банк №61

a) math.h
b) conio.h
c) stdlib.h
d) iostream.h
e) No correct answer

26. In C, what is the result of `cout<<sqrt(pow(5,4));`? банк №62

a) 5
b) 625
c) 25
d) 15
e) No correct answer

27. In C, show the unary increment operation: банк №63

a) /*
b) --
c) //
d) ++
e) No correct answer

28. In C, show the unary decrement operation: банк №64

a) */
b) --
c) //
d) ++
e) No correct answer

29. How is the decimal logarithm lg x written in C? банк №65

a) ln (x)
b) log10(x)
c) lg(x)
d) log (10x)
e) No correct answer

30. In C, if x=2 initially, what is the value of x after x*=(x+1)? банк №66

a) 4
b) 6
c) 5
d) 3
e) No correct answer

31. In C, which function returns the absolute value of a real-type argument? банк №67

a) fabc (x)
b) labs (x)
c) absx (x)
d) fabs (x)
e) No correct answer

32. In C. Which directive is included to use input-output commands? банк №68

a) # include <iostream.h>
b) # include <math.h>
c) # include <conio.h>
d) # include <stdio.h>
e) No correct answer

33. In C. The function of the output operator: банк №69

a) getch()
b) cin>>
c) cout<<
d) scanf()
e) No correct answer

34. The input operator in C++: банк №70

a) getch()
b) printf()
c) cout<<
d) cin>>
e) No correct answer

35. In C. What does the getch(); statement do? банк №71

a) returns the value of a function
b) marks the end of the program
c) waits for any key to be pressed, but does not display it on the screen
d) clears the screen
e) No correct answer

36. In C. Which header file defines the getch() function? банк №83

a) iostream.h
b) conio.h
c) stdio.h
d) stdlib.h
e) No correct answer

37. The screen-clearing function in C: банк №85

a) clrscr();
b) cls();
c) clean();
d) clear screen();
e) No correct answer

38. In C. Which header file is included to use the screen-clearing function? банк №86

a) stdio.h
b) iostream.h
c) conio.h
d) stdlib.h
e) No correct answer

39. In C. Which header file is included to use the random number function? банк №87

a) stdio.h
b) conio.h
c) iostream.h
d) stdlib.h
e) No correct answer

40. In C, which number does «false» correspond to? банк №88

a) -1
b) it is not equated to any number
c) 0
d) 1
e) No correct answer

41. In C, which number does «true» correspond to? банк №89

a) 1
b) it is not equated to any number
c) 0
d) -1
e) No correct answer

42. In C. What does 1 as the result of evaluating a condition mean? банк №90

a) the result of correct compilation of the program
b) false
c) true
d) boolean
e) No correct answer

43. In C. What does 0 as the result of evaluating a condition mean? банк №91

a) true
b) boolean
c) the result of correct compilation of the program
d) false
e) No correct answer

44. In C, is the expression (1 != 1) true or false? банк №92

a) meaningless
b) unknown
c) false
d) true
e) No correct answer

45. In C. An expression equivalent to s=s+x банк №96

a) s=+x
b) s+=x
c) s=x++
d) s=s++
e) No correct answer

46. How is the arccos(x) function written in C? банк №99

a) acos(x)
b) cosa(x)
c) acos{x}
d) acos[x]
e) No correct answer

47. What is the quoted phrase in the cout<< operator used for? банк №100

a) for clarity; it can also be left out;
b) to move to the next line;
c) so as not to move to the next line
d) to print a question mark;
e) No correct answer

48. Which logical operation is called “logical multiplication” (AND)? банк №101

a) ||
b) !
c) :
d) &&
e) No correct answer

49. Which logical operation is called “logical sum” (OR)? банк №102

a) ||
b) :
c) !
d) &&
e) No correct answer

50. What is the value of the expression k=(3/2)*(3%2)? банк №103

a) 3
b) 2
c) 1
d) 0.5
e) No correct answer

51. Given the declarations: What is the type of the value of the expression d+1? банк №104

```
int a,b,c,n; float d,e;
```

a) char
b) bool
c) int
d) float
e) No correct answer

52. The operator for entering the values of variables into the computer: банк №105

a) getch()
b) printf()
c) cin>>
d) cout<<
e) No correct answer

53. What is the value of the variable k in `k=ceil(3.226);`? банк №108

a) 3.5
b) 3
c) 2
d) 4
e) No correct answer

54. What is the value of the variable k in `k=floor(3.63);`? банк №109

a) 3.5
b) 2
c) 3
d) 4
e) No correct answer

55. Which function is used to find the square root of a value? банк №112

a) sqrt()
b) sgrt()
c) pov()
d) sqr()
e) No correct answer

56. How do we write x to the power of y in C? банк №113

a) sqrt(x,y)
b) pow(x,y)
c) povv(x,y)
d) sqr(x,y)
e) No correct answer

57. How do we write the 5th root of x? банк №114

a) sqrt(x,1/5)
b) pow(x,1/5.)
c) sqrt(x,5)
d) pow(x,1/5)
e) No correct answer

58. What is the printf() format for printing a string? банк №115

a) %o
b) %u
c) %f
d) %s
e) No correct answer

59. What is the printf() format for printing a real number? банк №116

a) %f
b) %o
c) %s
d) %u
e) No correct answer

60. The printf() format for printing an unsigned decimal integer: банк №117

a) %u
b) %o
c) %s
d) %f
e) No correct answer

61. The printf() format for printing an unsigned octal integer: банк №118

a) %s
b) %f
c) %u
d) %o
e) No correct answer

62. What is the printf() format for printing an unsigned hexadecimal integer? банк №119

a) %s
b) %x
c) %f
d) %u
e) No correct answer

63. What is the printf() format for printing a real number in exponential form? банк №120

a) %x
b) %f
c) %e
d) %s
e) No correct answer

64. What answer do we get? банк №133

```
int i=10;
cout <<  i++;
cout  << ++i;
```

a) 10, 10
b) 10, 12
c) 10, 11
d) 11, 12
e) No correct answer

65. Show the correct way to calculate x to the power of 4. банк №141

a) pow(x/4);
b) pow(x*4);
c) pow(x,4);
d) pow(x;4);
e) No correct answer

66. What is the mathematical library for calculating standard arithmetic functions called? банк №165

a) math.h
b) match.h
c) matem.h
d) stdlib.h
e) No correct answer

67. What will the code below print to the screen? банк №170

```
int const a = 5; a++; cout << a;
```

a) the code shows an error
b) 5
c) 0
d) 6
e) No correct answer

68. The function that finds the absolute value of integers: банк №174

a) abs(x)
b) fabc(x)
c) fabs(x)
d) abc(x)
e) No correct answer

69. How is arccosine written in C++? банк №175

a) acos{x}
b) asin[x]
c) acos(x)
d) atan(x)
e) No correct answer

70. The function that rounds a number up: банк №176

a) log(x)
b) floor(x)
c) fabs(x)
d) ceil(x)
e) No correct answer

71. The function that rounds a number down: банк №177

a) fabs(x)
b) exp(x)
c) floor(x)
d) ceil(x)
e) No correct answer

72. Which of the following is NOT a comparison sign? банк №179

a) <
b) =
c) ==
d) !=
e) No correct answer

73. The increment operation is applied: банк №186

a) to constants
b) to expressions
c) to variables
d) to strings
e) No correct answer

74. Which numbers is the % operation applied to? банк №187

a) integer and real
b) fractional
c) real
d) integer
e) No correct answer

75. Logical “and” is denoted as follows: банк №188

a) ||
b) &
c) &&
d) |
e) No correct answer

76. Logical “or” is denoted as follows: банк №189

a) &&
b) ||
c) &
d) |
e) No correct answer

77. Logical “not equal” is denoted as follows: банк №190

a) |=
b) =!
c) =|
d) !=
e) No correct answer

78. What is the output? 20 сек · B1·2

```
int x = 7;
x += 4;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

79. What is the output? 20 сек · B1·3

```
int a = 17, b = 5;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

80. What is the output? 20 сек · B2·2

```
int x = 5;
cout << x++ + ++x;
```

Напиши, что выведет программа.

81. What is the output? 20 сек · B2·3

```
int a = 7, b = 2;
double x = a / b;
double y = (double)a / b;
cout << x << " " << y;
```

Напиши, что выведет программа.

82. What is the output? 20 сек · B3·2

```
int x = 10;
x -= 3;
x *= 2;
x /= 2;
cout << x;
```

Напиши, что выведет программа.

83. What is the output? 20 сек · B3·3

```
int x = 10;
cout << x-- << " " << --x;
```

Напиши, что выведет программа.

84. What is the output? 20 сек · B4·2

```
int a = 4;
a *= 3;
a -= 2;
a /= 2;
cout << a;
```

Напиши, что выведет программа.

85. What is the output? 20 сек · B4·3

```
int a = 19, b = 4;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

86. What is the output? 20 сек · B5·2

```
int x = 6;
x++;
x *= 3;
cout << x;
```

Напиши, что выведет программа.

87. What is the output? 20 сек · B5·3

```
int a = 5, b = 2;
cout << a * b + a % b;
```

Напиши, что выведет программа.

88. What is the output? 20 сек · B6·2

```
int x = 8;
x /= 2;
x += 7;
cout << x;
```

Напиши, что выведет программа.

89. What is the output? 20 сек · B6·3

```
int x = 5;
cout << ++x << " " << x++;
```

Напиши, что выведет программа.

90. What is the output? 20 сек · B7·2

```
int x = 3;
x = x + 4 * 2;
cout << x;
```

Напиши, что выведет программа.

91. What is the output? 20 сек · B7·3

```
int a = 20, b = 6;
cout << a % b + a / b;
```

Напиши, что выведет программа.

92. What is the output? 20 сек · B8·2

```
int x = 4;
x *= 2;
x++;
x -= 3;
cout << x;
```

Напиши, что выведет программа.

93. What is the output? 20 сек · B8·3

```
int a = 15, b = 4;
cout << (a % b) * 2;
```

Напиши, что выведет программа.

94. What is the output? 20 сек · B9·2

```
int x = 9;
x -= 4;
x *= 3;
cout << x;
```

Напиши, что выведет программа.

95. What is the output? 20 сек · B9·3

```
int a = 14, b = 5;
cout << a / b * b + a % b;
```

Напиши, что выведет программа.

96. What is the output? 20 сек · B10·2

```
int x = 5;
x += 5;
x /= 2;
cout << x;
```

Напиши, что выведет программа.

97. What is the output? 20 сек · B10·3

```
int x = 6;
cout << x++ * 2;
```

Напиши, что выведет программа.

98. What is the output? 20 сек · B11·2

```
int x = 7;
x *= 2;
x -= 4;
cout << x;
```

Напиши, что выведет программа.

99. What is the output? 20 сек · B11·3

```
int a = 23, b = 7;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

100. What is the output? 20 сек · B12·2

```
int x = 3;
x = x * 4 - 2;
cout << x;
```

Напиши, что выведет программа.

101. What is the output? 20 сек · B12·3

```
int a = 18, b = 5;
cout << a % b * 2 + 1;
```

Напиши, что выведет программа.

102. What is the output? 20 сек · B13·2

```
int x = 6;
x++;
x++;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

103. What is the output? 20 сек · B13·3

```
int a = 21, b = 4;
cout << a / b + a % b;
```

Напиши, что выведет программа.

104. What is the output? 20 сек · B14·2

```
int x = 9;
x /= 3;
x += 8;
cout << x;
```

Напиши, что выведет программа.

105. What is the output? 20 сек · B14·3

```
int x = 5;
cout << ++x + x++;
```

Напиши, что выведет программа.

106. What is the output? 20 сек · B15·2

```
int x = 4;
x += 6;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

107. What is the output? 20 сек · B15·3

```
int a = 16, b = 3;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

108. What is the value of `double d = 7 / 2;`? новый

a) 3.5
b) 3
c) 4
d) 3.0 is impossible

109. Which expression correctly checks that `x` is in the range [1, 10]? новый

a) 1 <= x <= 10
b) x >= 1 || x <= 10
c) x >= 1 && x <= 10
d) x => 1 && x =< 10

110. What does `cout << (17 >> 2);` print? новый

a) 68
b) 4
c) 8
d) 15

111. What is the output? сложный

```
int a = 5, b = 0;
if (b != 0 && a / b > 1) cout << "A";
else if (a > 3 || ++b) cout << "B";
cout << b;
```

a) A0
b) B1
c) B0
d) Runtime error (division by zero)

112. What is the output? сложный

```
int x = 3;
x += x++ * 0 + 2;
int y = 10;
y = y / 3 * 3 + y % 3;
int z = (y, x, 7);
cout << y << " " << z;
```

a) 9 7
b) 10 7
c) 10 3
d) 10 10

113. What is the output? сложный

```
int a = 6, b = 4;
cout << (a & b) << (a | b) << (a ^ b) << (a && b);
```

a) 1111
b) 4621
c) 4610
d) 6421

114. What is the output? сложный

```
double x = 2.7;
int k = (int)x + (int)(x * 2) % 3;
cout << k << " " << ceil(-2.5) << " " << floor(-2.5);
```

a) 5 -3 -2
b) 4 -2 -3
c) 4 -3 -2
d) 3 -2 -3

115. What is the output? сложный

```
int i = 5;
int j = i++ * 2;
j += --i;
cout << j;
```

a) 16
b) 15
c) 14
d) 17

116. Which header file must be included to use `sqrt()` and `pow()`? новый

a) stdlib.h
b) conio.h
c) math.h
d) string.h

Ключи

1 — e	**2** — b	**3** — d	**4** — d	**5** — e	**6** — b
7 — a	**8** — b	**9** — a	**10** — c	**11** — d	**12** — b
13 — a	**14** — a	**15** — a	**16** — b	**17** — c	**18** — d
19 — c	**20** — b	**21** — d	**22** — b	**23** — c	**24** — d
25 — a	**26** — c	**27** — d	**28** — b	**29** — b	**30** — b
31 — d	**32** — a	**33** — c	**34** — d	**35** — c	**36** — b
37 — a	**38** — c	**39** — d	**40** — c	**41** — a	**42** — c
43 — d	**44** — c	**45** — b	**46** — a	**47** — a	**48** — d
49 — a	**50** — c	**51** — d	**52** — c	**53** — d	**54** — c
55 — a	**56** — b	**57** — b	**58** — d	**59** — a	**60** — a
61 — d	**62** — b	**63** — c	**64** — b	**65** — c	**66** — a
67 — a	**68** — a	**69** — c	**70** — d	**71** — c	**72** — b
73 — c	**74** — d	**75** — c	**76** — b	**77** — d	**78** — 22
79 — 3 2	**80** — 12	**81** — 3 3.5	**82** — 7 **(в ключе 12)**	**83** — 9 9 **(в ключе 19)**	**84** — 5
85 — 4 3	**86** — 21	**87** — 11	**88** — 11	**89** — 7 5 **(в ключе 6 6)**	**90** — 11
91 — 5	**92** — 6 **(в ключе 12)**	**93** — 6	**94** — 15	**95** — 14 **(в ключе 4 2)**	**96** — 5
97 — 12	**98** — 10	**99** — 3 2	**100** — 10	**101** — 7	**102** — 16
103 — 6	**104** — 11	**105** — 12	**106** — 20	**107** — 5 1	**108** — b
109 — c	**110** — b	**111** — c	**112** — b	**113** — b	**114** — b
115 — b	**116** — c				

17. Ключ преподавателя — `random(70)-20`, отвечай так. Но строго говоря `random(n)` в Borland даёт числа от 0 до n−1, поэтому `random(70)-20` даёт [−20; 49]. Чтобы попасть ровно в [−20; 50], нужно `random(71)-20` (в вариантах такого нет).

18. Ключ — [−5,15], отвечай так. Точно: `random(20)` даёт 0…19, значит x ∈ [−5; 14]. Преподаватель считает границы «на глаз».

26. `pow(5,4)` = 625, `sqrt(625)` = 25.

47. Формулировка странная (перевод с казахского), но в ключе — «для понятности, можно и не писать»: текст в кавычках просто подписывает вывод, программа работает и без него.

50. `3/2` = 1 (целое деление), `3%2` = 1, 1·1 = 1.

64. `i++` печатает старое значение 10 (потом i = 11), `++i` сначала увеличивает до 12 и печатает 12.

80. Неопределённое поведение (x меняется дважды). Borland 6 даёт 12 — как в ключе. Запомни ответ, но так не программируй.

82. 10−3 = 7, 7·2 = 14, 14/2 = 7. В ключе 12 — ошибка.

83. Неопределённое поведение: x меняется дважды в одном выражении. Borland 6 считает `cout << a << b` справа налево и печатает «9 9»; современный C++17 напечатает «10 8». В ключе «19» — неверно в любом случае.

89. Неопределённое поведение (x меняется дважды в одном `cout`). Borland 6 вычисляет справа налево: сначала `x++` (даёт 5, x = 6), потом `++x` (x = 7) → «7 5». В C++17 будет «6 6» — как в ключе. На тесте пиши ответ по ключу (6 6), но в коде так не пиши.

92. 4·2 = 8, ++ → 9, 9−3 = 6. В ключе 12 — ошибка.

95. 14/5 = 2, 2·5 = 10, 14%5 = 4, 10 + 4 = 14 (выводится одно число). В ключе «4 2» — ошибка.

105. Неопределённое поведение. Borland 6 даёт 12 — как в ключе.

108. 7/2 считается в целых = 3, потом 3 превращается в 3.0 (выводится как 3).

110. Сдвиг вправо на 2 = целочисленное деление на 4: 17/4 = 4.

111. В `&&` левая часть ложна → деление не выполняется. В `||` левая часть истинна → `++b` не выполняется, b остаётся 0.

112. y/3*3 + y%3 восстанавливает y = 10 всегда. Запятая возвращает последнее значение 7. (Значение x здесь не печатаем — там неопределённое поведение.)

113. 110 & 100 = 100 (4); 110 | 100 = 110 (6); 110 ^ 100 = 010 (2); 6 && 4 = 1.

114. (int)2.7 = 2; x·2 = 5.4 → 5, 5 % 3 = 2; 2 + 2 = 4. ceil округляет вверх (−2.5 → −2), floor вниз (−2.5 → −3).

115. i++ отдаёт 5 (после этого i = 6), j = 10. Затем --i делает i = 5, j = 10 + 5 = 15. Здесь всё однозначно: каждое изменение i — в своём операторе.

ЛЕКЦИЯ 3 ИЗ 15

Ветвления if, switch, goto и цикл for

Научиться писать программы, которые принимают решения и повторяют действия заданное число раз.

3.1 Управляющие конструкции

Без них программа выполняется сверху вниз строго по порядку (линейно). **Управляющие конструкции** меняют порядок выполнения:

- **ветвление** — `if`, `switch` (выбрать путь);
- **цикл** — `for`, `while`, `do…while` (повторить);
- **переход** — `goto`, `break`, `continue`, `return` (прыгнуть).

3.2 Условный оператор if

```
if (условие)            if (условие)
    оператор;               оператор1;
                        else
                            оператор2;
```

- **Сокращённая форма** (`if` без `else`): если условие ложно, ничего не делаем.
- **Полная форма** (`if…else`): выполняется ровно одна из двух ветвей.
- Если в ветви **несколько** операторов, их обязательно заключают в `{ }` (составной оператор, блок).

```cpp
int age = 17;
if (age >= 18)
    cout << "adult";
else
    cout << "minor";
cout << endl;
int t = 30;
if (t > 25) {
    cout << "hot, ";
    cout << "drink water";
}
```

Вывод

```
minor
hot, drink water
```

Ловушка: забыли фигурные скобки

Без `{ }` к `if` относится **только один** следующий оператор, отступы для компилятора ничего не значат:

НЕПРАВИЛЬНО

```
int x = 0;
if (x > 5)
    cout << "big ";
    cout << "really";   // выполнится ВСЕГДА
```

ПРАВИЛЬНО

```
int x = 0;
if (x > 5) {
    cout << "big ";
    cout << "really";   // только если x > 5
}
```

Ловушка: точка с запятой после условия

`if (x > 5);` — здесь `;` это **пустой оператор**, он и есть тело if. Следующий блок выполнится всегда.

Лесенка else-if: выбор из нескольких вариантов

C++

```
int score = 78;
char grade;
if (score >= 90) grade = 'A';
else if (score >= 75) grade = 'B';
else if (score >= 60) grade = 'C';
else grade = 'F';
cout << grade;
```

Вывод

```
B
```

Порядок проверок важен

Условия проверяются сверху вниз, выполняется **первое** истинное. Если начать с `score >= 60`, то и 95 баллов получат 'C'.

Вложенные if и «висячий else»

`else` относится к **ближайшему** `if` без else, независимо от отступов:

C++

```
int a = 5, b = -3;
if (a > 0)
    if (b > 0) cout << "both positive";
    else cout << "a>0, b<=0";
```

Вывод

```
a>0, b<=0
```

Лайфхак

Если нужно, чтобы else относился к внешнему if, ставь фигурные скобки вокруг внутреннего: `if (a > 0) { if (b > 0) … } else …`.

3.3 Оператор выбора switch

`switch` выбирает ветку по **значению** целочисленного выражения (int, char, enum).

```
switch (выражение) {
    case константа1: операторы; break;
    case константа2: операторы; break;
    ...
    default: операторы;      // если ни один case не подошёл (необязателен)
}
```

```C++
int day = 3;
switch (day) {
    case 1: cout << "Mon"; break;
    case 2: cout << "Tue"; break;
    case 3: cout << "Wed"; break;
    case 6:
    case 7: cout << "Weekend"; break;   // несколько case на одно действие
    default: cout << "Other";
}
```

Вывод
```
Wed
```

Как работает switch на самом деле

`switch` **прыгает** на подходящую метку `case` и выполняет **всё ниже**, пока не встретит `break` или конец switch. `break` — выход из switch. Если break забыли, выполнение **«проваливается» (fall-through)** в следующие case.

```C++
int n = 2, s = 0;
switch (n) {
    case 1: s += 1;
    case 2: s += 10;    // вход здесь
    case 3: s += 100;   // провалились
    default: s += 1000; // и сюда
}
cout << s;
```

Вывод
```
1110
```

Любимый тип вопроса

«What is the output?» со switch без break. Алгоритм: найди case, куда прыгнули, и выполняй всё вниз до первого `break`. Не складывай напечатанные числа! `cout << 7` и затем `cout << 9` дают на экране **79**, а не 16 (такая ошибка есть даже в ключе 20-секундного теста).

Ограничения switch:

- выражение и метки — только **целые** типы (`int` , `char` , `enum`); `double` и строки нельзя;
- метки `case` — **константы**, не переменные и не диапазоны (`case x > 5:` — ошибка);
- две одинаковые метки — ошибка компиляции;
- `default` может стоять где угодно, но выполняется, только если не подошёл ни один case.

Когда что использовать	
`if`	диапазоны (`x > 10`), сложные условия (`&&` , `||`), вещественные числа, строки
`switch`	выбор по конкретным значениям одной переменной (меню, номер дня, символ операции)

```C++
// Калькулятор: switch по символу
double a = 12, b = 5;
char op = '/';
switch (op) {
    case '+': cout << a + b; break;
    case '-': cout << a - b; break;
    case '*': cout << a * b; break;
    case '/': if (b != 0) cout << a / b; else cout << "div by 0"; break;
    default: cout << "unknown op";
}
```

Вывод
```
2.4
```

3.4 Оператор goto и метки

`goto метка;` — безусловный переход к строке с меткой `метка:` .

```C++
int i = 1;
again:
cout << i << " ";
i++;
if (i <= 3) goto again;
```

Вывод
```
1 2 3
```

Почему goto не любят

С `goto` программа превращается в «спагетти»: трудно понять, откуда пришли в строку. Структурное программирование запрещает его. Единственное оправданное применение — выход сразу из нескольких вложенных циклов. Нельзя прыгать внутрь блока через объявление переменной с инициализацией.

3.5 Цикл for (цикл с параметром, со счётчиком)

```
for (инициализация; условие; изменение)
    тело;
```

Порядок выполнения:

1. **Инициализация** — один раз в начале (`int i = 1`).
2. Проверка **условия**; если ложно, цикл заканчивается.
3. Выполняется **тело**.

4. Выполняется **изменение** (`i++`), и снова шаг 2.

```cpp
for (int i = 1; i <= 5; i++) cout << i << " ";
cout << endl;
for (int i = 10; i > 0; i -= 3) cout << i << " ";
cout << endl;
for (int i = 1; i <= 100; i *= 3) cout << i << " ";
```

Вывод

```
1 2 3 4 5
10 7 4 1
1 3 9 27 81
```

Сколько раз выполнится цикл?

Для `for (i = a; i <= b; i += h)` число повторений = `(b - a) / h + 1` (целая часть). Для `i < b` сначала проверь, попадает ли граница. Пример из теста: `for (x=1; x<=10; x=x+2)` → x = 1, 3, 5, 7, 9 → **5 раз**.

Значение счётчика после цикла

Цикл заканчивается, когда условие стало **ложным**. Поэтому после `for (a = 0; a < 10; a++)` переменная a = **10**, а не 9. (В банке тестов здесь ошибка в ключе.)

```cpp
int a;
for (a = 0; a < 10; a++) ;
cout << a;
```

Вывод

```
10
```

Типичные шаблоны с for:

```cpp
int n = 5;
int sum = 0, fact = 1;
for (int i = 1; i <= n; i++) {
    sum += i;       // сумма 1..n
    fact *= i;      // факториал n!
}
cout << sum << " " << fact << endl;
int cnt = 0;
for (int x = 1; x <= 30; x++)
    if (x % 3 == 0 && x % 2 != 0) cnt++;    // нечётные кратные 3
cout << cnt;
```

Вывод

```
15 120
5
```

Любую часть заголовка можно опустить, но `;` остаются: `for (;;)` — бесконечный цикл (выходят через `break`).

Частые ошибки с for

- `for (i = 0; i < 10; i++);` — `;` после скобки: тело пустое, цикл «прокрутится» вхолостую, а следующий оператор выполнится один раз.
- Запятая вместо `;`: `for (i = 0, i < 10, i++)` — ошибка.
- Счётчик меняется в неверную сторону: `for (i = 10; i > 0; i++)` — бесконечный цикл (на самом деле до переполнения).
- Сравнение вещественного счётчика на равенство: `for (x = 0; x != 1; x += 0.1)` может не остановиться (0.1 хранится неточно). Используй `x <= 1 + 1e-9`.
- Переменная, объявленная в `for (int i…)`, видна только внутри цикла.

Вложенные циклы

Внутренний цикл полностью выполняется на **каждом** шаге внешнего.

```cpp
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) cout << "*";
    cout << endl;
}
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) cout << i * j << "\t";
    cout << endl;
}
```

Вывод

```
*
**
***
1       2       3
2       4       6
3       6       9
```

Фишка / совет

Общее число повторений вложенного цикла = произведение: внешний 10 × внутренний 20 = 200 выполнений тела.

Разбор задач

Задача 3.1. Максимум из трёх лёгкая

Даны a = 7, b = 12, c = 9. Найди наибольшее с помощью if.

Решение

```cpp
int a = 7, b = 12, c = 9;
int mx = a;
if (b > mx) mx = b;
if (c > mx) mx = c;
cout << mx;
```

Вывод

```
12
```

Приём «предполагаем, что максимум — первый, и обновляем» работает для любого количества чисел.

Задача 3.2. Високосный год лёгкая

Год високосный, если делится на 4 и не делится на 100, либо делится на 400. Проверь 1900, 2000, 2024.

Решение

```cpp
int y[3] = {1900, 2000, 2024};
for (int i = 0; i < 3; i++) {
    int g = y[i];
    if ((g % 4 == 0 && g % 100 != 0) || g % 400 == 0) cout << g << " leap" << endl;
    else cout << g << " not leap" << endl;
}
```

Вывод

```
1900 not leap
2000 leap
2024 leap
```

Задача 3.3. Квадратное уравнение средняя

Реши $ax^2 + bx + c = 0$ для a = 1, b = −5, c = 6 с разбором всех случаев дискриминанта.

Решение

```cpp
double a = 1, b = -5, c = 6;
double d = b * b - 4 * a * c;
if (d > 0) {
    double x1 = (-b + sqrt(d)) / (2 * a);
    double x2 = (-b - sqrt(d)) / (2 * a);
    cout << "x1=" << x1 << " x2=" << x2;
} else if (d == 0) {
    cout << "x=" << -b / (2 * a);
} else {
    cout << "no real roots";
}
```

Вывод

```
x1=3 x2=2
```

Обрати внимание на скобки `(2 * a)`: `-b / 2 * a` — это (−b/2)·a.

Задача 3.4. Меню через switch средняя

По номеру месяца m = 4 выведи время года и число дней (февраль — 28).

Решение

```cpp
int m = 4;
switch (m) {
    case 12: case 1: case 2: cout << "winter"; break;
    case 3: case 4: case 5: cout << "spring"; break;
    case 6: case 7: case 8: cout << "summer"; break;
    case 9: case 10: case 11: cout << "autumn"; break;
    default: cout << "error";
}
int days;
switch (m) {
    case 2: days = 28; break;
    case 4: case 6: case 9: case 11: days = 30; break;
    default: days = 31;
}
cout << " " << days;
```

Вывод

```
spring 30
```

Задача 3.5. Сумма ряда средняя

Найди S = 1 + 1/2 + 1/3 + … + 1/10 и S2 = 1 − 1/2 + 1/3 − … (10 слагаемых).

Решение

```C++
double s = 0, s2 = 0;
int sign = 1;
for (int i = 1; i <= 10; i++) {
    s += 1.0 / i;          // 1.0 — иначе 1/i = 0 для i > 1!
    s2 += sign * 1.0 / i;
    sign = -sign;
}
printf("%.4f %.4f", s, s2);
```

Вывод

```
2.9290 0.6456
```

Задача 3.6. Простые числа и делители сложная

Выведи все простые числа от 2 до 30 и количество делителей числа 36.

Решение

```C++
for (int n = 2; n <= 30; n++) {
    int prime = 1;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) { prime = 0; break; }
    if (prime) cout << n << " ";
}
int k = 0;
for (int d = 1; d <= 36; d++) if (36 % d == 0) k++;
cout << endl << k;
```

Вывод

```
2 3 5 7 11 13 17 19 23 29
9
```

Фишка: делители достаточно проверять до √n (`d * d <= n`), это намного быстрее.

Задача 3.7. Таблица истинности сложная

Выведи таблицу для `(a && !b) || c` по всем 8 комбинациям a, b, c ∈ {0, 1}.

Решение

```cpp
for (int a = 0; a <= 1; a++)
    for (int b = 0; b <= 1; b++)
        for (int c = 0; c <= 1; c++)
            cout << a << b << c << " " << ((a && !b) || c) << endl;
```

Вывод

```
000 0
001 1
010 0
011 1
100 1
101 1
110 0
111 1
```

Контрольные вопросы из силлабуса (устно)

1. **Что такое условный оператор?** — Оператор, выполняющий действия в зависимости от истинности условия; нужен для ветвления.
2. **Синтаксис if?** — `if (условие) оператор; [else оператор;]`, несколько операторов — в `{ }`.
3. **if, if...else, вложенный if?** — Сокращённая форма, полная форма, if внутри другого if (else — к ближайшему if).
4. **Назначение switch?** — Множественный выбор по значению целочисленного выражения.
5. **case, break, default?** — Метка значения; выход из switch; ветка «во всех остальных случаях».
6. **Отличия if и switch?** — if — любые условия, диапазоны, вещественные; switch — сравнение одного целого выражения с константами, удобнее для меню.
7. **Цикл со счётчиком?** — Цикл, число повторений которого известно заранее (`for`).
8. **Синтаксис for?** — `for (инициализация; условие; изменение) тело;`.
9. **Части заголовка for?** — Инициализация выполняется 1 раз; условие проверяется перед каждой итерацией; изменение — после каждой итерации.
10. **Вложенные циклы?** — Цикл внутри цикла; для таблиц, матриц, перебора пар.
11. **Как возникает бесконечный цикл?** — Условие никогда не становится ложным: `for(;;)`, счётчик меняется не в ту сторону, забыли изменить переменную в while.

Тест по лекции 3

1. Which word is used to write the selection (choice) operator? банк №18

a) switch
b) return
c) for
d) if
e) class

2. Which statement is used to exit a switch? банк №29

a) do
b) case
c) default
d) return
e) break

3. Which is the correct condition for checking that an array index is odd in C? банк №41

a) if (i%2=1)
b) if (i%2!=1)
c) if (i%2==1)
d) if (i%2==0)
e) No correct answer

4. In C, the correct condition for checking that an array index is even: банк №59

a) if (i%2==0)
b) if (a[i]%2==0)
c) if (i%2==1)
d) if (i%2=0)
e) No correct answer

5. In C. How many times does the loop repeat in the program below? банк №72

```
for (x=1; x <= 10; x=x+2)
```

a) 20
b) 5
c) 10
d) 15
e) No correct answer

6. In C. What is the value of k as a result of running the program? банк №76

```
k=0; for (x=1;x <=10; x++) if (x % 3==0) k++;
```

a) 3
b) 4
c) 5
d) 2
e) No correct answer

7. In C. What is the value of k as a result of running the program? банк №77

```
k=0; for (x=1;x <=10; x++) if (x % 5==0) k++;
```

a) 3
b) 4
c) 5
d) 2
e) No correct answer

8. In C. What is the value of k as a result of running the program? банк №78

```
k=0; for (x=1;x <=10; x++) if (x % 9==0) k++;
```

a) 4
b) 1
c) 3
d) 2
e) No correct answer

9. In C. The notation of the loop with a parameter: банк №81

a) for (initial value; final value; step)
b) do {loop body}while (condition)
c) switch () {}
d) while (condition) {loop body}
e) No correct answer

10. Which is the conditional operator? банк №110

a) if
b) for
c) while
d) switch
e) No correct answer

11. Which one is called the selection operator? банк №111

a) for
b) if
c) while
d) switch
e) No correct answer

12. What is a branching operator? банк №128

a) an operator that checks the fulfillment of only one condition
b) an operator that checks that conditions are not met
c) an operator responsible for the execution of any program in C
d) a division of an algorithm into several paths, branching depending on whether a certain condition is met or not
e) No correct answer

13. What values does i take in the statement `for(i=1;i<10;i+=2)` банк №140

a) 1 3 5 7 9
b) 2 4 6 8 10
c) 1 3 5 7 9 10
d) 1 2 3 4 5
e) No correct answer

14. What values does i take? банк №164

```
for (int i=100; i>0; i-=10)
```

a) 90,70,50,30,10
b) 100,90,80,70,60,50,40,30,20,10
c) 50,40,30,20,10
d) 100,80,60,40,20
e) No correct answer

15. What values does i take? банк №166

```
for (int i=100; i<=150; i+=10)
```

a) 100,110,120,130,140
b) 110,110,120,130,140,160
c) 110,120,130,140,150
d) 100,110,120,130,140,150
e) No correct answer

16. After the code below is executed, what is the final value of the variable a? банк №172

```
int a; for(a = 0; a < 10; a++) cout<<a;
```

a) 9
b) 0
c) 10
d) 1
e) No correct answer

17. If int n=3, what will the result be? банк №181

```
switch(n) {
case 2: cout << "aaa"; break;
case 3: cout << "bbb"; break;
default: cout << "vvv"; break; }
```

a) vvv
b) bbb
c) the code will not run
d) aaa
e) No correct answer

18. If int n=45, what will the result be? банк №182

```
switch(n) {
case 23: cout << "aaa";
case 45: cout << "bbb";
default: cout << "vvv";
break; }
```

a) bbbvvv
b) bbb
c) vvv
d) aaa
e) No correct answer

19. What is the output? 20 сек · B1·4

```
int x = 12;
if (x % 2 == 0 && x > 10)
    cout << x / 2;
else
    cout << x * 2;
```

Напиши, что выведет программа.

20. What is the output? 20 сек · B1·5

```
int x = 2;
switch (x) {
    case 1: cout << 5; break;
    case 2: cout << 7;
    case 3: cout << 9; break;
    default: cout << 0;
}
```

Напиши, что выведет программа.

21. What is the output? 20 сек · B2·4

```
int x = 8;
if (x > 5) {
    x += 2;
    if (x > 10)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

22. What is the output? 20 сек · B2·5

```
int x = 3, y = 2;
switch (x) {
    case 1: y += 5; break;
    case 2: y *= 2; break;
    case 3: y += 4;
    case 4: y *= 2; break;
}
cout << y;
```

Напиши, что выведет программа.

23. What is the output? 20 сек · B3·4

```
int x = 15;
if (x % 3 == 0)
    x /= 3;
else
    x += 3;
if (x < 10)
    x += 5;
cout << x;
```

Напиши, что выведет программа.

24. What is the output? 20 сек · B3·5

```
int x = 5;
for (int i = 1; i <= 4; i++)
    x += i;
cout << x;
```

Напиши, что выведет программа.

25. What is the output? 20 сек · B4·4

```
int x = 7;
if (x < 5 || x > 10)
    cout << "A";
else
    cout << "B";
```

Напиши, что выведет программа.

26. What is the output? 20 сек · B4·5

```
int s = 0;
for (int i = 1; i <= 6; i++)
    if (i % 2 == 0)
        s += i;
cout << s;
```

Напиши, что выведет программа.

27. What is the output? 20 сек · B5·4

```
int x = 9;
if (x % 2 == 0)
    cout << x / 2;
else
    cout << x + 2;
```

Напиши, что выведет программа.

28. What is the output? 20 сек · B5·5

```
int x = 2;
switch (x) {
    case 1: x += 10; break;
    case 2: x += 3;
    case 3: x *= 2; break;
    default: x = 0;
}
cout << x;
```

Напиши, что выведет программа.

29. What is the output? 20 сек · B6·4

```
int x = 14;
if (x % 2 == 0) {
    x /= 2;
    if (x > 5)
        x -= 2;
}
cout << x;
```

Напиши, что выведет программа.

30. What is the output? 20 сек · B6·5

```
int x = 4;
for (int i = 1; i <= 3; i++)
    x += i * 2;
cout << x;
```

Напиши, что выведет программа.

31. What is the output? 20 сек · B7·4

```
int x = 11;
if (x > 10)
    x -= 5;
else
    x += 5;
cout << x;
```

Напиши, что выведет программа.

32. What is the output? 20 сек · B7·5

```
int s = 0;
for (int i = 2; i <= 8; i += 2)
    s += i * i;
cout << s;
```

Напиши, что выведет программа.

33. What is the output? 20 сек · B8·4

```
int x = 6;
if (x > 3 && x < 10)
    cout << x * 2;
else
    cout << x / 2;
```

Напиши, что выведет программа.

34. What is the output? 20 сек · B8·5

```
int x = 3;
switch (x) {
    case 1: cout << 1; break;
    case 2: cout << 2; break;
    case 3: cout << 3;
    case 4: cout << 4; break;
    default: cout << 5;
}
```

Напиши, что выведет программа.

35. What is the output? 20 сек · B9·4

```
int x = 16;
if (x % 4 == 0)
    x /= 4;
if (x < 5)
    x += 10;
cout << x;
```

Напиши, что выведет программа.

36. What is the output? 20 сек · B9·5

```
int x = 1;
for (int i = 1; i <= 4; i++)
    x *= i;
cout << x;
```

Напиши, что выведет программа.

37. What is the output? 20 сек · B10·4

```
int x = 13;
if (x % 2 == 1 && x > 10)
    cout << x - 3;
else
    cout << x + 3;
```

Напиши, что выведет программа.

38. What is the output? 20 сек · B10·5

```
int s = 0;
for (int i = 1; i <= 5; i++)
    s += i * 2;
cout << s;
```

Напиши, что выведет программа.

39. What is the output? 20 сек · B11·4

```
int x = 5;
if (x > 3) {
    x += 4;
    if (x % 2 == 1)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

40. What is the output? 20 сек · B11·5

```
int s = 0;
for (int i = 1; i <= 7; i += 2)
    s += i;
cout << s;
```

Напиши, что выведет программа.

41. What is the output? 20 сек · B12·4

```
int x = 10;
if (x < 5)
    cout << 1;
else if (x < 10)
    cout << 2;
else
    cout << 3;
```

Напиши, что выведет программа.

42. What is the output? 20 сек · B12·5

```
int x = 2;
for (int i = 1; i <= 4; i++) {
    x += i;
}
cout << x;
```

Напиши, что выведет программа.

43. What is the output? 20 сек · B13·4

```
int x = 18;
if (x % 3 == 0) {
    x /= 3;
    if (x % 2 == 1)
        x += 5;
}
cout << x;
```

Напиши, что выведет программа.

44. What is the output? 20 сек · B13·5

```
int s = 0;
for (int i = 1; i <= 5; i++)
    if (i % 2 == 1)
        s += i;
cout << s;
```

Напиши, что выведет программа.

45. What is the output? 20 сек · B14·4

```
int x = 4;
if (x % 2 == 0)
    x *= 3;
else
    x += 3;
cout << x;
```

Напиши, что выведет программа.

46. What is the output? 20 сек · B14·5

```
int s = 0;
for (int i = 2; i <= 10; i += 2)
    s += i;
cout << s;
```

Напиши, что выведет программа.

47. What is the output? 20 сек · B15·4

```
int x = 20;
if (x > 10) {
    x -= 7;
    if (x % 2 == 1)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

48. What is the output? 20 сек · B15·5

```
int x = 1;
for (int i = 1; i <= 3; i++)
    x = x * 2 + i;
cout << x;
```

Напиши, что выведет программа.

49. What is printed? новый

```
int x = 10;
if (x = 0) cout << "zero";
else cout << "nonzero";
cout << x;
```

a) zero0
b) nonzero0
c) nonzero10
d) zero10

50. How many times is "Hi" printed? `for (int i = 0; i < 10; i += 3) cout << "Hi";` новый

a) 3
b) 4
c) 10
d) 5

51. Which of these is NOT allowed as a `case` label? новый

a) `case 'A':`
b) `case 5:`
c) `case x > 5:`
d) `case 2 + 3 * 4:`

52. What is the output? сложный

```
int s = 0;
for (int i = 1; i <= 4; i++)
    for (int j = i; j <= 4; j++)
        s++;
cout << s;
```

a) 16
b) 8
c) 10
d) 4

53. What is the output? сложный

```
int k = 0;
for (int i = 0; i < 5; i++) {
    switch (i % 3) {
        case 0: k += 1;
        case 1: k += 10; break;
        default: k += 100;
    }
}
cout << k;
```

a) 153
b) 142
c) 111
d) 131

54. What is the output? сложный

```
int a = 3, b = 7;
if (a > b)
    if (a > 0) cout << "X";
else
    cout << "Y";
cout << "Z";
```

a) YZ
b) XZ
c) Z
d) XYZ

55. What is the output? сложный

```
int i, s = 0;
for (i = 1; i < 20; i *= 2) ;
s = i;
for (i = 10; i > 0; i -= 4) s += i;
cout << s << " " << i;
```

a) 48 -2
b) 50 -2
c) 50 2
d) 34 0

56. Which statement is used to exit a `switch` block? новый

a) continue
b) exit
c) break
d) goto default

Ключи

1 — a	**2** — e	**3** — c	**4** — a	**5** — b	**6** — a
7 — d	**8** — b	**9** — a	**10** — a	**11** — d	**12** — d
13 — a	**14** — b	**15** — d	**16** — c **(в ключе a)**	**17** — b	**18** — a
19 — 6	**20** — 79 **(в ключе 16)**	**21** — 10	**22** — 12	**23** — 10 **(в ключе 17)**	**24** — 15 **(в ключе 10)**
25 — B	**26** — 12	**27** — 11	**28** — 10	**29** — 5	**30** — 16
31 — 6	**32** — 120	**33** — 12	**34** — 34	**35** — 14	**36** — 24
37 — 10	**38** — 30	**39** — 18	**40** — 16	**41** — 3	**42** — 12
43 — 6 **(в ключе 11)**	**44** — 9	**45** — 12	**46** — 30	**47** — 26	**48** — 19 **(в ключе 14)**
49 — b	**50** — b	**51** — c	**52** — c	**53** — b	**54** — c
55 — b	**56** — c				

16. Ошибка в ключе! Цикл печатает 0…9, но после последней итерации выполняется `a++` → a = 10, проверка `10 < 10` ложна, цикл заканчивается. Значит a = **10**. В ключе стоит 9 — на тесте выбирай по ситуации: если система проверяет по ключу, ответ «9».

18. Нет `break` после `case 45` — выполнение «проваливается» в `default`: печатается bbb, затем vvv.

20. В ключе «16» — сложили 7+9. Но `cout` ПЕЧАТАЕТ 7, потом (нет break) печатает 9 → на экране «79».

23. 15 % 3 == 0 → x = 5; 5 < 10 → x = 10. В ключе 17 — ошибка.

24. 5 + 1 + 2 + 3 + 4 = 15. В ключе 10 — ошибка.

43. 18/3 = 6, 6 — чётное, +5 не делаем → 6. В ключе 11 — ошибка.

48. x: 1 → 1·2+1 = 3 → 3·2+2 = 8 → 8·2+3 = 19. В ключе 14 — ошибка.

49. `x = 0` — присваивание, его значение 0 = ложь → ветка else. При этом x уже стал 0.

50. i = 0, 3, 6, 9 → 4 раза.

51. Метка — константное выражение. `2 + 3 * 4` — константа (14), а `x > 5` зависит от переменной.

52. 4 + 3 + 2 + 1 = 10.

53. i % 3 даёт 0, 1, 2, 0, 1. Для 0: +1 и провал в case 1: +10 (= 11). Для 1: +10. Для 2: default +100. Итого 11 + 10 + 100 + 11 + 10 = 142.

54. Висячий else относится к ближайшему (внутреннему) if. Внешнее условие 3 > 7 ложно, поэтому весь вложенный if пропускается. Печатается только Z.

55. Первый цикл (пустое тело из-за `;`): 1, 2, 4, 8, 16, 32 → i = 32. Второй: 10 + 6 + 2 = 18, i становится −2. s = 32 + 18 = 50.

ЛЕКЦИЯ 4 ИЗ 15

Циклы while и do...while, break и continue, одномерные и двумерные массивы

Научиться выбирать подходящий цикл и обрабатывать наборы данных в одномерных и двумерных массивах.

4.1 Цикл while (с предусловием)

```
while (условие)
    тело;
```

Сначала проверяется условие; пока оно **истинно**, выполняется тело. Если условие ложно с самого начала, тело **не выполнится ни разу**.

C++

```cpp
int n = 2468, sum = 0, cnt = 0;
while (n > 0) {
    sum += n % 10;   // последняя цифра
    n /= 10;         // отрезаем её
    cnt++;
}
cout << "sum=" << sum << " digits=" << cnt;
```

Вывод

```
sum=20 digits=4
```

Когда while

Когда число повторений **заранее неизвестно**: обработка цифр числа, «пока не введут 0», «пока точность не станет меньше ε», алгоритм Евклида.

C++

```cpp
// НОД по алгоритму Евклида
int a = 48, b = 18;
while (b != 0) {
    int r = a % b;
    a = b;
    b = r;
}
cout << "gcd=" << a;
```

Вывод

```
gcd=6
```

4.2 Цикл do...while (с постусловием)

```
do {
    тело;
} while (условие);     // ← точка с запятой обязательна!
```

Сначала выполняется тело, потом проверяется условие. Поэтому тело выполнится **хотя бы один раз**.

```cpp
int x = 100;
do {
    cout << "once ";
} while (x < 10);      // условие ложно, но тело уже выполнилось
int k = 1;
do {
    k *= 3;
} while (k < 50);
cout << k;
```

Вывод
```
once 81
```

Когда do...while

Когда действие нужно сделать минимум один раз: меню («показать меню, пока не выбран выход»), ввод с проверкой («вводить, пока число не станет положительным»).

4.3 Сравнение трёх циклов

	`for`	`while`	`do…while`
Тип	с параметром (счётчиком)	с **пред**условием	с **пост**условием
Когда проверка	до тела	до тела	после тела
Минимум выполнений	0	0	**1**
Когда удобен	число повторений известно	число неизвестно	нужно минимум 1 раз
Выход из цикла	условие ложно	условие ложно	условие ложно

Особенность C++, которую путают с Паскалем

Все три цикла C++ продолжаются, пока условие **истинно**, и заканчиваются, когда оно стало ложным. «Выход по истинному условию» — это `repeat…until` из Паскаля, которого в C++ **нет**. В банке тестов есть вопрос, где по ключу правильный ответ `do…while`; отвечай по ключу, но знай, как на самом деле.

Любой цикл можно переписать через другой:

```cpp
int i;
for (i = 1; i <= 3; i++) cout << i;     // for
cout << " ";
i = 1;
while (i <= 3) { cout << i; i++; }      // while
cout << " ";
i = 1;
do { cout << i; i++; } while (i <= 3);  // do-while
```

Вывод
```
123 123 123
```

Бесконечный while

Забыли изменить переменную условия внутри тела: `while (i < 10) cout << i;` — i не меняется, цикл никогда не закончится. Ещё ловушка: `while (i < 10);` с точкой с запятой — пустое тело, бесконечный цикл.

4.4 break и continue

- `break` — **немедленно выйти** из ближайшего цикла (или switch).
- `continue` — **пропустить остаток тела** и перейти к следующей итерации (в `for` при этом выполнится изменение счётчика).

```C++
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   // чётные пропускаем
    if (i > 7) break;           // после 7 выходим
    cout << i << " ";
}
```

Вывод
```
1 3 5 7
```

continue в while

В `while` после `continue` сразу проверяется условие. Если увеличение счётчика стоит **после** continue, оно пропустится, и цикл зациклится:

НЕПРАВИЛЬНО
```
int i = 0;
while (i < 5) {
    if (i == 2) continue;   // i навсегда = 2
    cout << i;
    i++;
}
```

ПРАВИЛЬНО
```
int i = 0;
while (i < 5) {
    i++;
    if (i == 2) continue;
    cout << i;              // 1345
}
```

Фишка / совет

`break` выходит только из **одного** (внутреннего) цикла. Для выхода из двух вложенных используют флаг или `goto`.

4.5 Одномерные массивы

Массив — набор элементов **одного типа**, расположенных в памяти подряд, под общим именем. Номер элемента — **индекс**, он начинается с **0**.

```
тип имя[размер];          int a[5];        → a[0], a[1], a[2], a[3], a[4]
```

Инициализация:

```cpp
int a[5] = {4, 8, 15, 16, 23};   // все элементы
int b[5] = {1, 2};               // остальные = 0 → {1, 2, 0, 0, 0}
int c[] = {7, 7, 7};             // размер посчитается сам = 3
int z[100] = {0};                // всё обнулить
//---
cout << a[2] << " " << b[3] << " " << sizeof(c) / sizeof(int) << " " << z[99];
```

Вывод

```
15 0 3 0
```

Выход за границы массива

У `int a[5]` последний элемент — `a[4]`. Обращение к `a[5]` компилятор **не проверяет**: программа прочитает или испортит чужую память. Самая частая ошибка — `for (i = 0; i <= n; i++)` вместо `i < n`.

Размер — константа

`int n = 10; int a[n];` — в Borland ошибка: размер обычного массива должен быть известен при компиляции. Пиши `const int N = 10; int a[N];` или `#define N 10`.

Массивы нельзя присваивать и сравнивать целиком

`b = a;` — ошибка компиляции. `if (a == b)` сравнивает **адреса**, а не содержимое. Копируй и сравнивай поэлементно в цикле.

Стандартные алгоритмы обработки

```cpp
const int N = 8;
int a[N] = {5, -3, 12, 0, 7, -8, 12, 4};
int sum = 0, mx = a[0], imx = 0, neg = 0;
for (int i = 0; i < N; i++) {
    sum += a[i];                          // сумма
    if (a[i] > mx) { mx = a[i]; imx = i; }    // максимум и его индекс
    if (a[i] < 0) neg++;                  // количество отрицательных
}
cout << "sum=" << sum << " avg=" << (double)sum / N << " max=" << mx << " at " << imx << "
neg=" << neg;
```

Вывод

```
sum=29 avg=3.625 max=12 at 2 neg=2
```

Фишка / совет

Максимум **всегда** начинай с первого элемента (`mx = a[0]`), а не с 0: если все числа отрицательные, `mx = 0` даст неверный ответ.

Линейный поиск

```cpp
int a[6] = {9, 4, 7, 1, 7, 3};
int key = 7, pos = -1;
for (int i = 0; i < 6; i++)
    if (a[i] == key) { pos = i; break; }   // первое вхождение
cout << (pos >= 0 ? "found at " : "not found ") << pos;
```

Вывод
```
found at 2
```

Сортировка пузырьком

Идея: соседние элементы сравниваются, и если они стоят неправильно, меняются местами. После каждого прохода самый большой элемент «всплывает» в конец.

```cpp
const int N = 6;
int a[N] = {5, 1, 4, 2, 8, 0};
for (int i = 0; i < N - 1; i++)
    for (int j = 0; j < N - 1 - i; j++)
        if (a[j] > a[j + 1]) {
            int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
        }
for (int i = 0; i < N; i++) cout << a[i] << " ";
```

Вывод
```
0 1 2 4 5 8
```

Лайфхак

Для сортировки **по убыванию** поменяй знак: `if (a[j] < a[j + 1])`. Ещё один простой метод — **выбором**: найти минимум в оставшейся части и поставить его на место i.

Переворот и сдвиг массива

```cpp
int a[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5 / 2; i++) {          // переворот: меняем i и n-1-i
    int t = a[i]; a[i] = a[4 - i]; a[4 - i] = t;
}
for (int i = 0; i < 5; i++) cout << a[i];
```

Вывод
```
54321
```

4.6 Двумерные массивы (матрицы)

`int m[3][4];` — 3 строки, 4 столбца. Элемент `m[i][j]`: i — номер строки, j — номер столбца. В памяти строки лежат подряд.

```cpp
int m[3][3] = { {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9} };
int diag = 0, side = 0, below = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << m[i][j] << "\t";
        if (i == j) diag += m[i][j];          // главная диагональ
        if (i + j == 2) side += m[i][j];      // побочная: i + j == n - 1
        if (j < i) below += m[i][j];          // ниже главной
    }
    cout << endl;
}
cout << diag << " " << side << " " << below;
```

C++

Вывод

```
1       2       3
4       5       6
7       8       9
15 15 19
```

Область	Условие на индексы (n×n)
Главная диагональ	`i == j`
Побочная диагональ	`i + j == n - 1`
Выше главной	`j > i`
Ниже главной	`j < i`

Как спрашивают на экзамене

В банке есть вопросы «что считает программа?» с `s += a[i][i]` (сумма главной диагонали) и с `if (j < i) s += a[i][j]` (сумма элементов **ниже** главной диагонали). Двумерный массив ещё называют **матрицей**.

```cpp
// Суммы по строкам и максимум в каждом столбце
int a[2][3] = { {3, 9, 1}, {7, 2, 8} };
for (int i = 0; i < 2; i++) {
    int s = 0;
    for (int j = 0; j < 3; j++) s += a[i][j];
    cout << "row" << i << "=" << s << " ";
}
for (int j = 0; j < 3; j++) {
    int mx = a[0][j];
    for (int i = 1; i < 2; i++) if (a[i][j] > mx) mx = a[i][j];
    cout << "col" << j << "max=" << mx << " ";
}
```

C++

Вывод

```
row0=13 row1=17 col0max=7 col1max=9 col2max=8
```

Транспонирование

Поменять строки и столбцы: `t[j][i] = a[i][j]`. Для квадратной матрицы на месте — меняй только элементы выше диагонали (`j > i`), иначе всё поменяется обратно.

Разбор задач

Задача 4.1. Ввод до нуля лёгкая

Пользователь вводит числа, пока не введёт 0. Посчитай сумму и количество (без нуля). Напиши через do...while.

Решение

```
int x, s = 0, k = 0;
do {
    cin >> x;
    if (x != 0) { s += x; k++; }
} while (x != 0);
cout << s << " " << k;

Ввод: 5 3 -2 0   →   Вывод: 6 3
```

Задача 4.2. Переворот числа лёгкая

Получи из n = 12345 число 54321.

Решение

```C++
int n = 12345, r = 0;
while (n > 0) {
    r = r * 10 + n % 10;
    n /= 10;
}
cout << r;
```

> **Вывод**
> 54321

Так же проверяется **палиндром**: число равно своему перевёртышу.

Задача 4.3. Второй максимум и среднее положительных средняя

В массиве {4, −2, 9, 9, 0, 7, −5, 3} найди второе по величине **различное** значение и среднее положительных.

Решение

```cpp
int a[8] = {4, -2, 9, 9, 0, 7, -5, 3};
int m1 = a[0], m2 = -32768;
int sp = 0, kp = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] > m1) { m2 = m1; m1 = a[i]; }
    else if (a[i] < m1 && a[i] > m2) m2 = a[i];
    if (a[i] > 0) { sp += a[i]; kp++; }
}
cout << m1 << " " << m2 << " " << (double)sp / kp;
```

> **Вывод**
> 9 7 6.4

Задача 4.4. Удаление элемента средняя

Удали из массива {10, 20, 30, 40, 50} элемент с индексом 1 (сдвигом влево) и выведи результат.

Решение

```cpp
int a[5] = {10, 20, 30, 40, 50};
int n = 5, del = 1;
for (int i = del; i < n - 1; i++) a[i] = a[i + 1];
n--;
for (int i = 0; i < n; i++) cout << a[i] << " ";
```

> **Вывод**
> 10 30 40 50

Задача 4.5. Таблица умножения в матрицу и след сложная

Заполни матрицу 4×4 по правилу a[i][j] = (i+1)·(j+1), выведи её и найди сумму побочной диагонали и сумму элементов выше главной.

Решение

```C++
const int N = 4;
int a[N][N];
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        a[i][j] = (i + 1) * (j + 1);
int side = 0, up = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        cout << a[i][j] << "\t";
        if (i + j == N - 1) side += a[i][j];
        if (j > i) up += a[i][j];
    }
    cout << endl;
}
cout << side << " " << up;
```

Вывод

```
1       2       3       4
2       4       6       8
3       6       9       12
4       8       12      16
20 35
```

Задача 4.6. Частота элементов сложная

Для массива {3, 1, 3, 2, 1, 3} выведи, сколько раз встречается каждое значение (без повторов в выводе).

Решение

```C++
int a[6] = {3, 1, 3, 2, 1, 3};
for (int i = 0; i < 6; i++) {
    int seen = 0;
    for (int j = 0; j < i; j++) if (a[j] == a[i]) { seen = 1; break; }
    if (seen) continue;                 // уже считали
    int c = 0;
    for (int j = 0; j < 6; j++) if (a[j] == a[i]) c++;
    cout << a[i] << ":" << c << " ";
}
```

Вывод

```
3:3 1:2 2:1
```

Контрольные вопросы из силлабуса (устно)

1. **Цикл с предусловием?** — Условие проверяется перед телом (`while`), тело может не выполниться ни разу.
2. **Цикл с постусловием?** — Условие после тела (`do…while`), тело выполняется хотя бы раз.
3. **Разница while и do…while?** — Момент проверки условия и минимальное число выполнений (0 и 1).
4. **Когда while?** — Число повторений неизвестно и может быть равно нулю.
5. **Когда do…while?** — Действие нужно выполнить минимум один раз (меню, ввод с проверкой).
6. **break и continue?** — break — выход из цикла; continue — переход к следующей итерации.
7. **Что такое массив?** — Набор однотипных элементов под одним именем, доступ по индексу с 0.
8. **Объявление и инициализация одномерного?** — `int a[5] = {1,2,3,4,5};`, недостающие элементы = 0.
9. **Доступ к элементам?** — По индексу `a[i]`, 0 ≤ i < размер.
10. **Двумерный массив?** — Массив массивов, таблица (матрица) `a[строки][столбцы]`.
11. **Объявление двумерного?** — `int m[2][3] = {{1,2,3},{4,5,6}};`.
12. **Вложенные циклы для матрицы?** — Внешний по строкам i, внутренний по столбцам j.
13. **Плюсы и минусы массивов?** — Быстрый доступ по индексу, компактно; минусы: фиксированный размер, нет проверки границ, вставка/удаление требуют сдвига.

Тест по лекции 4

1. What is an array? банк №2

a) a type that defines the size
b) a pointer that uses dynamic memory
c) a sequence of values of the same type under one name
d) a set of types that defines a reference
e) data located in cells

2. Which option correctly declares an array of 10 real numbers? банк №16

a) int a[1..10];
b) float a(10);
c) float a[10];
d) float a[1..10];
e) int a[10];

3. The correct declaration of an array a of 12 integers: банк №24

a) char a[12];
b) float a[10];
c) float a[12];
d) int a[12];
e) int a(12);

4. What does `float a[10];` mean? банк №28

a) declaring a 10x10 matrix a
b) declaring a sequence of integers from 1 to 10
c) declaring an array a of 10 real numbers
d) declaring a sequence of real numbers from 1 to 10
e) No correct answer

5. In C. How many times does the loop repeat in the program below? банк №73

```
x=2; while (x<10) {y=x*x; x=x+2;}
```

a) 5
b) 10
c) 4
d) 3
e) No correct answer

6. In C. How many times does the loop repeat in the program below? банк №74

```
x=7; while (x <=10) { cout<<x; x++; }
```

a) 7
b) 4
c) 6
d) 5
e) No correct answer

7. In C. How many times does the loop repeat in the program below? банк №75

```
x=7; while (x <10) { cout<<x; x++; }
```

a) 6
b) 4
c) 3
d) 5
e) No correct answer

8. In C. The notation of the pre-condition loop: банк №79

a) for (final value; initial value; step)
b) do {loop body}while (condition)
c) while (condition) {loop body}
d) for (initial value; final value; step)
e) No correct answer

9. In C. The notation of the post-condition loop: банк №80

a) for (final value; initial value; step)
b) while (condition) {loop body}
c) for (initial value; final value; step)
d) do {loop body}while (condition)
e) No correct answer

10. How many types of loop operators are there in C? банк №82

a) 2
b) 3
c) 4
d) 1
e) No correct answer

11. In C. An array is банк №84

a) a set of any elements
b) a set of elements of different types
c) a sequence of variables
d) a set of elements of the same type
e) No correct answer

12. In which loop operator do we exit the loop when the condition is true? банк №106

a) switch
b) do {} while(condition)
c) while (condition) { }
d) for
e) No correct answer

13. In which loop operator do we exit the loop when the condition is not true? банк №107

a) for
b) do {} while(condition)
c) while (condition) { }
d) switch
e) No correct answer

14. Array indexes start from which number: банк №137

a) 1
b) -1
c) 0
d) 2
e) No correct answer

15. What result does the program below give? банк №139

```
int a[3][3];
for(int i=0;i<3;i++)
for(int j=0;j<3;j++)
cin>>a[i][j];
int m=a[1][1],s=0;
for(int i=0;i<3;i++)
s+=a[i][i];
cout<<s;
```

a) the sum of all elements of the matrix
b) the sum of the even elements of the array
c) the sum of the odd elements of the array
d) the sum of the elements of the main diagonal of the matrix
e) No correct answer

16. What does the program below calculate? Банк №159

```
s=0;
for(i=0; i<3; i++)
for(j=0; j<3; j++)
if (j<i) s+=a[i][j];
printf("\ns=%i",s);
getch();
```

a) the sum of the elements of the secondary diagonal of the given array
b) the sum of the elements located above the main diagonal of the given array
c) the sum of the elements of the main diagonal of the given array
d) the sum of the elements located below the main diagonal of the given array
e) No correct answer

17. What does the program fragment calculate? Банк №167

```
for (i=0; i<=n; i++)
cin>>a[i];
s2=0;
for (i=0; i<=n; i++)
if (i %2== 0) s2=s2+a[i];
cout<<s2;
```

a) the sum of all elements
b) determines the number of all elements equal to 0
c) finds the sum of only the even-numbered elements
d) finds the sum of only the odd-numbered elements
e) No correct answer

18. What does the program fragment calculate? Банк №168

```
for (i=0; i<=n; i++)
cin>>a[i];
s2=0;
for (i=0; i<=n; i++)
if (i %2!= 0) s2=s2+a[i];
cout<<s2;
```

a) the sum of all elements
b) finds the sum of only the odd-numbered elements
c) finds the sum of only the even-numbered elements
d) determines the number of all elements not equal to 0
e) No correct answer

19. Which of the operators below is NOT a loop operator of C++? банк №173

a) while
b) do while
c) for
d) repeat until
e) No correct answer

20. What else do we call an array that uses two indexes? банк №180

a) one-dimensional array
b) heterogeneous array
c) dynamic array
d) matrix
e) No correct answer

21. The number used to work with a single element of an array is called .... банк №183

a) matrix
b) arithmetic mean
c) type
d) index
e) No correct answer

22. Array elements are related by the fact that their ... are the same. банк №184

a) names
b) values
c) types
d) sizes
e) No correct answer

23. The process of arranging array elements in order is called .... банк №185

a) changing
b) sorting
c) searching
d) checking
e) No correct answer

24. What is the output? 20 сек · B1·6

```
int x = 3, s = 0;
while (x < 15) {
    s += x;
    x += 4;
}
cout << s;
```

Напиши, что выведет программа.

25. What is the output? 20 сек · B1·7

```
int x = 10;
do {
    x -= 3;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

26. What is the output? 20 сек · B2·6

```
int x = 64;
while (x > 4)
    x /= 2;
cout << x;
```

Напиши, что выведет программа.

27. What is the output? 20 сек · B2·7

```
int x = 2;
do {
    x *= 2;
} while (x < 10);
cout << x;
```

Напиши, что выведет программа.

28. What is the output? 20 сек · B3·6

```
int n = 2468;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

29. What is the output? 20 сек · B3·7

```
int n = 5;
int s = 0;
do {
    s += n;
    n -= 2;
} while (n > 0);
cout << s;
```

Напиши, что выведет программа.

30. What is the output? 20 сек · B4·6

```
int x = 1, s = 0;
while (x < 20) {
    s += x;
    x *= 2;
}
cout << s;
```

Напиши, что выведет программа.

31. What is the output? 20 сек · B4·7

```
int x = 10;
do {
    x /= 2;
} while (x > 1);
cout << x;
```

Напиши, что выведет программа.

32. What is the output? 20 сек · B5·6

```
int x = 3, s = 0;
while (x <= 12) {
    s += x;
    x += 3;
}
cout << s;
```

Напиши, что выведет программа.

33. What is the output? 20 сек · B5·7

```
int x = 1;
do {
    x = x * 2 + 1;
} while (x < 10);
cout << x;
```

Напиши, что выведет программа.

34. What is the output? 20 сек · B6·6

```
int x = 100;
while (x > 10)
    x /= 3;
cout << x;
```

Напиши, что выведет программа.

35. What is the output? 20 сек · B6·7

```
int x = 7;
do {
    x -= 2;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

36. What is the output? 20 сек · B7·6

```
int x = 2, s = 0;
while (x <= 10) {
    s += x;
    x += 2;
}
cout << s;
```

Напиши, что выведет программа.

37. What is the output? 20 сек · B7·7

```
int x = 20;
do {
    x -= 6;
} while (x > 5);
cout << x;
```

Напиши, что выведет программа.

38. What is the output? 20 сек · B8·6

```
int x = 1;
while (x < 30)
    x *= 3;
cout << x;
```

Напиши, что выведет программа.

39. What is the output? 20 сек · B8·7

```
int x = 3;
do {
    x += 4;
} while (x < 15);
cout << x;
```

Напиши, что выведет программа.

40. What is the output? 20 сек · B9·6

```
int n = 532;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

41. What is the output? 20 сек · B9·7

```
int x = 12;
do {
    x -= 5;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

42. What is the output? 20 сек · B10·6

```
int x = 50;
while (x > 5) {
    x -= 7;
}
cout << x;
```

Напиши, что выведет программа.

43. What is the output? 20 сек · B10·7

```
int x = 1;
do {
    x *= 3;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

44. What is the output? 20 сек · B11·6

```
int x = 81;
while (x > 3)
    x /= 3;
cout << x;
```

Напиши, что выведет программа.

45. What is the output? 20 сек · B11·7

```
int x = 2, s = 0;
do {
    s += x;
    x += 3;
} while (x < 12);
cout << s;
```

Напиши, что выведет программа.

46. What is the output? 20 сек · B12·6

```
int x = 5, s = 0;
while (x > 0) {
    s += x;
    x -= 2;
}
cout << s;
```

Напиши, что выведет программа.

47. What is the output? 20 сек · B12·7

```
int x = 5;
do {
    x *= 2;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

48. What is the output? 20 сек · B13·6

```
int n = 864;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

49. What is the output? 20 сек · B13·7

```
int x = 9;
do {
    x -= 4;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

50. What is the output? 20 сек · B14·6

```
int x = 2, s = 0;
while (x < 20) {
    s += x;
    x *= 2;
}
cout << s;
```

Напиши, что выведет программа.

51. What is the output? 20 сек · B14·7

```
int x = 15;
do {
    x /= 2;
} while (x > 1);
cout << x;
```

Напиши, что выведет программа.

52. What is the output? 20 сек · B15·6

```
int x = 3, s = 0;
while (x <= 15) {
    s += x;
    x += 3;
}
cout << s;
```

Напиши, что выведет программа.

53. What is the output? 20 сек · B15·7

```
int x = 4;
do {
    x += 5;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

54. How many times does the body run? `int i = 10; while (i < 10) i++;` новый

a) 1 **b)** 0
c) 10 **d)** infinite

55. How many times does the body run? `int i = 10; do i++; while (i < 10);` новый

a) 0 **b)** 1
c) 10 **d)** infinite

56. What is the last valid index of `double t[12];`? новый

a) 12 **b)** 11
c) 13 **d)** 0

57. What is the output? сложный

```
int a[5] = {2, 4, 6};
int s = 0;
for (int i = 0; i < 5; i++) s += a[i] * i;
cout << s;
```

a) 12
b) 16
c) 28
d) garbage (uninitialized)

58. What is the output? сложный

```
int i = 0, s = 0;
while (i < 10) {
    i++;
    if (i % 3 == 0) continue;
    if (i == 8) break;
    s += i;
}
cout << s << " " << i;
```

a) 28 10
b) 19 8
c) 27 8
d) 19 10

59. What is the output? сложный

```
int m[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
int s = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        if (i + j == 2 || i == j) s += m[i][j];
cout << s;
```

a) 30
b) 25
c) 15
d) 20

60. What is the output? сложный

```
int a[6] = {6, 3, 8, 1, 9, 2};
int k = 0;
for (int i = 0; i < 5; i++)
    if (a[i] > a[i + 1]) {
        int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t;
        k++;
    }
cout << a[5] << " " << a[0] << " " << k;
```

a) 9 1 5
b) 9 3 4
c) 2 3 3
d) 9 3 3

61. Which loop always executes its body at least once? новый

a) for
b) while
c) do...while
d) none of them

Ключи

1 – c	**2** – c	**3** – d	**4** – c	**5** – c	**6** – b
7 – c	**8** – c	**9** – d	**10** – b	**11** – d	**12** – b
13 – c	**14** – c	**15** – d	**16** – d	**17** – c	**18** – b
19 – d	**20** – d	**21** – d	**22** – c	**23** – b	**24** – 21 **(в ключе 27)**
25 – -2	**26** – 4	**27** – 16	**28** – 20 **(в ключе 15)**	**29** – 9 **(в ключе 20)**	**30** – 31
31 – 1	**32** – 30	**33** – 15	**34** – 3	**35** – -1	**36** – 30
37 – 2	**38** – 81 **(в ключе 27)**	**39** – 15	**40** – 10	**41** – -3	**42** – 1
43 – 27	**44** – 3	**45** – 26 **(в ключе 20)**	**46** – 9	**47** – 20	**48** – 18
49 – -3	**50** – 30	**51** – 1	**52** – 45	**53** – 24	**54** – b
55 – b	**56** – b	**57** – b	**58** – b	**59** – b	**60** – d
61 – c					

12. Отвечай `do {} while(condition)` — так в ключе. Но в C++ ОБА цикла продолжают работу, пока условие истинно, и выходят, когда оно ложно. Выход «по истинному условию» — это `repeat…until` из Паскаля. Вопрос перенесён из Паскаля неудачно.

24. s = 3 + 7 + 11 = 21 (при x = 15 условие x < 15 ложно). В ключе 27 — ошибка.

28. Сумма цифр 2468 = 2+4+6+8 = 20. В ключе 15 — ошибка.

29. s = 5 + 3 + 1 = 9 (дальше n = −1, цикл закончился). В ключе 20 — ошибка.

38. 1 → 3 → 9 → 27 → 81: при x = 27 условие 27 < 30 ещё истинно, поэтому умножаем ещё раз. В ключе 27 — ошибка.

45. s = 2 + 5 + 8 + 11 = 26. В ключе 20 — ошибка.

57. Недостающие элементы равны 0: {2, 4, 6, 0, 0}. 2·0 + 4·1 + 6·2 = 16.

58. Складываем 1, 2, 4, 5, 7 (3 и 6 пропущены continue) = 19; при i = 8 выходим.

59. Главная: 1+5+9, побочная: 3+5+7. Центр 5 входит в обе, но прибавляется один раз (условие через ||): 15 + 15 − 5 = 25.

60. Это один проход пузырька. i=0: 6>3 → обмен (3 6 8 1 9 2); i=1: 6<8; i=2: 8>1 → обмен (3 6 1 8 9 2); i=3: 8<9; i=4: 9>2 → обмен (3 6 1 8 2 9). Три обмена, максимум 9 «всплыл» в конец, a[0] = 3.

ЛЕКЦИЯ 5 ИЗ 15

Функции: объявление, параметры по значению и по ссылке, const, значения по умолчанию, inline, массивы, рекурсия

Научиться разбивать программу на функции, правильно передавать в них данные и понимать, какие переменные функция может изменить.

5.1 Зачем нужны функции

Функция — именованный фрагмент программы, который выполняет определённую задачу и может вызываться много раз. Плюсы:

- код не повторяется (написал один раз — вызываешь сколько угодно);
- программу легче читать: `printReport()` понятнее, чем 40 строк;
- ошибку исправляешь в одном месте;
- функцию можно проверять отдельно.

5.2 Определение, вызов, return

```
тип_результата имя(список_параметров)    ← заголовок
{
    тело;
    return значение;                      ← если тип не void
}
```

```C++
int square(int x)        // определение функции
{
    return x * x;
}
void hello()             // void — ничего не возвращает
{
    cout << "Hello! ";
}
//---
hello();                         // вызов функции без результата
int r = square(7);               // вызов в выражении
cout << r << " " << square(3) + square(4);
```

Вывод

```
Hello! 49 25
```

- `return выражение;` — **возвращает значение** в то место, откуда вызвали функцию, и **сразу завершает** её.
- В `void`-функции можно написать `return;` (без значения), чтобы выйти досрочно.
- Функция возвращает **только одно** значение. Чтобы «вернуть» несколько, используют ссылки или указатели (ниже).

Забыли return

Если в функции с типом `int` нет `return` на каком-то пути, результат будет мусором. Borland выдаст предупреждение «Function should return a value».

5.3 Прототип: объявление и определение

Компилятор читает файл сверху вниз. Если функция **вызывается раньше**, чем написана, её нужно **объявить** заранее — написать **прототип** (заголовок с `;`).

```C++
#include <iostream.h>
int maxOf(int a, int b);     // прототип (объявление): тип, имя, типы параметров

int main()
{
    cout << maxOf(8, 3);     // вызов ДО определения — работает благодаря прототипу
    return 0;
}

int maxOf(int a, int b)      // определение (реализация)
{
    return a > b ? a : b;
}
```

Вывод

```
8
```

	Объявление (прототип)	Определение
Что это	заголовок с `;`	заголовок + тело `{ }`
Сколько раз	можно много раз	ровно один раз
Зачем	сообщить компилятору тип результата, число и типы аргументов	реализовать действие

Запомни

Прототип помогает компилятору проверить количество и типы аргументов при вызове. В прототипе имена параметров можно не писать: `int maxOf(int, int);`. Прототипы библиотечных функций лежат в заголовочных файлах: `sqrt` в `math.h`, `strlen` в `string.h`, `rand` в `stdlib.h`.

5.4 Параметры и аргументы

- **Формальные параметры** — переменные в заголовке функции (`int a, int b`).
- **Фактические параметры (аргументы)** — значения, которые подставляются при вызове (`maxOf(8, 3)`).

Количество, порядок и типы аргументов должны соответствовать параметрам.

5.5 Передача по значению

По умолчанию функция получает **копию** аргумента. Изменения внутри функции **не влияют** на исходную переменную.

```cpp
void tryChange(int v)
{
    v = v * 100;
    cout << "inside: " << v << endl;
}
//---
int a = 5;
tryChange(a);
cout << "outside: " << a;
```

Вывод
```
inside: 500
outside: 5
```

5.6 Передача по ссылке

Ссылка (`&` в объявлении параметра) — это **другое имя той же переменной**. Функция работает с оригиналом, поэтому изменения видны снаружи.

```cpp
void doubleIt(int &v)    // & — параметр-ссылка
{
    v = v * 2;
}
void swapInts(int &x, int &y)
{
    int t = x; x = y; y = t;
}
//---
int a = 5, b = 9;
doubleIt(a);
swapInts(a, b);
cout << a << " " << b;
```

Вывод
```
9 10
```

Что такое ссылка

`int &r = x;` — r становится **псевдонимом** x: `r = 7` меняет x. Ссылку обязательно инициализируют при создании, и «перевесить» на другую переменную её нельзя.

По ссылке нельзя передать константу

`doubleIt(5);` или `doubleIt(a + 1);` — **ошибка**: нечего изменять. По ссылке передаются только переменные.

Передача через указатель (старый стиль C)

То же самое, но явно передаём **адрес**:

```cpp
void incPtr(int *p) { (*p)++; }
//---
int k = 41;
incPtr(&k);            // передаём адрес
cout << k;
```

Вывод
42

Способ	Заголовок	Вызов	Меняет оригинал?
по значению	`f(int x)`	`f(a)`	нет (работает с копией)
по ссылке	`f(int &x)`	`f(a)`	**да**
через указатель	`f(int *x)`	`f(&a)`	**да** (через `*x`)
по константной ссылке	`f(const int &x)`	`f(a)`	нет (и не копирует)

Когда что использовать

- Нужно **изменить** переменную или вернуть несколько результатов → по ссылке.
- Большой объект (строка, структура, объект класса) только **читаем** → `const &`: быстро, без копирования, и защищено от изменений.
- Маленькое значение (int, double) только читаем → по значению.

5.7 Спецификатор const у параметров

`const` запрещает функции менять параметр. Компилятор выдаст ошибку, если попытаться.

```cpp
double average(const int a[], int n)   // массив только для чтения
{
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    // a[0] = 100;  ← ошибка компиляции: a — const
    return (double)s / n;
}
//---
int m[4] = {3, 4, 5, 8};
cout << average(m, 4);
```

Вывод
5

5.8 Параметры по умолчанию

Параметру можно задать значение по умолчанию: если аргумент не передали, берётся оно. Такие параметры должны стоять **в конце** списка.

```C++
double power(double x, int n = 2)
{
    double r = 1;
    for (int i = 0; i < n; i++) r *= x;
    return r;
}
void line(char c = '-', int len = 10)
{
    for (int i = 0; i < len; i++) cout << c;
    cout << endl;
}
//---
cout << power(5) << " " << power(2, 10) << endl;
line();
line('*', 5);
```

Вывод

```
25 1024
----------
*****
```

Частая ошибка

`void f(int a = 1, int b);` — ошибка: после параметра со значением по умолчанию все следующие тоже должны его иметь. Пропустить средний аргумент при вызове нельзя: `line(, 5)` — ошибка.

Осторожно

Значение по умолчанию пишут **в прототипе** (если он есть), а не повторяют в определении.

5.9 inline-функции

`inline` — просьба компилятору **подставить тело** функции прямо в место вызова вместо настоящего вызова. Это экономит время на вызов для маленьких, часто вызываемых функций.

```C++
inline int cube(int x) { return x * x * x; }
//---
int s = 0;
for (int i = 1; i <= 3; i++) s += cube(i);
cout << s;
```

Вывод

```
36
```

Запомни

`inline` — только рекомендация: компилятор может её проигнорировать (например, для функций с циклами или рекурсией). Работает функция как обычная. Методы, написанные прямо внутри класса, автоматически считаются inline.

5.10 Массивы как параметры

Массив **всегда** передаётся по адресу (передаётся указатель на первый элемент), а не копируется. Поэтому функция **может изменить** элементы исходного массива. Размер массива функция не знает, его передают отдельным параметром.

```cpp
void fillSquares(int a[], int n)        // int a[] ≡ int *a
{
    for (int i = 0; i < n; i++) a[i] = i * i;
}
void print(const int a[], int n)        // const — только чтение
{
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}
int sumMatrix(int m[][3], int rows)     // у матрицы указывают все размеры, кроме первого
{
    int s = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < 3; j++) s += m[i][j];
    return s;
}
//---
int a[5];
fillSquares(a, 5);
print(a, 5);
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
cout << sumMatrix(m, 2);
```

Вывод

```
0 1 4 9 16
21
```

sizeof внутри функции

В функции `sizeof(a)` для параметра-массива даёт размер **указателя** (4), а не всего массива. Поэтому размер всегда передают отдельно.

5.11 Локальные, глобальные и статические переменные. Область видимости

- **Локальная** — объявлена внутри функции или блока `{ }`. Видна только там, создаётся при входе и уничтожается при выходе.
- **Глобальная** — объявлена вне всех функций. Видна во всех функциях ниже объявления, живёт всё время работы программы, по умолчанию равна 0.
- **static локальная** — видна только в функции, но **сохраняет значение** между вызовами (создаётся один раз).

```C++
int counter = 0;            // глобальная
void tick()
{
    static int calls = 0;   // создаётся 1 раз, помнит значение
    int local = 0;          // каждый раз заново
    calls++; local++; counter++;
    cout << calls << local << " ";
}
//---
tick(); tick(); tick();
cout << "counter=" << counter;
```

Вывод
```
11 21 31 counter=3
```

Если локальная переменная имеет то же имя, что глобальная, она её **закрывает**; к глобальной можно обратиться через `::имя`.

```C++
int x = 1;
//---
int x = 2;
{
    int x = 3;
    cout << x << ::x;
}
cout << x;
```

Вывод
```
312
```

Фишка / совет

Глобальные переменные удобны, но опасны: любая функция может их испортить. Передавай данные через параметры.

5.12 Перегрузка функций

Несколько функций с **одним именем**, но **разными параметрами** (числом или типами). Компилятор выбирает нужную по аргументам.

```C++
int area(int side) { return side * side; }
int area(int a, int b) { return a * b; }
double area(double r) { return 3.14 * r * r; }
//---
cout << area(4) << " " << area(3, 5) << " " << area(1.0);
```

Вывод
```
16 15 3.14
```

Частая ошибка

Функции, отличающиеся **только типом результата**, перегрузить нельзя: `int f(int)` и `double f(int)` — ошибка.

5.13 Рекурсия

Рекурсия — функция, которая вызывает саму себя. У рекурсивной функции обязательно есть:

1. **Базовый случай** — условие остановки (без него получится бесконечная рекурсия и переполнение стека).
2. **Рекурсивный шаг** — вызов себя для «меньшей» задачи.

```C++
int fact(int n)
{
    if (n <= 1) return 1;           // базовый случай
    return n * fact(n - 1);         // рекурсивный шаг
}
int fib(int n)                      // 0 1 1 2 3 5 8 13 ...
{
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
int sumDigits(int n)
{
    return n < 10 ? n : n % 10 + sumDigits(n / 10);
}
//---
cout << fact(5) << " " << fib(10) << " " << sumDigits(9875);
```

Вывод

```
120 55 29
```

Как считается `fact(4)`: `4 · fact(3)` → `4 · 3 · fact(2)` → `4 · 3 · 2 · fact(1)` → `4 · 3 · 2 · 1` = 24. Вызовы складываются в **стек** и «раскручиваются» обратно.

Виды рекурсии: **прямая** (функция вызывает сама себя), **косвенная** (A вызывает B, B вызывает A).

```C++
void countdown(int n)
{
    if (n == 0) return;
    cout << n << " ";     // печать ДО вызова — по убыванию
    countdown(n - 1);
    cout << n << " ";     // печать ПОСЛЕ вызова — по возрастанию
}
//---
countdown(3);
```

Вывод

```
3 2 1 1 2 3
```

Как решать «что выведет рекурсия»

Рисуй «лесенку» вызовов: что печатается до рекурсивного вызова, идёт на спуске; что после, идёт на подъёме, в обратном порядке.

Разбор задач

Задача 5.1. Функция «простое ли число» лёгкая

Напиши `bool isPrime(int n)` и выведи простые числа от 1 до 20.

Решение

```C++
bool isPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return false;   // return сразу выходит из функции
    return true;
}
//---
for (int i = 1; i <= 20; i++) if (isPrime(i)) cout << i << " ";
```

Вывод

```
2 3 5 7 11 13 17 19
```

Задача 5.2. Несколько результатов через ссылки средняя

Напиши функцию, которая по массиву возвращает (через ссылки) минимум, максимум и среднее.

Решение

```C++
void stats(const int a[], int n, int &mn, int &mx, double &avg)
{
    mn = mx = a[0];
    int s = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
        s += a[i];
    }
    avg = (double)s / n;
}
//---
int a[6] = {7, -2, 10, 4, 4, 1};
int lo, hi; double av;
stats(a, 6, lo, hi, av);
cout << lo << " " << hi << " " << av;
```

Вывод

```
-2 10 4
```

Задача 5.3. Ввод, вывод и обработка массива функциями (лабораторная из силлабуса) средняя

Сделай три функции: заполнение массива, вывод, и функцию, которая возвращает количество элементов больше среднего. Исходный массив изменять разрешено только функции заполнения.

Решение

```C++
void fill(int a[], int n)              // меняет массив — без const
{
    for (int i = 0; i < n; i++) a[i] = (i * 7 + 3) % 10;   // вместо cin, для примера
}
void show(const int a[], int n)          // const: менять запрещено
{
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}
int aboveAvg(const int a[], int n)
{
    double s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    double avg = s / n;
    int k = 0;
    for (int i = 0; i < n; i++) if (a[i] > avg) k++;
    return k;
}
//---
int a[8];
fill(a, 8);
show(a, 8);
cout << aboveAvg(a, 8);
```

Вывод

```
3 0 7 4 1 8 5 2
4
```

Задача 5.4. Рекурсия: степень, НОД, двоичная запись сложная

Напиши рекурсивные функции: быстрое возведение в степень, НОД (Евклид) и вывод числа в двоичной системе.

Решение

```C++
long power(long x, int n)
{
    if (n == 0) return 1;
    long h = power(x, n / 2);
    return n % 2 == 0 ? h * h : h * h * x;
}
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
void bin(int n)
{
    if (n > 1) bin(n / 2);       // сначала старшие разряды
    cout << n % 2;
}
//---
cout << power(3, 13) << " " << gcd(84, 36) << " ";
bin(37);
```

> **Вывод**
> 1594323 12 100101

Задача 5.5. По значению или по ссылке? сложная

Что выведет программа? Объясни каждую строку.

```C++
void f(int a, int &b, int *c)
{
    a += 10; b += 10; *c += 10;
}
//---
int x = 1, y = 2, z = 3;
f(x, y, &z);
f(y, z, &x);
cout << x << " " << y << " " << z;
```

Решение

> **Вывод**
> 11 12 23

1-й вызов: a — копия x (x не меняется), y += 10 → 12, z += 10 → 13. 2-й вызов: a — копия y (не меняется), z += 10 → 23, x += 10 → 11.

Контрольные вопросы из силлабуса (устно)

1. **Что такое функция и зачем?** — Именованный блок кода, выполняющий задачу; убирает повторы, упрощает чтение и отладку.

2. **Объявление и определение?** — Объявление (прототип) — заголовок с `;`; определение — заголовок с телом.
3. **Прототип?** — Объявление функции до её использования; сообщает тип результата, число и типы параметров.
4. **Параметры и аргументы?** — Формальные (в заголовке) и фактические (при вызове).
5. **Как вызвать функцию?** — `имя(аргументы)`, результат можно использовать в выражении.
6. **Назначение return?** — Вернуть значение в точку вызова и завершить функцию.
7. **Передача по значению?** — Функция получает копию, оригинал не меняется.
8. **Передача по ссылке?** — Функция получает псевдоним переменной (`int &x`) и может её изменить.
9. **Разница?** — Копия против оригинала; по ссылке нельзя передать константу; ссылка не копирует данные.
10. **Когда по ссылке?** — Нужно изменить аргумент, вернуть несколько результатов, не копировать большой объект (`const &`).
11. **Что такое ссылка?** — Альтернативное имя существующей переменной, инициализируется сразу.
12. **Как изменить оригинал?** — Объявить параметр как `тип &имя` и присвоить ему значение.

Тест по лекции 5

1. What is the purpose of the return (expression) operation? банк №7

a) there is no such operator in C
b) it is used when a function returns no value
c) its purpose is undefined; by law it must be at the end of the program
d) returns the value of the expression to the operator that called the function
e) it adds beauty to the program

2. What is a variable called that is known only inside a function and is defined within it? банк №9

a) known variable
b) unknown variable
c) recursive variable
d) local variable
e) global variable

3. What is a variable called that is declared outside any block or function? банк №11

a) known variable
b) base variable
c) global variable
d) local variable
e) unary variable

4. What is the void keyword used for in a function header? банк №12

a) to show that the function stores a value
b) to show that the function returns a value
c) the void keyword is not used in a function header
d) to show that the function returns no value
e) No correct answer

5. A defined sequence of operations and descriptions that performs some sequence of actions is called ....? банк №121

a) function
b) recursion
c) procedure
d) code
e) No correct answer

6. What are the parameters at the beginning of a function description called? банк №122

a) global
b) argument
c) formal
d) actual
e) No correct answer

7. What do we call a function that calls itself? банк №124

a) procedure
b) function
c) recursion
d) operator
e) No correct answer

8. What is the purpose of the return (expression) operation? банк №125

a) its purpose is undefined; by law it must be at the end of the program
b) there is no such operator in C
c) it is used when a function returns no value
d) returns the value of the expression to the operator that called the function
e) No correct answer

9. ..... helps the compiler determine the number and type of arguments passed to a function. банк №126

a) file prototype
b) local prototype
c) function prototype
d) function identifier
e) No correct answer

10. What is the void keyword used for in a function header? банк №127

a) to show that the function returns a value
b) to show that the function returns no value
c) to show that the function stores a value
d) the void keyword is not used in a function header
e) No correct answer

11. If `return ;` is written at the end of the program, what type does the function return? банк №138

a) float
b) void
c) int
d) char
e) No correct answer

12. What does the function below determine? банк №160

```
int m(int x, int y)
{ int t;
if (x<y) t=x;
else t=y;
return t;}
```

a) a function that determines the smaller of two real numbers
b) a function that determines the equality of two integers
c) a function that determines the smaller of two integers
d) a function that determines the larger of two integers
e) No correct answer

13. What does the function below determine? банк №161

```
int m(int x, int y)
{ if (x>y)
return (x);
else return (y);}
```

a) a function that determines the larger of two integers
b) a function that determines the smaller of two integers
c) a function that determines the smaller of two real numbers
d) a function that determines the equality of two integers
e) No correct answer

14. What is the value of the variable a after the function is called? банк №169

```
int inc2(int vv)
{
vv++;
return vv;
}
void main()
{int a,b=5;
a=inc2(b);
}
```

a) 5
b) 7
c) 6
d) 8
e) No correct answer

15. Which function header passes the argument by reference? новый

a) void f(int x)
b) void f(int &x)
c) void f(int *x)
d) void f(const int x)

16. Which declaration with default parameters is correct? новый

a) void f(int a = 1, int b);
b) void f(int a = 1, int b, int c = 3);
c) void f(int a, int b = 2, int c = 3);
d) void f(int = 1, b);

17. What is a function that calls itself called? новый

a) inline
b) overloaded
c) recursive
d) static

18. What is the output? сложный

```
int f(int n)
{
    static int s = 0;
    s += n;
    return s;
}
//---
f(1); f(2);
cout << f(3);
```

a) 3
b) 6
c) 0
d) 1

19. What is the output? сложный

```
void g(int a[], int n) { for (int i = 0; i < n; i++) a[i] *= 2; }
void h(int v) { v *= 2; }
//---
int a[3] = {1, 2, 3};
int v = 5;
g(a, 3); h(v); h(a[0]);
cout << a[0] + a[1] + a[2] << " " << v;
```

a) 6 5
b) 24 10
c) 12 5
d) 12 10

20. What is the output? сложный

```
int r(int n)
{
    if (n <= 0) return 0;
    return n + r(n - 2);
}
//---
cout << r(7) << " " << r(6);
```

a) 28 21
b) 16 12
c) 15 12
d) 16 6

21. What is the output? сложный

```
void p(int n)
{
    if (n == 0) return;
    p(n / 10);
    cout << n % 10 << "-";
}
//---
p(352);
```

a) 2-5-3-
b) 3-5-2-
c) 352-
d) 3-5-2

22. What is the output? сложный

```
int x = 10;
void m(int &x) { x++; ::x += 5; }
//---
int y = 1;
m(y);
m(x);
cout << x << " " << y;
```

a) 16 2
b) 21 2
c) 20 2
d) 16 7

Ключи

1 – d	**2** – d	**3** – c	**4** – d	**5** – a	**6** – c
7 – c	**8** – d	**9** – c	**10** – b	**11** – b	**12** – c
13 – a	**14** – c	**15** – b	**16** – c	**17** – c	**18** – b
19 – c	**20** – b	**21** – b	**22** – b		

15. `int *x` — это указатель (передача адреса), а ссылка обозначается `&` в объявлении.

18. static-переменная помнит значение между вызовами: 1, потом 3, потом 6.

19. Массив передаётся по адресу, поэтому g удваивает элементы (2 + 4 + 6 = 12). h получает копии, и ни v, ни a[0] не меняются.

20. r(7) = 7 + 5 + 3 + 1 = 16; r(6) = 6 + 4 + 2 = 12.

21. Сначала рекурсивно уходим до старшей цифры, а печатаем на «подъёме» — в прямом порядке цифр.

22. Первый вызов: y → 2, глобальный x → 15. Во втором вызове параметр-ссылка — это сам глобальный x: x++ → 16, затем ::x += 5 → 21.

ЛЕКЦИЯ 6 ИЗ 15

Тип char и строки: массивы символов, string.h и класс string

Научиться обрабатывать текст: отдельные символы, строки-массивы char и строки класса string.

6.1 Тип char: символ — это число

`char` занимает 1 байт и хранит **код** символа по таблице **ASCII**. Поэтому символ можно сравнивать, складывать и выводить как число.

Символы	Коды
`'0'`…`'9'`	48…57
`'A'`…`'Z'`	65…90
`'a'`…`'z'`	97…122
пробел `' '`	32
`'\0'` (конец строки)	0

```cpp
char c = 'A';
cout << c << " " << (int)c << " " << (char)(c + 2) << " " << (char)99 << endl;
char d = '7';
int digit = d - '0';                    // символ-цифра → число
char low = 'G' + ('a' - 'A');           // большая → маленькая (разница 32)
cout << digit * 2 << " " << low << " " << ('b' > 'a');
```

Вывод

```
A 65 C c
14 g 1
```

Три приёма, которые нужны постоянно

- `ch - '0'` — превратить символ-цифру в число (`'7' - '0'` = 7).
- `ch >= 'a' && ch <= 'z'` — проверка «маленькая латинская буква».
- `ch - 'a' + 'A'` (или `ch - 32`) — в верхний регистр.

'5' и 5 — разные вещи

`'5'` — символ с кодом 53, `5` — число пять, `"5"` — строка из двух байтов: '5' и '\0'. Одинарные кавычки — для **одного** символа, двойные — для строки.

Готовые функции для символов лежат в `ctype.h` : `isdigit(c)` , `isalpha(c)` , `isupper(c)` , `islower(c)` , `isspace(c)` , `toupper(c)` , `tolower(c)` .

```C++
#include <iostream.h>
#include <ctype.h>
int main()
{
    char s[] = "Hello, World 2026!";
    int letters = 0, digits = 0, upper = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (isalpha(s[i])) letters++;
        if (isdigit(s[i])) digits++;
        if (isupper(s[i])) upper++;
        s[i] = toupper(s[i]);
    }
    cout << letters << " " << digits << " " << upper << " " << s;
    return 0;
}
```

Вывод
```
10 4 2 HELLO, WORLD 2026!
```

6.2 Строка в стиле C: массив символов с '\0'

В C/C++ строка — это **массив char**, в конце которого стоит нулевой символ `'\0'` (признак конца строки). Все строковые функции идут по массиву, пока не встретят `'\0'`.

```
char s[10] = "Hello";

индекс:  0    1    2    3    4    5     6..9
символ: 'H'  'e'  'l'  'l'  'o'  '\0'  (свободно)
```

Объявление:

```C++
char a[20] = "Borland";        // 7 букв + '\0', остальное свободно
char b[] = "C++";              // размер посчитается сам: 4 (3 + '\0')
char c[6] = {'H','e','l','l','o','\0'};
//---
cout << a << " " << b << " " << c << " " << sizeof(b) << " " << strlen(b);
```

Вывод
```
Borland C++ Hello 4 3
```

sizeof и strlen

`sizeof(b)` — сколько байт занимает **массив** (вместе с '\0' и свободным местом). `strlen(b)` — сколько **символов** до '\0'. Для `char a[20] = "Borland"` : sizeof = 20, strlen = 7.

Нет места для '\0'

`char s[5] = "Hello";` — 5 букв + '\0' = 6 байт, а места 5. Размер массива всегда **длина + 1**.

Ввод строк

```
char name[50];
cin >> name;              // читает ДО ПРОБЕЛА: "Ivan Petrov" → "Ivan"
cin.getline(name, 50);    // читает всю строку с пробелами (до Enter)
gets(name);               // старый способ из stdio.h (опасен: не проверяет размер)
```

> **Ловушка cin >> и getline**
>
> После `cin >> x;` в буфере остаётся символ Enter. Следующий `cin.getline` сразу прочитает пустую строку. Лечится `cin.ignore();` перед getline.

Обработка строки по символам

C++

```
char s[] = "programming in c plus plus";
int spaces = 0, vowels = 0;
for (int i = 0; s[i]; i++) {              // s[i] == 0 → конец строки
    if (s[i] == ' ') spaces++;
    if (strchr("aeiou", s[i])) vowels++;  // есть ли символ в строке гласных
    if (i == 0 || s[i - 1] == ' ') s[i] = s[i] - 32;   // каждое слово с большой
}
cout << s << endl << "words=" << spaces + 1 << " vowels=" << vowels;
```

Вывод

```
Programming In C Plus Plus
words=5 vowels=6
```

6.3 Функции string.h

Функция	Что делает	Пример → результат
`strlen(s)`	длина (без '\0')	`strlen("Salem")` → 5
`strcpy(d, s)`	копирует s в d	d = "Salem"
`strncpy(d, s, k)`	копирует первые k символов s в d	
`strcat(d, s)`	приписывает s в конец d, результат в d	"ab" + "cd" → "abcd"
`strncat(d, s, k)`	приписывает первые k символов s	
`strcmp(a, b)`	сравнивает: 0 — равны, <0 — a раньше, >0 — a позже	`strcmp("Salem","Salem")` → 0
`strncmp(a, b, k)`	сравнивает только первые k символов	
`stricmp(a, b)`	сравнивает **без учёта регистра**	`stricmp("ABC","abc")` → 0
`strchr(s, ch)`	ищет символ слева направо, возвращает указатель или NULL	
`strrchr(s, ch)`	ищет символ **с конца**	
`strstr(s1, s2)`	ищет подстроку s2 в s1	
`strrev(s)`	переворачивает строку (Borland)	"abc" → "cba"
`strupr(s)` / `strlwr(s)`	в верхний / нижний регистр (Borland)	
`strnset(s, ch, k)`	заменяет первые k символов на ch	
`strset(s, ch)`	заменяет все символы на ch	

```C++
char a[40] = "Hello", b[20] = "World", c[40];
strcpy(c, a);                 // c = "Hello"
strcat(c, ", ");
strcat(c, b);                 // c = "Hello, World"
cout << c << " " << strlen(c) << endl;
cout << strcmp("abc", "abd") << " " << strcmp("b", "a") << " " << strcmp("Salem", "Salem") << "
" << stricmp("ABC", "abc") << endl;
char *p = strstr(c, "Wor");
cout << p << " at " << p - c << endl;
char r[10] = "stack";
strrev(r);
cout << r << " " << strupr(r) << endl;
char t[10] = "******";
strncpy(t, "abc", 2);
cout << t;
```

Вывод

```
Hello, World 12
-1 1 0 0
World at 7
kcats KCATS
ab****
```

strcmp сравнивает по алфавиту (по кодам символов)

Посимвольно слева направо, до первого различия. `"abc" < "abd"`, `"Z" < "a"` (большие буквы имеют меньшие коды), `"app" < "apple"` (короче — раньше). Borland возвращает разность кодов или ±1, поэтому проверяй знак (`< 0`, `== 0`, `> 0`), а не конкретное число.

Три главные ошибки со строками-массивами

1. **Присваивание** `s = "text";` — ошибка компиляции (массив нельзя присвоить). Нужно `strcpy(s, "text");`.
2. **Сравнение** `if (s == "yes")` сравнивает **адреса**, всегда ложно. Нужно `if (strcmp(s, "yes") == 0)`.
3. **Переполнение**: `strcat` и `strcpy` не проверяют размер. Если в d не хватает места, портится соседняя память.

НЕПРАВИЛЬНО

```
char s[10];
s = "hi";                 // ошибка
if (s == "hi") { }        // сравнивает адреса
```

ПРАВИЛЬНО

```
char s[10];
strcpy(s, "hi");
if (strcmp(s, "hi") == 0) { }
```

6.4 Класс string (C++)

В C++ есть удобный класс `string` (`#include <string>`). Он сам следит за памятью, его можно присваивать, сравнивать через `==` и складывать через `+`.

C++

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string a = "Object", b = "Oriented";
    string s = a + "-" + b;           // склейка через +
    s += " Programming";
    cout << s << endl;
    cout << s.length() << " " << s[0] << " " << s.substr(7, 8) << endl;
    cout << s.find("Pro") << " " << (a == "Object") << " " << (a < b) << endl;
    s.replace(0, 6, "OBJ");
    s.erase(3, 1);
    s.insert(0, ">> ");
    cout << s << endl;
    const char *cs = s.c_str();       // обратно в строку C
    cout << strlen(cs);
    return 0;
}
```

Вывод

```
Object-Oriented Programming
27 O Oriented
16 1 1
>> OBJOriented Programming
26
```

Действие	Массив char	string
Присвоить	`strcpy(s, "x")`	`s = "x"`
Склеить	`strcat(s, t)`	`s + t`, `s += t`
Сравнить	`strcmp(s, t) == 0`	`s == t`, `s < t`
Длина	`strlen(s)`	`s.length()` / `s.size()`
Найти	`strstr(s, t)`	`s.find(t)` (или `string::npos`, если нет)
Часть строки	вручную	`s.substr(pos, len)`
Ввод с пробелами	`cin.getline(s, N)`	`getline(cin, s)`

Что выбрать

В новом коде удобнее `string`: меньше ошибок с памятью. Но на экзамене по Borland спрашивают именно функции `string.h`, поэтому знай оба способа. В визуальных проектах C++ Builder используется ещё свой тип **AnsiString** (`Edit1->Text`), он похож на string, но индексы в нём **начинаются с 1**.

6.5 Работа с файлами (stdio.h)

```
FILE *f = fopen("data.txt", "w");    // открыть: "r" чтение, "w" запись (создать/стереть), "a" дописать
fprintf(f, "%d %s\n", 5, "abc");    // записать форматированно
fputs("line\n", f);                 // записать строку
fscanf(f, "%d", &x);                // прочитать форматированно
fgets(buf, 100, f);                 // прочитать строку
fclose(f);                          // закрыть файл (обязательно!)
```

C++

```
FILE *f = fopen("demo.txt", "w");
for (int i = 1; i <= 3; i++) fprintf(f, "%d %d\n", i, i * i);
fputs("end\n", f);
fclose(f);
f = fopen("demo.txt", "r");
int a, b, sum = 0;
while (fscanf(f, "%d %d", &a, &b) == 2) sum += b;
char line[20];
fgets(line, 20, f);
fclose(f);
cout << sum << " " << line;
```

Вывод

```
14 end
```

Частая ошибка

- Не проверили, открылся ли файл: `if (f == NULL) { cout << "error"; }`.
- Забыли `fclose` — данные могут не записаться на диск.
- В `fscanf` забыли `&`: `fscanf(f, "%d", x)` — нужно `&x` (для строк `char[]` амперсанд не нужен).

Разбор задач

Задача 6.1. Подсчёт символа лёгкая

Сколько раз в строке "banana bandana" встречается буква 'a'? Замени все 'a' на '*'.

Решение

```C++
char s[] = "banana bandana";
int k = 0;
for (int i = 0; s[i] != '\0'; i++)
    if (s[i] == 'a') { k++; s[i] = '*'; }
cout << k << " " << s;
```

Вывод
```
6 b*n*n* b*nd*n*
```

Задача 6.2. Палиндром лёгкая

Проверь, является ли строка "level" палиндромом (без strrev).

Решение

```C++
char s[] = "level";
int n = strlen(s), ok = 1;
for (int i = 0; i < n / 2; i++)
    if (s[i] != s[n - 1 - i]) { ok = 0; break; }
cout << (ok ? "palindrome" : "not palindrome");
```

Вывод
```
palindrome
```

Задача 6.3. Самое длинное слово средняя

Найди самое длинное слово в строке "C plus plus is a powerful language".

Решение

```cpp
char s[] = "C plus plus is a powerful language";
int best = 0, bestStart = 0, start = 0;
int n = strlen(s);
for (int i = 0; i <= n; i++) {
    if (s[i] == ' ' || s[i] == '\0') {
        int len = i - start;
        if (len > best) { best = len; bestStart = start; }
        start = i + 1;
    }
}
for (int i = bestStart; i < bestStart + best; i++) cout << s[i];
cout << " " << best;
```

Вывод

```
powerful 8
```

Фишка: цикл до `i <= n`, чтобы обработать и последнее слово (на '\0').

Задача 6.4. Строка в число и обратно средняя

Без `atoi` преобразуй строку "40721" в число и прибавь 1. Потом преобразуй число 905 в строку.

Решение

```cpp
char s[] = "40721";
long v = 0;
for (int i = 0; s[i]; i++) v = v * 10 + (s[i] - '0');
cout << v + 1 << " ";
int n = 905;
char buf[12];
int len = 0;
do { buf[len++] = '0' + n % 10; n /= 10; } while (n > 0);
buf[len] = '\0';
strrev(buf);
cout << buf;
```

Вывод

```
40722 905
```

Задача 6.5. Двумя способами (лабораторная): шифр Цезаря средняя

Зашифруй "Hello World" сдвигом на 3 (z → c), сначала массивом char, потом через класс string.

Решение

```C++
#include <iostream>
#include <string>
using namespace std;
char shift(char c, int k)
{
    if (c >= 'a' && c <= 'z') return 'a' + (c - 'a' + k) % 26;
    if (c >= 'A' && c <= 'Z') return 'A' + (c - 'A' + k) % 26;
    return c;
}
int main()
{
    char a[] = "Hello World";
    for (int i = 0; a[i]; i++) a[i] = shift(a[i], 3);
    string s = "Hello World";
    for (int i = 0; i < (int)s.length(); i++) s[i] = shift(s[i], 3);
    cout << a << endl << s << endl << (s == a ? "same" : "diff");
    return 0;
}
```

Вывод

```
Khoor Zruog
Khoor Zruog
same
```

Задача 6.6. Слова в обратном порядке сложная

Выведи слова строки "one two three four" в обратном порядке: "four three two one".

Решение

```C++
char s[] = "one two three four";
int end = strlen(s);
for (int i = end - 1; i >= -1; i--) {
    if (i == -1 || s[i] == ' ') {
        for (int j = i + 1; j < end; j++) cout << s[j];
        if (i != -1) cout << " ";
        end = i;
    }
}
```

Вывод

```
four three two one
```

Контрольные вопросы из силлабуса (устно)

1. **Тип char?** — Целочисленный тип размером 1 байт, хранит код символа (ASCII).

2. **Как хранится символ?** — Как число-код: 'A' = 65, '0' = 48.
3. **Символ и строка?** — Символ — один char в одинарных кавычках; строка — последовательность символов в двойных кавычках, заканчивается '\0'.
4. **Escape-последовательности в символах?** — Запись непечатаемых символов: '\n', '\t', '\0', '\\', '\''.
5. **Что такое строка в C++?** — Массив char с '\0' в конце или объект класса string.
6. **Класс string?** — Стандартный класс для строк: сам управляет памятью, поддерживает =, +, ==, методы length, find, substr.
7. **Объявление и инициализация string?** — `string s = "text";`, `string t(s);`, `string e;`.
8. **Ввод и вывод строк?** — `cin >> s` (до пробела), `getline(cin, s)` / `cin.getline(buf, N)` (вся строка), `cout << s`.
9. **Склейка?** — `strcat(a, b)` для char[]; `a + b`, `a += b` для string.
10. **Сравнение?** — `strcmp(a, b)` (0 — равны); для string — `==`, `<`.
11. **Длина?** — `strlen(a)`; `s.length()`.
12. **Типичные операции?** — Копирование, склейка, сравнение, поиск символа/подстроки, замена, смена регистра, переворот.

Тест по лекции 6

1. Which function is used to determine the length of a string? банк №129

a) strlen
b) strcmp
c) strcat
d) strcpy
e) No correct answer

2. Which header file contains the prototype of the strlen() function? банк №131

a) <string.h>
b) <conio.h>
c) <iostream.h>
d) <stdlib.h>
e) No correct answer

3. string.h is a library of the following functions: банк №132

a) graphics functions
b) mathematical functions
c) basic input / output library
d) string functions
e) No correct answer

4. Which of the following functions reads a string from a file? банк №135

a) fputs()
b) fgetc()
c) fread()
d) fgets()
e) No correct answer

5. Which of the following functions writes a string to a file? банк №136

a) fgets()
b) fputs()
c) fwrite()
d) fread()
e) No correct answer

6. Which of the following functions writes data to a file? банк №142

a) fgets ()
b) fread ()
c) fscanf ()
d) fprintf ()
e) No correct answer

7. Which of the following functions reads data from a file? банк №143

a) fgets ()
b) fwrite ()
c) fscanf()
d) fprintf ()
e) No correct answer

8. What does the strcat(stroka1, stroka2) function do? банк №144

a) It compares the strings stroka1 and stroka2
b) It joins the strings stroka1 and stroka2 and assigns the result to the variable stroka1
c) It searches for the second given string inside the first string
d) It searches the first string for any character of the second string
e) No correct answer

9. What does the strcmp(stroka1,stroka2) function do? банк №145

a) it joins two strings
b) it is used to compare two strings
c) it searches the first string for any character of the second string
d) it searches for the second given string inside the first string
e) No correct answer

10. What is the result of this code? банк №146

```
main() { printf("%d\n",strcmp("Salem","Salem")); getch(); }
```

a) 7
b) 0
c) 2
d) 1
e) No correct answer

11. What does the strcpy(str1,str2) function do? банк №147

a) it is used to make a copy of a string; here the string in variable str2 is copied to variable str1
b) it searches for the second given string inside the first string
c) it is used to compare two strings
d) it joins two strings
e) No correct answer

12. What does the strstr(str1, str2) function do? банк №148

a) it searches for the second given string inside the first string
b) it joins two strings
c) it is used to compare two strings
d) it is used to make a copy of a string
e) No correct answer

13. What does the strrev(str) function do? банк №149

a) it is used to compare strings
b) it is used to delete a string
c) it is used to make a copy of a string
d) reverses all characters of the given string
e) No correct answer

14. The strncpy(S1,CS2,k) function: банк №150

a) copies the last k characters of string CS2 to S1
b) copies the first k characters of string CS2 to S1
c) deletes k characters from strings S1 and CS2
d) deletes the first k characters of string CS2 from S1
e) No correct answer

15. The strnset(S,ch,k) function: банк №151

a) repeats the character ch k times in string S
b) removes the characters ch and k from string S
c) deletes the character ch k times in string S
d) replaces the character ch in string S with character k
e) No correct answer

16. The strncat(S1,CS2,k) function: банк №152

a) appends k characters to strings CS2 and S1
b) appends the last k characters of string CS2 to string S1
c) repeats the character CS2 k times in string S1
d) appends the first k characters of string CS2 to string S1
e) No correct answer

17. The strlwr(S) function: банк №153

a) converts the characters of string S to Roman numerals
b) converts the characters of string S to lowercase letters
c) converts the characters of string S to Arabic numerals
d) converts the characters of string S to uppercase letters
e) No correct answer

18. The strupr(S) function: банк №154

a) converts the characters of string S to lowercase letters
b) converts the characters of string S to Arabic numerals
c) converts the characters of string S to uppercase letters
d) converts the characters of string S to Roman numerals
e) No correct answer

19. What is the strncmp(CS1,CS2,k) function intended for? банк №155

a) appends the first k characters of string CS1 to string CS2
b) compares only the first k characters of strings CS1 and CS2
c) compares only the last k characters of strings CS1 and CS2
d) appends the last k characters of string CS1 to string CS2
e) No correct answer

20. The stricmp(CS1,CS2) function: банк №156

a) it is used to compare two strings
b) takes into account the difference between uppercase and lowercase letter codes when comparing
c) it joins two strings
d) ignores the difference between uppercase and lowercase letter codes when comparing
e) No correct answer

21. The strchr(CS,ch) function: банк №157

a) scans string CS from left to right until the character ch is found
b) it joins two strings
c) it is used to compare two strings
d) scans string CS from the end to the beginning until the character ch is found
e) No correct answer

22. The strrchr(CS,ch) function: банк №158

a) scans string CS from left to right until the character ch is found
b) it is used to compare two strings
c) scans string CS from the end to the beginning until the character ch is found
d) it joins two strings
e) No correct answer

23. To close a file after you have finished working with it, the following function is used: банк №162

a) fclose()
b) fgets()
c) fexit()
d) fgetc()
e) No correct answer

24. Show the function for opening a file in C. банк №163

a) fclose()
b) fnew()
c) fexit()
d) fopen()
e) No correct answer

25. What is the result of the code below? банк №178

```
char s1[ 50 ] = "jack";
char s2[ 50 ] = "jill";
char s3[ 50 ] , *sptr;
cout << strlen( s1 ) + strlen( s2 ) << endl;
```

a) 7
b) 8
c) 10
d) 9
e) No correct answer

26. How many bytes does `char s[] = "Hello";` occupy? новый

a) 5
b) 6
c) 4
d) 10

27. Which is the correct way to compare two C-strings `a` and `b` for equality? новый

a) if (a == b)
b) if (a = b)
c) if (strcmp(a, b) == 0)
d) if (strcmp(a, b) == 1)

28. What does `cin >> name;` read from the input "John Smith"? новый

a) John Smith
b) John
c) Smith
d) nothing

29. What is the output? сложный

```
char s[] = "abcdef";
char *p = s + 2;
*p = 'X';
p += 2;
cout << s << " " << p << " " << strlen(p);
```

a) abcdef ef 2
b) abXdef ef 2
c) abXdef e 1
d) aXcdef def 3

30. What is the output? сложный

```
char s[20] = "data";
s[2] = '\0';
cout << s << " " << strlen(s) << " " << sizeof(s);
```

a) data 4 20
b) da 2 20
c) da 2 3
d) d 1 20

31. What is the output? сложный

```
char w[] = "kazakhstan";
int k = 0;
for (int i = 0; w[i]; i++)
    if (w[i] == 'a') k++;
    else w[i] = w[i] - 'a' + 'A';
cout << k << " " << w;
```

a) 3 KAZAKHSTAN
b) 4 KaZaKHSTaN
c) 3 KaZaKHSTaN
d) 3 kazakhstan

32. What is the output? сложный

```
char a[10] = "12", b[10] = "9";
cout << strcmp(a, b) << " " << (atoi(a) > atoi(b));
```

a) 1 1
b) -8 1
c) -1 0
d) 0 1

Ключи

1 – a	**2** – a	**3** – d	**4** – d	**5** – b	**6** – d
7 – c	**8** – b	**9** – b	**10** – b	**11** – a	**12** – a
13 – d	**14** – b	**15** – a	**16** – d	**17** – b	**18** – c
19 – b	**20** – d	**21** – a	**22** – c	**23** – a	**24** – d
25 – b	**26** – b	**27** – c	**28** – b	**29** – b	**30** – b
31 – c	**32** – b				

10. `strcmp` возвращает 0, если строки одинаковые. В казахском оригинале ключ ошибочно «1», в английском файле уже исправлено на 0.

26. 5 букв + завершающий '\0'.

28. Оператор `>>` читает до первого пробела.

29. p указывает на s[2], меняем его на 'X'. Потом p сдвигается на s[4]; вывод char* печатает строку с этого места до '\0': "ef".

30. '\0' в середине обрывает строку для всех строковых функций и для cout. sizeof — размер массива, он не меняется.

31. Буквы 'a' (их 3) остаются как есть, остальные маленькие буквы переводятся в большие формулой `c - 'a' + 'A'`.

32. Строки сравниваются посимвольно: '1' (49) < '9' (57), поэтому "12" < "9" и результат отрицательный (Borland возвращает разность кодов 49 − 57 = −8). Как числа 12 > 9.

ЛЕКЦИЯ 7 ИЗ 15

Ключевые понятия и принципы объектно-ориентированного программирования

Понять идею ООП и четыре принципа так, чтобы объяснить их своими словами и показать на коде.

7.1 От процедур к объектам

Процедурный подход (лекции 1–6): программа — это **данные** отдельно и **функции** отдельно. Функции получают данные через параметры.

Объектно-ориентированный подход (ООП): программа — набор **объектов**, которые взаимодействуют. Каждый объект **объединяет** свои данные и действия над ними. Мы моделируем предметы реального мира: студент, книга, банковский счёт, кнопка на форме.

	Процедурный	Объектно-ориентированный
Главное	функции (действия)	объекты (данные + действия)
Данные	отдельно, часто глобальные	спрятаны внутри объектов
Связь	данные передаются в функции	объекты вызывают методы друг друга
Повторное использование	копирование функций	наследование классов
Подходит для	небольших расчётных программ	больших систем, интерфейсов
Пример	`printStudent(name, age)`	`student.print()`

C++

```cpp
// Процедурно: данные и функции раздельно
struct Account { double balance; };
void deposit(Account &a, double sum) { a.balance += sum; }

// ООП: данные и действия вместе, баланс спрятан
class BankAccount {
    double balance;                 // скрыт (private по умолчанию)
public:
    BankAccount() { balance = 0; }
    void deposit(double s) { if (s > 0) balance += s; }
    double getBalance() { return balance; }
};
//---
Account a = {100};
deposit(a, 50);
BankAccount b;
b.deposit(50);
b.deposit(-1000);                   // объект сам защищает себя от ошибки
cout << a.balance << " " << b.getBalance();
```

Вывод

```
150 50
```

7.2 Основные понятия

- **Класс** — это **описание (шаблон, чертёж)** объектов: какие у них данные и что они умеют. Сам класс в памяти не хранит конкретных значений.
- **Объект (экземпляр класса)** — конкретная «вещь», созданная по классу. Объектов одного класса может быть сколько угодно, у каждого свои значения.
- **Атрибут (поле, свойство, данные-член)** — переменная внутри класса: `name`, `age`.
- **Метод (функция-член)** — функция внутри класса, описывает **поведение**: `bark()`, `deposit()`.
- **Состояние** объекта — текущие значения всех его полей. **Поведение** — набор его методов.

> **Аналогия**
>
> Класс — чертёж дома, объекты — сами дома, построенные по этому чертежу. Чертёж один, а дома разные: у каждого свой адрес, цвет крыши, жильцы (состояние). Но двери у всех открываются одинаково (поведение).

```cpp
#include <iostream>
#include <string>
using namespace std;

class Dog {
public:
    string name;            // атрибуты
    string breed;
    void bark() { cout << name << ": Woof!" << endl; }    // метод
};

int main()
{
    Dog a, b;               // два объекта одного класса
    a.name = "Buddy"; a.breed = "Golden Retriever";
    b.name = "Rex";   b.breed = "Shepherd";
    a.bark();
    b.bark();
    cout << a.breed << " / " << b.breed;
    return 0;
}
```

Вывод

```
Buddy: Woof!
Rex: Woof!
Golden Retriever / Shepherd
```

7.3 Принцип 1. Инкапсуляция

Инкапсуляция — объединение данных и методов в одном классе **плюс сокрытие** внутреннего устройства. Снаружи объектом пользуются только через **открытый интерфейс** (public-методы), а данные закрыты (private) и меняются только через методы с проверками.

Зачем: никто не испортит данные случайно (отрицательный возраст, баланс −1000), а внутреннее устройство можно менять, не ломая чужой код.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Dog {
private:
    string name;                        // закрыто
public:
    string getName() { return name; }   // геттер — прочитать
    void setName(string n)              // сеттер — изменить с проверкой
    {
        if (n.length() > 0) name = n;
        else cout << "Empty name rejected" << endl;
    }
};

int main()
{
    Dog d;
    d.setName("Buddy");
    d.setName("");          // проверка не пропустит
    // d.name = "X";        // ОШИБКА компиляции: name is private
    cout << d.getName();
    return 0;
}
```

Вывод

```
Empty name rejected
Buddy
```

7.4 Принцип 2. Абстракция

Абстракция — выделение **главного** и скрытие несущественных деталей. Пользователь объекта знает, **что** он делает, но не обязан знать, **как**.

Пример из лабораторной: метод `performAction()` сам вызывает закрытые `walk()` и `fetch()`, а пользователь вызывает только одну команду.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Dog {
    string name;
    void walk()  { cout << name << " walks. "; }     // детали скрыты
    void fetch() { cout << name << " fetches the ball. "; }
public:
    Dog(string n) { name = n; }
    void performAction()                             // простой интерфейс
    {
        walk();
        fetch();
        cout << "Done!" << endl;
    }
};

int main()
{
    Dog d("Buddy");
    d.performAction();
    // d.walk();   // ошибка: walk() — private
    return 0;
}
```

Вывод

```
Buddy walks. Buddy fetches the ball. Done!
```

Инкапсуляция и абстракция: в чём разница

Абстракция — про **дизайн**: решаем, какие свойства и действия важны, и показываем простой интерфейс («что делает»). **Инкапсуляция** — про **реализацию**: прячем данные и детали внутри класса и защищаем их (`private` , «как устроено»). Абстракция отвечает на вопрос «что показать», инкапсуляция — «как спрятать».

7.5 Принцип 3. Наследование

Наследование — создание нового класса (**производного**, потомка) на основе существующего (**базового**, родителя). Потомок получает все поля и методы родителя и добавляет свои. Отношение «**является**» (is-a): пудель **является** собакой.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Dog {
protected:                          // доступно потомкам
    string name;
public:
    Dog(string n) { name = n; }
    void bark() { cout << name << ": Woof!" << endl; }
};

class Poodle : public Dog {         // Poodle наследует Dog
    string coatType;                // своё новое поле
public:
    Poodle(string n, string c) : Dog(n) { coatType = c; }
    void doesGrooming() { cout << name << " gets a " << coatType << " haircut" << endl; }
};

int main()
{
    Poodle p("Fifi", "curly");
    p.bark();           // метод родителя
    p.doesGrooming();   // свой метод
    return 0;
}
```

Вывод

```
Fifi: Woof!
Fifi gets a curly haircut
```

Плюсы: код родителя пишется один раз, иерархия отражает реальный мир, можно расширять без изменения старого кода.

7.6 Принцип 4. Полиморфизм

Полиморфизм («много форм») — один и тот же вызов ведёт себя по-разному в зависимости от того, какой объект на самом деле его получил. В C++ это делается **виртуальными функциями** (`virtual`).

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() { cout << "..." << endl; }   // virtual!
};
class Dog : public Animal {
public:
    void makeSound() { cout << "Woof" << endl; }           // переопределение
};
class Cat : public Animal {
public:
    void makeSound() { cout << "Meow" << endl; }
};

int main()
{
    Dog d; Cat c; Animal a;
    Animal *zoo[3] = { &d, &c, &a };
    for (int i = 0; i < 3; i++)
        zoo[i]->makeSound();     // один вызов — разное поведение
    return 0;
}
```

Вывод

```
Woof
Meow
...
```

Без virtual полиморфизма нет

Если убрать `virtual`, через указатель `Animal*` всегда вызовется `Animal::makeSound()`: компилятор выберет метод по типу **указателя**, а не по реальному объекту. Это любимый вопрос на экзамене (подробнее в лекции 12).

Виды полиморфизма:

- **статический (на этапе компиляции)** — перегрузка функций и операций: `area(int)` и `area(double)`;
- **динамический (во время выполнения)** — виртуальные функции.

7.7 Взаимодействие объектов

Объекты общаются, вызывая методы друг друга и передавая друг другу объекты:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Engine {
public:
    int power;
    Engine(int p) { power = p; }
};
class Car {
    string model;
    Engine engine;                      // объект внутри объекта (композиция: «имеет»)
public:
    Car(string m, int p) : engine(p) { model = m; }
    void race(Car &other)               // объект получает другой объект
    {
        cout << (engine.power > other.engine.power ? model : other.model) << " wins" << endl;
    }
};

int main()
{
    Car a("Audi", 250), b("BMW", 300);
    a.race(b);
    return 0;
}
```

Вывод

```
BMW wins
```

«Является» или «имеет»?

Пудель **является** собакой → наследование. Машина **имеет** двигатель → объект-поле (композиция). Не наследуй Car от Engine!

7.8 Преимущества и применения ООП

Преимущества: **модульность** (каждый класс — отдельный кирпичик), **повторное использование** (наследование, библиотеки классов), **надёжность** (инкапсуляция защищает данные), **гибкость** (полиморфизм позволяет добавлять новые типы без изменения старого кода), **понятность** (модель похожа на реальный мир).

Где используется: графические интерфейсы (все кнопки, формы в C++ Builder — объекты VCL), игры (персонажи, предметы), банковские и учётные системы, моделирование.

Разбор задач

Задача 7.1. Класс Rectangle лёгкая

Опиши класс прямоугольника с закрытыми сторонами, методом set с проверкой (стороны > 0), методами area и perimeter.

Решение

```C++
class Rectangle {
    double w, h;
public:
    void set(double a, double b)
    {
        if (a > 0 && b > 0) { w = a; h = b; }
        else { w = h = 1; }
    }
    double area() { return w * h; }
    double perimeter() { return 2 * (w + h); }
};
//---
Rectangle r;
r.set(3, 4.5);
cout << r.area() << " " << r.perimeter() << endl;
r.set(-2, 5);
cout << r.area();
```

Вывод

```
13.5 15
1
```

Задача 7.2. Лабораторная из силлабуса целиком: Animal → Dog → Poodle, Cat средняя

Собери в одной программе все пункты задания: Dog с name и breed, приватным name и get/set, `performAction` с приватными walk/fetch, Poodle с coatType и doesGrooming, Cat и полиморфный `makeSound`.

Решение

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class Animal {
public:
    virtual void makeSound() { cout << "Some sound" << endl; }
    virtual ~Animal() {}
};

class Dog : public Animal {
    string name;                                    // private
    void walk()  { cout << name << " is walking" << endl; }
    void fetch() { cout << name << " fetches the ball" << endl; }
protected:
    string breed;
public:
    Dog(string n, string b) { name = n; breed = b; }
    string getName() { return name; }
    void setName(string n) { if (n != "") name = n; }
    void bark() { cout << name << " (" << breed << "): Woof!" << endl; }
    void performAction() { walk(); fetch(); }
    void makeSound() { cout << "Bark" << endl; }
};

class Poodle : public Dog {
    string coatType;
public:
    Poodle(string n, string c) : Dog(n, "Poodle") { coatType = c; }
    void doesGrooming() { cout << getName() << " has " << coatType << " coat groomed" <<
endl; }
};

class Cat : public Animal {
public:
    void makeSound() { cout << "Meow" << endl; }
};

int main()
{
    Dog myDog("Buddy", "Golden Retriever"), other("Max", "Beagle");
    myDog.bark(); other.bark();
    myDog.setName("Buddy Jr");
    cout << myDog.getName() << endl;
    myDog.performAction();
    Poodle p("Fifi", "curly");
    p.bark(); p.doesGrooming();
    Cat c;
    Animal *list[3] = { &myDog, &c, &p };
    for (int i = 0; i < 3; i++) list[i]->makeSound();
    return 0;
}
```

Вывод

```
Buddy (Golden Retriever): Woof!
Max (Beagle): Woof!
Buddy Jr
Buddy Jr is walking
Buddy Jr fetches the ball
Fifi (Poodle): Woof!
Fifi has curly coat groomed
Bark
Meow
Bark
```

Обрати внимание: Poodle не переопределил `makeSound`, поэтому вызывается версия Dog («Bark»). А `name` в Dog — private, поэтому Poodle берёт имя через `getName()`.

Задача 7.3. Определи принцип сложная

Для каждой ситуации назови принцип ООП: (1) поле `salary` закрыто, изменить можно только через `raise(percent)`; (2) `Shape *s` — для круга `s->area()` считает πr^2, для квадрата a^2; (3) `Student` и `Teacher` получают `name` и `print()` от `Person`; (4) пользователь вызывает `car.start()`, не зная про свечи и топливный насос; (5) две функции `print(int)` и `print(string)`.

Решение

1 — инкапсуляция; 2 — полиморфизм (динамический, через virtual); 3 — наследование; 4 — абстракция; 5 — полиморфизм (статический, перегрузка).

Контрольные вопросы из силлабуса (устно)

1. **Что такое ООП?** — Подход, при котором программа строится из объектов, объединяющих данные и поведение и взаимодействующих друг с другом.
2. **Основные понятия ООП?** — Класс, объект, атрибут (поле), метод; принципы: инкапсуляция, наследование, полиморфизм, абстракция.
3. **Класс?** — Пользовательский тип-шаблон, описывающий поля и методы объектов.
4. **Объект?** — Экземпляр класса со своим состоянием.
5. **Инкапсуляция?** — Объединение данных и методов в классе и сокрытие внутреннего устройства (private), доступ через методы.
6. **Наследование?** — Создание нового класса на основе существующего с получением его полей и методов.
7. **Полиморфизм?** — Один интерфейс — разные реализации; одинаковый вызов работает по-разному для разных объектов.
8. **Абстракция?** — Выделение существенных характеристик объекта и игнорирование лишних деталей.
9. **Инкапсуляция vs абстракция?** — Абстракция решает, что показать (интерфейс), инкапсуляция прячет и защищает реализацию.
10. **Преимущества ООП?** — Модульность, повторное использование, защита данных, расширяемость, понятность.

11. **Процедурный vs ООП?** — В процедурном данные отдельно от функций; в ООП они объединены в объекты.
12. **Как моделировать реальные сущности?** — Сущность → класс, её характеристики → поля, её действия → методы, конкретные предметы → объекты.

Тест по лекции 7

1. A class is best described as: новый

a) a variable that stores several values
b) a blueprint (template) that describes the data and behavior of objects
c) a function that returns an object
d) a library of functions

2. An object is: новый

a) a description of a class
b) an instance of a class
c) a method of a class
d) a keyword of C++

3. Hiding the internal data of a class and providing access only through methods is called: новый

a) inheritance
b) polymorphism
c) encapsulation
d) recursion

4. Creating a new class based on an existing class is called: новый

a) inheritance
b) encapsulation
c) abstraction
d) overloading

5. The ability of the same function call to behave differently for objects of different classes is called: новый

a) encapsulation
b) polymorphism
c) inheritance
d) abstraction

6. Showing only essential features of an object and hiding unnecessary details is called: новый

a) abstraction
b) polymorphism
c) inheritance
d) iteration

7. Which pair is an example of the "is-a" relationship (inheritance)? новый

a) Car — Engine
b) Poodle — Dog
c) Book — Page
d) Computer — Keyboard

8. In OOP, a function declared inside a class is called: новый

a) a field
b) an attribute
c) a method
d) an object

9. The set of current values of all fields of an object is its: новый

a) behavior
b) state
c) interface
d) method

10. Which kind of polymorphism is function overloading? новый

a) static (compile-time)
b) dynamic (run-time)
c) virtual
d) it is not polymorphism

11. What is the output? сложный

```
class A {
public:
    void show() { cout << "A"; }
};
class B : public A {
public:
    void show() { cout << "B"; }
};
//---
B b;
A *p = &b;
p->show();
b.show();
```

a) BB
b) AB
c) BA
d) AA

12. What is the output? сложный

```
class A {
public:
    virtual void show() { cout << "A"; }
};
class B : public A {
public:
    void show() { cout << "B"; }
};
//---
B b;
A *p = &b;
A a = b;
p->show();
a.show();
```

a) BB
b) BA
c) AB
d) AA

13. Which line causes a compilation error? сложный

```
class Counter {
    int value;                // line 1
public:
    void inc() { value++; }   // line 2
};
int main() {
    Counter c;                // line 3
    c.inc();                  // line 4
    c.value = 10;             // line 5
}
```

a) line 2
b) line 3
c) line 4
d) line 5

Ключи

1 – b **2** – b **3** – c **4** – a **5** – b **6** – a
7 – b **8** – c **9** – b **10** – a **11** – b **12** – b
13 – d

7. Пудель является собакой. Остальные пары — «имеет» (композиция).

11. Метод не виртуальный, поэтому через указатель типа A* вызывается A::show(). Напрямую у объекта b — B::show().

12. p указывает на объект B, функция виртуальная → B. A `A a = b;` создаёт **копию только части A** (срезка объекта), поэтому a — настоящий A.

13. По умолчанию члены класса — private. Изнутри класса (line 2) обращаться можно, снаружи (line 5) нельзя.

ЛЕКЦИЯ 8 ИЗ 15

Классы и объекты в C++

Уверенно описывать классы, создавать объекты, обращаться к их членам и понимать, кто и к чему имеет доступ.

8.1 Структура класса

```
class ИмяКласса {
private:            // закрытая часть (по умолчанию)
    поля;
    служебные методы;
public:             // открытая часть — интерфейс
    методы;
protected:          // для наследников (лекция 12)
    ...
};                  // ← точка с запятой обязательна!
```

C++

```
class Student {
private:
    char name[30];
    int age;
    double gpa;
public:
    void set(const char *n, int a, double g)
    {
        strcpy(name, n);
        age = (a > 0) ? a : 18;
        gpa = g;
    }
    void print() { cout << name << ", " << age << ", GPA " << gpa << endl; }
    bool isExcellent() { return gpa >= 3.67; }
};
//---
Student s1, s2;
s1.set("Aigerim", 19, 3.9);
s2.set("Daniyar", -5, 3.1);
s1.print();
s2.print();
cout << s1.isExcellent() << s2.isExcellent();
```

Вывод

```
Aigerim, 19, GPA 3.9
Daniyar, 18, GPA 3.1
10
```

- **Данные-члены (поля)** — переменные класса: `name`, `age`, `gpa`.
- **Функции-члены (методы)** — функции класса: `set`, `print`. Внутри метода поля доступны **напрямую**, без точки: метод всегда работает с тем объектом, для которого его вызвали.

Забытая точка с запятой

`class A { … }` без `;` в конце даёт странные ошибки в **следующей** строке. Ставь `};` всегда.

8.2 Спецификаторы доступа

Спецификатор	Кто имеет доступ
`public`	все: и методы класса, и любой внешний код
`private`	**только** методы этого класса (и друзья — `friend`)
`protected`	методы этого класса **и его наследников**

- В `class` по умолчанию всё **private**, в `struct` — **public**. Других отличий у class и struct в C++ нет.
- Спецификаторы можно повторять в любом порядке, действуют до следующего.
- Доступ проверяется для **класса**, а не для объекта: метод может читать private-поля **другого** объекта того же класса.

```C++
class Point {
    int x, y;                       // private по умолчанию
public:
    void set(int a, int b) { x = a; y = b; }
    int dist2(Point other)          // имеет доступ к other.x, хотя он private
    {
        return (x - other.x) * (x - other.x) + (y - other.y) * (y - other.y);
    }
};
struct P2 { int x, y; };            // public по умолчанию
//---
Point a, b;
a.set(0, 0); b.set(3, 4);
P2 q = {7, 8};
cout << a.dist2(b) << " " << q.x + q.y;
```

Вывод

```
25 15
```

8.3 Методы вне класса и оператор ::

Большие методы удобно **объявить** в классе, а **определить** снаружи, указав принадлежность через `::` (оператор разрешения области видимости). Так в C++ Builder и делают: класс в `.h`, реализация в `.cpp`.

```C++
class Circle {
    double r;
public:
    void setR(double v);          // только объявление
    double area();
    double length();
};
void Circle::setR(double v) { r = (v >= 0) ? v : 0; }   // Класс::метод
double Circle::area() { return 3.14159 * r * r; }
double Circle::length() { return 2 * 3.14159 * r; }
//---
Circle c;
c.setR(2);
cout << c.area() << " " << c.length();
```

Вывод
```
12.5664 12.5664
```

Запомни

Методы, **написанные прямо внутри** класса, автоматически считаются `inline`. Определённые снаружи — обычные функции.

8.4 Создание объектов и доступ к членам

```C++
class Box {
public:
    int w, h;
    int area() { return w * h; }
};
//---
Box a;                  // объект в стеке — уничтожится сам в конце блока
a.w = 2; a.h = 3;       // доступ через точку
Box *p = new Box;       // объект в динамической памяти (куча)
p->w = 4; p->h = 5;     // доступ через указатель — стрелка ->
(*p).w = 6;             // то же, что p->w = 6
Box arr[3];             // массив объектов
for (int i = 0; i < 3; i++) { arr[i].w = i + 1; arr[i].h = 10; }
cout << a.area() << " " << p->area() << " " << arr[2].area();
delete p;               // объект из new обязательно удаляем
```

Вывод
```
6 30 30
```

Как создан	Обращение	Удаление
`Box a;`	`a.w`, `a.area()`	автоматически
`Box *p = new Box;`	`p->w`, `p->area()`	`delete p;`
`Box *m = new Box[n];`	`m[i].w`	`delete [] m;`

Утечка памяти

Каждому `new` должен соответствовать `delete` (для массива — `delete []`). Иначе память не освобождается до конца программы. Обращаться к объекту после `delete` нельзя.

8.5 Указатель this

Внутри любого метода есть скрытый указатель **this** — адрес **того объекта**, для которого метод вызван. Он нужен, когда:

- имя параметра совпадает с именем поля (`this->age = age`);
- метод должен вернуть сам объект (для цепочки вызовов);
- объект передаёт себя в другую функцию.

```cpp
class Person {
    int age;
public:
    Person &setAge(int age)        // параметр закрыл поле
    {
        this->age = age;           // this->age — поле, age — параметр
        return *this;              // возвращаем сам объект
    }
    Person &older(int k) { age += k; return *this; }
    int getAge() { return age; }
    bool isSame(Person *p) { return this == p; }
};
//---
Person p, q;
p.setAge(20).older(3).older(2);    // цепочка вызовов
cout << p.getAge() << " " << p.isSame(&p) << p.isSame(&q);
```

Вывод

```
25 10
```

Без this параметр «съедает» поле

`void setAge(int age) { age = age; }` — параметр присваивается **сам себе**, поле не меняется. Пиши `this->age = age;` или называй параметр иначе.

8.6 Статические члены класса

static-поле — одно на **весь класс**, общее для всех объектов (а обычное поле у каждого объекта своё). Его нужно **определить вне класса**: `int Cls::count = 0;`.

static-метод — вызывается через имя класса `Cls::method()`, работает только со static-полями (у него нет `this`).

```C++
class Student {
    int id;
public:
    static int count;               // объявление
    Student() { count++; id = count; }
    int getId() { return id; }
    static int total() { return count; }   // нет this, видит только static
};
int Student::count = 0;             // определение — обязательно!
//---
Student a, b, c;
cout << a.getId() << b.getId() << c.getId() << " " << Student::total() << " " << a.count;
```

Вывод
```
123 3 3
```

Частая ошибка

Забыли строку `int Student::count = 0;` — ошибка линковщика «Undefined symbol Student::count». В static-методе нельзя обращаться к обычным полям (`id`) и к `this`.

8.7 Объекты как параметры и результат

```C++
class Money {
public:
    int tg;
    Money add(Money other) { Money r; r.tg = tg + other.tg; return r; }   // возврат объекта
};
void bonusByValue(Money m) { m.tg += 1000; }     // копия — оригинал не меняется
void bonusByRef(Money &m)  { m.tg += 1000; }     // ссылка — меняется оригинал
void show(const Money &m)  { cout << m.tg << " "; }   // const & — без копии, только чтение
//---
Money a, b;
a.tg = 500; b.tg = 200;
bonusByValue(a); show(a);
bonusByRef(a);   show(a);
Money c = a.add(b); show(c);
```

Вывод
```
500 1500 1700
```

Фишка / совет

Объекты лучше передавать по ссылке: копирование большого объекта медленное. Если менять не нужно — `const Класс &`.

8.8 Константные методы

Метод с `const` после скобок **обещает не менять** поля объекта. Только такие методы можно вызывать у константных объектов и через `const &`.

```C++
class Temp {
    double c;
public:
    Temp(double v) { c = v; }
    double getF() const { return c * 9 / 5 + 32; }   // не меняет объект
    void heat(double d) { c += d; }
};
void report(const Temp &t) { cout << t.getF(); }      // t.heat(1) здесь — ошибка
//---
Temp t(25);
t.heat(5);
report(t);
```

Вывод

```
86
```

Фишка / совет

Все геттеры делай `const`. Это требование из лабораторной про Book (`string getTitle() const;`).

8.9 Взаимодействие объектов разных классов

```C++
class Book {
public:
    char title[30];
    int pages;
};
class Reader {
    char name[20];
    int readPages;
public:
    Reader(const char *n) { strcpy(name, n); readPages = 0; }
    void read(const Book &b) { readPages += b.pages; cout << name << " read " << b.title <<
endl; }
    int total() const { return readPages; }
};
//---
Book b1 = {"Abai", 320}, b2 = {"C++ Primer", 970};
Reader r("Aruzhan");
r.read(b1);
r.read(b2);
cout << r.total();
```

Вывод

```
Aruzhan read Abai
Aruzhan read C++ Primer
1290
```

Разбор задач

Задача 8.1. Класс Time лёгкая

Класс с полями часов и минут (private), методами `set(h, m)` с проверкой, `addMinutes(k)` и `print()` в формате ЧЧ:ММ.

Решение

```C++
class Time {
    int h, m;
public:
    void set(int hh, int mm)
    {
        h = (hh >= 0 && hh < 24) ? hh : 0;
        m = (mm >= 0 && mm < 60) ? mm : 0;
    }
    void addMinutes(int k)
    {
        int t = (h * 60 + m + k) % (24 * 60);
        h = t / 60; m = t % 60;
    }
    void print() const
    {
        if (h < 10) cout << 0;
        cout << h << ":";
        if (m < 10) cout << 0;
        cout << m << endl;
    }
};
//---
Time t;
t.set(22, 45);
t.print();
t.addMinutes(90);
t.print();
```

Вывод

```
22:45
00:15
```

Задача 8.2. Счёт в банке со static-счётчиком средняя

Класс Account: номер (выдаётся автоматически), баланс, методы deposit, withdraw (нельзя уйти в минус, вернуть false), transfer на другой счёт; static-поле — сколько счетов создано.

Решение

```C++
class Account {
    int number;
    double balance;
    static int created;
public:
    Account() { number = ++created; balance = 0; }
    void deposit(double s) { if (s > 0) balance += s; }
    bool withdraw(double s)
    {
        if (s <= 0 || s > balance) return false;
        balance -= s;
        return true;
    }
    bool transfer(Account &to, double s)
    {
        if (!withdraw(s)) return false;
        to.deposit(s);
        return true;
    }
    void print() const { cout << "#" << number << ": " << balance << endl; }
    static int count() { return created; }
};
int Account::created = 0;
//---
Account a, b;
a.deposit(1000);
cout << a.transfer(b, 300) << a.transfer(b, 5000) << endl;
a.print(); b.print();
cout << Account::count();
```

Вывод

```
10
#1: 700
#2: 300
2
```

Задача 8.3. Массив объектов в динамической памяти сложная

Создай через `new` массив из n = 4 объектов Student (имя, балл), заполни, найди лучшего и средний балл, освободи память.

Решение

```C++
class Student {
public:
    char name[20];
    int score;
};
//---
int n = 4;
Student *g = new Student[n];
const char *names[4] = {"Ali", "Bota", "Erlan", "Zhanna"};
int scores[4] = {78, 95, 64, 88};
for (int i = 0; i < n; i++) { strcpy(g[i].name, names[i]); g[i].score = scores[i]; }
int best = 0, sum = 0;
for (int i = 0; i < n; i++) {
    sum += g[i].score;
    if (g[i].score > g[best].score) best = i;
}
cout << g[best].name << " " << (double)sum / n;
delete [] g;
```

Вывод

```
Bota 81.25
```

Контрольные вопросы из силлабуса (устно)

1. **Класс в C++?** — Пользовательский тип, объединяющий данные (поля) и функции (методы).
2. **Объект?** — Переменная типа класса, экземпляр.
3. **Связь класса и объекта?** — Класс — описание, объект — его конкретная реализация в памяти; объектов одного класса много.
4. **Данные-члены и функции-члены?** — Поля (переменные) и методы (функции) класса.
5. **Как объявить класс?** — `class Имя { спецификаторы: поля; методы; };`.
6. **public, private, protected?** — Доступ всем; только методам класса; классу и наследникам.
7. **Инкапсуляция в классе C++?** — Поля private, доступ через public-методы.
8. **Как создать объект?** — `Имя obj;`, `Имя *p = new Имя;`, `Имя arr[N];`.
9. **Как обратиться к членам?** — `obj.член`, через указатель `p->член`.
10. **Указатель this?** — Скрытый указатель на текущий объект в методе; нужен при совпадении имён и для возврата `*this`.
11. **static-члены?** — Общие для всех объектов класса; static-поле определяется вне класса, static-метод вызывается `Класс::метод()`.
12. **Взаимодействие объектов одного класса?** — Через методы, принимающие другой объект (по ссылке); метод имеет доступ к private-полям другого объекта того же класса.

Тест по лекции 8

1. By default, members of a `class` are: новый

a) public
b) private
c) protected
d) static

2. By default, members of a `struct` in C++ are: новый

a) public
b) private
c) protected
d) const

3. Which operator accesses a member through a pointer to an object? новый

a) .
b) ::
c) ->
d) &

4. What is `this` inside a member function? новый

a) the name of the class
b) a pointer to the object for which the method was called
c) a copy of the object
d) a static member

5. How is the method `area` of class `Circle` defined outside the class? новый

a) double Circle.area() { ... }
b) double area::Circle() { ... }
c) double Circle::area() { ... }
d) double Circle->area() { ... }

6. A static data member of a class is: новый

a) shared by all objects of the class
b) created separately for each object
c) accessible only through `this`
d) automatically public

7. Which keyword protects a member from outside access but allows access from derived classes? новый

a) private
b) public
c) protected
d) static

8. What is the output? сложный

```
class C {
public:
    static int n;
    int k;
    C() { n++; k = n * 10; }
};
int C::n = 0;
//---
C a, b;
C c;
cout << a.k + b.k + c.k << " " << C::n;
```

a) 30 3
b) 60 3
c) 90 3
d) 60 1

9. What is the output? сложный

```
class V {
public:
    int x;
    void set(int x) { x = x; }
    void set2(int x) { this->x = x; }
};
//---
V v;
v.x = 1;
v.set(5);
cout << v.x;
v.set2(7);
cout << v.x;
```

a) 57
b) 17
c) 55
d) 11

10. What is the output? сложный

```
class Box {
public:
    int v;
};
void f(Box b) { b.v = 100; }
void g(Box &b) { b.v += 5; }
void h(Box *b) { b->v *= 2; }
//---
Box x;
x.v = 1;
f(x); g(x); h(&x);
cout << x.v;
```

a) 100
b) 210
c) 12
d) 6

11. Which line does NOT compile? сложный

```
class T {
    int a;
public:
    int b;
    int getA() const { return a; }
    void setA(int v) const { a = v; }   // Line X
};
```

a) int a;

b) int b;

c) int getA() const { return a; }

d) void setA(int v) const { a = v; }

Ключи

1 — b **2** — a **3** — c **4** — b **5** — c **6** — a
7 — c **8** — b **9** — b **10** — c **11** — d

8. a.k = 10, b.k = 20, c.k = 30 (k вычисляется в момент создания каждого объекта). Счётчик n общий, он равен 3.

9. В `set` параметр x присваивается сам себе, поле не меняется. В `set2` через `this->x` меняется поле.

10. f получает копию (оригинал не меняется), g — ссылку: 1 + 5 = 6, h — указатель: 6 · 2 = 12.

11. const-метод не может изменять поля объекта.

ЛЕКЦИЯ 9 ИЗ 15

Интегрированная среда разработки Borland C++ Builder

Ориентироваться в окнах C++ Builder, понимать, из каких файлов состоит проект, уметь компилировать, запускать и отлаживать программу.

9.1 Что такое IDE

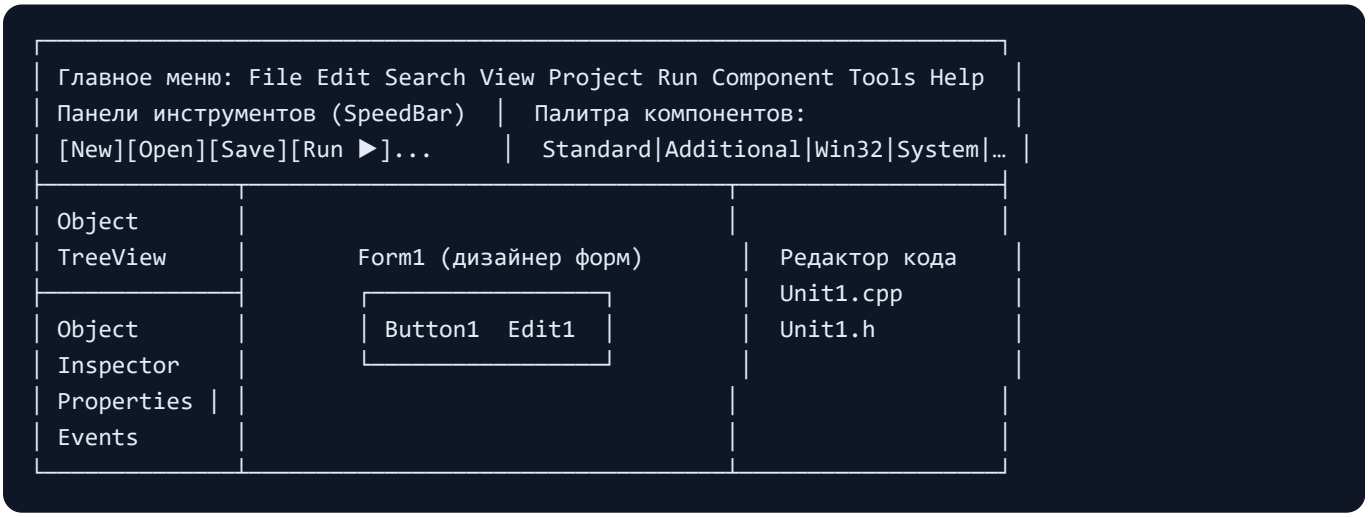
IDE (Integrated Development Environment, интегрированная среда разработки) — программа, которая объединяет в одном окне всё, что нужно разработчику:

- **редактор** исходного кода (с подсветкой и подсказками);
- **компилятор** и **компоновщик (линкер)** — превращают текст в `.exe`;
- **отладчик** — пошаговое выполнение и просмотр переменных;
- визуальные инструменты: **дизайнер форм**, **палитра компонентов**, **инспектор объектов**;
- **менеджер проекта**, справку.

Borland C++ Builder (сейчас Embarcadero C++Builder) — IDE для **визуальной** разработки Windows-приложений на C++ по технологии **RAD** (Rapid Application Development — быстрая разработка приложений): интерфейс собирается мышкой из готовых компонентов библиотеки **VCL** (Visual Component Library), а программист пишет только код реакций на события.

> **Консоль или окна**
>
> В C++ Builder можно делать и консольные программы (**File** → **New** → **Other** → **Console Wizard**, как в лекциях 1–8), и оконные приложения (**File** → **New** → **Application**), о которых лекции 10–15.

9.2 Главные окна C++ Builder



Окно	Назначение
Главное меню	все команды: File (создать, открыть, сохранить), Project (опции, Build), Run (запуск, отладка), Component (установка компонентов)
Панели инструментов (SpeedBar)	кнопки для самых частых команд: открыть, сохранить, запустить ▶
Палитра компонентов (Component Palette)	все компоненты VCL, разложенные по вкладкам: Standard (Button, Label, Edit, Memo, CheckBox, RadioButton, ListBox, ComboBox, Panel, MainMenu), Additional (BitBtn, SpeedButton, StringGrid, Image, Chart), Win32 (PageControl, ProgressBar, TreeView), System (Timer, MediaPlayer), Dialogs (OpenDialog, SaveDialog, ColorDialog)
Дизайнер форм (Form Designer)	окно будущей программы; на него кладут компоненты, двигают и меняют размер мышкой
Инспектор объектов (Object Inspector)	свойства (вкладка **Properties**) и события (вкладка **Events**) выделенного компонента
Редактор кода (Code Editor)	текст программы: `Unit1.cpp`, `Unit1.h`; вкладки внизу переключают файлы
Object TreeView	дерево компонентов формы: кто внутри кого лежит
Менеджер проекта (Project Manager)	список файлов проекта, добавление и удаление модулей
Окно сообщений (Messages)	ошибки и предупреждения компилятора; двойной щелчок переходит к строке с ошибкой

Полезные клавиши

F12 — переключиться между формой и кодом. **F11** — показать инспектор объектов. **Ctrl+Alt+F11** — менеджер проекта. **Ctrl+Space** — подсказка кода. **Ctrl+S / Shift+Ctrl+S** — сохранить файл / всё.

9.3 Инспектор объектов подробнее

- Слева **имя свойства**, справа **значение**. Изменение сразу видно на форме.
- Свойства бывают: текстовые/числовые (`Caption`, `Width`), логические (`Visible: true/false`), перечисления из списка (`Color: clRed`, `Position: poScreenCenter`), составные с «+» (`Font` раскрывается в `Name`, `Size`, `Style`), с кнопкой «…» (открывают редактор, например `Items` у ListBox или `Picture` у Image).
- На вкладке **Events** перечислены события (`OnClick`, `OnCreate`, `OnKeyPress`…). **Двойной щелчок** по пустому полю создаёт **обработчик** в коде.
- Свойство **Name** — имя объекта в программе (`Button1`). Меняй его до того, как написал код, и давай понятные имена: `btnCalc`, `edtA`, `lblResult`.

9.4 Из каких файлов состоит проект

Файл	Что в нём
`Project1.bpr` (в новых версиях `.cbproj`)	файл проекта: настройки, список модулей
`Project1.cpp`	главный файл с функцией `WinMain`: создаёт формы и запускает приложение (генерируется автоматически)
`Unit1.cpp`	**ваш код**: реализация методов формы, обработчики событий
`Unit1.h`	объявление класса формы `TForm1`: компоненты и методы
`Unit1.dfm`	описание формы: какие компоненты лежат и какие у них свойства (то, что настроено мышкой)
`Project1.res`	ресурсы: иконка приложения, версия
`.obj`	объектные файлы, результат компиляции модулей
`.exe`	готовая программа
`.tds`	отладочная информация

C++ BUILDER

```cpp
// Project1.cpp — создаётся автоматически
#include <vcl.h>
#pragma hdrstop
USEFORM("Unit1.cpp", Form1);
WINAPI WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
{
    try {
        Application->Initialize();
        Application->CreateForm(__classid(TForm1), &Form1);   // создаём главную форму
        Application->Run();                                    // цикл обработки событий
    }
    catch (Exception &exception) {
        Application->ShowException(&exception);
    }
    return 0;
}
```

Сохранение проекта

Сохраняй каждый проект **в отдельную папку** через **File → Save All**. Сначала Builder спрашивает имя модуля (Unit1.cpp), потом имя проекта (Project1.bpr), и они **не должны совпадать**. Имя проекта станет именем `.exe`.

9.5 Компиляция, сборка, запуск

Команда	Клавиши	Что делает
Compile Unit	Alt+F9	компилирует только текущий модуль (проверка ошибок)
Make	Ctrl+F9	перекомпилирует изменённые модули и собирает `.exe`
Build	Project → Build	полностью пересобирает весь проект
Run	**F9**	собирает (если нужно) и запускает программу
Program Reset	Ctrl+F2	аварийно остановить запущенную программу

Этапы превращения кода в программу:

1. **Препроцессор** обрабатывает `#include`, `#define`.
2. **Компилятор** переводит каждый `.cpp` в объектный файл `.obj` и сообщает о синтаксических ошибках.
3. **Компоновщик (linker)** соединяет `.obj` с библиотеками в `.exe` («Unresolved external» — ошибка линковки).
4. **Запуск** и проверка работы.

9.6 Виды ошибок

Вид	Когда обнаруживается	Пример
Синтаксическая	при компиляции	`int x = 5` (нет `;`), `cout << "hi` (нет кавычки), `Cout`
Семантическая	при компиляции (или линковке)	использование необъявленной переменной, несоответствие типов: `int *p = 5.5;`, вызов несуществующего метода
Логическая	**не обнаруживается** компилятором — программа работает, но результат неверный	`avg = sum / n` в целых, `<=` вместо `<`, `=` вместо `==`
Ошибка времени выполнения (runtime)	во время работы	деление на ноль, выход за границы массива, обращение по NULL

> **Как спрашивают на экзамене**
>
> Синтаксическая — нарушены правила записи языка. Семантическая — запись правильная, но смысл недопустим (типы, имена). Логическая — программа компилируется и работает, но делает не то, что нужно. Логические ошибки ищут отладкой и тестированием.

9.7 Отладка (Debugging)

Отладка — поиск и исправление ошибок, прежде всего логических. Инструменты C++ Builder:

Инструмент	Клавиши	Назначение
Точка останова (Breakpoint)	F5 или щелчок слева от строки	программа остановится перед этой строкой
Step Over	**F8**	выполнить строку целиком (не заходя в функции)
Trace Into	**F7**	выполнить строку, заходя внутрь вызываемых функций
Run to Cursor	F4	выполнить до строки с курсором
Watch (наблюдение)	Ctrl+F5	постоянно показывать значение переменной/выражения
Evaluate/Modify	Ctrl+F7	посмотреть и даже изменить значение во время паузы
Подсказка	навести мышь на переменную	показать её текущее значение
Call Stack	View → Debug Windows	цепочка вызовов функций

Как искать логическую ошибку

1. Поставь breakpoint перед подозрительным местом. 2. Запусти F9. 3. Иди по шагам F8 и смотри в Watch, как меняются переменные. 4. Найди строку, после которой значение стало неверным. Без отладчика подойдёт и «печать на экран»: временно выводи промежуточные значения.

C++
```cpp
// Логическая ошибка: среднее считается неверно
int a[4] = {2, 3, 4, 4};
int sum = 0;
for (int i = 0; i < 4; i++) sum += a[i];
double wrong = sum / 4;          // целое деление: 13/4 = 3
double right = sum / 4.0;        // 3.25
cout << wrong << " " << right;
```

Вывод
```
3 3.25
```

Разбор задач

Задача 9.1. Найди и классифицируй ошибки лёгкая

Сколько ошибок и каких видов в этом фрагменте?

```
int main()
{
    int n = 5
    int s;
    for (int i = 1; i < n; i++) s += i;
    cout << "Sum 1..5 = " << S;
}
```

Решение

1. `int n = 5` без `;` — **синтаксическая**.
2. `S` вместо `s` — **семантическая** (необъявленный идентификатор, регистр важен).
3. `s` не инициализирована (`int s = 0;`) — **логическая**: результат будет мусором.
4. `i < n` вместо `i <= n` — **логическая**: сумма до 4, а не до 5.

Синтаксическую и семантическую найдёт компилятор, логические — только тестирование и отладка.

Задача 9.2. Пошаговая отладка «в уме» средняя

Поставь breakpoint на строку цикла и нажимай F8. Запиши, какие значения покажет Watch для `i`, `p` после каждого шага.

```C++
int p = 1;
for (int i = 1; i <= 4; i++)
    p = p * 2 + i;
cout << p;
```

Решение

> **Вывод**
> 42

шаг	i	p после шага
1	1	1·2 + 1 = 3
2	2	3·2 + 2 = 8
3	3	8·2 + 3 = 19
4	4	19·2 + 4 = 42

После 4-го шага i становится 5, условие `i <= 4` ложно, цикл заканчивается, и печатается 42. Именно так отладчик помогает: видно значение на каждом шаге, и сразу понятно, где оно «поехало».

Контрольные вопросы из силлабуса (устно)

1. **Что такое IDE?** — Интегрированная среда разработки: редактор, компилятор, линкер, отладчик и визуальные инструменты в одной программе.
2. **Что такое Borland C++ Builder?** — IDE для быстрой визуальной разработки (RAD) Windows-приложений на C++ с библиотекой компонентов VCL.
3. **Основные компоненты IDE?** — Главное меню, панели инструментов, палитра компонентов, дизайнер форм, инспектор объектов, редактор кода, менеджер проекта, окно сообщений.
4. **Редактор кода?** — Окно для написания и редактирования исходного текста (.cpp, .h) с подсветкой и подсказками.
5. **Object Inspector?** — Окно свойств (Properties) и событий (Events) выбранного компонента.
6. **Component Palette?** — Панель с вкладками готовых компонентов VCL для размещения на форме.
7. **Form Designer?** — Визуальный редактор формы, где компоненты располагают мышкой.
8. **Проект?** — Набор файлов (.bpr, .cpp, .h, .dfm, .res), из которых собирается одна программа.
9. **Этапы компиляции и запуска?** — Препроцессор → компиляция в .obj → компоновка в .exe → запуск (F9).
10. **Отладка?** — Поиск и устранение ошибок с помощью точек останова, пошагового выполнения и наблюдения за переменными.
11. **Синтаксические, семантические, логические ошибки?** — Нарушение правил записи; недопустимый смысл (типы, имена); неверный алгоритм при правильной программе.

Тест по лекции 9

1. What does IDE stand for? новый

a) Internal Debugging Engine
b) Integrated Development Environment
c) Interactive Design Editor
d) Integrated Data Exchange

2. Which window of C++ Builder shows the properties and events of the selected component? новый

a) Component Palette
b) Code Editor
c) Object Inspector
d) Project Manager

3. Where are ready-made components such as TButton and TEdit selected? новый

a) Component Palette
b) Object Inspector
c) Messages window
d) Watch list

4. Which file stores the description of a form (its components and their property values)? новый

a) Unit1.cpp
b) Unit1.h
c) Unit1.dfm
d) Project1.res

5. Which key compiles (if needed) and runs the program in C++ Builder? новый

a) F7
b) F8
c) F9
d) F12

6. Which key switches between the form and its code? новый

a) F9 **b)** F12
c) F5 **d)** Ctrl+F9

7. A program compiles and runs, but prints a wrong result. What kind of error is it? новый

a) syntax error **b)** semantic error
c) logical error **d)** linker error

8. Which debugging command executes the current line without entering called functions? новый

a) Trace Into (F7) **b)** Step Over (F8)
c) Run to Cursor (F4) **d)** Program Reset (Ctrl+F2)

9. What is a breakpoint? новый

a) an error that stops compilation
b) a marked line where program execution pauses in the debugger
c) the end of the main function
d) a `break` statement in a loop

10. Which library of visual components is used in Borland C++ Builder? новый

a) STL **b)** MFC
c) VCL **d)** Qt

11. Missing semicolon after a statement is an example of: новый

a) a syntax error **b)** a logical error
c) a runtime error **d)** a semantic error

12. Division by zero during program execution is: новый

a) a syntax error **b)** a runtime error
c) a linker error **d)** not an error

Ключи

1 – b	**2** – c	**3** – a	**4** – c	**5** – c	**6** – b
7 – c	**8** – b	**9** – b	**10** – c	**11** – a	**12** – b

ЛЕКЦИЯ 10 ИЗ 15

Создание проекта. Работа с формами. Дизайнер форм

Создать оконное приложение: спроектировать форму мышкой, связать её с кодом и открыть вторую форму.

10.1 Проект в C++ Builder

Проект — набор файлов, из которых собирается одна программа (`.exe`). Создание: **File** → **New** → **Application**. Builder сразу создаёт:

- форму `Form1` (класс `TForm1`) с модулем `Unit1.cpp` / `Unit1.h` / `Unit1.dfm`;
- главный файл `Project1.cpp` с `WinMain`;
- файл проекта `Project1.bpr`.

Модуль (unit) — пара файлов `.cpp` + `.h` (и `.dfm`, если это форма). Каждой форме соответствует свой модуль.

Порядок работы над любым оконным приложением

1. **File** → **New** → **Application**, сразу **File** → **Save All** в отдельную папку.
2. Положи компоненты на форму (палитра → щелчок → щелчок по форме).
3. Настрой свойства в Object Inspector (Name, Caption, Text...).
4. Создай обработчики событий (двойной щелчок по кнопке → `OnClick`).
5. Напиши код в обработчиках.
6. Запусти **F9**, проверь, исправь.

10.2 Форма — основа графического интерфейса

Форма — окно программы. Это объект класса, производного от `TForm`. На форме размещаются все элементы интерфейса (**GUI — Graphical User Interface**, графический интерфейс пользователя): кнопки, поля ввода, надписи, списки.

Builder сам пишет класс формы. Вот как он выглядит после того, как на форму положили Label, Edit и Button:

C++ BUILDER

```cpp
// Unit1.h — объявление класса формы
#ifndef Unit1H
#define Unit1H
#include <Classes.hpp>
#include <Controls.hpp>
#include <StdCtrls.hpp>
#include <Forms.hpp>

class TForm1 : public TForm                    // наша форма наследует TForm
{
__published:    // компоненты и обработчики, которыми управляет IDE
    TLabel *Label1;
    TEdit *Edit1;
    TButton *Button1;
    void __fastcall Button1Click(TObject *Sender);
private:        // ваши закрытые поля и методы
public:         // ваши открытые поля и методы
    __fastcall TForm1(TComponent* Owner);        // конструктор формы
};
extern PACKAGE TForm1 *Form1;                  // глобальный указатель на форму
#endif
```

C++ BUILDER

```cpp
// Unit1.cpp — реализация
#include <vcl.h>
#pragma hdrstop
#include "Unit1.h"
#pragma package(smart_init)
#pragma resource "*.dfm"                       // подключить описание формы
TForm1 *Form1;

__fastcall TForm1::TForm1(TComponent* Owner) : TForm(Owner)
{
}

void __fastcall TForm1::Button1Click(TObject *Sender)
{
    Label1->Caption = "Hello, " + Edit1->Text + "!";
}
```

Что здесь важно понимать:

- Форма — это **класс** (лекции 7–8!), а компоненты — **указатели на объекты**-поля этого класса, поэтому к ним обращаются через `->`: `Label1->Caption`.
- Раздел `__published` заполняет IDE. Не удаляй оттуда строки вручную: компонент удаляй на форме, а пустой обработчик Builder уберёт сам при сохранении.
- `__fastcall` — соглашение о вызове, обязательное для методов VCL.
- `Sender` — указатель на объект, который вызвал событие (какая кнопка нажата).

10.3 Дизайнер форм: размещение компонентов

- **Положить компонент**: щелчок по значку в палитре → щелчок по форме (или двойной щелчок по значку — компонент встанет в центр).
- **Выделить несколько**: Shift + щелчок или обвести рамкой.

- **Переместить**: перетащить мышкой; точно — стрелками с **Ctrl**.
- **Изменить размер**: тянуть за маркеры; точно — стрелками с **Shift**.
- **Выровнять группу**: контекстное меню → **Align** (по левому краю, по центру…), **Size** (одинаковый размер), палитра выравнивания View → Alignment Palette.
- **Порядок перехода по Tab**: Edit → **Tab Order**.
- **Передний/задний план**: Bring to Front / Send to Back.
- **Заблокировать**: Edit → Lock Controls (чтобы случайно не сдвинуть).
- Свойства расположения: `Left`, `Top` (координаты от левого верхнего угла формы), `Width`, `Height`; **Align** (`alTop`, `alBottom`, `alClient` — растянуть на всё свободное место); **Anchors** (к каким краям «привязан» при изменении размеров формы).

> **Визуальные и невизуальные компоненты**
>
> Визуальные (Button, Label, Edit) видны во время работы. Невизуальные (Timer, OpenDialog, MainMenu) на форме выглядят значком, а во время работы не видны как отдельный элемент.

10.4 Самые нужные компоненты (вкладка Standard)

Компонент	Для чего	Главные свойства / методы
`TLabel`	надпись (вывод текста)	`Caption`, `Font`, `AutoSize`
`TEdit`	поле ввода одной строки	`Text`, `MaxLength`, `ReadOnly`, `Clear()`
`TButton`	кнопка	`Caption`, событие `OnClick`
`TMemo`	многострочный текст	`Lines->Add(…)`, `Lines->Clear()`, `Lines->Count`
`TListBox`	список строк	`Items->Add(…)`, `ItemIndex`, `Items->Strings[i]`
`TComboBox`	выпадающий список	`Items`, `Text`, `ItemIndex`
`TCheckBox`	флажок «да/нет»	`Checked`
`TRadioButton` / `TRadioGroup`	выбор одного из нескольких	`Checked` / `ItemIndex`
`TPanel`, `TGroupBox`	контейнеры для группировки	`Caption`, `Align`
`TImage`	картинка	`Picture->LoadFromFile(…)`
`TTimer` (System)	событие через интервал	`Interval` (мс), `Enabled`, `OnTimer`

Превращение текста в числа и обратно

Всё, что вводит пользователь в `Edit`, — это **текст** (`AnsiString`). Для расчётов его переводят в число:

Функция	Что делает
`StrToInt(s)`	строка → int (ошибка, если там не число)
`StrToFloat(s)`	строка → double (в русских настройках дробная часть через **запятую**)
`IntToStr(n)`	int → строка
`FloatToStr(x)`	double → строка
`FormatFloat("0.00", x)`	double → строка с двумя знаками
`ShowMessage(s)`	окно с сообщением

C++ BUILDER
```
void __fastcall TForm1::btnSumClick(TObject *Sender)
{
    int a = StrToInt(edtA->Text);
    int b = StrToInt(edtB->Text);
    lblResult->Caption = "Sum = " + IntToStr(a + b);
}
```

Сложили строки, а не числа

`lblResult->Caption = edtA->Text + edtB->Text;` при вводе 2 и 3 покажет **«23»**: `+` для строк — склейка. Сначала `StrToInt`, потом сложение.

Неверный ввод

Если в Edit введено «abc», `StrToInt` выбросит исключение «'abc' is not a valid integer value». Надёжно: `StrToIntDef(edtA->Text, 0)` (вернёт 0) или блок `try { … } catch (...) { ShowMessage("Enter a number"); }`.

10.5 Пример: приложение «Калькулятор ИМТ»

Форма: два Edit (рост в см, вес в кг), кнопка, Label для результата.

C++ BUILDER
```
void __fastcall TForm1::btnCalcClick(TObject *Sender)
{
    try {
        double h = StrToFloat(edtHeight->Text) / 100;   // см → м
        double w = StrToFloat(edtWeight->Text);
        double bmi = w / (h * h);
        AnsiString verdict;
        if (bmi < 18.5) verdict = "underweight";
        else if (bmi < 25) verdict = "normal";
        else verdict = "overweight";
        lblResult->Caption = "BMI = " + FormatFloat("0.0", bmi) + " (" + verdict + ")";
        lblResult->Font->Color = (verdict == "normal") ? clGreen : clRed;
    }
    catch (...) {
        ShowMessage("Please enter numbers");
    }
}
```

10.6 Приложение с двумя формами

Задание из силлабуса: на главной форме кнопка, по нажатию открывается вторая форма.

1. **File → New → Form** — появится `Form2` (`Unit2.cpp`, `Unit2.h`).
2. В `Unit1.cpp` подключить заголовок второй формы: `#include "Unit2.h"` (или **File → Include Unit Hdr**, Alt+F11).
3. В обработчике кнопки показать форму.

Стандартный способ — форма создаётся автоматически при старте (Project → Options → Forms → список *Auto-create forms*), мы её только показываем:

C++ BUILDER

```
#include "Unit2.h"
void __fastcall TForm1::btnOpenClick(TObject *Sender)
{
    Form2->Show();          // немодально: можно переключаться между окнами
}
void __fastcall TForm1::btnDialogClick(TObject *Sender)
{
    if (Form2->ShowModal() == mrOk)    // модально: пока Form2 открыта, Form1 недоступна
        ShowMessage("OK pressed");
}
```

Без стандартного способа: создаём форму сами во время работы (тогда её нужно убрать из Auto-create):

C++ BUILDER

```
void __fastcall TForm1::btnCreateClick(TObject *Sender)
{
    TForm2 *f = new TForm2(this);   // this — владелец (Owner)
    f->Caption = "Created at runtime";
    f->ShowModal();
    delete f;                       // модальную форму после закрытия удаляем сами
}
```

	`Show()`	`ShowModal()`
Вид окна	немодальное	модальное (диалог)
Другие формы	доступны	заблокированы, пока окно открыто
Код после вызова	выполняется сразу	выполняется **после закрытия** окна
Результат	нет	`ModalResult` : `mrOk`, `mrCancel`…

Кнопки «OK / Отмена» для модальной формы

У кнопок на Form2 задай свойство **ModalResult** = `mrOk` / `mrCancel`, тогда нажатие само закроет форму и вернёт это значение из `ShowModal()`.

10.7 Компоненты, созданные во время работы

Компонент можно создать **программно** (задание из лекции 12 силлабуса):

```
C++ BUILDER
void __fastcall TForm1::btnAddClick(TObject *Sender)
{
    TButton *b = new TButton(this);      // владелец — форма (она удалит кнопку сама)
    b->Parent = Panel1;                  // ОБЯЗАТЕЛЬНО: в каком контейнере отображать
    b->Left = 10;
    b->Top = 10 + 30 * Panel1->ControlCount;
    b->Caption = "Button " + IntToStr(Panel1->ControlCount);
    b->OnClick = DynClick;               // назначаем обработчик (метод формы)
}
void __fastcall TForm1::DynClick(TObject *Sender)
{
    TButton *b = (TButton*)Sender;       // кто нажат
    ShowMessage("You clicked " + b->Caption);
}
```

Без Parent не видно

Компонент без `Parent` создаётся в памяти, но **не появляется** на экране. **Owner** (в конструкторе) отвечает за удаление, **Parent** — за отображение.

Разбор задач

Задача 10.1. Конвертер валют лёгкая

Форма: Edit (сумма в тенге), ComboBox (USD, EUR, RUB), кнопка «Convert», Label. Курсы: 1 USD = 500, 1 EUR = 540, 1 RUB = 6. Напиши обработчик.

Решение

```
C++ BUILDER
void __fastcall TForm1::btnConvertClick(TObject *Sender)
{
    double kzt = StrToFloat(edtSum->Text);
    double rate;
    switch (cbCur->ItemIndex) {          // 0 — USD, 1 — EUR, 2 — RUB
        case 0: rate = 500; break;
        case 1: rate = 540; break;
        case 2: rate = 6; break;
        default: ShowMessage("Choose a currency"); return;
    }
    lblRes->Caption = FormatFloat("0.00", kzt / rate) + " " + cbCur->Text;
}
```

`ItemIndex` = −1, если ничего не выбрано, поэтому есть `default`.

Задача 10.2. Список дел средняя

Edit, кнопки «Add», «Delete», «Clear» и ListBox. Пустые строки не добавлять, удалять выделенную строку.

Решение

C++ BUILDER

```
void __fastcall TForm1::btnAddClick(TObject *Sender)
{
    if (edtTask->Text.Trim() == "") return;   // Trim убирает пробелы
    lbTasks->Items->Add(edtTask->Text);
    edtTask->Clear();
    edtTask->SetFocus();                      // курсор снова в поле ввода
}
void __fastcall TForm1::btnDeleteClick(TObject *Sender)
{
    if (lbTasks->ItemIndex >= 0)
        lbTasks->Items->Delete(lbTasks->ItemIndex);
}
void __fastcall TForm1::btnClearClick(TObject *Sender)
{
    lbTasks->Items->Clear();
}
```

Задача 10.3. Две формы: передача данных средняя

На Form1 — Edit с именем и кнопка. Form2 открывается модально и показывает в Label «Welcome, имя». По кнопке OK на Form2 возвращает mrOk, и Form1 пишет «Confirmed».

Решение

C++ BUILDER

```
// Unit1.cpp
#include "Unit2.h"
void __fastcall TForm1::btnNextClick(TObject *Sender)
{
    Form2->lblWelcome->Caption = "Welcome, " + edtName->Text;   // доступ к компоненту другой формы
    if (Form2->ShowModal() == mrOk)
        lblStatus->Caption = "Confirmed";
}
// На Form2 у кнопки btnOk в инспекторе: ModalResult = mrOk
```

Контрольные вопросы из силлабуса (устно)

1. **Проект в C++Builder?** — Набор файлов (.bpr, Project1.cpp, модули .cpp/.h/.dfm, .res), из которых собирается одно приложение.
2. **Шаги создания проекта?** — File → New → Application, Save All в отдельную папку, разместить компоненты, задать свойства, написать обработчики, F9.
3. **Что такое форма?** — Окно приложения, объект класса-наследника TForm, контейнер для компонентов.

4. **Назначение формы?** — Основа графического интерфейса: на ней размещают элементы для ввода, вывода и управления.
5. **Form Designer?** — Визуальный редактор, где форму проектируют мышкой.
6. **Как добавить компонент?** — Выбрать его в палитре и щёлкнуть по форме (или двойной щелчок по значку).
7. **Как изменить положение и размер?** — Мышкой, стрелками с Ctrl/Shift, свойствами Left, Top, Width, Height, Align.
8. **Связь формы и кода?** — Форма — класс TForm1 в Unit1.h; компоненты — его поля; события — методы в Unit1.cpp; внешний вид хранится в Unit1.dfm.
9. **Что такое GUI?** — Графический интерфейс: взаимодействие через окна, кнопки, мышь.
10. **Этапы простого визуального приложения?** — Проект → форма и компоненты → свойства → обработчики событий → компиляция и тестирование.

Тест по лекции 10

1. How do you create a new windowed (GUI) application in C++ Builder? новый

a) File → New → Console Wizard
b) File → New → Application
c) Project → Build
d) Component → New Component

2. Which file contains the C++ code of event handlers of Form1? новый

a) Unit1.dfm
b) Unit1.cpp
c) Project1.res
d) Project1.bpr

3. Which property of `TEdit` contains the text entered by the user? новый

a) Caption
b) Text
c) Lines
d) Value

4. Which property of `TLabel` holds the text displayed on it? новый

a) Caption
b) Text
c) Name
d) Items

5. Which function converts the text of an Edit into an integer? новый

a) IntToStr
b) StrToInt
c) atoi
d) FloatToStr

6. What does `Label1->Caption = Edit1->Text + Edit2->Text;` show if the user typed 2 and 3? новый

a) 5
b) 23
c) 2+3
d) an error

7. Which method opens a form so that the other forms of the application are blocked until it is closed? новый

a) Show()
b) ShowModal()
c) Open()
d) Visible = true

8. What must be set so that a button created at run time appears on the form? новый

a) its Owner
b) its Parent
c) its Name
d) its Tag

9. What do you add to Unit1.cpp to be able to use Form2 from Form1? новый

a) #include "Unit2.cpp"
b) #include "Unit2.h"
c) #include <Form2>
d) nothing

10. Which property value makes a component fill all free space of its container? новый

a) alTop
b) alNone
c) alClient
d) alLeft

11. What does the abbreviation GUI mean? новый

a) General User Input
b) Graphical User Interface
c) Global Unit Index
d) Graphical Unit Inspector

12. Which part of the form class is managed by the IDE (components and event handlers)? новый

a) private
b) public
c) __published
d) protected

Ключи

1 — b	**2** — b	**3** — b	**4** — a	**5** — b	**6** — b
7 — b	**8** — b	**9** — b	**10** — c	**11** — b	**12** — c

6. Text — строки; `+` для строк склеивает их.

ЛЕКЦИЯ 11 ИЗ 15

Работа с формой: свойства, методы и события

Понимать разницу между свойством, методом и событием и уметь управлять формой как во время проектирования, так и во время работы программы.

11.1 Свойство, метод, событие

Понятие	Что это	Пример	Как использовать
Свойство (property)	характеристика объекта, **значение**	`Caption`, `Width`, `Color`	читать и присваивать: `Form1->Width = 400;`
Метод (method)	**действие**, которое объект умеет выполнять (функция класса)	`Show()`, `Close()`, `Clear()`	вызывать со скобками: `Edit1->Clear();`
Событие (event)	**ситуация**, на которую объект может реагировать	`OnClick`, `OnCreate`, `OnKeyPress`	написать обработчик — метод, который вызовется автоматически

Как не путать

Свойство — **существительное** («ширина», «цвет»), метод — **глагол** («показать», «закрыть»), событие — **«когда...»** («когда нажали», «когда создали»).

Свойство — не просто поле

В VCL свойство (`__property`) выглядит как переменная, но при присваивании может **вызывать метод**. Например, `Form1->Color = clRed;` не только запоминает цвет, но и сразу перерисовывает форму. Как это устроено, увидим в лекции 14.

11.2 Свойства во время проектирования и во время работы

Во время проектирования (design-time) — в **Object Inspector**: выделить компонент → изменить значение справа. Результат сохраняется в `.dfm` и действует сразу при запуске.

Во время работы (run-time) — **программно**, присваиванием в коде:

C++ BUILDER

```
void __fastcall TForm1::btnBigClick(TObject *Sender)
{
    Width = Width + 50;                 // внутри методов формы можно без Form1->
    Caption = "Width: " + IntToStr(Width);
    Color = clSkyBlue;
    btnBig->Enabled = (Width < 800);    // выключить кнопку, когда окно большое
}
```

Внутри методов формы «Form1->» не нужен

Обработчик — это метод класса `TForm1`, поэтому `Width` означает `this->Width`. `Form1->` нужен только **из другой формы** или из обычной функции.

11.3 Главные свойства формы

Свойство	Смысл	Примеры значений
`Name`	имя объекта в коде	`Form1`, `frmMain`
`Caption`	текст в заголовке окна	`"Calculator"`
`Width`, `Height`	ширина и высота окна (пиксели)	`400`, `300`
`ClientWidth`, `ClientHeight`	размеры **внутренней** области (без рамки и заголовка)	
`Left`, `Top`	положение окна на экране	
`Color`	цвет фона	`clBtnFace`, `clWhite`, `clRed`, `clSkyBlue`
`Position`	где появится окно	`poDesigned`, `poScreenCenter`, `poDesktopCenter`
`Visible`	видна ли форма	`true` / `false`
`Enabled`	реагирует ли на мышь и клавиатуру	`true` / `false`
`BorderStyle`	вид рамки	`bsSizeable` (можно тянуть), `bsSingle` (нельзя), `bsDialog`, `bsNone`
`BorderIcons`	кнопки в заголовке	`biSystemMenu`, `biMinimize`, `biMaximize`
`WindowState`	состояние	`wsNormal`, `wsMinimized`, `wsMaximized`
`FormStyle`	стиль	`fsNormal`, `fsStayOnTop` (поверх всех окон)
`Font`	шрифт формы (наследуют компоненты)	`Font->Size = 12`
`AlphaBlend`, `AlphaBlendValue`	прозрачность окна	`true`, `0…255`
`KeyPreview`	форма получает нажатия клавиш раньше компонентов	`true` / `false`
`ActiveControl`	какой компонент в фокусе при открытии	`Edit1`

Visible и Enabled

`Visible = false` — компонент **не виден**. `Enabled = false` — **виден, но серый** и не реагирует. На экзамене их часто путают.

Свойства только для чтения (read-only)

Некоторые свойства можно только **читать**: их значение вычисляет сама VCL. Обычно их нет в Object Inspector, они доступны только в коде. Они показывают текущее состояние объекта.

Свойство	Что показывает
`ComponentCount`	сколько компонентов принадлежит форме (владелец)
`ControlCount`	сколько элементов лежит внутри контейнера (родитель)
`Components[i]`, `Controls[i]`	доступ к i-му компоненту / элементу
`Handle`	системный дескриптор окна Windows
`ClassName()` (метод)	имя класса объекта: `"TButton"`

C++ BUILDER

```
void __fastcall TForm1::btnInfoClick(TObject *Sender)
{
    Memo1->Lines->Clear();
    Memo1->Lines->Add("Components: " + IntToStr(ComponentCount));
    for (int i = 0; i < ComponentCount; i++)
        Memo1->Lines->Add(Components[i]->Name + " : " + Components[i]->ClassName());
    // ComponentCount = 5;   ← ошибка компиляции: свойство только для чтения
}
```

11.4 Методы формы и компонентов

Метод	Что делает
`Show()`	показать форму (немодально)
`ShowModal()`	показать модально, вернуть ModalResult
`Hide()`	скрыть (то же, что `Visible = false`)
`Close()`	закрыть форму (для главной — завершить программу)
`Repaint()`, `Refresh()`	перерисовать
`SetFocus()`	передать фокус ввода компоненту
`BringToFront()`, `SendToBack()`	на передний / задний план
`Edit1->Clear()`	очистить поле
`Memo1->Lines->Add(s)`	добавить строку
`Application->Terminate()`	завершить приложение

11.5 События и обработчики

Событие — сигнал о том, что что-то произошло: щелчок, нажатие клавиши, создание формы, тик таймера. Программа в Windows **управляется событиями (event-driven)**: большую часть времени она ждёт, а когда событие наступает, VCL вызывает назначенный **обработчик**.

Обработчик события (event handler) — метод формы, который выполняется при наступлении события. Создание: Object Inspector → вкладка **Events** → двойной щелчок в поле нужного события.

Двойной щелчок по самому компоненту создаёт обработчик его **главного** события (для кнопки — `OnClick`).

C++ BUILDER

```
void __fastcall TForm1::Button1Click(TObject *Sender)   // ИмяКомпонентаСобытие
{
    // код реакции на щелчок
}
```

Параметр Sender и общий обработчик

`Sender` — указатель на объект, который **вызвал** событие. Благодаря ему один обработчик можно назначить нескольким компонентам (в инспекторе выбрать у других кнопок то же событие из выпадающего списка).

C++ BUILDER

```
// Один обработчик для кнопок "0".."9" калькулятора
void __fastcall TForm1::DigitClick(TObject *Sender)
{
    TButton *b = dynamic_cast<TButton*>(Sender);   // безопасное приведение типа
    if (b) edtDisplay->Text = edtDisplay->Text + b->Caption;
}
```

Tag

У каждого компонента есть целое свойство `Tag`, которое VCL никак не использует. Удобно хранить в нём номер: у кнопок «+», «−», «×» поставить Tag = 1, 2, 3 и в общем обработчике делать `switch (((TComponent*)Sender)->Tag)`.

Основные события формы и компонентов

Событие	Когда возникает
`OnCreate`	форма создаётся (один раз при запуске) — начальные настройки
`OnShow`	форма показывается
`OnActivate`	форма становится активной
`OnPaint`	форму нужно перерисовать (рисование на `Canvas`)
`OnResize`	изменился размер формы
`OnClick`, `OnDblClick`	щелчок / двойной щелчок
`OnMouseDown`, `OnMouseUp`, `OnMouseMove`	кнопка мыши нажата / отпущена / мышь двигается (есть X, Y)
`OnKeyPress`	нажата символьная клавиша (параметр `char &Key`)
`OnKeyDown`, `OnKeyUp`	нажата / отпущена любая клавиша (`WORD &Key`, коды `VK_…`)
`OnChange`	изменился текст (Edit) или выбор (ComboBox)
`OnEnter`, `OnExit`	компонент получил / потерял фокус
`OnCloseQuery`	попытка закрыть форму — можно запретить (`CanClose = false`)
`OnClose`	форма закрывается
`OnDestroy`	форма уничтожается
`OnTimer` (у TTimer)	прошёл `Interval` миллисекунд

Порядок при запуске: `OnCreate` → `OnShow` → `OnActivate` → `OnPaint`. При закрытии: `OnCloseQuery` → `OnClose` → `OnDestroy`.

C++ BUILDER

```
void __fastcall TForm1::FormCreate(TObject *Sender)
{
    Caption = "My app";
    Position = poScreenCenter;
    edtA->Text = "0";
}
void __fastcall TForm1::FormCloseQuery(TObject *Sender, bool &CanClose)
{
    CanClose = (Application->MessageBox("Exit?", "Confirm", MB_YESNO) == IDYES);
}
void __fastcall TForm1::edtAKeyPress(TObject *Sender, char &Key)
{
    if ((Key < '0' || Key > '9') && Key != 8)   // 8 — Backspace
        Key = 0;                                  // запретить ввод не-цифр
}
void __fastcall TForm1::FormMouseMove(TObject *Sender, TShiftState Shift, int X, int Y)
{
    Caption = "X=" + IntToStr(X) + " Y=" + IntToStr(Y);
}
void __fastcall TForm1::FormResize(TObject *Sender)
{
    btnOk->Left = ClientWidth - btnOk->Width - 10;   // кнопка всегда у правого края
}
```

Удаление обработчика

Нельзя просто стереть метод в `Unit1.cpp`: в `.dfm` и `.h` останется ссылка, и проект не соберётся. Правильно: **очистить тело** обработчика (оставить пустые `{ }`) и сохранить — Builder сам уберёт пустой метод.

11.6 Интерактивное приложение: таймер-секундомер

Компоненты: Label `lblTime`, кнопки `btnStart`, `btnStop`, `btnReset`, Timer `tmr` (Interval = 100, Enabled = false).

```cpp
// в private части класса TForm1 (Unit1.h): int tenths;
void __fastcall TForm1::FormCreate(TObject *Sender)
{
    tenths = 0;
    lblTime->Caption = "0.0";
}
void __fastcall TForm1::btnStartClick(TObject *Sender) { tmr->Enabled = true; }
void __fastcall TForm1::btnStopClick(TObject *Sender)  { tmr->Enabled = false; }
void __fastcall TForm1::btnResetClick(TObject *Sender)
{
    tmr->Enabled = false;
    tenths = 0;
    lblTime->Caption = "0.0";
}
void __fastcall TForm1::tmrTimer(TObject *Sender)          // каждые 100 мс
{
    tenths++;
    lblTime->Caption = IntToStr(tenths / 10) + "." + IntToStr(tenths % 10);
}
```
C++ BUILDER

Разбор задач

Задача 11.1. Убегающая кнопка лёгкая

Кнопка «Catch me» при наведении мыши (`OnMouseMove`) прыгает в случайное место формы, не выходя за её границы.

Решение

```cpp
void __fastcall TForm1::btnCatchMouseMove(TObject *Sender, TShiftState Shift, int X, int Y)
{
    btnCatch->Left = random(ClientWidth - btnCatch->Width);
    btnCatch->Top = random(ClientHeight - btnCatch->Height);
}
void __fastcall TForm1::FormCreate(TObject *Sender)
{
    randomize();
}
```
C++ BUILDER

`ClientWidth – Width` — чтобы кнопка не вылезла за правый край.

Задача 11.2. Настройка формы из программы средняя

По кнопкам: «Center» — окно в центр экрана, «Max/Normal» — переключать развёрнутое и обычное состояние, «On top» — CheckBox «поверх всех окон», TrackBar — прозрачность.

Решение

C++ BUILDER

```
void __fastcall TForm1::btnCenterClick(TObject *Sender)
{
    Left = (Screen->Width - Width) / 2;
    Top = (Screen->Height - Height) / 2;
}
void __fastcall TForm1::btnMaxClick(TObject *Sender)
{
    WindowState = (WindowState == wsMaximized) ? wsNormal : wsMaximized;
}
void __fastcall TForm1::chkTopClick(TObject *Sender)
{
    FormStyle = chkTop->Checked ? fsStayOnTop : fsNormal;
}
void __fastcall TForm1::tbAlphaChange(TObject *Sender)
{
    AlphaBlend = true;
    AlphaBlendValue = tbAlpha->Position;     // TrackBar: Min = 50, Max = 255
}
```

Задача 11.3. Калькулятор с одним обработчиком средняя

Два Edit, четыре кнопки `+ − × ÷` с Tag 1…4, один обработчик `OpClick`, результат в Label. Защита от деления на 0 и от неверного ввода.

Решение

C++ BUILDER

```
void __fastcall TForm1::OpClick(TObject *Sender)
{
    double a, b, r;
    try {
        a = StrToFloat(edtA->Text);
        b = StrToFloat(edtB->Text);
    }
    catch (...) { ShowMessage("Enter numbers"); return; }
    switch (((TComponent*)Sender)->Tag) {
        case 1: r = a + b; break;
        case 2: r = a - b; break;
        case 3: r = a * b; break;
        case 4:
            if (b == 0) { ShowMessage("Division by zero"); return; }
            r = a / b; break;
    }
    lblRes->Caption = FloatToStr(r);
}
```

Контрольные вопросы из силлабуса (устно)

1. **Свойство объекта?** — Именованная характеристика (данные) объекта: Caption, Width, Color.
2. **Метод?** — Действие, которое объект умеет выполнять (функция класса): Show, Close, Clear.
3. **Разница свойства и метода?** — Свойство хранит/возвращает значение и присваивается; метод вызывается и выполняет действие.
4. **Изменение через Object Inspector?** — Выделить компонент, на вкладке Properties изменить значение (design-time).
5. **Программное изменение?** — Присвоить в коде: `Form1->Caption = "Text";` (run-time).
6. **Главные свойства формы?** — Caption, Name, Width, Height, Color, Position, Visible, Enabled, BorderStyle, WindowState, Font.
7. **Событие?** — Ситуация (действие пользователя или системы), на которую объект может отреагировать.
8. **Обработчик события?** — Метод, автоматически вызываемый при наступлении события.
9. **Как связать событие с методом?** — Вкладка Events → двойной щелчок (создать) или выбрать существующий метод из списка.
10. **Частые события формы?** — OnCreate, OnShow, OnClick, OnResize, OnPaint, OnKeyPress, OnClose, OnCloseQuery.
11. **Обработка действий пользователя?** — Через обработчики событий мыши и клавиатуры, используя Sender и параметры (X, Y, Key).

Тест по лекции 11

1. `Width`, `Caption` and `Color` of a form are its: новый

a) methods
b) events
c) properties
d) handlers

2. `Show()`, `Close()` and `Hide()` of a form are its: новый

a) properties
b) methods
c) events
d) fields

3. `OnClick`, `OnCreate` and `OnKeyPress` are: новый

a) properties
b) methods
c) events
d) constructors

4. Which form event is the best place for initial settings when the program starts? новый

a) OnClick
b) OnPaint
c) OnCreate
d) OnClose

5. Which property makes a component visible but disabled (grayed out)? новый

a) Visible = false
b) Enabled = false
c) ReadOnly = false
d) Hide()

6. Which property value places the form in the center of the screen when it opens? новый

a) poDesigned
b) poScreenCenter
c) wsMaximized
d) alClient

7. What does the `Sender` parameter of an event handler contain? новый

a) the name of the form
b) a pointer to the object that generated the event
c) the key that was pressed
d) the mouse coordinates

8. In the handler `FormCloseQuery(TObject *Sender, bool &CanClose)`, how do you prevent the form from closing? новый

a) Close();
b) CanClose = false;
c) Sender = NULL;
d) return 0;

9. Which property of a TTimer sets the period between `OnTimer` events? новый

a) Period
b) Delay
c) Interval
d) Tick

10. In the `OnKeyPress(TObject *Sender, char &Key)` handler, what does `Key = 0;` do? новый

a) closes the program
b) cancels the entered character
c) presses the key again
d) clears the Edit

11. Which is a read-only property of a form? новый

a) Caption
b) Width
c) ComponentCount
d) Color

12. Which value of `BorderStyle` does NOT allow the user to resize the window? новый

a) bsSizeable
b) bsSingle
c) bsSizeToolWin
d) wsNormal

Ключи

1 – c	**2** – b	**3** – c	**4** – c	**5** – b	**6** – b
7 – b	**8** – b	**9** – c	**10** – b	**11** – c	**12** – b

ЛЕКЦИЯ 12 ИЗ 15

Иерархия классов: наследование, переопределение, виртуальные функции

Строить иерархии классов, понимать, что наследуется и с каким доступом, и где нужен virtual. Знать иерархию VCL.

12.1 Наследование и иерархия

Наследование — механизм, при котором новый класс (**производный, дочерний, потомок**) создаётся на основе существующего (**базового, родительского, предка**) и получает его поля и методы.
Иерархия классов — «дерево» классов, связанных наследованием.

```
            Person                  ← базовый (общие поля: name, age)
           /      \
      Student     Teacher           ← производные (свои поля: gpa / salary)
         |
    Graduate                        ← наследник наследника
```

Синтаксис:

```
class Производный : спецификатор_наследования Базовый {
    // новые поля и методы
};
```

```C++
class Person {
protected:
    char name[20];
    int age;
public:
    void setPerson(const char *n, int a) { strcpy(name, n); age = a; }
    void print() { cout << name << ", " << age; }
};
class Student : public Person {      // Student является Person
    double gpa;
public:
    void setGpa(double g) { gpa = g; }
    void printAll() { print(); cout << ", GPA " << gpa << endl; }   // использует метод базы
    bool canGetGrant() { return gpa >= 3.0 && age < 25; }           // protected-поле age
доступно
};
//---
Student s;
s.setPerson("Madina", 20);   // унаследованный метод
s.setGpa(3.4);
s.printAll();
cout << s.canGetGrant();
```

Вывод
```
Madina, 20, GPA 3.4
1
```

Что даёт наследование: повторное использование кода (Person пишем один раз), расширение (в Student добавили gpa), единый интерфейс для полиморфизма, иерархия отражает реальный мир.

> **Что НЕ наследуется**
>
> Конструкторы, деструктор и оператор присваивания не наследуются: у производного класса свои (но они вызывают базовые). Private-члены базы **есть** внутри объекта-наследника, но напрямую из его методов **недоступны**.

12.2 Одиночное и множественное наследование

- **Одиночное**: один базовый класс (`class B : public A`). В VCL используется только оно.
- **Множественное**: несколько базовых (`class C : public A, public B`). Объект получает члены обоих.

C++
```
class Printer { public: void printDoc() { cout << "print "; } };
class Scanner { public: void scanDoc()  { cout << "scan "; } };
class MFU : public Printer, public Scanner {      // множественное
public:
    void copyDoc() { scanDoc(); printDoc(); }
};
//---
MFU m;
m.copyDoc();
```

Вывод
```
scan print
```

> **Ромб (проблема множественного наследования)**
>
> Если A наследуют B и C, а D наследует B и C, в D окажется **две копии** A, и обращение к членам A станет неоднозначным. Решение — **виртуальное наследование** (`class B : virtual public A`). Поэтому множественное наследование используют осторожно.

12.3 Доступ при наследовании

Спецификатор наследования (`public`, `protected`, `private`) определяет, каким станет доступ к унаследованным членам **в производном классе**:

Член в базе ↓ / наследование →	`public`	`protected`	`private`
`public`	public	protected	private
`protected`	protected	protected	private
`private`	**недоступен**	**недоступен**	**недоступен**

Как запомнить

Наследование «понижает» доступ до своего уровня, но не повышает. Private базы не виден наследнику **никогда**. Почти всегда используют `public` (отношение «является»). Если спецификатор не указан, для `class` по умолчанию берётся **private**-наследование, для `struct` — public.

```
class A { public: int x; protected: int y; private: int z; };
class B : public A {
    void f() { x = 1; y = 2; /* z = 3; ← ошибка: private базы */ }
};
class C : private A { };
int main() {
    B b; b.x = 5;      // можно: x остался public
    // b.y = 5;        // ошибка: y protected
    C c; // c.x = 5;   // ошибка: при private-наследовании x стал private
}
```

12.4 Конструкторы и деструкторы при наследовании

Порядок: при создании объекта **сначала** работает конструктор **базового** класса, **потом** производного. Деструкторы — в **обратном** порядке. Параметры базовому конструктору передают через **список инициализации**: `Derived(…) : Base(…) { }`.

C++

```
class Base {
public:
    Base(int v) { cout << "Base(" << v << ") "; }
    ~Base() { cout << "~Base "; }
};
class Derived : public Base {
public:
    Derived(int v) : Base(v * 10) { cout << "Derived(" << v << ") "; }
    ~Derived() { cout << "~Derived "; }
};
//---
{
    Derived d(5);
    cout << "| ";
}                   // здесь d уничтожается
```

Вывод

```
Base(50) Derived(5) | ~Derived ~Base
```

Аналогия

Дом строят снизу вверх (сначала фундамент — база), а разбирают сверху вниз (сначала крыша — наследник).

12.5 Переопределение методов

Производный класс может написать **свой** вариант метода с тем же именем и параметрами — это **переопределение (overriding)**. Вызвать версию базы можно через `Base::method()`.

```C++
class Shape {
public:
    void info() { cout << "I am a shape" << endl; }
};
class Square : public Shape {
public:
    void info()
    {
        Shape::info();                      // вызов версии базового класса
        cout << "...and a square" << endl;
    }
};
//---
Square q;
q.info();
```

Вывод

```
I am a shape
...and a square
```

Перегрузка ≠ переопределение

Перегрузка (overloading) — одно имя, **разные параметры**, обычно в одном классе.
Переопределение (overriding) — то же имя и **те же параметры** в классе-наследнике.

12.6 Виртуальные функции и полиморфизм

Указатель (или ссылка) на базовый класс может указывать на объект **производного** класса. Какой метод вызовется?

- метод **обычный** → по типу **указателя** (раннее, статическое связывание, на этапе компиляции);
- метод **virtual** → по типу **реального объекта** (позднее, динамическое связывание, во время работы).

```C++
class Animal {
public:
    void name() { cout << "animal "; }              // обычный
    virtual void sound() { cout << "... "; }        // виртуальный
};
class Cow : public Animal {
public:
    void name() { cout << "cow "; }
    void sound() { cout << "Moo "; }                // virtual наследуется автоматически
};
//---
Cow c;
Animal *p = &c;
p->name();     // по типу указателя → animal
p->sound();    // по реальному объекту → Moo
Animal &r = c;
r.sound();
```

Вывод
```
animal Moo Moo
```

Мощь полиморфизма: один код работает с объектами разных классов.

C++
```cpp
class Shape {
public:
    virtual double area() = 0;                  // чисто виртуальная функция
    virtual const char *name() { return "shape"; }
    virtual ~Shape() { }                        // виртуальный деструктор
};
class Circle : public Shape {
    double r;
public:
    Circle(double v) : r(v) { }
    double area() { return 3.14159 * r * r; }
    const char *name() { return "circle"; }
};
class Rect : public Shape {
    double a, b;
public:
    Rect(double x, double y) : a(x), b(y) { }
    double area() { return a * b; }
    const char *name() { return "rect"; }
};
//---
Shape *figs[3] = { new Circle(1), new Rect(2, 3), new Circle(2) };
double total = 0;
for (int i = 0; i < 3; i++) {
    cout << figs[i]->name() << " " << figs[i]->area() << endl;
    total += figs[i]->area();
}
cout << "total " << total << endl;
for (int i = 0; i < 3; i++) delete figs[i];   // благодаря virtual ~Shape вызовутся нужные деструкторы
```

Вывод
```
circle 3.14159
rect 6
circle 12.5664
total 21.708
```

Абстрактный класс

- **Чисто виртуальная функция**: `virtual тип f() = 0;` — у неё нет реализации в базе, наследники **обязаны** её определить.
- Класс, в котором есть хотя бы одна чисто виртуальная функция, — **абстрактный**: создать его объект **нельзя** (`Shape s;` — ошибка), можно только указатели и ссылки на него.
- Абстрактный класс задаёт **общий интерфейс** для всей иерархии.

Виртуальный деструктор

Если удалять объект наследника через указатель на базу (`delete figs[i]`), а деструктор базы **не** виртуальный, вызовется только деструктор базы: ресурсы наследника не освободятся. Правило: **есть virtual-методы — делай деструктор virtual**.

12.7 Иерархия классов VCL

Все компоненты C++ Builder выстроены в одну иерархию. Корень — **TObject**.

```
TObject                          базовый класс всех классов VCL: создание/удаление, ClassName(),
ClassParent(), InheritsFrom()
 └─ TPersistent                  умеет сохранять/загружать свойства (в .dfm), Assign()
     └─ TComponent               «компонент»: Name, Owner, Components[], может лежать в палитре
         ├─ (невизуальные)       TTimer, TOpenDialog, TMainMenu …
         └─ TControl             визуальный элемент: Left, Top, Width, Height, Visible, Parent,
OnClick
             ├─ TGraphicControl рисуется на родителе, без окна Windows, не получает фокус:
TLabel, TImage, TShape, TSpeedButton
             └─ TWinControl      имеет окно (Handle), получает фокус, может содержать другие
элементы
                 ├─ TButtonControl → TButton, TCheckBox, TRadioButton
                 ├─ TCustomEdit    → TEdit, TMemo
                 ├─ TCustomPanel   → TPanel
                 ├─ TCustomGroupBox→ TGroupBox
                 └─ TScrollingWinControl → TCustomForm → TForm → TForm1 (ваша форма)
```

Класс	Что добавляет
`TObject`	основа: память, информация о типе во время выполнения (RTTI): `ClassName()`, `ClassParent()`, `InheritsFrom()`
`TPersistent`	сохранение свойств в поток, копирование `Assign`
`TComponent`	владение (`Owner` , `Components`), имя `Name` , работа в дизайнере
`TControl`	видимость и положение на экране, события мыши
`TWinControl`	окно Windows, фокус ввода, клавиатура, может быть контейнером
`TGraphicControl`	лёгкий визуальный элемент без собственного окна

Программа, которая выводит «родословную» компонента (задание из силлабуса):

C++ BUILDER

```
void __fastcall TForm1::btnLineageClick(TObject *Sender)
{
    Memo1->Lines->Clear();
    TControl *items[3] = { Button1, Edit1, Label1 };
    for (int i = 0; i < 3; i++) {
        AnsiString line = items[i]->Name + ": ";
        for (TClass c = items[i]->ClassType(); c != NULL; c = c->ClassParent())
            line += AnsiString(c->ClassName()) + " <- ";
        Memo1->Lines->Add(line);
    }
}
```

```
Button1: TButton <- TButtonControl <- TWinControl <- TControl <- TComponent <- TPersistent <-
TObject <-
Edit1:   TEdit <- TCustomEdit <- TWinControl <- TControl <- TComponent <- TPersistent <-
TObject <-
Label1:  TLabel <- TCustomLabel <- TGraphicControl <- TControl <- TComponent <- TPersistent <-
TObject <-
```

12.8 Контейнеры, Owner и Parent

Контейнер — компонент, внутри которого лежат другие (`TPanel`, `TGroupBox`, `TPageControl` с вкладками `TTabSheet`, сама форма). Контейнер управляет **видимым расположением** детей: координаты детей считаются от его левого верхнего угла; если скрыть или сдвинуть контейнер, всё содержимое скрывается и сдвигается вместе с ним.

	Owner (владелец)	**Parent (родитель)**
Задаётся	в конструкторе `new TButton(Owner)`	свойством `->Parent = …`
Отвечает за	**удаление** из памяти (владелец удаляет свои компоненты)	**отображение**: внутри какого контейнера рисуется
Списки	`Components[]`, `ComponentCount`	`Controls[]`, `ControlCount`
Обычно	форма	форма, панель, groupbox

C++ BUILDER

```
void __fastcall TForm1::btnAddEditClick(TObject *Sender)
{
    TEdit *e = new TEdit(this);      // владелец — форма
    e->Parent = GroupBox1;           // показывать внутри GroupBox1
    e->Top = 20 + 25 * GroupBox1->ControlCount;
    e->Left = 10;
    e->Text = "Edit #" + IntToStr(GroupBox1->ControlCount);
}
void __fastcall TForm1::btnRemoveLastClick(TObject *Sender)
{
    int n = GroupBox1->ControlCount;
    if (n > 0) delete GroupBox1->Controls[n - 1];    // удалить последний элемент контейнера
}
void __fastcall TForm1::btnHideGroupClick(TObject *Sender)
{
    GroupBox1->Visible = !GroupBox1->Visible;        // скрывает и всё содержимое
}
```

Проверка типа во время выполнения:

```
void __fastcall TForm1::btnClearEditsClick(TObject *Sender)
{
    for (int i = 0; i < ComponentCount; i++) {
        TEdit *e = dynamic_cast<TEdit*>(Components[i]);   // NULL, если это не TEdit
        if (e) e->Clear();
        if (Components[i]->InheritsFrom(__classid(TButtonControl)))
            ((TButtonControl*)Components[i])->Enabled = true;
    }
}
```
C++ BUILDER

Разбор задач

Задача 12.1. Транспорт лёгкая

Базовый класс Vehicle (скорость, метод `move()` печатает скорость), производные Car и Bicycle со своими полями. Создай объекты и вызови методы.

Решение

```cpp
class Vehicle {
protected:
    int speed;
public:
    Vehicle(int s) : speed(s) { }
    void move() { cout << "moving at " << speed << " km/h" << endl; }
};
class Car : public Vehicle {
    int doors;
public:
    Car(int s, int d) : Vehicle(s), doors(d) { }
    void info() { cout << "Car with " << doors << " doors, "; move(); }
};
class Bicycle : public Vehicle {
public:
    Bicycle() : Vehicle(20) { }
    void ring() { cout << "Ring! "; move(); }
};
//---
Car c(120, 4);
Bicycle b;
c.info();
b.ring();
```
C++

Вывод
```
Car with 4 doors, moving at 120 km/h
Ring! moving at 20 km/h
```

Задача 12.2. Зарплата: полиморфизм средняя

Абстрактный класс Employee с чисто виртуальным `salary()`. Наследники: Fixed (оклад), Hourly (часы × ставка), Manager (оклад + 10 % за каждого подчинённого). Посчитай фонд оплаты для массива указателей.

Решение

```C++
class Employee {
public:
    virtual double salary() = 0;
    virtual ~Employee() { }
};
class Fixed : public Employee {
    double s;
public:
    Fixed(double v) : s(v) { }
    double salary() { return s; }
};
class Hourly : public Employee {
    int h; double rate;
public:
    Hourly(int hh, double r) : h(hh), rate(r) { }
    double salary() { return h * rate; }
};
class Manager : public Fixed {
    int team;
public:
    Manager(double v, int t) : Fixed(v), team(t) { }
    double salary() { return Fixed::salary() * (1 + 0.1 * team); }
};
//---
Employee *staff[3] = { new Fixed(300000), new Hourly(120, 2500), new Manager(400000, 3) };
double sum = 0;
for (int i = 0; i < 3; i++) { cout << staff[i]->salary() << endl; sum += staff[i]-
>salary(); }
cout << "total " << sum;
for (int i = 0; i < 3; i++) delete staff[i];
```

Вывод

```
300000
300000
520000
total 1.12e+06
```

Фишка: `Manager::salary()` использует версию базы `Fixed::salary()` и добавляет надбавку. Большие double cout печатает в экспоненциальной форме; для красивого вывода используй `printf("%.0f", sum)`.

Задача 12.3. Порядок вызовов сложная

Что выведет программа?

```C++
class A {
public:
    A() { cout << "A"; }
    virtual ~A() { cout << "a"; }
    virtual void f() { cout << "1"; }
};
class B : public A {
public:
    B() { cout << "B"; }
    ~B() { cout << "b"; }
    void f() { cout << "2"; }
};
class C : public B {
public:
    C() { cout << "C"; }
    ~C() { cout << "c"; }
};
//---
A *p = new C;
p->f();
delete p;
```

Решение

> **Вывод**
> `ABC2cba`

Конструкторы от базы к наследнику: A, B, C. `f` виртуальная; в C она не переопределена, поэтому берётся ближайшая — из B: «2». Деструктор виртуальный, поэтому удаляется весь объект: c, b, a.

Контрольные вопросы из силлабуса (устно)

1. **Иерархия классов?** — Совокупность классов, связанных наследованием, в виде дерева от общего к частному.
2. **Наследование?** — Создание класса на основе существующего с получением его членов.
3. **Базовый класс?** — Класс, от которого наследуют (родитель).
4. **Производный класс?** — Класс-наследник, который получает члены базы и добавляет свои.
5. **Преимущества наследования?** — Повторное использование кода, расширяемость, единый интерфейс, полиморфизм.
6. **Одиночное наследование?** — Один базовый класс.
7. **Множественное?** — Несколько базовых классов (`class C : public A, public B`).
8. **Как наследование влияет на доступ?** — public-члены базы доступны всем (при public-наследовании), protected — только наследникам, private — недоступны даже наследникам; спецификатор наследования может понизить доступ.
9. **Переопределение метода?** — Новая реализация метода базы с той же сигнатурой в наследнике.

10. **Виртуальный метод?** — Метод с `virtual`, вызов которого определяется реальным типом объекта во время выполнения.
11. **Связь с полиморфизмом?** — Указатель на базу + виртуальные методы → один вызов работает по-разному для разных наследников.
12. **Как представить иерархию в программе?** — Объявить базовый класс с общими членами и производные через `class D : public B`; в VCL: TObject → TPersistent → TComponent → TControl → TWinControl / TGraphicControl.

Тест по лекции 12

1. How do you declare class `Dog` publicly derived from class `Animal`? новый

a) class Dog extends Animal { };
b) class Dog :: Animal { };
c) class Dog : public Animal { };
d) class Dog inherits public Animal { };

2. Which members of a base class are NOT accessible in a derived class? новый

a) public
b) protected
c) private
d) static public

3. In which order are constructors called when an object of a derived class is created? новый

a) derived first, then base
b) base first, then derived
c) only the derived constructor is called
d) in random order

4. A class that contains at least one pure virtual function is called: новый

a) a friend class
b) a static class
c) an abstract class
d) a template class

5. How is a pure virtual function declared? новый

a) virtual void f() {}
b) void f() = 0;
c) virtual void f() = 0;
d) abstract void f();

6. Which class is the root of the VCL class hierarchy? новый

a) TComponent
b) TControl
c) TObject
d) TForm

7. Which VCL class introduces the `Owner` and `Name` of components? новый

a) TObject
b) TPersistent
c) TComponent
d) TGraphicControl

8. Which VCL class is the ancestor of controls that have their own window handle and can receive focus? новый

a) TGraphicControl
b) TWinControl
c) TPersistent
d) TCustomLabel

9. TLabel is derived (through TCustomLabel) from: новый

a) TWinControl
b) TGraphicControl
c) TButtonControl
d) TForm

10. With `private` inheritance, public members of the base class become in the derived class: новый

a) public
b) protected
c) private
d) inaccessible

11. What is the output? сложный

```
class P {
public:
    P() { cout << "P"; }
    ~P() { cout << "p"; }
};
class Q : public P {
public:
    Q() { cout << "Q"; }
    ~Q() { cout << "q"; }
};
//---
{
    Q a, b;
}
```

a) PQqpPQqp
b) PQPQqpqp
c) PQPQpqpq
d) QPQPqpqp

12. What is the output? сложный

```
class A {
public:
    virtual int f() { return 1; }
    int g() { return f() * 10; }
};
class B : public A {
public:
    int f() { return 2; }
};
//---
B b;
A a;
A *p = &b;
cout << a.g() << " " << p->g() << " " << b.g();
```

a) 10 10 10
b) 10 20 20
c) 10 10 20
d) 20 20 20

13. What is the output? сложный

```
class Base {
public:
    Base() { show(); }
    virtual void show() { cout << "Base "; }
};
class Der : public Base {
public:
    Der() { show(); }
    void show() { cout << "Der "; }
};
//---
Der d;
```

a) Der Der
b) Base Der
c) Base Base
d) Der Base

14. Which line causes a compilation error? сложный

```
class Shape { public: virtual double area() = 0; };
class Sq : public Shape { double a; public: Sq(double x) : a(x) {} double area() { return
a*a; } };
Shape *p = new Sq(2);    // line 1
Sq s(3);                 // line 2
Shape &r = s;            // line 3
Shape t;                 // line 4
```

a) line 1
b) line 2
c) line 3
d) line 4

Ключи

1 – c **2** – c **3** – b **4** – c **5** – c **6** – c
7 – c **8** – b **9** – b **10** – c **11** – b **12** – b
13 – b **14** – d

11. Объекты создаются по очереди (a, потом b), каждый: P, затем Q. Уничтожаются в обратном порядке создания (сначала b, потом a), каждый: q, затем p.

12. g() не виртуальный, но внутри него f() виртуальная, поэтому выбирается по реальному объекту: у a — 1, у b — 2.

13. Пока работает конструктор Base, объект ещё не стал Der, поэтому виртуальный вызов из конструктора базы попадает в Base::show(). Потом конструктор Der вызывает Der::show().

14. Объект абстрактного класса создать нельзя. Указатели и ссылки на абстрактный класс можно.

ЛЕКЦИЯ 13 ИЗ 15

Пользовательский класс. Конструктор и деструктор

Научиться правильно создавать и уничтожать объекты: инициализировать их конструкторами и освобождать ресурсы деструктором.

13.1 Проектирование пользовательского класса

Как превратить задачу в класс:

1. **Сущность** → имя класса (`Book`, `Student`, `Matrix`).
2. **Характеристики** → закрытые поля (`private`).
3. **Как создаётся** → конструкторы (какие данные обязательны?).
4. **Что умеет** → открытые методы (`public`).
5. **Какие правила** → проверки в конструкторах и сеттерах (год > 1000, баланс ≥ 0).
6. **Какие ресурсы** держит (динамическая память, файлы) → деструктор.

13.2 Конструктор

Конструктор — специальный метод, который **автоматически** вызывается при создании объекта и **инициализирует** его поля.

Особенности:

- имя совпадает с именем класса;
- **нет типа результата** (даже `void` не пишется);
- обычно `public` (иначе объект нельзя создать снаружи);
- может иметь параметры и быть **перегружен** (несколько конструкторов);
- вызывать его вручную как обычный метод нельзя: он срабатывает сам при создании объекта.

Конструктор по умолчанию

Вызывается **без аргументов**. Если в классе **нет ни одного** конструктора, компилятор создаёт пустой конструктор по умолчанию сам (поля тогда содержат мусор).

```C++
class Counter {
    int value;
public:
    Counter() { value = 0; cout << "Counter() "; }      // по умолчанию
    void inc() { value++; }
    int get() { return value; }
};
//---
Counter c;          // вызов Counter()
c.inc(); c.inc();
cout << c.get();
```

Вывод

```
Counter() 2
```

Ловушка «самая досадная»

`Counter c();` — это **не** создание объекта, а **объявление функции** с, которая возвращает Counter. Объект без аргументов создаётся **без скобок**: `Counter c;`.

Конструктор с параметрами

```C++
class Point {
    int x, y;
public:
    Point(int a, int b) { x = a; y = b; }
    void print() { cout << "(" << x << "," << y << ") "; }
};
//---
Point p(3, 4);           // вызов Point(int, int)
Point q = Point(-1, 2);  // тоже можно
p.print(); q.print();
// Point r;              // ОШИБКА: конструктора без параметров теперь нет
```

Вывод

```
(3,4) (-1,2)
```

Конструктор по умолчанию пропадает

Как только ты написал **любой** конструктор, компилятор больше **не создаёт** конструктор по умолчанию. Тогда `Point r;` и массив `Point arr[10];` перестанут компилироваться. Решение — написать его самому или задать параметрам значения по умолчанию: `Point(int a = 0, int b = 0)`.

Список инициализации

Поля можно инициализировать **до** входа в тело конструктора, через двоеточие. Это обязательный способ для **const-полей**, **полей-ссылок**, **полей-объектов без конструктора по умолчанию** и для вызова **конструктора базового класса**.

```C++
class Circle {
    const double PI;     // const-поле можно задать только в списке инициализации
    double r;
public:
    Circle(double radius) : PI(3.14159), r(radius > 0 ? radius : 1) { }
    double area() { return PI * r * r; }
};
//---
Circle c(2), bad(-5);
cout << c.area() << " " << bad.area();
```

Вывод

```
12.5664 3.14159
```

Перегрузка конструкторов

```C++
class Fraction {
    int num, den;
public:
    Fraction() : num(0), den(1) { }                    // 0/1
    Fraction(int n) : num(n), den(1) { }               // n/1
    Fraction(int n, int d) : num(n), den(d != 0 ? d : 1) { }
    void print() { cout << num << "/" << den << " "; }
};
//---
Fraction a, b(5), c(3, 4), d(1, 0);
a.print(); b.print(); c.print(); d.print();
```

> **Вывод**
> `0/1 5/1 3/4 1/1`

Конструктор копирования

Создаёт объект **как копию** другого объекта того же класса. Сигнатура: `ИмяКласса(const ИмяКласса &other)`. Если его не написать, компилятор создаст свой — он копирует поля «побайтно».

Когда вызывается конструктор копирования:

1. `Point b = a;` или `Point b(a);` — создание объекта из другого;
2. передача объекта в функцию **по значению**;
3. возврат объекта из функции по значению (компилятор иногда это оптимизирует).

```C++
class Item {
    int id;
public:
    Item(int i) : id(i) { cout << "make" << id << " "; }
    Item(const Item &o) : id(o.id + 100) { cout << "copy" << id << " "; }
    int getId() const { return id; }
};
void byValue(Item it) { cout << "[in " << it.getId() << "] "; }
void byRef(const Item &it) { cout << "[ref " << it.getId() << "] "; }
//---
Item a(1);
Item b = a;          // конструктор копирования
byValue(a);          // ещё одна копия
byRef(a);            // копии нет
```

> **Вывод**
> `make1 copy101 copy101 [in 101] [ref 1]`

13.3 Деструктор

Деструктор — специальный метод, который **автоматически** вызывается при **уничтожении** объекта. Нужен, чтобы **освободить ресурсы**: динамическую память (`delete`), закрыть файлы, соединения.

Особенности:

- имя = `~` + имя класса: `~Book()`;
- нет параметров и нет типа результата;

- **только один** в классе (перегружать нельзя);
- вызывается сам: для локального объекта — при выходе из блока `{ }`; для объекта из `new` — при `delete`; для глобального — при завершении программы.

```C++
class Logger {
    char name[10];
public:
    Logger(const char *n) { strcpy(name, n); cout << "+" << name << " "; }
    ~Logger() { cout << "-" << name << " "; }
};
//---
Logger a("A");
{
    Logger b("B");
    Logger *c = new Logger("C");
    delete c;           // явное удаление → деструктор C
}                       // конец блока → деструктор B
Logger d("D");
cout << "| ";
```

Вывод

```
+A +B +C -C -B +D | -D -A
```

После `|` заканчивается `main`, и срабатывают деструкторы `-D -A`: локальные объекты уничтожаются **в обратном порядке** создания.

Время жизни объекта

- **Автоматический** (локальный): от объявления до `}` своего блока.
- **Динамический**: от `new` до `delete` (если delete забыть — утечка памяти, деструктор не вызовется).
- **Статический/глобальный**: от начала до конца программы.
- **Временный**: `Point(1, 2).print();` — уничтожается в конце выражения.

Класс с динамической памятью: зачем нужны конструктор копирования и деструктор

```cpp
class IntArray {
    int *data;
    int size;
public:
    IntArray(int n) : size(n) { data = new int[n]; for (int i = 0; i < n; i++) data[i] = 0; }
    IntArray(const IntArray &o) : size(o.size)        // ГЛУБОКОЕ копирование
    {
        data = new int[size];
        for (int i = 0; i < size; i++) data[i] = o.data[i];
    }
    ~IntArray() { delete [] data; }                   // освобождаем память
    int &at(int i) { return data[i]; }
    int getSize() const { return size; }
};
//---
IntArray a(3);
a.at(0) = 7;
IntArray b = a;      // своя копия массива
b.at(0) = 99;        // а не изменится
cout << a.at(0) << " " << b.at(0) << " " << b.getSize();
```

Вывод

```
7 99 3
```

Поверхностное копирование

Без своего конструктора копирования `b = a` скопирует **указатель** `data`: оба объекта будут ссылаться на **один** массив. Изменение b испортит a, а в конце деструкторы **дважды** удалят одну память → крах программы. **Правило трёх**: если классу нужен деструктор, ему почти наверняка нужны и конструктор копирования, и оператор присваивания.

13.4 Где разместить члены: public, private, protected (задание СРСП)

Задание: класс с конструктором, целым полем и методом, меняющим поле. Что изменится, если переносить члены в разные секции?

```
class Test {
public:    Test() : value(0) { }      // public-конструктор: объект можно создать где угодно
private:   int value;                  // private-поле: снаружи t.value — ошибка
public:    void add(int k) { value += k; }
};
```

Член помещён в	Что происходит
конструктор в `private`	`Test t;` снаружи — **ошибка**: объект нельзя создать (так делают «одиночку» Singleton через static-метод)
конструктор в `protected`	снаружи создать нельзя, но можно создавать **наследников** (так делают базовые классы «только для наследования», похоже на абстрактный класс)
поле в `public`	снаружи можно написать `t.value = -100;` — инкапсуляция нарушена, проверки обходятся
поле в `private`	доступ только через методы (правильно)
поле в `protected`	снаружи нельзя, в методах наследников можно
метод в `private`	снаружи вызвать нельзя, только из других методов класса (служебный метод)
метод в `protected`	вызывают методы класса и наследников

13.5 Лабораторная работа: класс Book

Полное решение задания из силлабуса: приватные поля, два конструктора, константные геттеры, сеттеры с проверкой, методы displayBookInfo, checkOut, returnBook и демонстрация всех случаев.

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int publicationYear;
    string isbn;
    bool isAvailable;
public:
    Book() : title("Unknown"), author("Unknown"), publicationYear(0), isbn("Unknown"),
isAvailable(true) { }
    Book(string t, string a, int y, string i)
        : title(t), author(a), publicationYear(0), isbn(i), isAvailable(true)
    {
        setPublicationYear(y);              // переиспользуем проверку
    }
    // Геттеры — const: не меняют объект
    string getTitle() const { return title; }
    string getAuthor() const { return author; }
    int getPublicationYear() const { return publicationYear; }
    string getISBN() const { return isbn; }
    bool getAvailability() const { return isAvailable; }
    // Сеттеры с проверкой
    void setPublicationYear(int year)
    {
        const int CURRENT_YEAR = 2026;
        if (year >= 1000 && year <= CURRENT_YEAR) publicationYear = year;
        else cout << "Error: invalid year " << year << endl;
    }
    void setIsbn(const string &newIsbn) { isbn = newIsbn; }
    // Остальные методы
    void displayBookInfo() const
    {
        cout << "'" << title << "' by " << author << ", " << publicationYear
             << ", ISBN " << isbn << " - " << (isAvailable ? "Available" : "Checked Out") <<
endl;
    }
    void checkOut()
    {
        if (isAvailable) { isAvailable = false; cout << title << " has been successfully
checked out." << endl; }
        else cout << title << " is currently unavailable." << endl;
    }
    void returnBook()
    {
        if (!isAvailable) { isAvailable = true; cout << title << " has been successfully
returned." << endl; }
        else cout << title << " is already in the library." << endl;
    }
};

int main()
```

```
{
    Book b1;                                                    // по умолчанию
    Book b2("Abai Zholy", "M. Auezov", 1942, "978-601-1");      // с параметрами
    Book b3("Clean Code", "R. Martin", 3000, "978-013-2");      // неверный год
    b1.displayBookInfo();
    b2.displayBookInfo();
    b3.displayBookInfo();
    b2.checkOut();
    b2.checkOut();                  // повторно — недоступна
    b2.returnBook();
    b2.returnBook();                // уже в библиотеке
    b3.setPublicationYear(2008);
    b3.setIsbn("978-0132350884");
    b3.displayBookInfo();
    cout << "Getter: " << b2.getTitle() << " / " << b2.getAuthor() << endl;
    return 0;
}
```

Вывод

```
Error: invalid year 3000
'Unknown' by Unknown, 0, ISBN Unknown - Available
'Abai Zholy' by M. Auezov, 1942, ISBN 978-601-1 - Available
'Clean Code' by R. Martin, 0, ISBN 978-013-2 - Available
Abai Zholy has been successfully checked out.
Abai Zholy is currently unavailable.
Abai Zholy has been successfully returned.
Abai Zholy is already in the library.
'Clean Code' by R. Martin, 2008, ISBN 978-0132350884 - Available
Getter: Abai Zholy / M. Auezov
```

На что смотрит преподаватель (Grading Focus)

Поля private, методы public; два конструктора, причём параметризованный **не дублирует** проверку года, а вызывает сеттер; геттеры с `const`; checkOut и returnBook проверяют текущее состояние (крайние случаи); в main есть все демонстрации.

Разбор задач

Задача 13.1. Класс Student с конструкторами лёгкая

Конструктор по умолчанию («NoName», курс 1), с параметрами (имя, курс 1–4, иначе 1) и деструктор, который печатает «bye, имя».

Решение

```C++
class Student {
    char name[20];
    int course;
public:
    Student() : course(1) { strcpy(name, "NoName"); }
    Student(const char *n, int c) { strcpy(name, n); course = (c >= 1 && c <= 4) ? c : 1; }
    ~Student() { cout << "bye, " << name << endl; }
    void print() { cout << name << " (" << course << ")" << endl; }
};
//---
Student a;
Student b("Alikhan", 7);
a.print();
b.print();
```

Вывод

```
NoName (1)
Alikhan (1)
bye, Alikhan
bye, NoName
```

Деструкторы — в обратном порядке создания: сначала b, потом a.

Задача 13.2. Стек на динамическом массиве средняя

Класс Stack: конструктор с ёмкостью (new), push, pop, isEmpty, деструктор (delete []). Используй для переворота последовательности 1...5.

Решение

```C++
class Stack {
    int *buf;
    int cap, top;
public:
    Stack(int c) : cap(c), top(0) { buf = new int[cap]; }
    ~Stack() { delete [] buf; }
    bool push(int v) { if (top == cap) return false; buf[top++] = v; return true; }
    int pop() { return buf[--top]; }
    bool isEmpty() const { return top == 0; }
};
//---
Stack s(10);
for (int i = 1; i <= 5; i++) s.push(i);
while (!s.isEmpty()) cout << s.pop() << " ";
```

Вывод

```
5 4 3 2 1
```

Задача 13.3. Посчитай вызовы сложная

Сколько раз и в каком порядке сработают конструкторы и деструкторы?

```cpp
class T {
    int n;
public:
    T(int v) : n(v) { cout << "C" << n << " "; }
    T(const T &o) : n(o.n * 10) { cout << "K" << n << " "; }
    ~T() { cout << "D" << n << " "; }
};
void f(T x) { cout << "f "; }
//---
T a(1);
f(a);
T *p = new T(2);
{
    T b = a;
}
delete p;
cout << "end ";
```

Решение

> **Вывод**
> `C1 K10 f D10 C2 K10 D10 D2 end D1`

`f(a)` — копия по значению (K10), она уничтожается при выходе из f (D10). `new T(2)` → C2. В блоке `T b = a` → K10, конец блока → D10. `delete p` → D2. После `end` заканчивается main, и уничтожается a → D1.

Контрольные вопросы из силлабуса (устно)

1. **Пользовательский класс?** — Класс, который программист описывает сам для своей задачи.
2. **Основные части?** — Поля, методы, конструкторы, деструктор, спецификаторы доступа.
3. **Конструктор?** — Специальный метод с именем класса без типа результата, вызывается автоматически при создании объекта.
4. **Зачем конструктор?** — Инициализировать поля, проверить данные, выделить ресурсы.
5. **Конструктор по умолчанию?** — Без параметров (или все параметры по умолчанию); создаётся компилятором, только если других конструкторов нет.
6. **С параметрами?** — Принимает значения для инициализации: `Point(int x, int y)`.
7. **Копирования?** — `Класс(const Класс &other)`: создаёт объект-копию.
8. **Перегрузка конструкторов?** — Несколько конструкторов с разными параметрами.
9. **Когда вызывается конструктор?** — При создании объекта: объявление, `new`, передача/возврат по значению (копирования).
10. **Деструктор?** — Метод `~Класс()` без параметров, вызывается при уничтожении объекта.
11. **Зачем деструктор?** — Освободить ресурсы: память (delete), файлы.
12. **Когда вызывается деструктор?** — При выходе из блока, при `delete`, при завершении программы; в обратном порядке создания.

13. **Жизненный цикл объекта?** — Выделение памяти → конструктор → использование → деструктор → освобождение памяти.

Тест по лекции 13

1. What is the name of a constructor of class `Book`? новый

a) constructor
b) Book
c) ~Book
d) init

2. What return type does a constructor have? новый

a) void
b) int
c) no return type at all
d) the class type

3. How many destructors can a class have? новый

a) any number (they can be overloaded)
b) two
c) exactly one
d) none

4. Which is the correct signature of a copy constructor of class `A`? новый

a) A(A other)
b) A(const A &other)
c) A copy(const A &other)
d) void A(const A &other)

5. When is the destructor of a local object called? новый

a) never
b) when `main` starts
c) when execution leaves the block where the object was declared
d) immediately after the constructor

6. A class has only the constructor `Point(int x, int y)`. Which declaration does NOT compile? новый

a) Point p(1, 2);
b) Point *q = new Point(0, 0);
c) Point r;
d) Point s = Point(3, 4);

7. Which members MUST be initialized in the constructor initializer list? новый

a) static members
b) const members and references
c) public members
d) all int members

8. What does `A a();` declare? новый

a) an object a created with the default constructor
b) a function a that returns an object of type A
c) an array of A
d) a compilation error

9. What is the output? сложный

```
class X {
    int v;
public:
    X(int a = 5) : v(a) { cout << v; }
    ~X() { cout << -v; }
};
//---
{
    X a, b(2);
    X *c = new X(7);
    delete c;
}
```

a) 527-5-2-7

b) 527-7-2-5

c) 52-2-57-7

d) 27-7-2

10. What is the output? сложный

```
class Cnt {
public:
    static int alive;
    Cnt() { alive++; }
    Cnt(const Cnt &) { alive++; }
    ~Cnt() { alive--; }
};
int Cnt::alive = 0;
void use(Cnt c) { cout << Cnt::alive; }
//---
Cnt a;
{
    Cnt b, c;
    use(a);
}
cout << Cnt::alive;
```

a) 31

b) 41

c) 33

d) 40

11. What is the output? сложный

```
class P {
    int *d;
public:
    P(int v) { d = new int(v); }
    P(const P &o) { d = new int(*o.d); }
    ~P() { delete d; }
    void set(int v) { *d = v; }
    int get() { return *d; }
};
//---
P a(10);
P b(a);
b.set(20);
cout << a.get() + b.get();
```

a) 40
b) 20
c) 30
d) program crashes

Ключи

1 – b **2** – c **3** – c **4** – b **5** – c **6** – c
7 – b **8** – b **9** – b **10** – b **11** – c

9. Создание: a (5), b (2), c (7). `delete c` → −7. Конец блока: сначала b (−2), потом a (−5).

10. Внутри use живут a, b, c и копия-параметр — 4 объекта. После блока остался только a — 1.

11. Конструктор копирования делает глубокую копию (новая ячейка памяти), поэтому изменение b не затрагивает a: 10 + 20 = 30.

ЛЕКЦИЯ 14 ИЗ 15

Пользовательский класс: свойства

Делать классы, данные которых защищены, а доступ к ним идёт через свойства с проверкой. Понимать __property в C++ Builder.

14.1 Что такое свойство

Свойство (property) — характеристика объекта, которую снаружи можно **прочитать** и (иногда) **изменить**, но доступ к которой **контролирует сам класс**. Пользователь видит «значение», а внутри работают методы: проверка, пересчёт, перерисовка.

Свойство состоит из:

- **поля** для хранения (обычно `private`, по соглашению VCL с буквой `F`: `FAge`);
- **метода чтения (getter, аксессор)** — возвращает значение;
- **метода записи (setter, мутатор)** — принимает новое значение и проверяет его.

14.2 Почему данные прячут (инкапсуляция)

НЕПРАВИЛЬНО

```
class Student {
public:
    int age;          // открыто всем
};
// где-то в программе:
s.age = -20;          // никто не помешает
```

ПРАВИЛЬНО

```
class Student {
    int age;          // закрыто
public:
    int getAge() const { return age; }
    void setAge(int a) { if (a >= 16 && a <= 100) age = a; }
};
s.setAge(-20);        // проверка не пропустит
```

Преимущества закрытых данных с доступом через методы:

1. **Целостность**: в объект невозможно записать неверное значение.
2. **Гибкость**: можно поменять внутреннее хранение (например, хранить дату рождения вместо возраста), не меняя код, который пользуется классом.
3. **Контроль**: можно сделать свойство **только для чтения** (нет сеттера) или выполнять действия при изменении (лог, пересчёт, перерисовка).
4. **Отладка**: все изменения проходят через одно место — сеттер.

14.3 Геттеры и сеттеры в стандартном C++

```C++
class Product {
    char name[30];
    double price;
    int qty;
public:
    Product(const char *n, double p, int q) { strcpy(name, n); price = 0; qty = 0; setPrice(p);
setQty(q); }
    // getters (const — не меняют объект)
    const char *getName() const { return name; }
    double getPrice() const { return price; }
    int getQty() const { return qty; }
    double getTotal() const { return price * qty; }      // вычисляемое свойство, поля нет
    // setters с проверкой, возвращают успех
    bool setPrice(double p)
    {
        if (p <= 0) { cout << "bad price " << p << endl; return false; }
        price = p;
        return true;
    }
    bool setQty(int q)
    {
        if (q < 0) { cout << "bad qty " << q << endl; return false; }
        qty = q;
        return true;
    }
};
//---
Product p("Pen", 150, 10);
p.setPrice(-5);           // отклонено
p.setQty(25);
cout << p.getName() << ": " << p.getPrice() << " x " << p.getQty() << " = " << p.getTotal();
```

> **Вывод**
> ```
> bad price -5
> Pen: 150 x 25 = 3750
> ```

Виды свойств по доступу:

Вид	Как сделать	Пример
чтение и запись	getter + setter	`price`
только чтение (read-only)	только getter	`total` (вычисляется), `id` (задаётся в конструкторе)
только запись (write-only)	только setter	пароль: записать можно, прочитать нельзя
вычисляемое	getter считает из других полей, своего поля нет	`area` у прямоугольника, `age` по дате рождения

Правила хорошего сеттера

- Проверяй значение **до** присваивания; при ошибке сообщай (вернуть `false`, сообщение, исключение) и **не меняй** поле.
- В конструкторе используй те же сеттеры, чтобы проверка была в одном месте.
- Параметры-объекты передавай как `const &`: `void setName(const string &n)`.

14.4 Разделение интерфейса и реализации

В реальных проектах класс делят на два файла:

- **заголовочный `.h` — интерфейс**: объявление класса (что умеет);
- ** `.cpp` — реализация**: тела методов (как умеет).

Другие модули подключают только `.h` и не зависят от деталей реализации. Именно так устроены формы в C++ Builder (`Unit1.h` + `Unit1.cpp`).

```
// Rect.h — интерфейс
#ifndef RectH                 // защита от повторного подключения
#define RectH
class Rect {
    double FW, FH;
public:
    Rect(double w, double h);
    double getW() const;
    void setW(double w);
    double area() const;
};
#endif

// Rect.cpp — реализация
#include "Rect.h"
Rect::Rect(double w, double h) : FW(1), FH(1) { setW(w); if (h > 0) FH = h; }
double Rect::getW() const { return FW; }
void Rect::setW(double w) { if (w > 0) FW = w; }
double Rect::area() const { return FW * FH; }

// main.cpp — пользователь класса
#include "Rect.h"
...
```

Защита `#ifndef … #define … #endif`

Не даёт подключить один заголовок дважды (иначе ошибка «Multiple declaration»). Builder вставляет такую защиту в каждый `UnitN.h` сам.

14.5 Свойства в C++ Builder: `__property`

C++ Builder добавляет к языку ключевое слово **`__property`: объявляется** настоящее** свойство, с которым работают как с переменной, а компилятор сам подставляет вызовы методов чтения и записи.

```
__property тип Имя = { read = поле_или_метод_чтения, write = поле_или_метод_записи };
```

```C++
class TTemperature {
private:
    double FCelsius;                                    // поле хранения (F — от "field")
    void SetCelsius(double v) { if (v >= -273.15) FCelsius = v; }
    double GetFahrenheit() { return FCelsius * 9 / 5 + 32; }
    void SetFahrenheit(double f) { SetCelsius((f - 32) * 5 / 9); }
    double GetKelvin() { return FCelsius + 273.15; }
public:
    TTemperature() : FCelsius(0) { }
    __property double Celsius = { read = FCelsius, write = SetCelsius };       // чтение прямо
из поля
    __property double Fahrenheit = { read = GetFahrenheit, write = SetFahrenheit };
    __property double Kelvin = { read = GetKelvin };                            // только
чтение
};
//---
TTemperature t;
t.Celsius = 25;                 // на самом деле вызывается SetCelsius(25)
cout << t.Fahrenheit << " ";    // на самом деле GetFahrenheit()
t.Fahrenheit = 212;
cout << t.Celsius << " ";
t.Celsius = -500;               // сеттер не пропустит
cout << t.Celsius << " " << t.Kelvin << " ";
t.Celsius += 10;                // чтение, потом запись
cout << t.Celsius;
// t.Kelvin = 0;                // ошибка компиляции: нет write
```

Вывод

```
77 100 100 373.15 110
```

Часть	Значение
`read = FCelsius`	чтение напрямую из поля (быстро)
`read = GetFahrenheit`	чтение через метод (вычисление)
`write = SetCelsius`	запись через метод (проверка, побочные действия)
нет `write`	свойство **только для чтения**
нет `read`	свойство только для записи

Так устроены все свойства VCL

`Form1->Width = 500;` — это `__property int Width = {read = FWidth, write = SetWidth};`. Метод `SetWidth` не только меняет поле, но и **перерисовывает** окно. Поэтому свойства VCL «живые»: присвоил, и экран обновился.

Осторожно

`__property` — **расширение Borland/Embarcadero**, в стандартном C++ (Visual Studio, g++) его нет. Адрес свойства взять нельзя (`&t.Celsius` — ошибка), и передать свойство по ссылке в функцию тоже нельзя.

14.6 Свойства в Object Inspector: `__published`

Если класс — **компонент** (наследник `TComponent`), свойства в разделе ** `__published` **видны в** Object Inspector** и сохраняются в `.dfm`. Значение по умолчанию задают через `default`, чтобы не сохранять его в файл формы.

C++ BUILDER

```
class PACKAGE TScoreLabel : public TLabel
{
private:
    int FScore;
    int FMaxScore;
    void __fastcall SetScore(int Value);
    AnsiString __fastcall GetPercent();
protected:
public:
    __fastcall TScoreLabel(TComponent* Owner);
    __property AnsiString Percent = { read = GetPercent };           // только в коде (public)
__published:
    __property int Score = { read = FScore, write = SetScore, default = 0 };       // видно в инспекторе
    __property int MaxScore = { read = FMaxScore, write = FMaxScore, default = 100 };
};

__fastcall TScoreLabel::TScoreLabel(TComponent* Owner) : TLabel(Owner)
{
    FScore = 0;
    FMaxScore = 100;
}
void __fastcall TScoreLabel::SetScore(int Value)
{
    if (Value < 0) Value = 0;
    if (Value > FMaxScore) Value = FMaxScore;
    FScore = Value;
    Caption = IntToStr(FScore) + " / " + IntToStr(FMaxScore);    // свойство само обновляет вид
}
AnsiString __fastcall TScoreLabel::GetPercent()
{
    return FormatFloat("0.0", 100.0 * FScore / FMaxScore) + "%";
}
```

Как такой компонент создать и установить в палитру — в лекции 15.

Раздел класса	Видно в коде снаружи	Видно в Object Inspector
`private`	нет	нет
`protected`	только наследникам	нет
`public`	да	нет (только во время работы программы)
`__published`	да	**да** (и сохраняется в .dfm)

Свойства в программе: чтение и изменение

```
C++ BUILDER
void __fastcall TForm1::btnScoreClick(TObject *Sender)
{
    ScoreLabel1->MaxScore = 50;
    ScoreLabel1->Score = StrToIntDef(Edit1->Text, 0);   // SetScore сам ограничит и обновит
Caption
    Label2->Caption = ScoreLabel1->Percent;              // свойство только для чтения
}
```

Разбор задач

Задача 14.1. Банковский счёт со свойствами лёгкая

Класс Account: баланс только для чтения, владельца можно менять (не пустая строка), методы deposit/withdraw с проверками.

Решение

```
C++
class Account {
    char owner[30];
    double balance;
public:
    Account(const char *o) : balance(0) { strcpy(owner, "unknown"); setOwner(o); }
    const char *getOwner() const { return owner; }
    void setOwner(const char *o) { if (strlen(o) > 0 && strlen(o) < 30) strcpy(owner, o); }
    double getBalance() const { return balance; }          // только чтение: сеттера нет
    bool deposit(double s) { if (s <= 0) return false; balance += s; return true; }
    bool withdraw(double s) { if (s <= 0 || s > balance) return false; balance -= s; return
true; }
};
//---
Account a("Dana");
a.deposit(5000);
cout << a.withdraw(7000) << a.withdraw(1500) << " ";
a.setOwner("");
cout << a.getOwner() << " " << a.getBalance();
```

Вывод

```
01 Dana 3500
```

Задача 14.2. Свойства через __property средняя

Класс TCircle: свойство Radius (запись с проверкой > 0), свойства только для чтения Diameter, Area. Покажи изменение радиуса и попытку записать отрицательный.

Решение

```C++
class TCircle {
    double FRadius;
    void SetRadius(double r) { if (r > 0) FRadius = r; else cout << "radius must be > 0" << endl; }
    double GetDiameter() { return 2 * FRadius; }
    double GetArea() { return 3.14159 * FRadius * FRadius; }
public:
    TCircle(double r = 1) : FRadius(1) { SetRadius(r); }
    __property double Radius = { read = FRadius, write = SetRadius };
    __property double Diameter = { read = GetDiameter };
    __property double Area = { read = GetArea };
};
//---
TCircle c(2);
cout << c.Diameter << " " << c.Area << endl;
c.Radius = -3;
c.Radius = c.Radius * 5;
cout << c.Radius << " " << c.Diameter;
```

Вывод

```
4 12.5664
radius must be > 0
10 20
```

Задача 14.3. Класс Date с проверкой сложная

Класс Date: свойства day, month, year с проверкой корректности (учитывая високосный год), метод `next()` — следующий день.

Решение

```C++
class Date {
    int d, m, y;
    static bool leap(int y) { return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; }
    static int daysIn(int m, int y)
    {
        int t[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        return (m == 2 && leap(y)) ? 29 : t[m - 1];
    }
public:
    Date() : d(1), m(1), y(2000) { }
    bool set(int dd, int mm, int yy)
    {
        if (yy < 1 || mm < 1 || mm > 12 || dd < 1 || dd > daysIn(mm, yy)) return false;
        d = dd; m = mm; y = yy;
        return true;
    }
    void next()
    {
        if (++d > daysIn(m, y)) { d = 1; if (++m > 12) { m = 1; y++; } }
    }
    void print() const { cout << d << "." << m << "." << y << " "; }
};
//---
Date x;
cout << x.set(29, 2, 2025) << x.set(28, 2, 2024) << " ";
x.next(); x.print();
x.next(); x.print();
x.set(31, 12, 2026); x.next(); x.print();
```

Вывод

```
01 29.2.2024 1.3.2024 1.1.2027
```

Контрольные вопросы из силлабуса (устно)

1. **Свойство класса?** — Характеристика объекта, доступ к которой (чтение/запись) контролирует класс через методы.
2. **Зачем инкапсулировать данные?** — Чтобы защитить объект от неверных значений и скрыть устройство.
3. **Геттер и сеттер?** — Методы чтения (get) и записи (set) значения закрытого поля.
4. **Назначение геттера?** — Вернуть значение (часто `const`-метод), можно вычислять его.
5. **Назначение сеттера?** — Изменить значение с проверкой и побочными действиями.
6. **Как сеттер проверяет данные?** — Сравнивает новое значение с допустимым диапазоном, при ошибке не меняет поле и сообщает об ошибке.

7. **Открытое поле vs свойство?** — Открытое поле меняется без контроля; свойство идёт через методы с проверкой и может быть только для чтения.
8. **Как реализуют свойства?** — private-поле + public get/set; в C++ Builder — `__property … = { read = …, write = … }`.
9. **Доступ снаружи?** — Через get/set методы или как к переменной, если это `__property`.
10. **Преимущества?** — Целостность данных, гибкость реализации, контроль доступа, удобная отладка.

Тест по лекции 14

1. A method that returns the value of a private field is called: новый

a) a setter (mutator)
b) a getter (accessor)
c) a constructor
d) a destructor

2. How do you make a property read-only? новый

a) declare its field public
b) provide only a getter (or `read` without `write`)
c) provide only a setter
d) make the getter static

3. Where should the validation of a new property value be placed? новый

a) in the getter
b) in the setter
c) in the destructor
d) in `main`

4. Which C++ Builder keyword declares a property with `read` and `write` specifiers? новый

a) property
b) __property
c) __published
d) __fastcall

5. Properties declared in which section of a component are shown in the Object Inspector? новый

a) private
b) public
c) protected
d) __published

6. In `__property int Size = { read = FSize, write = SetSize };` what is called when you write `obj.Size = 5;`? новый

a) FSize is assigned directly
b) SetSize(5)
c) the getter
d) nothing, it is a compilation error

7. Which file usually contains the class declaration (interface)? новый

a) the header file (.h)
b) the implementation file (.cpp)
c) the form file (.dfm)
d) the project file (.bpr)

8. What is the purpose of `#ifndef RectH / #define RectH / #endif` in a header file? новый

a) to declare a constant

b) to prevent the header from being included twice

c) to speed up the program

d) to define a property

9. What is the output? сложный

```
class Box {
    int FV;
    void SetV(int v) { FV = (v > 10) ? 10 : v; }
public:
    Box() : FV(0) { }
    __property int V = { read = FV, write = SetV };
};
//---
Box b;
b.V = 7;
b.V = b.V + 5;
cout << b.V << " ";
b.V = 3;
b.V += b.V;
cout << b.V;
```

a) 12 6

b) 10 6

c) 10 10

d) 7 3

10. What is the output? сложный

```
class Acc {
    int bal;
public:
    Acc() : bal(100) { }
    int get() const { return bal; }
    bool take(int s) { if (s > bal) return false; bal -= s; return true; }
};
//---
Acc a;
int ok = 0;
for (int i = 1; i <= 5; i++)
    if (a.take(i * 15)) ok++;
cout << ok << " " << a.get();
```

a) 5 0

b) 3 10

c) 4 -50

d) 3 0

Ключи

1 — b	**2** — b	**3** — b	**4** — b	**5** — d	**6** — b
7 — a	**8** — b	**9** — b	**10** — b		

9. 7 + 5 = 12, сеттер обрезает до 10. Потом 3 + 3 = 6 (`+=` читает через read и записывает через write).

10. Снимается 15, 30, 45 (итого 90, осталось 10); 60 и 75 больше остатка, отклонены.

ЛЕКЦИЯ 15 ИЗ 15

Создание пользовательского компонента

Создать свой визуальный компонент на основе стандартного компонента VCL, добавить свойства, метод и событие, установить его в палитру и использовать на форме.

15.1 Компонент, объект, класс

Понятие	Что это	Пример
Класс	описание типа (поля, методы, свойства)	`TButton`
Объект	экземпляр класса в памяти во время работы	кнопка, созданная `new TButton(this)`
Компонент	класс-наследник **TComponent**, который можно положить на форму из **палитры**, настроить в **Object Inspector** и сохранить в `.dfm`	`TButton` в палитре Standard

Любой компонент — это класс, но не любой класс — компонент. Компонентом класс делает наследование от `TComponent` и регистрация в палитре.

Стандартные компоненты поставляются с C++ Builder (VCL). **Пользовательские** пишет программист, чтобы:

- **переиспользовать** готовое решение во многих проектах (поле для ввода только чисел, кнопка с подтверждением);
- **не повторять** одни и те же настройки и обработчики на каждой форме;
- **расширить** стандартный компонент новыми свойствами и поведением;
- настраивать его **мышкой** в Object Inspector, как стандартный.

15.2 С чего начинается компонент: выбор предка

Новый компонент почти всегда **наследует** существующий и добавляет своё (лекция 12).

Хочу сделать	Предок
немного изменить существующий компонент	сам этот компонент: `TEdit`, `TLabel`, `TButton`
то же, но скрыть часть свойств	`TCustomXxx` (`TCustomEdit`, `TCustomLabel`): свойства в них protected, публикуешь только нужные
свой визуальный элемент с фокусом и окном	`TWinControl` / `TCustomControl`
свой рисуемый элемент без фокуса	`TGraphicControl`
невизуальный компонент (как Timer)	`TComponent`

15.3 Мастер New Component

1. **Component → New Component...**
2. **Ancestor type** — предок, например `TLabel`.
3. **Class Name** — имя нового класса, по соглашению с `T`: `TGreetLabel`.
4. **Palette Page** — вкладка палитры, куда он попадёт: `Samples` (или своя, например `StudyPlay`).
5. **Unit file name** — где сохранить `GreetLabel.cpp`.
6. **OK** — Builder создаст заготовку модуля (`.h` + `.cpp`).

15.4 Готовый компонент: TGreetLabel (лабораторная из силлабуса)

Задание: свой визуальный компонент на основе стандартного, с **текстовым свойством**. Делаем надпись, у которой есть свойства `UserName` и `Greeting`; при их изменении `Caption` пересобирается сам. Добавим метод `Reset()` и событие `OnNameChange`.

C++ BUILDER

```
// GreetLabel.h
#ifndef GreetLabelH
#define GreetLabelH
#include <SysUtils.hpp>
#include <Classes.hpp>
#include <Controls.hpp>
#include <StdCtrls.hpp>

class PACKAGE TGreetLabel : public TLabel
{
private:
    AnsiString FUserName;                // поля хранения свойств
    AnsiString FGreeting;
    TNotifyEvent FOnNameChange;          // поле для события
    void __fastcall SetUserName(AnsiString Value);
    void __fastcall SetGreeting(AnsiString Value);
    void __fastcall UpdateCaption();
protected:
    virtual void __fastcall DoNameChange();          // «вызвать событие»
public:
    __fastcall TGreetLabel(TComponent* Owner);
    void __fastcall Reset();                          // публичный метод
__published:
    __property AnsiString UserName = { read = FUserName, write = SetUserName };
    __property AnsiString Greeting = { read = FGreeting, write = SetGreeting };
    __property TNotifyEvent OnNameChange = { read = FOnNameChange, write = FOnNameChange };
};
#endif
```

C++ BUILDER

```cpp
// GreetLabel.cpp
#include <vcl.h>
#pragma hdrstop
#include "GreetLabel.h"
#pragma package(smart_init)

// Проверка, что в классе нет чисто виртуальных функций (генерирует мастер)
static inline void ValidCtrCheck(TGreetLabel *)
{
    new TGreetLabel(NULL);
}

__fastcall TGreetLabel::TGreetLabel(TComponent* Owner) : TLabel(Owner)
{
    FGreeting = "Hello";                  // значения по умолчанию
    FUserName = "student";
    Font->Size = 12;
    UpdateCaption();
}

void __fastcall TGreetLabel::UpdateCaption()
{
    Caption = FGreeting + ", " + FUserName + "!";
}

void __fastcall TGreetLabel::SetUserName(AnsiString Value)
{
    Value = Value.Trim();
    if (Value == "") Value = "student";   // проверка значения
    if (Value == FUserName) return;       // ничего не изменилось
    FUserName = Value;
    UpdateCaption();
    DoNameChange();                       // сообщаем подписчикам
}

void __fastcall TGreetLabel::SetGreeting(AnsiString Value)
{
    FGreeting = Value;
    UpdateCaption();
}

void __fastcall TGreetLabel::DoNameChange()
{
    if (FOnNameChange) FOnNameChange(this);   // если обработчик назначен — вызвать
}

void __fastcall TGreetLabel::Reset()
{
    FGreeting = "Hello";
    UserName = "student";                 // через свойство → сработают проверка и событие
    UpdateCaption();
}

// Регистрация в палитре: имя namespace = имя модуля с большой буквы, остальные маленькие
namespace Greetlabel
```

```
{
    void __fastcall PACKAGE Register()
    {
        TComponentClass classes[1] = { __classid(TGreetLabel) };
        RegisterComponents("Samples", classes, 0);
    }
}
```

Разбор важных мест:

Строка	Зачем
`class PACKAGE TGreetLabel : public TLabel`	наследуем всё, что умеет Label; `PACKAGE` нужен для экспорта из пакета
`FUserName` + `SetUserName`	хранение и запись свойства с проверкой и обновлением вида
`__published: __property …`	свойства появятся в **Object Inspector**
`TNotifyEvent FOnNameChange`	событие — это свойство типа «указатель на метод» `void __fastcall (__closure *)(TObject *Sender)`
`if (FOnNameChange) FOnNameChange(this);`	вызвать обработчик, **только если** его назначили
`ValidCtrCheck`	проверка мастера: класс не абстрактный
`namespace Greetlabel { Register() }`	регистрирует компонент на вкладке палитры

Ошибки в Register

Имя `namespace` должно совпадать с именем файла модуля: первая буква большая, остальные **маленькие** (`GreetLabel.cpp` → `Greetlabel`). Иначе компонент не появится в палитре.

Событие = свойство-указатель на метод

Пользователь компонента в Object Inspector на вкладке Events дважды щёлкает `OnNameChange` , и Builder создаёт обработчик в форме. Компонент **не знает**, что будет в обработчике: он лишь «зовёт» его в нужный момент через `DoNameChange()` .

15.5 Установка компонента в палитру

1. Сохрани модуль компонента.
2. **Component → Install Component…**
3. Вкладка **Into existing package** → пакет ** `dclusr.bpk` **(пакет пользовательских компонентов)** **или** Into new package** (свой пакет `.bpk`).
4. Unit file name — твой `GreetLabel.cpp` → **OK**.
5. Builder скомпилирует пакет (`.bpl`) и сообщит, что компонент зарегистрирован.
6. На вкладке палитры **Samples** появится значок TGreetLabel — его можно класть на формы.

Пакет (package)

Пакет — особая DLL C++ Builder (`.bpl`), в которой хранятся компоненты. **Design-time** пакет (`dcl…`) устанавливается в IDE и даёт значки в палитре; **run-time** пакет нужен программе во время работы (или код компонента вкомпилируется в .exe).

15.6 Использование и тестирование

Тест до установки — удобно создать компонент в коде обычного приложения и проверить, не трогая палитру:

C++ BUILDER

```
// Unit1.cpp тестового приложения: #include "GreetLabel.h", GreetLabel.cpp добавлен в проект
void __fastcall TForm1::FormCreate(TObject *Sender)
{
    G = new TGreetLabel(this);             // G объявлен в private TForm1: TGreetLabel *G;
    G->Parent = this;
    G->Left = 20; G->Top = 20;
    G->OnNameChange = GreetChanged;        // назначаем обработчик события
}
void __fastcall TForm1::btnApplyClick(TObject *Sender)
{
    G->UserName = Edit1->Text;             // свойство: проверка и обновление Caption
    G->Greeting = cbGreeting->Text;
}
void __fastcall TForm1::btnResetClick(TObject *Sender)
{
    G->Reset();                            // метод компонента
}
void __fastcall TForm1::GreetChanged(TObject *Sender)
{
    Memo1->Lines->Add("Name changed to: " + G->UserName);
}
```

После установки — просто положи TGreetLabel с палитры, задай `UserName` и `Greeting` в Object Inspector (надпись на форме обновится сразу в дизайнере!), создай обработчик `OnNameChange` на вкладке Events.

Что проверить при тестировании:

- значения по умолчанию после создания;
- реакцию на неверные значения (пустое имя, пробелы);
- работу в дизайнере (изменение свойств в инспекторе) и во время выполнения;
- что событие срабатывает только при реальном изменении;
- сохранение и загрузку формы (закрыть и открыть проект — свойства на месте).

15.7 Второй пример: TNumEdit — поле только для чисел

C++ BUILDER

```
// NumEdit.h
class PACKAGE TNumEdit : public TEdit
{
private:
    int FMinValue, FMaxValue;
    int __fastcall GetValue();
    void __fastcall SetValue(int v);
protected:
    DYNAMIC void __fastcall KeyPress(char &Key);       // переопределяем реакцию на клавиши
public:
    __fastcall TNumEdit(TComponent* Owner);
__published:
    __property int Value = { read = GetValue, write = SetValue };
    __property int MinValue = { read = FMinValue, write = FMinValue, default = 0 };
    __property int MaxValue = { read = FMaxValue, write = FMaxValue, default = 100 };
};
```

C++ BUILDER

```
// NumEdit.cpp (фрагмент)
__fastcall TNumEdit::TNumEdit(TComponent* Owner) : TEdit(Owner)
{
    FMinValue = 0; FMaxValue = 100;
    Text = "0";
}
void __fastcall TNumEdit::KeyPress(char &Key)
{
    if ((Key < '0' || Key > '9') && Key != 8) Key = 0;   // только цифры и Backspace
    TEdit::KeyPress(Key);                                 // вызвать поведение предка (и
OnKeyPress)
}
int __fastcall TNumEdit::GetValue()
{
    return StrToIntDef(Text, FMinValue);
}
void __fastcall TNumEdit::SetValue(int v)
{
    if (v < FMinValue) v = FMinValue;
    if (v > FMaxValue) v = FMaxValue;
    Text = IntToStr(v);
}
```

Вызывай метод предка

При переопределении `KeyPress`, `Click`, `Paint` почти всегда нужно вызвать версию базового класса (`TEdit::KeyPress(Key)`), иначе пропадёт стандартное поведение и пользовательский обработчик `OnKeyPress` перестанет работать.

15.8 Этапы разработки компонента (итог)

1. **Постановка задачи**: что должен уметь компонент, какие свойства, методы, события.
2. **Выбор предка** (TEdit, TLabel, TCustom..., TGraphicControl, TComponent).
3. **Создание заготовки**: Component → New Component.

4. **Свойства**: поля `F…`, методы `Get…/Set…`, `__property` в `__published`.
5. **Методы**: публичные действия (`Reset`), переопределённые методы предка (`KeyPress`, `Paint`).
6. **События**: поле `TNotifyEvent FOnX`, `__property OnX`, вызов через `DoX()`.
7. **Регистрация**: функция `Register()` с `RegisterComponents("Вкладка", …)`.
8. **Тестирование** в коде (создание через `new`, `Parent`).
9. **Установка** в пакет: Component → Install Component.
10. **Использование** в проектах, исправление ошибок, переиспользование.

Разбор задач

Задача 15.1. Что выбрать предком? лёгкая

Для каждого компонента выбери предка: (1) кнопка, которая при нажатии спрашивает «Are you sure?»; (2) часы, показывающие время текстом; (3) круглый индикатор, нарисованный на форме; (4) невидимый компонент, который раз в минуту сохраняет данные.

Решение

1 — `TButton` (переопределить `Click`). 2 — `TLabel` (+ внутренний `TTimer` или наследовать `TCustomLabel`). 3 — `TGraphicControl` (переопределить `Paint`, рисовать на `Canvas`). 4 — `TComponent` (невизуальный, внутри таймер).

Задача 15.2. Кнопка с подтверждением средняя

Напиши компонент TConfirmButton (на основе TButton) со свойством `Question` (текст вопроса). При нажатии показывается вопрос; обычный `OnClick` срабатывает только при ответе «Yes».

Решение

C++ BUILDER

```
class PACKAGE TConfirmButton : public TButton
{
private:
    AnsiString FQuestion;
public:
    __fastcall TConfirmButton(TComponent* Owner) : TButton(Owner) { FQuestion = "Are you
sure?"; }
    DYNAMIC void __fastcall Click()
    {
        if (Application->MessageBox(FQuestion.c_str(), "Confirm", MB_YESNO |
MB_ICONQUESTION) == IDYES)
            TButton::Click();              // только тогда — стандартное поведение и
OnClick
    }
__published:
    __property AnsiString Question = { read = FQuestion, write = FQuestion };
};
```

Задача 15.3. Счётчик нажатий с событием сложная

Компонент TCounterButton (на основе TButton): свойство `Count` только для чтения (в разделе public), свойство `Limit` (в __published, чтобы задавать в инспекторе), при каждом нажатии Count++ и Caption = "Clicks: N"; при достижении Limit вызывается событие `OnLimit`, а Count сбрасывается.

Решение

C++ BUILDER

```
class PACKAGE TCounterButton : public TButton
{
private:
    int FCount, FLimit;
    TNotifyEvent FOnLimit;
public:
    __fastcall TCounterButton(TComponent* Owner) : TButton(Owner)
    {
        FCount = 0; FLimit = 10; FOnLimit = NULL;
        Caption = "Clicks: 0";
    }
    DYNAMIC void __fastcall Click()
    {
        FCount++;
        if (FCount >= FLimit) {
            if (FOnLimit) FOnLimit(this);     // событие
            FCount = 0;
        }
        Caption = "Clicks: " + IntToStr(FCount);
        TButton::Click();
    }
    __property int Count = { read = FCount };                // public, только чтение
__published:
    __property int Limit = { read = FLimit, write = FLimit, default = 10 };
    __property TNotifyEvent OnLimit = { read = FOnLimit, write = FOnLimit };
};
```

Контрольные вопросы из силлабуса (устно)

1. **Компонент в C++Builder?** — Класс-наследник TComponent, который размещают из палитры, настраивают в Object Inspector и сохраняют в .dfm.
2. **Класс, объект, компонент?** — Класс — описание; объект — экземпляр в памяти; компонент — особый класс, работающий в дизайнере.
3. **Пользовательский компонент?** — Компонент, созданный программистом (обычно наследник стандартного) и установленный в палитру.
4. **Зачем?** — Переиспользование, расширение стандартных компонентов, единообразие, настройка мышкой.
5. **Роль наследования?** — Новый компонент получает всё поведение предка и добавляет своё.
6. **Как выбрать предка?** — Самый близкий по поведению: готовый компонент, TCustom-версия, TWinControl/TGraphicControl или TComponent.

7. **Какие свойства добавить?** — Любые данные компонента: поле F + Get/Set + `__property` в `__published` (видно в инспекторе).
8. **Как добавить методы?** — Объявить в public (действия) или переопределить методы предка (KeyPress, Click, Paint) с вызовом базовой версии.
9. **События компонента?** — Свойства типа указателя на метод (TNotifyEvent), которые компонент вызывает в нужный момент.
10. **Регистрация?** — Функция `Register()` в namespace модуля с `RegisterComponents("Вкладка", classes, 0)`.
11. **Добавление в палитру?** — Component → Install Component → пакет (dclusr.bpk) → компиляция → значок на вкладке.
12. **Этапы создания и тестирования?** — Задача → предок → New Component → свойства, методы, события → Register → тест через new → установка → использование.

Тест по лекции 15

1. **Which class must a class inherit from (directly or indirectly) to become a component placed on the Component Palette?** новый

a) TObject
b) TPersistent
c) TComponent
d) TForm

2. **Which menu command starts the wizard for creating a new component?** новый

a) File → New → Application
b) Component → New Component
c) Project → Add to Project
d) Tools → Options

3. **Which function adds a component to a page of the Component Palette?** новый

a) InstallComponent
b) AddToPalette
c) RegisterComponents
d) CreateForm

4. **In which section must a property be declared to appear in the Object Inspector?** новый

a) private
b) protected
c) public
d) __published

5. **What is a component event in VCL?** новый

a) a public method
b) a private field
c) a property of a method-pointer type (for example, TNotifyEvent)
d) a constructor

6. **Which package is commonly used to install user components?** новый

a) vcl.bpl
b) dclusr.bpk
c) rtl.lib
d) Project1.bpr

7. Which ancestor is best for a component that only draws itself and does not need keyboard focus? новый

a) TWinControl
b) TGraphicControl
c) TComponent
d) TForm

8. Why do you call `TEdit::KeyPress(Key)` inside an overridden `KeyPress` of a component derived from TEdit? новый

a) it is required by the compiler
b) to keep the standard behavior and the user's OnKeyPress handler
c) to register the component
d) to delete the component

9. Which code correctly fires an optional event `FOnChange` of a component? новый

a) FOnChange();
b) if (FOnChange) FOnChange(this);
c) OnChange = this;
d) FOnChange = NULL;

10. How is the namespace for the Register function named for the unit file `MyButton.cpp`? новый

a) MyButton
b) MYBUTTON
c) Mybutton
d) mybutton

Ключи

1 — c **2** — b **3** — c **4** — d **5** — c **6** — b
7 — b **8** — b **9** — b **10** — c

ИТОГ КУРСА

Итог: всё повторение в одном месте

Быстро повторить весь курс перед рубежным контролем или экзаменом: кратко лекции, все задачи и все тесты с ключами.

Конспект в двух словах: каждая лекция на одной карточке

1. Парадигмы, алгоритмы, синтаксис C++, структура программы, типы данных, указатели

- **Алгоритм**: конечная последовательность точных шагов. Свойства: дискретность, понятность, определённость, конечность, массовость. Структуры: следование, ветвление, цикл.
- **Блок-схема**: овал (начало/конец), параллелограмм (ввод/вывод), прямоугольник (действие), ромб (условие).
- **Идентификатор**: латиница, цифры, `_`; начинается не с цифры; регистр важен; не ключевое слово.
- **Константы**: `017` = 15 (восьмеричная), `0x1F` = 31, `'A'` (символ), `"Hi"` (строка), `const double PI = 3.14;`.
- **Escape**: `\n` новая строка, `\t` табуляция, `\\`, `\"`, `\0` конец строки.
- **Программа**: `#include <iostream.h>` → `int main() { … return 0; }`. Выполнение начинается с `main`.
- **Типы**: char 1, short 2, int 4, float 4, double 8, bool 1 байт. Переполнение «заворачивает» значение.
- **enum**: `enum C {A, B=5, D};` → A=0, B=5, D=6.
- **Указатели**: `&x` — адрес, `int *p = &x;`, `*p` — значение по адресу; `a[i]` ≡ `*(a+i)`.

2. Операции и выражения

- `/` для двух целых — **целое деление** (7/2 = 3); `%` — остаток, только для целых. `(double)a/b` — дробное деление.
- `x++` отдаёт старое значение, потом увеличивает; `++x` — сначала увеличивает. Не меняй переменную дважды в одном выражении.
- `x op= y` ≡ `x = x op (y)` (правая часть целиком!).
- Сравнения `== != < > <= >=` дают 1/0. ** `=` — присваивание, не сравнение.**
- Логика: `&&` (И), `||` (ИЛИ), `!` (НЕ); сокращённое вычисление. `1<=x<=10` — ошибка, нужно `x>=1 && x<=10`.
- Битовые: `& | ^ ~ << >>`; `x<<k` = $x \cdot 2^k$, `x>>k` = $x/2^k$.
- `a ? b : c` — тернарная; `,` — возвращает последнее; `sizeof(type)`; `::x` — глобальная; `.` и `->`.
- Приоритет: унарные → `* / %` → `+ -` → сравнения → `&&` → `||` → `?:` → присваивания.
- Типы: short+int→int, long+float→float, float+double→double. `int k = 3.99` → 3.
- math.h: sqrt, pow, fabs, ceil (вверх), floor (вниз), log (ln), log10, sin/cos в радианах; корень n-й степени `pow(x, 1.0/n)`.
- Borland: `randomize(); random(n)` → 0…n−1. conio.h: `getch()`, `clrscr()`. printf: `%d %u %o %x %f %e %c %s`.

3. Ветвления if, switch, goto и цикл for

- `if (усл) A; else B;` — несколько операторов в `{ }`. `if (x = 5)` — присваивание (ошибка!). `;` после `if(...)` — пустое тело.
- Лесенка `else if`: выполняется первое истинное условие. `else` относится к ближайшему `if`.
- `switch(целое) { case K: …; break; default: … }` — прыгает на case и идёт вниз до `break` (fall-through). Метки — целые константы, без диапазонов.
- `goto метка;` — безусловный переход, лучше не использовать.
- `for (init; cond; step)`: init — 1 раз, cond — перед каждым шагом, step — после тела. После `for(a=0;a<10;a++)` a = 10.
- Число повторений `for(i=a;i<=b;i+=h)` = (b−a)/h + 1. Вложенные циклы: произведение повторений.
- Шаблоны: сумма (`s=0; s+=…`), произведение (`p=1; p*=…`), счётчик (`k++` по условию), максимум (`mx=первый; if(x>mx) mx=x`).

4. Циклы while и do...while, break и continue, одномерные и двумерные массивы

- `while (усл) {…}` — с предусловием, 0+ раз; `do {…} while (усл);` — с постусловием, 1+ раз (`;` в конце!). Все циклы C++ работают, пока условие истинно.
- `break` — выход из цикла/switch; `continue` — к следующей итерации (в while не пропусти изменение счётчика).
- Массив `int a[N];` — индексы 0…N−1, размер — константа. `int b[5] = {1,2};` → остальные 0. Выход за границы не проверяется.
- Массивы не присваиваются (`b = a` — ошибка) и не сравниваются целиком.
- Шаблоны: сумма/среднее, max/min с индексом (начинать с a[0]), поиск с break, пузырёк (2 цикла, `j < N-1-i`), переворот (`i < N/2`).
- Матрица `m[i][j]` (i — строка, j — столбец). Главная диагональ `i==j`, побочная `i+j==n-1`, выше `j>i`, ниже `j<i`.

5. Функции: объявление, параметры по значению и по ссылке, const, значения по умолчанию, inline, массивы, рекурсия

- Функция: `тип имя(параметры) { … return значение; }`; `void` — без результата. return возвращает значение и завершает функцию.
- **Прототип** `int f(int, int);` нужен, если функция вызывается выше определения; помогает проверить число и типы аргументов.
- Параметры: **формальные** (в заголовке) и **фактические** (при вызове).
- По значению — копия; по ссылке `int &x` — оригинал (нельзя передать константу); через указатель `int *x` + вызов `f(&a)`; `const &` — без копии и без изменений.
- По умолчанию: `void f(int a, int b = 2);` — только в конце списка, указываются в прототипе.
- `inline` — подстановка тела вместо вызова (рекомендация). Перегрузка — одно имя, разные параметры (не только тип результата).
- Массив передаётся по адресу (функция может его менять), размер передают отдельно; `const int a[]` защищает от изменений.
- Локальные — внутри блока; глобальные — вне функций (по умолчанию 0); `static` локальная помнит значение; `::x` — глобальная.
- Рекурсия: базовый случай + шаг; `fact(n) = n·fact(n−1)`, `fact(1) = 1`. Печать до вызова — на спуске, после — на подъёме.

6. Тип char и строки: массивы символов, string.h и класс string

- `char` — 1 байт, хранит код ASCII: '0' = 48, 'A' = 65, 'a' = 97. `c - '0'` — цифра в число; `c - 32` — в верхний регистр. ctype.h: isdigit, isalpha, toupper.
- Строка C — массив char с `'\0'` в конце; размер = длина + 1. `sizeof` — размер массива, `strlen` — число символов до '\0'.
- `cin >> s` читает до пробела; `cin.getline(s, N)` — всю строку.
- string.h: strlen, strcpy, strncpy, strcat, strncat, strcmp (0 — равны), strncmp, stricmp (без регистра), strchr, strrchr (с конца), strstr (подстрока), strrev, strupr, strlwr, strnset.
- Нельзя: `s = "x"` (нужен strcpy), `s == t` (нужен strcmp), выходить за размер.
- Класс string: `=`, `+`, `==`, `.length()`, `.substr(pos,len)`, `.find()`, `.insert()`, `.erase()`, `.c_str()`, `getline(cin, s)`.
- Файлы: `FILE *f = fopen(имя, "r"/"w"/"a")`, `fprintf` / `fscanf`, `fputs` / `fgets`, `fclose(f)`.

7. Ключевые понятия и принципы объектно-ориентированного программирования

- **ООП**: программа из объектов (данные + методы), а не из отдельных функций и данных.
- **Класс** — шаблон (тип); **объект** — экземпляр с собственным состоянием; **атрибут/поле** — данные; **метод** — функция класса. **Состояние** — значения полей, **поведение** — методы.
- **Инкапсуляция** — данные private, доступ через public-методы (геттеры/сеттеры с проверкой).
- **Абстракция** — простой интерфейс, детали скрыты (performAction вызывает walk/fetch).
- **Наследование** — `class Poodle : public Dog`; отношение «является». «Имеет» → поле-объект (композиция).
- **Полиморфизм** — статический (перегрузка) и динамический (`virtual` + указатель/ссылка на базовый класс). Без virtual вызов выбирается по типу указателя.

8. Классы и объекты в C++

- `class Имя { private: … public: … };` — точка с запятой в конце. Поля — данные, методы — функции класса.
- `public` — всем, `private` — только классу (по умолчанию в class), `protected` — классу и наследникам. В `struct` по умолчанию public.
- Методы вне класса: `тип Класс::метод(…) { … }`. Методы внутри класса — inline.
- Объекты: `A a;` → `a.x`; `A *p = new A;` → `p->x`, `delete p;`; `new A[n]` → `delete [] p`.
- `this` — указатель на текущий объект: `this->x = x;`, `return *this;` (цепочки).
- `static int count;` + `int Класс::count = 0;` вне класса; общий для всех объектов; static-метод без this.
- Объекты в функции: по значению (копия), по ссылке, `const &`. const-метод `int get() const` не меняет поля.

9. Интегрированная среда разработки Borland C++ Builder

- **IDE** — редактор + компилятор + линкер + отладчик + визуальные инструменты. **C++ Builder** — RAD-среда на C++ с библиотекой **VCL**.
- Окна: главное меню, SpeedBar, **Component Palette** (компоненты по вкладкам), **Form Designer** (форма), **Object Inspector** (Properties / Events), **Code Editor**, **Project Manager**, Messages. F12 — форма ↔ код.
- Файлы: `.bpr` (проект), `Project1.cpp` (WinMain), `Unit1.cpp` (ваш код), `Unit1.h` (класс формы), `Unit1.dfm` (описание формы), `.res`, `.obj`, `.exe`.
- Сборка: Alt+F9 compile, Ctrl+F9 make, Build all, **F9 run**, Ctrl+F2 reset. Этапы: препроцессор → компилятор (.obj) → линкер (.exe).
- Ошибки: синтаксические, семантические (их находит компилятор), логические (только тестирование и отладка), runtime.
- Отладка: breakpoint F5, **F8 step over**, **F7 trace into**, F4 run to cursor, Watch Ctrl+F5, Evaluate Ctrl+F7.

10. Создание проекта. Работа с формами. Дизайнер форм

- Оконное приложение: **File** → **New** → **Application**; сразу Save All в отдельную папку. Модуль формы: `Unit1.cpp` (код), `Unit1.h` (класс TForm1), `Unit1.dfm` (вид).
- Форма — класс-наследник `TForm`; компоненты — поля-указатели в `__published` → обращение через `->`: `Label1->Caption`.
- Дизайнер: палитра → щелчок по форме; Ctrl+стрелки — сдвиг, Shift+стрелки — размер; Align, Anchors, Tab Order.
- Компоненты: Label (Caption), Edit (Text), Button (OnClick), Memo (Lines), ListBox (Items, ItemIndex), CheckBox (Checked), Timer (Interval, OnTimer).
- Текст ↔ число: `StrToInt`, `StrToFloat`, `IntToStr`, `FloatToStr`, `FormatFloat("0.00", x)`; `ShowMessage`. `Text + Text` — склейка строк!
- Вторая форма: File → New → Form, `#include "Unit2.h"`, `Form2->Show()` (немодально) / `Form2->ShowModal()` (модально, ModalResult). Или `new TForm2(this)` + `delete`.
- Компонент в runtime: `new TButton(this)` (Owner), обязательно `->Parent = …`, назначить `OnClick`.

11. Работа с формой: свойства, методы и события

- **Свойство** — значение (Caption, Width, Color), **метод** — действие со скобками (Show(), Close(), Clear()), **событие** — «когда…» (OnClick, OnCreate).
- Свойства меняют в Object Inspector (design-time) или кодом `Width = 500;` (run-time). Внутри методов формы `Form1->` не нужен.
- Форма: Name, Caption, Width/Height, ClientWidth, Left/Top, Color, Position (poScreenCenter), Visible, Enabled, BorderStyle, WindowState, FormStyle (fsStayOnTop), Font, AlphaBlend. Read-only: ComponentCount, ControlCount, Handle.
- Методы: Show, ShowModal, Hide, Close, SetFocus, Repaint; `Application->Terminate()`.
- Обработчик: Events → двойной щелчок; `void __fastcall TForm1::Button1Click(TObject *Sender)`. Sender — кто вызвал; общий обработчик + Tag.
- События: OnCreate → OnShow → OnActivate → OnPaint; OnResize, OnClick, OnMouseMove (X, Y), OnKeyPress (`char &Key`, `Key = 0` отменяет), OnChange, OnCloseQuery (`CanClose`), OnClose, OnTimer (Interval).

12. Иерархия классов: наследование, переопределение, виртуальные функции

- `class D : public B { … };` — D наследует члены B. Отношение «является». Конструкторы, деструктор, operator= не наследуются.
- Одиночное и множественное (`: public A, public B`) наследование; проблема ромба → виртуальное наследование.
- Доступ: private базы недоступен всегда; при public-наследовании доступ сохраняется, при protected/private — понижается. По умолчанию для class — private-наследование.
- Конструкторы: база → наследник (параметры через `: Base(args)`); деструкторы — наоборот.
- Переопределение — та же сигнатура в наследнике; `Base::f()` вызывает версию базы. Перегрузка — другие параметры.
- `virtual` → вызов по реальному объекту (через указатель/ссылку на базу). `virtual f() = 0` — чисто виртуальная → класс абстрактный, объект создать нельзя. Есть virtual → деструктор тоже virtual.
- VCL: TObject → TPersistent → TComponent → TControl → TWinControl (Button, Edit, Panel, Form) / TGraphicControl (Label, Image). Owner — удаляет, Parent — показывает. `dynamic_cast`, `InheritsFrom`, `ClassName()`, `ClassParent()`.

13. Пользовательский класс. Конструктор и деструктор

- Конструктор: имя класса, без типа результата, вызывается при создании; можно перегружать. `A a;` (без скобок!), `A a(1, 2);`, `A *p = new A(1);`.
- По умолчанию — без аргументов; исчезает, если написан любой другой конструктор (выход: значения по умолчанию у параметров).
- Список инициализации `A(int x) : field(x), Base(x) { }` — обязателен для const-полей, ссылок, базового класса.
- Копирования: `A(const A &o)` — при `A b = a;`, передаче и возврате по значению. Для динамической памяти — глубокое копирование (правило трёх).
- Деструктор `~A()` — один, без параметров; при выходе из блока, `delete`, конце программы; порядок обратный созданию. Освобождает ресурсы.
- Время жизни: автоматический (блок), динамический (new…delete), статический (вся программа), временный (выражение).
- Book (лаба): private-поля, 2 конструктора, геттеры `const`, сеттер года с проверкой 1000…2026, checkOut/returnBook с проверкой состояния.

14. Пользовательский класс: свойства

- Свойство — характеристика объекта с контролируемым доступом: private-поле + getter (чтение) + setter (запись с проверкой).
- Плюсы: целостность данных, гибкость реализации, read-only/write-only/вычисляемые свойства, удобная отладка.
- Геттеры делай `const`; в конструкторе используй сеттеры (проверка в одном месте).
- Интерфейс в `.h` (с защитой `#ifndef X #define X … #endif`), реализация в `.cpp` (`Класс::метод`).
- C++ Builder: `__property тип Имя = { read = FПоле / GetМетод, write = SetМетод };` — работает как переменная; без write — только чтение. Расширение Borland.
- `__published` — свойство видно в Object Inspector и сохраняется в .dfm; `default = …`. VCL-свойства (Width, Caption) устроены так же: сеттер ещё и перерисовывает.

15. Создание пользовательского компонента

- Компонент — класс-наследник `TComponent`: есть в палитре, настраивается в Object Inspector, сохраняется в .dfm. Класс → объект → компонент.
- Предок: сам компонент (TEdit, TLabel, TButton), TCustomXxx, TWinControl (окно, фокус), TGraphicControl (рисование), TComponent (невизуальный).
- Мастер: **Component** → **New Component** (Ancestor, Class Name `TMy…`, Palette Page, Unit).
- Свойства: `FПоле` + `SetПоле` + `__property` в `__published`. Методы: public или переопределение (`DYNAMIC void __fastcall Click()`) с вызовом предка.
- Событие: `TNotifyEvent FOnX;` + `__property TNotifyEvent OnX = {read=FOnX, write=FOnX};` + `if (FOnX) FOnX(this);`.
- Регистрация: `namespace Unitname { void __fastcall PACKAGE Register() { TComponentClass c[1] = {__classid(TMy)}; RegisterComponents("Samples", c, 0); } }`.
- Установка: **Component** → **Install Component** → пакет `dclusr.bpk` → значок в палитре. Тест до установки: `new TMy(this)` + `Parent = this`.

Все задачи курса

Здесь собраны все задачи из лекций 1–15. Сначала попробуй решить сам, потом смотри решение.

Лекция 1. Парадигмы, алгоритмы, синтаксис C++, структура программы, типы данных, указатели

Задача 1.1. Перевод температуры лёгкая

Дана температура в градусах Цельсия `c = 36.6`. Выведи её в Фаренгейтах по формуле F = C·9/5 + 32.

Решение

```C++
double c = 36.6;
double f = c * 9 / 5 + 32;
cout << f;
```

> **Вывод**
> 97.88

Обрати внимание: если бы `c` была `int` и мы написали `9/5`, получили бы 1 (целое деление!). Поэтому делим вещественное `c * 9` на 5.

Задача 1.2. Цифры трёхзначного числа лёгкая

Дано трёхзначное число `n = 472`. Выведи его цифры через пробел и их сумму.

Решение

```C++
int n = 472;
int a = n / 100;        // сотни
int b = n / 10 % 10;    // десятки
int c = n % 10;         // единицы
cout << a << " " << b << " " << c << " sum=" << a + b + c;
```

> **Вывод**
> 4 7 2 sum=13

Приём: `n % 10` — последняя цифра, `n / 10` — «отрезать» последнюю цифру.

Задача 1.3. Обмен значений средняя

Поменяй местами значения `a = 3` и `b = 8`: (1) через третью переменную; (2) без неё.

Решение

```C++
int a = 3, b = 8, t;
t = a; a = b; b = t;            // способ 1
cout << a << " " << b << endl;
a = a + b; b = a - b; a = a - b; // способ 2 (арифметика)
cout << a << " " << b;
```

Вывод
```
8 3
3 8
```

Задача 1.4. Указатели средняя

Объяви `int x = 7, y = 3;` и указатель `p`. Через указатель: сначала увеличь x в 2 раза, потом переставь p на y и прибавь к y 10. Выведи x и y.

Решение

```C++
int x = 7, y = 3;
int *p = &x;
*p = *p * 2;
p = &y;
*p += 10;
cout << x << " " << y;
```

Вывод
```
14 13
```

Задача 1.5. Размер типов и переполнение сложная

Что выведет программа? Объясни каждое число.

```C++
unsigned char u = 255;
u++;
char c = 'A' + 2;
int big = 2147483647;
big++;
cout << (int)u << " " << c << " " << big;
```

Решение

Вывод
```
0 C -2147483648
```

`unsigned char` хранит 0…255, после 255 идёт 0. `'A'` имеет код 65, 65+2 = 67 — это `'C'`. Максимальный `int` плюс 1 переполняется в минимальный.

Лекция 2. Операции и выражения

Задача 2.1. Гипотенуза лёгкая

Даны катеты a = 3, b = 4. Найди гипотенузу и площадь треугольника.

Решение

```C++
double a = 3, b = 4;
double c = sqrt(a * a + b * b);
double s = a * b / 2;
cout << "c=" << c << " S=" << s;
```

Вывод

```
c=5 S=6
```

Задача 2.2. Запись формулы в C++ лёгкая

Запиши в линейном виде и посчитай при x = 2: $y = (x^2 + 3x) / (2\sqrt{x} + 1) + |x - 5|$.

Решение

```C++
double x = 2;
double y = (x * x + 3 * x) / (2 * sqrt(x) + 1) + fabs(x - 5);
printf("%.4f", y);
```

Вывод

```
5.6120
```

Числитель и знаменатель — **в скобках**! Без скобок `x*x + 3*x / 2*sqrt(x) + 1` посчитается совсем другое.

Задача 2.3. Чётность и знак без if средняя

Для n = −7 выведи: «even»/«odd» и «positive»/«negative»/«zero», используя только тернарный оператор.

Решение

```C++
int n = -7;
cout << (n % 2 == 0 ? "even" : "odd") << " "
     << (n > 0 ? "positive" : n < 0 ? "negative" : "zero");
```

Вывод

```
odd negative
```

Внимание: `-7 % 2` = −1, а не 1. Поэтому проверять нечётность через `n % 2 == 1` для отрицательных **нельзя**, а через `n % 2 != 0` или `n % 2 == 0` можно.

Задача 2.4. Пошаговое вычисление средняя

Что выведет программа? Пройди по шагам.

```cpp
int a = 10, b = 3;
a %= b + 1;
b *= a++ + 2;
int c = a > b ? a - b : b - a;
cout << a << " " << b << " " << c;
```

Решение

> **Вывод**
> `3 12 9`

`a %= 4` → a = 2. `b *= a++ + 2` → b = 3·(2 + 2) = 12, затем a = 3. `3 > 12` ложно → c = 12 − 3 = 9.

Задача 2.5. Деньги без потери копеек сложная

Сумма в тенге с копейками хранится как `double sum = 1234.56;`. Раздели её на купюры по 1000 и 100 и выведи остаток в копейках, без ошибок округления.

Решение

```cpp
double sum = 1234.56;
int kop = (int)(sum * 100 + 0.5);   // всё в целые копейки, +0.5 для округления
int k1000 = kop / 100000;
kop %= 100000;
int k100 = kop / 10000;
kop %= 10000;
cout << k1000 << " x1000, " << k100 << " x100, rest " << kop << " kop";
```

> **Вывод**
> `1 x1000, 2 x100, rest 3456 kop`

Фишка: вещественные числа хранятся неточно (1234.56·100 может оказаться 123455.9999…), поэтому переходим к целым с округлением `+0.5`.

Задача 2.6. Биты сложная

Число n = 45. С помощью битовых операций: проверь, нечётное ли оно; умножь на 8; найди остаток от деления на 4; установи бит №1 (`| 2`).

Решение

```cpp
int n = 45;   // 101101
cout << (n & 1) << " " << (n << 3) << " " << (n & 3) << " " << (n | 2);
```

> **Вывод**
> `1 360 1 47`

`n & (2^k - 1)` — это остаток от деления на 2^k.

Лекция 3. Ветвления if, switch, goto и цикл for

Задача 3.1. Максимум из трёх лёгкая

Даны a = 7, b = 12, c = 9. Найди наибольшее с помощью if.

Решение

```cpp
int a = 7, b = 12, c = 9;
int mx = a;
if (b > mx) mx = b;
if (c > mx) mx = c;
cout << mx;
```

> **Вывод**
> `12`

Приём «предполагаем, что максимум — первый, и обновляем» работает для любого количества чисел.

Задача 3.2. Високосный год лёгкая

Год високосный, если делится на 4 и не делится на 100, либо делится на 400. Проверь 1900, 2000, 2024.

Решение

```cpp
int y[3] = {1900, 2000, 2024};
for (int i = 0; i < 3; i++) {
    int g = y[i];
    if ((g % 4 == 0 && g % 100 != 0) || g % 400 == 0) cout << g << " leap" << endl;
    else cout << g << " not leap" << endl;
}
```

Вывод

```
1900 not leap
2000 leap
2024 leap
```

Задача 3.3. Квадратное уравнение средняя

Реши $ax^2 + bx + c = 0$ для a = 1, b = −5, c = 6 с разбором всех случаев дискриминанта.

Решение

```cpp
double a = 1, b = -5, c = 6;
double d = b * b - 4 * a * c;
if (d > 0) {
    double x1 = (-b + sqrt(d)) / (2 * a);
    double x2 = (-b - sqrt(d)) / (2 * a);
    cout << "x1=" << x1 << " x2=" << x2;
} else if (d == 0) {
    cout << "x=" << -b / (2 * a);
} else {
    cout << "no real roots";
}
```

Вывод

```
x1=3 x2=2
```

Обрати внимание на скобки `(2 * a)`: `-b / 2 * a` — это (−b/2)·a.

Задача 3.4. Меню через switch средняя

По номеру месяца m = 4 выведи время года и число дней (февраль — 28).

Решение

```cpp
int m = 4;
switch (m) {
    case 12: case 1: case 2: cout << "winter"; break;
    case 3: case 4: case 5: cout << "spring"; break;
    case 6: case 7: case 8: cout << "summer"; break;
    case 9: case 10: case 11: cout << "autumn"; break;
    default: cout << "error";
}
int days;
switch (m) {
    case 2: days = 28; break;
    case 4: case 6: case 9: case 11: days = 30; break;
    default: days = 31;
}
cout << " " << days;
```

Вывод

```
spring 30
```

Задача 3.5. Сумма ряда средняя

Найди S = 1 + 1/2 + 1/3 + … + 1/10 и S2 = 1 − 1/2 + 1/3 − … (10 слагаемых).

Решение

```cpp
double s = 0, s2 = 0;
int sign = 1;
for (int i = 1; i <= 10; i++) {
    s += 1.0 / i;          // 1.0 — иначе 1/i = 0 для i > 1!
    s2 += sign * 1.0 / i;
    sign = -sign;
}
printf("%.4f %.4f", s, s2);
```

Вывод

```
2.9290 0.6456
```

Задача 3.6. Простые числа и делители сложная

Выведи все простые числа от 2 до 30 и количество делителей числа 36.

Решение

```C++
for (int n = 2; n <= 30; n++) {
    int prime = 1;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) { prime = 0; break; }
    if (prime) cout << n << " ";
}
int k = 0;
for (int d = 1; d <= 36; d++) if (36 % d == 0) k++;
cout << endl << k;
```

Вывод

```
2 3 5 7 11 13 17 19 23 29
9
```

Фишка: делители достаточно проверять до √n (`d * d <= n`), это намного быстрее.

Задача 3.7. Таблица истинности сложная

Выведи таблицу для `(a && !b) || c` по всем 8 комбинациям a, b, c ∈ {0, 1}.

Решение

```C++
for (int a = 0; a <= 1; a++)
    for (int b = 0; b <= 1; b++)
        for (int c = 0; c <= 1; c++)
            cout << a << b << c << " " << ((a && !b) || c) << endl;
```

Вывод

```
000 0
001 1
010 0
011 1
100 1
101 1
110 0
111 1
```

Лекция 4. Циклы while и do...while, break и continue, одномерные и двумерные массивы

Задача 4.1. Ввод до нуля лёгкая

Пользователь вводит числа, пока не введёт 0. Посчитай сумму и количество (без нуля). Напиши через do...while.

Решение

```
int x, s = 0, k = 0;
do {
    cin >> x;
    if (x != 0) { s += x; k++; }
} while (x != 0);
cout << s << " " << k;

Ввод: 5 3 -2 0   →   Вывод: 6 3
```

Задача 4.2. Переворот числа лёгкая

Получи из n = 12345 число 54321.

Решение

```C++
int n = 12345, r = 0;
while (n > 0) {
    r = r * 10 + n % 10;
    n /= 10;
}
cout << r;
```

> **Вывод**
> 54321

Так же проверяется **палиндром**: число равно своему перевёртышу.

Задача 4.3. Второй максимум и среднее положительных средняя

В массиве {4, −2, 9, 9, 0, 7, −5, 3} найди второе по величине **различное** значение и среднее положительных.

Решение

```cpp
int a[8] = {4, -2, 9, 9, 0, 7, -5, 3};
int m1 = a[0], m2 = -32768;
int sp = 0, kp = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] > m1) { m2 = m1; m1 = a[i]; }
    else if (a[i] < m1 && a[i] > m2) m2 = a[i];
    if (a[i] > 0) { sp += a[i]; kp++; }
}
cout << m1 << " " << m2 << " " << (double)sp / kp;
```

Вывод

```
9 7 6.4
```

Задача 4.4. Удаление элемента средняя

Удали из массива {10, 20, 30, 40, 50} элемент с индексом 1 (сдвигом влево) и выведи результат.

Решение

```cpp
int a[5] = {10, 20, 30, 40, 50};
int n = 5, del = 1;
for (int i = del; i < n - 1; i++) a[i] = a[i + 1];
n--;
for (int i = 0; i < n; i++) cout << a[i] << " ";
```

Вывод

```
10 30 40 50
```

Задача 4.5. Таблица умножения в матрицу и след сложная

Заполни матрицу 4×4 по правилу a[i][j] = (i+1)·(j+1), выведи её и найди сумму побочной диагонали и сумму элементов выше главной.

Решение

```C++
const int N = 4;
int a[N][N];
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        a[i][j] = (i + 1) * (j + 1);
int side = 0, up = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        cout << a[i][j] << "\t";
        if (i + j == N - 1) side += a[i][j];
        if (j > i) up += a[i][j];
    }
    cout << endl;
}
cout << side << " " << up;
```

Вывод

```
1       2       3       4
2       4       6       8
3       6       9       12
4       8       12      16
20 35
```

Задача 4.6. Частота элементов сложная

Для массива {3, 1, 3, 2, 1, 3} выведи, сколько раз встречается каждое значение (без повторов в выводе).

Решение

```C++
int a[6] = {3, 1, 3, 2, 1, 3};
for (int i = 0; i < 6; i++) {
    int seen = 0;
    for (int j = 0; j < i; j++) if (a[j] == a[i]) { seen = 1; break; }
    if (seen) continue;                 // уже считали
    int c = 0;
    for (int j = 0; j < 6; j++) if (a[j] == a[i]) c++;
    cout << a[i] << ":" << c << " ";
}
```

Вывод

```
3:3 1:2 2:1
```

Лекция 5. Функции: объявление, параметры по значению и по ссылке, const, значения по умолчанию, inline, массивы, рекурсия

Задача 5.1. Функция «простое ли число» лёгкая

Напиши `bool isPrime(int n)` и выведи простые числа от 1 до 20.

Решение

```C++
bool isPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return false;   // return сразу выходит из функции
    return true;
}
//---
for (int i = 1; i <= 20; i++) if (isPrime(i)) cout << i << " ";
```

Вывод

```
2 3 5 7 11 13 17 19
```

Задача 5.2. Несколько результатов через ссылки средняя

Напиши функцию, которая по массиву возвращает (через ссылки) минимум, максимум и среднее.

Решение

```C++
void stats(const int a[], int n, int &mn, int &mx, double &avg)
{
    mn = mx = a[0];
    int s = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
        s += a[i];
    }
    avg = (double)s / n;
}
//---
int a[6] = {7, -2, 10, 4, 4, 1};
int lo, hi; double av;
stats(a, 6, lo, hi, av);
cout << lo << " " << hi << " " << av;
```

Вывод

```
-2 10 4
```

Задача 5.3. Ввод, вывод и обработка массива функциями (лабораторная из силлабуса) средняя

Сделай три функции: заполнение массива, вывод, и функцию, которая возвращает количество элементов больше среднего. Исходный массив изменять разрешено только функции заполнения.

Решение

```C++
void fill(int a[], int n)              // меняет массив — без const
{
    for (int i = 0; i < n; i++) a[i] = (i * 7 + 3) % 10;   // вместо cin, для примера
}
void show(const int a[], int n)          // const: менять запрещено
{
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}
int aboveAvg(const int a[], int n)
{
    double s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    double avg = s / n;
    int k = 0;
    for (int i = 0; i < n; i++) if (a[i] > avg) k++;
    return k;
}
//---
int a[8];
fill(a, 8);
show(a, 8);
cout << aboveAvg(a, 8);
```

Вывод

```
3 0 7 4 1 8 5 2
4
```

Задача 5.4. Рекурсия: степень, НОД, двоичная запись сложная

Напиши рекурсивные функции: быстрое возведение в степень, НОД (Евклид) и вывод числа в двоичной системе.

Решение

```C++
long power(long x, int n)
{
    if (n == 0) return 1;
    long h = power(x, n / 2);
    return n % 2 == 0 ? h * h : h * h * x;
}
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
void bin(int n)
{
    if (n > 1) bin(n / 2);       // сначала старшие разряды
    cout << n % 2;
}
//---
cout << power(3, 13) << " " << gcd(84, 36) << " ";
bin(37);
```

Вывод
```
1594323 12 100101
```

Задача 5.5. По значению или по ссылке? сложная

Что выведет программа? Объясни каждую строку.

```C++
void f(int a, int &b, int *c)
{
    a += 10; b += 10; *c += 10;
}
//---
int x = 1, y = 2, z = 3;
f(x, y, &z);
f(y, z, &x);
cout << x << " " << y << " " << z;
```

Решение

Вывод
```
11 12 23
```

1-й вызов: a — копия x (x не меняется), y += 10 → 12, z += 10 → 13. 2-й вызов: a — копия y (не меняется), z += 10 → 23, x += 10 → 11.

Лекция 6. Тип char и строки: массивы символов, string.h и класс string

Задача 6.1. Подсчёт символа лёгкая

Сколько раз в строке "banana bandana" встречается буква 'a'? Замени все 'a' на '*'.

Решение

```C++
char s[] = "banana bandana";
int k = 0;
for (int i = 0; s[i] != '\0'; i++)
    if (s[i] == 'a') { k++; s[i] = '*'; }
cout << k << " " << s;
```

Вывод

```
6 b*n*n* b*nd*n*
```

Задача 6.2. Палиндром лёгкая

Проверь, является ли строка "level" палиндромом (без strrev).

Решение

```C++
char s[] = "level";
int n = strlen(s), ok = 1;
for (int i = 0; i < n / 2; i++)
    if (s[i] != s[n - 1 - i]) { ok = 0; break; }
cout << (ok ? "palindrome" : "not palindrome");
```

Вывод

```
palindrome
```

Задача 6.3. Самое длинное слово средняя

Найди самое длинное слово в строке "C plus plus is a powerful language".

Решение

```cpp
char s[] = "C plus plus is a powerful language";
int best = 0, bestStart = 0, start = 0;
int n = strlen(s);
for (int i = 0; i <= n; i++) {
    if (s[i] == ' ' || s[i] == '\0') {
        int len = i - start;
        if (len > best) { best = len; bestStart = start; }
        start = i + 1;
    }
}
for (int i = bestStart; i < bestStart + best; i++) cout << s[i];
cout << " " << best;
```

Вывод

```
powerful 8
```

Фишка: цикл до `i <= n`, чтобы обработать и последнее слово (на '\0').

Задача 6.4. Строка в число и обратно средняя

Без `atoi` преобразуй строку "40721" в число и прибавь 1. Потом преобразуй число 905 в строку.

Решение

```cpp
char s[] = "40721";
long v = 0;
for (int i = 0; s[i]; i++) v = v * 10 + (s[i] - '0');
cout << v + 1 << " ";
int n = 905;
char buf[12];
int len = 0;
do { buf[len++] = '0' + n % 10; n /= 10; } while (n > 0);
buf[len] = '\0';
strrev(buf);
cout << buf;
```

Вывод

```
40722 905
```

Задача 6.5. Двумя способами (лабораторная): шифр Цезаря средняя

Зашифруй "Hello World" сдвигом на 3 (z → c), сначала массивом char, потом через класс string.

Решение

```C++
#include <iostream>
#include <string>
using namespace std;
char shift(char c, int k)
{
    if (c >= 'a' && c <= 'z') return 'a' + (c - 'a' + k) % 26;
    if (c >= 'A' && c <= 'Z') return 'A' + (c - 'A' + k) % 26;
    return c;
}
int main()
{
    char a[] = "Hello World";
    for (int i = 0; a[i]; i++) a[i] = shift(a[i], 3);
    string s = "Hello World";
    for (int i = 0; i < (int)s.length(); i++) s[i] = shift(s[i], 3);
    cout << a << endl << s << endl << (s == a ? "same" : "diff");
    return 0;
}
```

Вывод

```
Khoor Zruog
Khoor Zruog
same
```

Задача 6.6. Слова в обратном порядке сложная

Выведи слова строки "one two three four" в обратном порядке: "four three two one".

Решение

```C++
char s[] = "one two three four";
int end = strlen(s);
for (int i = end - 1; i >= -1; i--) {
    if (i == -1 || s[i] == ' ') {
        for (int j = i + 1; j < end; j++) cout << s[j];
        if (i != -1) cout << " ";
        end = i;
    }
}
```

Вывод

```
four three two one
```

Лекция 7. Ключевые понятия и принципы объектно-ориентированного программирования

Задача 7.1. Класс Rectangle лёгкая

Опиши класс прямоугольника с закрытыми сторонами, методом set с проверкой (стороны > 0), методами area и perimeter.

Решение

```C++
class Rectangle {
    double w, h;
public:
    void set(double a, double b)
    {
        if (a > 0 && b > 0) { w = a; h = b; }
        else { w = h = 1; }
    }
    double area() { return w * h; }
    double perimeter() { return 2 * (w + h); }
};
//---
Rectangle r;
r.set(3, 4.5);
cout << r.area() << " " << r.perimeter() << endl;
r.set(-2, 5);
cout << r.area();
```

Вывод

```
13.5 15
1
```

Задача 7.2. Лабораторная из силлабуса целиком: Animal → Dog → Poodle, Cat средняя

Собери в одной программе все пункты задания: Dog с name и breed, приватным name и get/set, `performAction` с приватными walk/fetch, Poodle с coatType и doesGrooming, Cat и полиморфный `makeSound`.

Решение

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class Animal {
public:
    virtual void makeSound() { cout << "Some sound" << endl; }
    virtual ~Animal() {}
};

class Dog : public Animal {
    string name;                                  // private
    void walk()  { cout << name << " is walking" << endl; }
    void fetch() { cout << name << " fetches the ball" << endl; }
protected:
    string breed;
public:
    Dog(string n, string b) { name = n; breed = b; }
    string getName() { return name; }
    void setName(string n) { if (n != "") name = n; }
    void bark() { cout << name << " (" << breed << "): Woof!" << endl; }
    void performAction() { walk(); fetch(); }
    void makeSound() { cout << "Bark" << endl; }
};

class Poodle : public Dog {
    string coatType;
public:
    Poodle(string n, string c) : Dog(n, "Poodle") { coatType = c; }
    void doesGrooming() { cout << getName() << " has " << coatType << " coat groomed" <<
endl; }
};

class Cat : public Animal {
public:
    void makeSound() { cout << "Meow" << endl; }
};

int main()
{
    Dog myDog("Buddy", "Golden Retriever"), other("Max", "Beagle");
    myDog.bark(); other.bark();
    myDog.setName("Buddy Jr");
    cout << myDog.getName() << endl;
    myDog.performAction();
    Poodle p("Fifi", "curly");
    p.bark(); p.doesGrooming();
    Cat c;
    Animal *list[3] = { &myDog, &c, &p };
    for (int i = 0; i < 3; i++) list[i]->makeSound();
    return 0;
}
```

Вывод

```
Buddy (Golden Retriever): Woof!
Max (Beagle): Woof!
Buddy Jr
Buddy Jr is walking
Buddy Jr fetches the ball
Fifi (Poodle): Woof!
Fifi has curly coat groomed
Bark
Meow
Bark
```

Обрати внимание: Poodle не переопределил `makeSound`, поэтому вызывается версия Dog («Bark»). А `name` в Dog — private, поэтому Poodle берёт имя через `getName()`.

Задача 7.3. Определи принцип сложная

Для каждой ситуации назови принцип ООП: (1) поле `salary` закрыто, изменить можно только через `raise(percent)`; (2) `Shape *s` — для круга `s->area()` считает πr^2, для квадрата a^2; (3) `Student` и `Teacher` получают `name` и `print()` от `Person`; (4) пользователь вызывает `car.start()`, не зная про свечи и топливный насос; (5) две функции `print(int)` и `print(string)`.

Решение

1 — инкапсуляция; 2 — полиморфизм (динамический, через virtual); 3 — наследование; 4 — абстракция; 5 — полиморфизм (статический, перегрузка).

Лекция 8. Классы и объекты в C++

Задача 8.1. Класс Time лёгкая

Класс с полями часов и минут (private), методами `set(h, m)` с проверкой, `addMinutes(k)` и `print()` в формате ЧЧ:ММ.

Решение

```C++
class Time {
    int h, m;
public:
    void set(int hh, int mm)
    {
        h = (hh >= 0 && hh < 24) ? hh : 0;
        m = (mm >= 0 && mm < 60) ? mm : 0;
    }
    void addMinutes(int k)
    {
        int t = (h * 60 + m + k) % (24 * 60);
        h = t / 60; m = t % 60;
    }
    void print() const
    {
        if (h < 10) cout << 0;
        cout << h << ":";
        if (m < 10) cout << 0;
        cout << m << endl;
    }
};
//---
Time t;
t.set(22, 45);
t.print();
t.addMinutes(90);
t.print();
```

Вывод

```
22:45
00:15
```

Задача 8.2. Счёт в банке со static-счётчиком средняя

Класс Account: номер (выдаётся автоматически), баланс, методы deposit, withdraw (нельзя уйти в минус, вернуть false), transfer на другой счёт; static-поле — сколько счетов создано.

Решение

```C++
class Account {
    int number;
    double balance;
    static int created;
public:
    Account() { number = ++created; balance = 0; }
    void deposit(double s) { if (s > 0) balance += s; }
    bool withdraw(double s)
    {
        if (s <= 0 || s > balance) return false;
        balance -= s;
        return true;
    }
    bool transfer(Account &to, double s)
    {
        if (!withdraw(s)) return false;
        to.deposit(s);
        return true;
    }
    void print() const { cout << "#" << number << ": " << balance << endl; }
    static int count() { return created; }
};
int Account::created = 0;
//---
Account a, b;
a.deposit(1000);
cout << a.transfer(b, 300) << a.transfer(b, 5000) << endl;
a.print(); b.print();
cout << Account::count();
```

Вывод

```
10
#1: 700
#2: 300
2
```

Задача 8.3. Массив объектов в динамической памяти сложная

Создай через `new` массив из n = 4 объектов Student (имя, балл), заполни, найди лучшего и средний балл, освободи память.

Решение

```C++
class Student {
public:
    char name[20];
    int score;
};
//---
int n = 4;
Student *g = new Student[n];
const char *names[4] = {"Ali", "Bota", "Erlan", "Zhanna"};
int scores[4] = {78, 95, 64, 88};
for (int i = 0; i < n; i++) { strcpy(g[i].name, names[i]); g[i].score = scores[i]; }
int best = 0, sum = 0;
for (int i = 0; i < n; i++) {
    sum += g[i].score;
    if (g[i].score > g[best].score) best = i;
}
cout << g[best].name << " " << (double)sum / n;
delete [] g;
```

> **Вывод**
> Bota 81.25

Лекция 9. Интегрированная среда разработки Borland C++ Builder

Задача 9.1. Найди и классифицируй ошибки лёгкая

Сколько ошибок и каких видов в этом фрагменте?

```
int main()
{
    int n = 5
    int s;
    for (int i = 1; i < n; i++) s += i;
    cout << "Sum 1..5 = " << S;
}
```

Решение

1. `int n = 5` без `;` — **синтаксическая**.
2. `S` вместо `s` — **семантическая** (необъявленный идентификатор, регистр важен).
3. `s` не инициализирована (`int s = 0;`) — **логическая**: результат будет мусором.
4. `i < n` вместо `i <= n` — **логическая**: сумма до 4, а не до 5.

Синтаксическую и семантическую найдёт компилятор, логические — только тестирование и отладка.

Задача 9.2. Пошаговая отладка «в уме» средняя

Поставь breakpoint на строку цикла и нажимай F8. Запиши, какие значения покажет Watch для `i`, `p` после каждого шага.

```cpp
int p = 1;
for (int i = 1; i <= 4; i++)
    p = p * 2 + i;
cout << p;
```

Решение

> **Вывод**
> 42

шаг	i	p после шага
1	1	1·2 + 1 = 3
2	2	3·2 + 2 = 8
3	3	8·2 + 3 = 19
4	4	19·2 + 4 = 42

После 4-го шага i становится 5, условие `i <= 4` ложно, цикл заканчивается, и печатается 42. Именно так отладчик помогает: видно значение на каждом шаге, и сразу понятно, где оно «поехало».

Лекция 10. Создание проекта. Работа с формами. Дизайнер форм

Задача 10.1. Конвертер валют лёгкая

Форма: Edit (сумма в тенге), ComboBox (USD, EUR, RUB), кнопка «Convert», Label. Курсы: 1 USD = 500, 1 EUR = 540, 1 RUB = 6. Напиши обработчик.

Решение

```cpp
void __fastcall TForm1::btnConvertClick(TObject *Sender)
{
    double kzt = StrToFloat(edtSum->Text);
    double rate;
    switch (cbCur->ItemIndex) {          // 0 – USD, 1 – EUR, 2 – RUB
        case 0: rate = 500; break;
        case 1: rate = 540; break;
        case 2: rate = 6; break;
        default: ShowMessage("Choose a currency"); return;
    }
    lblRes->Caption = FormatFloat("0.00", kzt / rate) + " " + cbCur->Text;
}
```

`ItemIndex` = −1, если ничего не выбрано, поэтому есть `default`.

Задача 10.2. Список дел средняя

Edit, кнопки «Add», «Delete», «Clear» и ListBox. Пустые строки не добавлять, удалять выделенную строку.

Решение

C++ BUILDER

```
void __fastcall TForm1::btnAddClick(TObject *Sender)
{
    if (edtTask->Text.Trim() == "") return;   // Trim убирает пробелы
    lbTasks->Items->Add(edtTask->Text);
    edtTask->Clear();
    edtTask->SetFocus();                      // курсор снова в поле ввода
}
void __fastcall TForm1::btnDeleteClick(TObject *Sender)
{
    if (lbTasks->ItemIndex >= 0)
        lbTasks->Items->Delete(lbTasks->ItemIndex);
}
void __fastcall TForm1::btnClearClick(TObject *Sender)
{
    lbTasks->Items->Clear();
}
```

Задача 10.3. Две формы: передача данных средняя

На Form1 — Edit с именем и кнопка. Form2 открывается модально и показывает в Label «Welcome, имя». По кнопке OK на Form2 возвращает mrOk, и Form1 пишет «Confirmed».

Решение

C++ BUILDER

```
// Unit1.cpp
#include "Unit2.h"
void __fastcall TForm1::btnNextClick(TObject *Sender)
{
    Form2->lblWelcome->Caption = "Welcome, " + edtName->Text;   // доступ к компоненту
другой формы
    if (Form2->ShowModal() == mrOk)
        lblStatus->Caption = "Confirmed";
}
// На Form2 у кнопки btnOk в инспекторе: ModalResult = mrOk
```

Лекция 11. Работа с формой: свойства, методы и события

Задача 11.1. Убегающая кнопка лёгкая

Кнопка «Catch me» при наведении мыши (`OnMouseMove`) прыгает в случайное место формы, не выходя за её границы.

Решение

C++ BUILDER
```
void __fastcall TForm1::btnCatchMouseMove(TObject *Sender, TShiftState Shift, int X, int Y)
{
    btnCatch->Left = random(ClientWidth - btnCatch->Width);
    btnCatch->Top = random(ClientHeight - btnCatch->Height);
}
void __fastcall TForm1::FormCreate(TObject *Sender)
{
    randomize();
}
```

`ClientWidth – Width` — чтобы кнопка не вылезла за правый край.

Задача 11.2. Настройка формы из программы средняя

По кнопкам: «Center» — окно в центр экрана, «Max/Normal» — переключать развёрнутое и обычное состояние, «On top» — CheckBox «поверх всех окон», TrackBar — прозрачность.

Решение

C++ BUILDER
```
void __fastcall TForm1::btnCenterClick(TObject *Sender)
{
    Left = (Screen->Width - Width) / 2;
    Top = (Screen->Height - Height) / 2;
}
void __fastcall TForm1::btnMaxClick(TObject *Sender)
{
    WindowState = (WindowState == wsMaximized) ? wsNormal : wsMaximized;
}
void __fastcall TForm1::chkTopClick(TObject *Sender)
{
    FormStyle = chkTop->Checked ? fsStayOnTop : fsNormal;
}
void __fastcall TForm1::tbAlphaChange(TObject *Sender)
{
    AlphaBlend = true;
    AlphaBlendValue = tbAlpha->Position;     // TrackBar: Min = 50, Max = 255
}
```

Задача 11.3. Калькулятор с одним обработчиком средняя

Два Edit, четыре кнопки `+ − × ÷` с Tag 1…4, один обработчик `OpClick`, результат в Label. Защита от деления на 0 и от неверного ввода.

Решение

C++ BUILDER

```
void __fastcall TForm1::OpClick(TObject *Sender)
{
    double a, b, r;
    try {
        a = StrToFloat(edtA->Text);
        b = StrToFloat(edtB->Text);
    }
    catch (...) { ShowMessage("Enter numbers"); return; }
    switch (((TComponent*)Sender)->Tag) {
        case 1: r = a + b; break;
        case 2: r = a - b; break;
        case 3: r = a * b; break;
        case 4:
            if (b == 0) { ShowMessage("Division by zero"); return; }
            r = a / b; break;
    }
    lblRes->Caption = FloatToStr(r);
}
```

Лекция 12. Иерархия классов: наследование, переопределение, виртуальные функции

Задача 12.1. Транспорт лёгкая

Базовый класс Vehicle (скорость, метод `move()` печатает скорость), производные Car и Bicycle со своими полями. Создай объекты и вызови методы.

Решение

```C++
class Vehicle {
protected:
    int speed;
public:
    Vehicle(int s) : speed(s) { }
    void move() { cout << "moving at " << speed << " km/h" << endl; }
};
class Car : public Vehicle {
    int doors;
public:
    Car(int s, int d) : Vehicle(s), doors(d) { }
    void info() { cout << "Car with " << doors << " doors, "; move(); }
};
class Bicycle : public Vehicle {
public:
    Bicycle() : Vehicle(20) { }
    void ring() { cout << "Ring! "; move(); }
};
//---
Car c(120, 4);
Bicycle b;
c.info();
b.ring();
```

Вывод

```
Car with 4 doors, moving at 120 km/h
Ring! moving at 20 km/h
```

Задача 12.2. Зарплата: полиморфизм средняя

Абстрактный класс Employee с чисто виртуальным `salary()`. Наследники: Fixed (оклад), Hourly (часы × ставка), Manager (оклад + 10 % за каждого подчинённого). Посчитай фонд оплаты для массива указателей.

Решение

```C++
class Employee {
public:
    virtual double salary() = 0;
    virtual ~Employee() { }
};
class Fixed : public Employee {
    double s;
public:
    Fixed(double v) : s(v) { }
    double salary() { return s; }
};
class Hourly : public Employee {
    int h; double rate;
public:
    Hourly(int hh, double r) : h(hh), rate(r) { }
    double salary() { return h * rate; }
};
class Manager : public Fixed {
    int team;
public:
    Manager(double v, int t) : Fixed(v), team(t) { }
    double salary() { return Fixed::salary() * (1 + 0.1 * team); }
};
//---
Employee *staff[3] = { new Fixed(300000), new Hourly(120, 2500), new Manager(400000, 3) };
double sum = 0;
for (int i = 0; i < 3; i++) { cout << staff[i]->salary() << endl; sum += staff[i]-
>salary(); }
cout << "total " << sum;
for (int i = 0; i < 3; i++) delete staff[i];
```

Вывод

```
300000
300000
520000
total 1.12e+06
```

Фишка: `Manager::salary()` использует версию базы `Fixed::salary()` и добавляет надбавку. Большие double cout печатает в экспоненциальной форме; для красивого вывода используй `printf("%.0f", sum)`.

Задача 12.3. Порядок вызовов сложная

Что выведет программа?

```C++
class A {
public:
    A() { cout << "A"; }
    virtual ~A() { cout << "a"; }
    virtual void f() { cout << "1"; }
};
class B : public A {
public:
    B() { cout << "B"; }
    ~B() { cout << "b"; }
    void f() { cout << "2"; }
};
class C : public B {
public:
    C() { cout << "C"; }
    ~C() { cout << "c"; }
};
//---
A *p = new C;
p->f();
delete p;
```

Решение

> **Вывод**
> `ABC2cba`

Конструкторы от базы к наследнику: A, B, C. `f` виртуальная; в C она не переопределена, поэтому берётся ближайшая — из B: «2». Деструктор виртуальный, поэтому удаляется весь объект: c, b, a.

Лекция 13. Пользовательский класс. Конструктор и деструктор

Задача 13.1. Класс Student с конструкторами лёгкая

Конструктор по умолчанию («NoName», курс 1), с параметрами (имя, курс 1–4, иначе 1) и деструктор, который печатает «bye, имя».

Решение

```C++
class Student {
    char name[20];
    int course;
public:
    Student() : course(1) { strcpy(name, "NoName"); }
    Student(const char *n, int c) { strcpy(name, n); course = (c >= 1 && c <= 4) ? c : 1; }
    ~Student() { cout << "bye, " << name << endl; }
    void print() { cout << name << " (" << course << ")" << endl; }
};
//---
Student a;
Student b("Alikhan", 7);
a.print();
b.print();
```

Вывод

```
NoName (1)
Alikhan (1)
bye, Alikhan
bye, NoName
```

Деструкторы — в обратном порядке создания: сначала b, потом a.

Задача 13.2. Стек на динамическом массиве средняя

Класс Stack: конструктор с ёмкостью (new), push, pop, isEmpty, деструктор (delete []). Используй для переворота последовательности 1…5.

Решение

```C++
class Stack {
    int *buf;
    int cap, top;
public:
    Stack(int c) : cap(c), top(0) { buf = new int[cap]; }
    ~Stack() { delete [] buf; }
    bool push(int v) { if (top == cap) return false; buf[top++] = v; return true; }
    int pop() { return buf[--top]; }
    bool isEmpty() const { return top == 0; }
};
//---
Stack s(10);
for (int i = 1; i <= 5; i++) s.push(i);
while (!s.isEmpty()) cout << s.pop() << " ";
```

Вывод

```
5 4 3 2 1
```

Задача 13.3. Посчитай вызовы сложная

Сколько раз и в каком порядке сработают конструкторы и деструкторы?

```cpp
class T {
    int n;
public:
    T(int v) : n(v) { cout << "C" << n << " "; }
    T(const T &o) : n(o.n * 10) { cout << "K" << n << " "; }
    ~T() { cout << "D" << n << " "; }
};
void f(T x) { cout << "f "; }
//---
T a(1);
f(a);
T *p = new T(2);
{
    T b = a;
}
delete p;
cout << "end ";
```

Решение

> **Вывод**
> `C1 K10 f D10 C2 K10 D10 D2 end D1`

`f(a)` — копия по значению (K10), она уничтожается при выходе из f (D10). `new T(2)` → C2. В блоке `T b = a` → K10, конец блока → D10. `delete p` → D2. После `end` заканчивается main, и уничтожается a → D1.

Лекция 14. Пользовательский класс: свойства

Задача 14.1. Банковский счёт со свойствами лёгкая

Класс Account: баланс только для чтения, владельца можно менять (не пустая строка), методы deposit/withdraw с проверками.

Решение

```C++
class Account {
    char owner[30];
    double balance;
public:
    Account(const char *o) : balance(0) { strcpy(owner, "unknown"); setOwner(o); }
    const char *getOwner() const { return owner; }
    void setOwner(const char *o) { if (strlen(o) > 0 && strlen(o) < 30) strcpy(owner, o); }
    double getBalance() const { return balance; }          // только чтение: сеттера нет
    bool deposit(double s) { if (s <= 0) return false; balance += s; return true; }
    bool withdraw(double s) { if (s <= 0 || s > balance) return false; balance -= s; return
true; }
};
//---
Account a("Dana");
a.deposit(5000);
cout << a.withdraw(7000) << a.withdraw(1500) << " ";
a.setOwner("");
cout << a.getOwner() << " " << a.getBalance();
```

Вывод

```
01 Dana 3500
```

Задача 14.2. Свойства через __property средняя

Класс TCircle: свойство Radius (запись с проверкой > 0), свойства только для чтения Diameter, Area. Покажи изменение радиуса и попытку записать отрицательный.

Решение

```C++
class TCircle {
    double FRadius;
    void SetRadius(double r) { if (r > 0) FRadius = r; else cout << "radius must be > 0" << endl; }
    double GetDiameter() { return 2 * FRadius; }
    double GetArea() { return 3.14159 * FRadius * FRadius; }
public:
    TCircle(double r = 1) : FRadius(1) { SetRadius(r); }
    __property double Radius = { read = FRadius, write = SetRadius };
    __property double Diameter = { read = GetDiameter };
    __property double Area = { read = GetArea };
};
//---
TCircle c(2);
cout << c.Diameter << " " << c.Area << endl;
c.Radius = -3;
c.Radius = c.Radius * 5;
cout << c.Radius << " " << c.Diameter;
```

Вывод

```
4 12.5664
radius must be > 0
10 20
```

Задача 14.3. Класс Date с проверкой сложная

Класс Date: свойства day, month, year с проверкой корректности (учитывая високосный год), метод `next()` — следующий день.

Решение

```C++
class Date {
    int d, m, y;
    static bool leap(int y) { return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; }
    static int daysIn(int m, int y)
    {
        int t[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        return (m == 2 && leap(y)) ? 29 : t[m - 1];
    }
public:
    Date() : d(1), m(1), y(2000) { }
    bool set(int dd, int mm, int yy)
    {
        if (yy < 1 || mm < 1 || mm > 12 || dd < 1 || dd > daysIn(mm, yy)) return false;
        d = dd; m = mm; y = yy;
        return true;
    }
    void next()
    {
        if (++d > daysIn(m, y)) { d = 1; if (++m > 12) { m = 1; y++; } }
    }
    void print() const { cout << d << "." << m << "." << y << " "; }
};
//---
Date x;
cout << x.set(29, 2, 2025) << x.set(28, 2, 2024) << " ";
x.next(); x.print();
x.next(); x.print();
x.set(31, 12, 2026); x.next(); x.print();
```

Вывод

```
01 29.2.2024 1.3.2024 1.1.2027
```

Лекция 15. Создание пользовательского компонента

Задача 15.1. Что выбрать предком? лёгкая

Для каждого компонента выбери предка: (1) кнопка, которая при нажатии спрашивает «Are you sure?»; (2) часы, показывающие время текстом; (3) круглый индикатор, нарисованный на форме; (4) невидимый компонент, который раз в минуту сохраняет данные.

Решение

1 — `TButton` (переопределить `Click`). 2 — `TLabel` (+ внутренний `TTimer` или наследовать `TCustomLabel`). 3 — `TGraphicControl` (переопределить `Paint`, рисовать на `Canvas`). 4 — `TComponent` (невизуальный, внутри таймер).

Задача 15.2. Кнопка с подтверждением средняя

Напиши компонент TConfirmButton (на основе TButton) со свойством `Question` (текст вопроса). При нажатии показывается вопрос; обычный `OnClick` срабатывает только при ответе «Yes».

Решение

C++ BUILDER

```
class PACKAGE TConfirmButton : public TButton
{
private:
    AnsiString FQuestion;
public:
    __fastcall TConfirmButton(TComponent* Owner) : TButton(Owner) { FQuestion = "Are you
sure?"; }
    DYNAMIC void __fastcall Click()
    {
        if (Application->MessageBox(FQuestion.c_str(), "Confirm", MB_YESNO |
MB_ICONQUESTION) == IDYES)
            TButton::Click();              // только тогда — стандартное поведение и
OnClick
    }
__published:
    __property AnsiString Question = { read = FQuestion, write = FQuestion };
};
```

Задача 15.3. Счётчик нажатий с событием сложная

Компонент TCounterButton (на основе TButton): свойство `Count` только для чтения (в разделе public), свойство `Limit` (в __published, чтобы задавать в инспекторе), при каждом нажатии Count++ и Caption = "Clicks: N"; при достижении Limit вызывается событие `OnLimit`, а Count сбрасывается.

Решение

C++ BUILDER

```
class PACKAGE TCounterButton : public TButton
{
private:
    int FCount, FLimit;
    TNotifyEvent FOnLimit;
public:
    __fastcall TCounterButton(TComponent* Owner) : TButton(Owner)
    {
        FCount = 0; FLimit = 10; FOnLimit = NULL;
        Caption = "Clicks: 0";
    }
    DYNAMIC void __fastcall Click()
    {
        FCount++;
        if (FCount >= FLimit) {
            if (FOnLimit) FOnLimit(this);     // событие
            FCount = 0;
        }
        Caption = "Clicks: " + IntToStr(FCount);
        TButton::Click();
    }
    __property int Count = { read = FCount };                // public, только чтение
__published:
    __property int Limit = { read = FLimit, write = FLimit, default = 10 };
    __property TNotifyEvent OnLimit = { read = FOnLimit, write = FOnLimit };
};
```

Итоговые задачи (на всё сразу)

Задача И.1. Журнал оценок средняя

Есть оценки 4 студентов по 3 предметам (матрица). Найди средний балл каждого студента, лучшего студента и предмет с самым низким средним. Сделай через функции.

Решение

```C++
const int S = 4, P = 3;
double rowAvg(const int m[][P], int r)
{
    int s = 0;
    for (int j = 0; j < P; j++) s += m[r][j];
    return (double)s / P;
}
double colAvg(const int m[][P], int c)
{
    int s = 0;
    for (int i = 0; i < S; i++) s += m[i][c];
    return (double)s / S;
}
//---
int g[S][P] = { {85, 90, 78}, {92, 88, 95}, {70, 65, 80}, {88, 94, 91} };
int best = 0;
for (int i = 0; i < S; i++) {
    printf("student %d: %.2f\n", i + 1, rowAvg(g, i));
    if (rowAvg(g, i) > rowAvg(g, best)) best = i;
}
int worst = 0;
for (int j = 1; j < P; j++) if (colAvg(g, j) < colAvg(g, worst)) worst = j;
cout << "best: " << best + 1 << ", hardest subject: " << worst + 1;
```

Вывод

```
student 1: 84.33
student 2: 91.67
student 3: 71.67
student 4: 91.00
best: 2, hardest subject: 1
```

Задача И.2. Подсчёт слов и частота букв средняя

В строке "the quick brown fox jumps over the lazy dog" посчитай количество слов, самую частую букву и проверь, есть ли в строке все 26 букв (панграмма).

Решение

```C++
char s[] = "the quick brown fox jumps over the lazy dog";
int cnt[26] = {0}, words = 0;
for (int i = 0; s[i]; i++) {
    if (s[i] >= 'a' && s[i] <= 'z') cnt[s[i] - 'a']++;
    if (s[i] != ' ' && (i == 0 || s[i - 1] == ' ')) words++;
}
int best = 0, all = 1;
for (int k = 0; k < 26; k++) {
    if (cnt[k] > cnt[best]) best = k;
    if (cnt[k] == 0) all = 0;
}
cout << words << " " << (char)('a' + best) << " " << cnt[best] << " " << (all ? "pangram" :
"no");
```

Вывод

```
9 o 4 pangram
```

Приём `cnt[s[i] - 'a']++` — «массив-счётчик», индекс = номер буквы.

Задача И.3. Иерархия фигур с сортировкой сложная

Абстрактный класс Shape (виртуальные `area()` и `name()`), наследники Circle, Rect, Triangle (по трём сторонам, формула Герона). Создай массив указателей, отсортируй фигуры по площади пузырьком и выведи.

Решение

```C++
class Shape {
public:
    virtual double area() = 0;
    virtual const char *name() = 0;
    virtual ~Shape() { }
};
class Circle : public Shape {
    double r;
public:
    Circle(double v) : r(v) { }
    double area() { return 3.14159 * r * r; }
    const char *name() { return "circle"; }
};
class Rect : public Shape {
    double a, b;
public:
    Rect(double x, double y) : a(x), b(y) { }
    double area() { return a * b; }
    const char *name() { return "rect"; }
};
class Triangle : public Shape {
    double a, b, c;
public:
    Triangle(double x, double y, double z) : a(x), b(y), c(z) { }
    double area() { double p = (a + b + c) / 2; return sqrt(p * (p - a) * (p - b) * (p -
c)); }
    const char *name() { return "triangle"; }
};
//---
const int N = 4;
Shape *f[N] = { new Rect(3, 4), new Circle(1), new Triangle(3, 4, 5), new Circle(2) };
for (int i = 0; i < N - 1; i++)
    for (int j = 0; j < N - 1 - i; j++)
        if (f[j]->area() > f[j + 1]->area()) { Shape *t = f[j]; f[j] = f[j + 1]; f[j + 1] =
t; }
for (int i = 0; i < N; i++) printf("%-9s %.2f\n", f[i]->name(), f[i]->area());
for (int i = 0; i < N; i++) delete f[i];
```

Вывод

```
circle    3.14
triangle  6.00
rect      12.00
circle    12.57
```

Сортируем **указатели**, сами объекты не копируются. Благодаря `virtual` у каждого элемента вызывается своя `area()`.

Задача И.4. Класс «Матрица» с конструктором копирования сложная

Класс Matrix n×n в динамической памяти: конструктор (единичная матрица), конструктор копирования (глубокое копирование), деструктор, метод `set`, метод `multiply` (возвращает новую матрицу) и `trace()` (сумма главной диагонали).

Решение

```C++
class Matrix {
    int n;
    int *a;
public:
    Matrix(int size) : n(size)
    {
        a = new int[n * n];
        for (int i = 0; i < n * n; i++) a[i] = (i / n == i % n) ? 1 : 0;
    }
    Matrix(const Matrix &o) : n(o.n)
    {
        a = new int[n * n];
        for (int i = 0; i < n * n; i++) a[i] = o.a[i];
    }
    ~Matrix() { delete [] a; }
    void set(int i, int j, int v) { a[i * n + j] = v; }
    int get(int i, int j) const { return a[i * n + j]; }
    Matrix multiply(const Matrix &b) const
    {
        Matrix r(n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                int s = 0;
                for (int k = 0; k < n; k++) s += get(i, k) * b.get(k, j);
                r.set(i, j, s);
            }
        return r;
    }
    int trace() const { int s = 0; for (int i = 0; i < n; i++) s += get(i, i); return s; }
};
//---
Matrix A(2);
A.set(0, 0, 1); A.set(0, 1, 2); A.set(1, 0, 3); A.set(1, 1, 4);
Matrix B = A;               // копия
B.set(0, 0, 10);            // A не меняется
Matrix C = A.multiply(A);   // A²
cout << A.trace() << " " << B.trace() << " " << C.get(0, 0) << " " << C.get(1, 1) << " " <<
C.trace();
```

Вывод

```
5 14 7 22 29
```

Большой итоговый тест

Все вопросы курса в одном месте: банк преподавателя (190 вопросов, на английском), 20-секундные задачи на вывод и новые вопросы, включая сложные с кодом. Ключи после каждой лекции-блока.

Блок 1. Парадигмы, алгоритмы, синтаксис C++, структура программы, типы данных, указатели

1. What is an identifier? банк №1

a) dynamic memory
b) an array made up of a set of elements
c) the name of an object in a program
d) the way a program is passed
e) words intended for the compiler

2. In C, which function does the execution of any program start from? банк №3

a) clrscr()
b) random
c) It is different in each program
d) main()
e) randomize()

3. Where are library functions located? банк №4

a) in library files
b) on the Internet
c) on a CD
d) in the computer's RAM
e) No correct answer

4. A sequence of characters made up of Latin letters and digits, which must begin with a letter, is called ... банк №5

a) constant
b) function
c) comment
d) identifier
e) No correct answer

5. Which keyword is used to declare constants in C? банк №6

a) int
b) const
c) procedure
d) var
e) All of the above are correct

6. What is a variable? банк №13

a) dividing memory classes into several parts
b) an expression made of an operand, an operation sign and brackets, expressing a value of a defined type
c) a memory area with a certain name, in which data of a defined type is stored
d) storing data of an undefined type in memory
e) No correct answer

7. What is the #define directive used for? банк №14

a) to set the value of a constant
b) for macros
c) for a symbol that controls conditional compilation
d) an expression made of an operand, an operation sign and brackets, expressing a value of a defined type
e) No correct answer

8. What is the extension of source (header) files that contain functions or defined data, and what are these files called? банк №15

a) ".c", ".c files"
b) ".f", ".f files"
c) ".htp", ".htp files"
d) ".h", ".h files"
e) No correct answer

9. What is a variable? банк №17

a) a quantity that takes different values while the program is running
b) the fulfillment of a certain condition of an algorithm
c) dividing a memory class into several areas
d) a sequence of programs
e) No correct answer

10. The correct declaration of an integer variable a: банк №21

a) float a;
b) a int;
c) int a;
d) integer a;
e) char a;

11. What characters is the body of a function enclosed in? банк №25

a) * *
b) / /
c) { }
d) ( )
e) /* */

12. What characters is a comment written inside in C? банк №26

a) * *
b) { }
c) (* *)
d) /* */
e) < >

13. What is a lexeme? банк №31

a) the smallest meaningful unit of a language
b) phrases
c) sentences
d) a page change
e) the body of a program

14. What does the statement "int n;" mean? банк №32

a) variable n takes real values
b) the body of a program
c) that the specifier n is given
d) variable n takes integer values
e) No correct answer

15. In C, uppercase and lowercase letters denoted by the same character ... банк №33

a) change depending on the language version
b) are considered different
c) change depending on the compiler
d) are considered the same
e) All of the above are correct

16. Names given to variables, constants, data types and functions are called ... банк №34

a) identifiers
b) pointers
c) keywords
d) directives
e) No correct answer is given

17. How is the single-precision real data type declared? банк №36

a) int
b) char
c) double
d) float
e) No correct answer

18. How is the double-precision real data type declared? банк №37

a) int
b) char
c) double
d) float
e) No correct answer

19. How is the character data type declared? банк №38

a) float
b) char
c) double
d) int
e) No correct answer

20. What are keywords? банк №39

a) operations
b) ordinary identifiers
c) functions
d) words entered into a special reserved set
e) No correct answer

21. The first system software product written in C? банк №40

a) UNIX
b) NC
c) Windows XP
d) MS-DOS
e) No correct answer

22. Which fragment declaring variables in C is correct? банк №42

a) main () int x, y; {float a, b; char c;}
b) main () { int x, y; float a, b; char c;}
c) {main (); int x, y; float, a, b; char c;}
d) main () int x, y; float a, b; char x;
e) No correct answer

23. How is the logical type defined in C? банк №43

a) char
b) The logical type is not defined in C
c) string
d) bool
e) No correct answer

24. What values can a logical quantity take in C? банк №44

a) true, false
b) + and -
c) 0, 255
d) 0 and 1
e) No correct answer

25. Who founded the C language? банк №93

a) Bill Gates
b) Dennis Ritchie
c) J. Neumann
d) Bjarne Stroustrup
e) No correct answer

26. In C. The control character "\n" банк №94

a) move to the next page
b) write the \ symbol n times
c) a blank space
d) move to a new line
e) No correct answer

27. In C. The control character "\t" банк №95

a) horizontal tab
b) write the \ symbol t times
c) move to the next page
d) move to a new line
e) No correct answer

28. Which of these is NOT part of the C alphabet? банк №97

a) Special symbols
b) Uppercase letters of the Russian alphabet
c) Lowercase and uppercase letters of the Latin alphabet
d) Arabic digits 0-9
e) No correct answer

29. In C. The range of values of the char type банк №98

a) -128...127
b) -65365...65365
c) -256...255
d) 0...65365
e) No correct answer

30. In C, which function does the execution of any program start from? банк №123

a) random
b) main()
c) clrscr()
d) randomize()
e) No correct answer

31. What characters is the body of a function enclosed in? банк №130

a) {}
b) <>
c) ()
d) []
e) No correct answer

32. In C, in which part of the program are variables declared? банк №134

a) anywhere, only before the variable is used
b) only at the beginning
c) at the end
d) nowhere
e) No correct answer

33. Which of the following data types does NOT belong to C++? банк №171

a) double
b) int
c) real
d) float
e) No correct answer

34. What is a finite sequence of steps used to solve a problem? 20 сек · B1·1

Ответь одним словом или символом.

35. Which function is the starting point of a C++ program? 20 сек · B2·1

Ответь одним словом или символом.

36. Which symbol ends a C++ statement? 20 сек · B3·1

Ответь одним словом или символом.

37. Which operator is used for assignment? 20 сек · B4·1

Ответь одним словом или символом.

38. Which keyword is used to define an integer variable? 20 сек · B5·1

Ответь одним словом или символом.

39. Which header is commonly used for cin and cout? 20 сек · B6·1

Ответь одним словом или символом.

40. Which operator is used for equality comparison? 20 сек · B7·1

Ответь одним словом или символом.

41. Which operator is used for input with cin? 20 сек · B8·1

Ответь одним словом или символом.

42. Which keyword is used to return a value from a function? 20 сек · B9·1

Ответь одним словом или символом.

43. Which keyword is used to declare a constant? 20 сек · B10·1

Ответь одним словом или символом.

44. Which data type is commonly used for whole numbers? 20 сек · B11·1

Ответь одним словом или символом.

45. Which function is used to display data on the screen? 20 сек · B12·1

Ответь одним словом или символом.

46. Which operator is used for remainder after integer division? 20 сек · B13·1

Ответь одним словом или символом.

47. Which operator is used for output with cout? 20 сек · B14·1

Ответь одним словом или символом.

48. Which keyword is used to create a condition in C++? 20 сек · B15·1

Ответь одним словом или символом.

49. Which of the following is a valid C++ identifier? новый

a) 2ndValue
b) _total2
c) my-sum
d) float

50. What is the value of `x` after `int x = 012;`? новый

a) 12
b) 10
c) 2
d) Compilation error

51. Which flowchart symbol is used for a condition? новый

a) rectangle
b) parallelogram
c) rhombus (diamond)
d) oval

52. Which property of an algorithm means "it must finish after a finite number of steps"? новый

a) discreteness
b) finiteness
c) mass character
d) determinism

53. What is the output? сложный

```
enum Level { LOW = 2, MID, HIGH = MID * 3 };
//---
cout << LOW + MID + HIGH;
```

a) 6
b) 11
c) 14
d) Compilation error

54. What is the output? сложный

```
int a[4] = {5, 10, 15, 20};
int *p = a + 1;
cout << *p + *(p + 2) << " " << p[1];
```

a) 25 15
b) 30 15
c) 30 10
d) 15 20

55. What is the output? сложный

```
int x = 5;
int *p = &x;
int *q = p;
*q = *p + 3;
x = x * 2;
cout << *p;
```

a) 8
b) 10
c) 16
d) 5

56. What is the output? сложный

```
short s = -32768;
s--;
cout << s;
```

a) -32769
b) 32767
c) 0
d) Compilation error

57. How many lexemes (tokens) are in `y = x * 25;`? новый

a) 4
b) 5
c) 6
d) 7

Ключи

1 — c	**2** — d	**3** — a	**4** — d	**5** — b	**6** — c
7 — a	**8** — d	**9** — a	**10** — c	**11** — c	**12** — d
13 — a	**14** — d	**15** — b	**16** — a	**17** — d	**18** — c
19 — b	**20** — d	**21** — a	**22** — b	**23** — d	**24** — a
25 — b	**26** — d	**27** — a	**28** — b	**29** — a	**30** — b
31 — a	**32** — a	**33** — c	**34** — Algorithm	**35** — main	**36** — ;
37 — =	**38** — int	**39** — iostream	**40** — ==	**41** — >>	**42** — return
43 — const	**44** — int	**45** — cout	**46** — %	**47** — <<	**48** — if
49 — b	**50** — b	**51** — c	**52** — b	**53** — c	**54** — b
55 — c	**56** — b	**57** — c			

23. В чистом C раньше логического типа не было (только `_Bool` с 1999 г.), а в C++ есть `bool`. Тест про C++, поэтому `bool`.

49. Идентификатор может начинаться с `_` или буквы; цифра в начале, дефис и ключевые слова запрещены.

50. Ведущий ноль — восьмеричная запись: 1·8 + 2 = 10.

53. LOW = 2, MID = 3, HIGH = 3·3 = 9, сумма 14.

54. p указывает на a[1] = 10; *(p+2) = a[3] = 20; 10 + 20 = 30; p[1] = a[2] = 15.

55. p и q указывают на x. *q = 5 + 3 → x = 8, потом x = 16;* p читает x.

56. Переполнение `short` вниз: после минимального значения идёт максимальное.

57. `y`, `=`, `x`, `*`, `25`, `;` — шесть лексем.

Блок 2. Операции и выражения

58. Which header file contains the prototype of the rand() function? банк №8

a) #include <conio.h>
b) #include <stdio.h>
c) #include <iostream.h>
d) #include <math.h>
e) #include <stdlib.h>

59. If float and double types are mixed, what is the result? банк №10

a) int;
b) double
c) float;
d) short;
e) long;

60. If long and float types are mixed, what is the result? банк №19

a) int
b) double
c) char
d) float
e) No correct answer

61. How is the remainder (modulo) operation written? банк №20

a) /
b) <<
c) &
d) %
e) \

62. In which option is sizeof written correctly? банк №22

a) sizeof [type]
b) sizeof: type
c) sizeof type
d) sizeof_ type
e) sizeof (type)

63. Which option correctly shows logical negation? банк №23

a) & (ampersand)
b) ! (exclamation mark)
c) ? (question mark)
d) \ (backslash)
e) = (equals sign)

64. If short and int types are mixed, the result is ... банк №27

a) int
b) short
c) float
d) double
e) char

65. What does the *= operation mean? банк №30

a) comment
b) multiply and assign
c) multiply each element
d) assignment
e) multiplication

66. What is the printf() format for printing a real number? банк №35

a) %f
b) %u
c) %o
d) %x
e) No correct answer

67. In C, what is the value of the expression 5/2? банк №45

a) 1
b) 2.5
c) 2
d) 5
e) No correct answer

68. In C, what is the value of the expression 7/2? банк №46

a) 5
b) 2
c) 3.5
d) 3
e) No correct answer

69. In C, what is the value of the expression 7/3? банк №47

a) 3
b) 2
c) 3.3
d) 2.5
e) No correct answer

70. In C, what is the value of the expression 5%2? банк №48

a) 1
b) 2
c) 3
d) 2.5
e) No correct answer

71. In C, what is the value of the expression 7%2? банк №49

a) 1
b) 3.5
c) 3
d) 4
e) No correct answer

72. In C, which operation finds the quotient of dividing an integer by an integer (like the div function in Pascal)? банк №50

a) /
b) &
c) \
d) %
e) No correct answer

73. In C, which expression does NOT increase a variable's value by 1? банк №51

a) x+=1
b) x*=1
c) x++
d) x=x+1
e) No correct answer

74. In C, how is the statement that produces a random integer in the range [-20, 50] written? банк №52

a) random(50)-20;
b) random(30)+20;
c) random(70)-20;
d) random(50);
e) No correct answer

75. In C, as a result of x= random(20)-5, what range is the value of x in? банк №53

a) [5,15]
b) [-5,20]
c) [5,20]
d) [-5,15]
e) No correct answer

76. In Borland C++ Builder, which statement starts the random number generator so that different random numbers are produced on every run of the program? банк №54

a) getch();
b) main();
c) randomize();
d) time();
e) No correct answer

77. In C, after `x=0; x++; x=x+1; x=x*x;` what is the value of x? банк №55

a) 3
b) 4
c) 2
d) 0
e) No correct answer

78. In C, after `x=0; x++; x=x*1; x=x*x;` what is the value of x? банк №56

a) 2
b) 0
c) 4
d) 1
e) No correct answer

79. In C, which statement is n%=3 equivalent to? банк №57

a) n=n*3
b) n=n%3
c) n=n\3
d) n=n/3
e) No correct answer

80. In C, show the correct program for calculating the volume of a cube with a given side: банк №58

a) v=a*a;
b) v=pow (a,4);
c) v=pow (a,3);
d) v=pow (a,2);
e) No correct answer

81. In C, what does the program `{ float x=60, y; y=atan(x*3.14/180); cout<< y;}` calculate? банк №60

a) arccotangent
b) tangent
c) cotangent
d) arctangent
e) No correct answer

82. In C, which file defines the standard function `pow(x,y);`? банк №61

a) math.h
b) conio.h
c) stdlib.h
d) iostream.h
e) No correct answer

83. In C, what is the result of `cout<<sqrt(pow(5,4));`? банк №62

a) 5
b) 625
c) 25
d) 15
e) No correct answer

84. In C, show the unary increment operation: банк №63

a) /*
b) --
c) //
d) ++
e) No correct answer

85. In C, show the unary decrement operation: банк №64

a) */
b) --
c) //
d) ++
e) No correct answer

86. How is the decimal logarithm lg x written in C? банк №65

a) ln (x)
b) log10(x)
c) lg(x)
d) log (10x)
e) No correct answer

87. In C, if x=2 initially, what is the value of x after x*=(x+1)? банк №66

a) 4
b) 6
c) 5
d) 3
e) No correct answer

88. In C, which function returns the absolute value of a real-type argument? банк №67

a) fabc (x)
b) labs (x)
c) absx (x)
d) fabs (x)
e) No correct answer

89. In C. Which directive is included to use input-output commands? банк №68

a) # include <iostream.h>
b) # include <math.h>
c) # include <conio.h>
d) # include <stdio.h>
e) No correct answer

90. In C. The function of the output operator: банк №69

a) getch()
b) cin>>
c) cout<<
d) scanf()
e) No correct answer

91. The input operator in C++: банк №70

a) getch()
b) printf()
c) cout<<
d) cin>>
e) No correct answer

92. In C. What does the getch(); statement do? банк №71

a) returns the value of a function
b) marks the end of the program
c) waits for any key to be pressed, but does not display it on the screen
d) clears the screen
e) No correct answer

93. In C. Which header file defines the getch() function? банк №83

a) iostream.h
b) conio.h
c) stdio.h
d) stdlib.h
e) No correct answer

94. The screen-clearing function in C: банк №85

a) clrscr();
b) cls();
c) clean();
d) clear screen();
e) No correct answer

95. In C. Which header file is included to use the screen-clearing function? банк №86

a) stdio.h
b) iostream.h
c) conio.h
d) stdlib.h
e) No correct answer

96. In C. Which header file is included to use the random number function? банк №87

a) stdio.h
b) conio.h
c) iostream.h
d) stdlib.h
e) No correct answer

97. In C, which number does «false» correspond to? банк №88

a) -1
b) it is not equated to any number
c) 0
d) 1
e) No correct answer

98. In C, which number does «true» correspond to? банк №89

a) 1
b) it is not equated to any number
c) 0
d) -1
e) No correct answer

99. In C. What does 1 as the result of evaluating a condition mean? банк №90

a) the result of correct compilation of the program
b) false
c) true
d) boolean
e) No correct answer

100. In C. What does 0 as the result of evaluating a condition mean? банк №91

a) true
b) boolean
c) the result of correct compilation of the program
d) false
e) No correct answer

101. In C, is the expression (1 != 1) true or false? банк №92

a) meaningless
b) unknown
c) false
d) true
e) No correct answer

102. In C. An expression equivalent to s=s+x банк №96

a) s=+x
b) s+=x
c) s=x++
d) s=s++
e) No correct answer

103. How is the arccos(x) function written in C? банк №99

a) acos(x)
b) cosa(x)
c) acos{x}
d) acos[x]
e) No correct answer

104. What is the quoted phrase in the cout<< operator used for? банк №100

a) for clarity; it can also be left out;
b) to move to the next line;
c) so as not to move to the next line
d) to print a question mark;
e) No correct answer

105. Which logical operation is called “logical multiplication” (AND)? банк №101

a) ||
b) !
c) :
d) &&
e) No correct answer

106. Which logical operation is called “logical sum” (OR)? банк №102

a) ||
b) :
c) !
d) &&
e) No correct answer

107. What is the value of the expression k=(3/2)*(3%2)? банк №103

a) 3
b) 2
c) 1
d) 0.5
e) No correct answer

108. Given the declarations: What is the type of the value of the expression d+1? банк №104

```
int a,b,c,n; float d,e;
```

a) char
b) bool
c) int
d) float
e) No correct answer

109. The operator for entering the values of variables into the computer: банк №105

a) getch()
b) printf()
c) cin>>
d) cout<<
e) No correct answer

110. What is the value of the variable k in `k=ceil(3.226);`? банк №108

a) 3.5
b) 3
c) 2
d) 4
e) No correct answer

111. What is the value of the variable k in `k=floor(3.63);`? банк №109

a) 3.5
b) 2
c) 3
d) 4
e) No correct answer

112. Which function is used to find the square root of a value? банк №112

a) sqrt()
b) sgrt()
c) pov()
d) sqr()
e) No correct answer

113. How do we write x to the power of y in C? банк №113

a) sqrt(x,y)
b) pow(x,y)
c) povv(x,y)
d) sqr(x,y)
e) No correct answer

114. How do we write the 5th root of x? банк №114

a) sqrt(x,1/5)
b) pow(x,1/5.)
c) sqrt(x,5)
d) pow(x,1/5)
e) No correct answer

115. What is the printf() format for printing a string? банк №115

a) %o
b) %u
c) %f
d) %s
e) No correct answer

116. What is the printf() format for printing a real number? банк №116

a) %f
b) %o
c) %s
d) %u
e) No correct answer

117. The printf() format for printing an unsigned decimal integer: банк №117

a) %u
b) %o
c) %s
d) %f
e) No correct answer

118. The printf() format for printing an unsigned octal integer: банк №118

a) %s
b) %f
c) %u
d) %o
e) No correct answer

119. What is the printf() format for printing an unsigned hexadecimal integer? банк №119

a) %s
b) %x
c) %f
d) %u
e) No correct answer

120. What is the printf() format for printing a real number in exponential form? банк №120

a) %x
b) %f
c) %e
d) %s
e) No correct answer

121. What answer do we get? банк №133

```
int i=10;
cout <<  i++;
cout  << ++i;
```

a) 10, 10
b) 10, 12
c) 10, 11
d) 11, 12
e) No correct answer

122. Show the correct way to calculate x to the power of 4. банк №141

a) pow(x/4);
b) pow(x*4);
c) pow(x,4);
d) pow(x;4);
e) No correct answer

123. What is the mathematical library for calculating standard arithmetic functions called? банк №165

a) math.h
b) match.h
c) matem.h
d) stdlib.h
e) No correct answer

124. What will the code below print to the screen? банк №170

```
int const a = 5; a++; cout << a;
```

a) the code shows an error
b) 5
c) 0
d) 6
e) No correct answer

125. The function that finds the absolute value of integers: банк №174

a) abs(x)
b) fabc(x)
c) fabs(x)
d) abc(x)
e) No correct answer

126. How is arccosine written in C++? банк №175

a) acos{x}
b) asin[x]
c) acos(x)
d) atan(x)
e) No correct answer

127. The function that rounds a number up: банк №176

a) log(x)
b) floor(x)
c) fabs(x)
d) ceil(x)
e) No correct answer

128. The function that rounds a number down: банк №177

a) fabs(x)
b) exp(x)
c) floor(x)
d) ceil(x)
e) No correct answer

129. Which of the following is NOT a comparison sign? банк №179

a) <
b) =
c) ==
d) !=
e) No correct answer

130. The increment operation is applied: банк №186

a) to constants
b) to expressions
c) to variables
d) to strings
e) No correct answer

131. Which numbers is the % operation applied to? банк №187

a) integer and real
b) fractional
c) real
d) integer
e) No correct answer

132. Logical “and” is denoted as follows: банк №188

a) ||
b) &
c) &&
d) |
e) No correct answer

133. Logical “or” is denoted as follows: банк №189

a) &&
b) ||
c) &
d) |
e) No correct answer

134. Logical “not equal” is denoted as follows: банк №190

a) |=
b) =!
c) =|
d) !=
e) No correct answer

135. What is the output? 20 сек · B1·2

```
int x = 7;
x += 4;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

136. What is the output? 20 сек · B1·3

```
int a = 17, b = 5;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

137. What is the output? 20 сек · B2·2

```
int x = 5;
cout << x++ + ++x;
```

Напиши, что выведет программа.

138. What is the output? 20 сек · B2·3

```
int a = 7, b = 2;
double x = a / b;
double y = (double)a / b;
cout << x << " " << y;
```

Напиши, что выведет программа.

139. What is the output? 20 сек · B3·2

```
int x = 10;
x -= 3;
x *= 2;
x /= 2;
cout << x;
```

Напиши, что выведет программа.

140. What is the output? 20 сек · B3·3

```
int x = 10;
cout << x-- << " " << --x;
```

Напиши, что выведет программа.

141. What is the output? 20 сек · B4·2

```
int a = 4;
a *= 3;
a -= 2;
a /= 2;
cout << a;
```

Напиши, что выведет программа.

142. What is the output? 20 сек · B4·3

```
int a = 19, b = 4;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

143. What is the output? 20 сек · B5·2

```
int x = 6;
x++;
x *= 3;
cout << x;
```

Напиши, что выведет программа.

144. What is the output? 20 сек · B5·3

```
int a = 5, b = 2;
cout << a * b + a % b;
```

Напиши, что выведет программа.

145. What is the output? 20 сек · B6·2

```
int x = 8;
x /= 2;
x += 7;
cout << x;
```

Напиши, что выведет программа.

146. What is the output? 20 сек · B6·3

```
int x = 5;
cout << ++x << " " << x++;
```

Напиши, что выведет программа.

147. What is the output? 20 сек · B7·2

```
int x = 3;
x = x + 4 * 2;
cout << x;
```

Напиши, что выведет программа.

148. What is the output? 20 сек · B7·3

```
int a = 20, b = 6;
cout << a % b + a / b;
```

Напиши, что выведет программа.

149. What is the output? 20 сек · B8·2

```
int x = 4;
x *= 2;
x++;
x -= 3;
cout << x;
```

Напиши, что выведет программа.

150. What is the output? 20 сек · B8·3

```
int a = 15, b = 4;
cout << (a % b) * 2;
```

Напиши, что выведет программа.

151. What is the output? 20 сек · B9·2

```
int x = 9;
x -= 4;
x *= 3;
cout << x;
```

Напиши, что выведет программа.

152. What is the output? 20 сек · B9·3

```
int a = 14, b = 5;
cout << a / b * b + a % b;
```

Напиши, что выведет программа.

153. What is the output? 20 сек · B10·2

```
int x = 5;
x += 5;
x /= 2;
cout << x;
```

Напиши, что выведет программа.

154. What is the output? 20 сек · B10·3

```
int x = 6;
cout << x++ * 2;
```

Напиши, что выведет программа.

155. What is the output? 20 сек · B11·2

```
int x = 7;
x *= 2;
x -= 4;
cout << x;
```

Напиши, что выведет программа.

156. What is the output? 20 сек · B11·3

```
int a = 23, b = 7;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

157. What is the output? 20 сек · B12·2

```
int x = 3;
x = x * 4 - 2;
cout << x;
```

Напиши, что выведет программа.

158. What is the output? 20 сек · B12·3

```
int a = 18, b = 5;
cout << a % b * 2 + 1;
```

Напиши, что выведет программа.

159. What is the output? 20 сек · B13·2

```
int x = 6;
x++;
x++;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

160. What is the output? 20 сек · B13·3

```
int a = 21, b = 4;
cout << a / b + a % b;
```

Напиши, что выведет программа.

161. What is the output? 20 сек · B14·2

```
int x = 9;
x /= 3;
x += 8;
cout << x;
```

Напиши, что выведет программа.

162. What is the output? 20 сек · B14·3

```
int x = 5;
cout << ++x + x++;
```

Напиши, что выведет программа.

163. What is the output? 20 сек · B15·2

```
int x = 4;
x += 6;
x *= 2;
cout << x;
```

Напиши, что выведет программа.

164. What is the output? 20 сек · B15·3

```
int a = 16, b = 3;
cout << a / b << " " << a % b;
```

Напиши, что выведет программа.

165. What is the value of `double d = 7 / 2;`? новый

a) 3.5
b) 3
c) 4
d) 3.0 is impossible

166. Which expression correctly checks that `x` is in the range [1, 10]? новый

a) 1 <= x <= 10
b) x >= 1 || x <= 10
c) x >= 1 && x <= 10
d) x => 1 && x =< 10

167. What does `cout << (17 >> 2);` print? новый

a) 68
b) 4
c) 8
d) 15

168. What is the output? сложный

```
int a = 5, b = 0;
if (b != 0 && a / b > 1) cout << "A";
else if (a > 3 || ++b) cout << "B";
cout << b;
```

a) A0
b) B1
c) B0
d) Runtime error (division by zero)

169. What is the output? сложный

```
int x = 3;
x += x++ * 0 + 2;
int y = 10;
y = y / 3 * 3 + y % 3;
int z = (y, x, 7);
cout << y << " " << z;
```

a) 9 7
b) 10 7
c) 10 3
d) 10 10

170. What is the output? сложный

```
int a = 6, b = 4;
cout << (a & b) << (a | b) << (a ^ b) << (a && b);
```

a) 1111
b) 4621
c) 4610
d) 6421

171. What is the output? сложный

```
double x = 2.7;
int k = (int)x + (int)(x * 2) % 3;
cout << k << " " << ceil(-2.5) << " " << floor(-2.5);
```

a) 5 -3 -2
b) 4 -2 -3
c) 4 -3 -2
d) 3 -2 -3

172. What is the output? сложный

```
int i = 5;
int j = i++ * 2;
j += --i;
cout << j;
```

a) 16
b) 15
c) 14
d) 17

173. Which header file must be included to use `sqrt()` and `pow()`? новый

a) stdlib.h
b) conio.h
c) math.h
d) string.h

Ключи

58 — e	**59** — b	**60** — d	**61** — d	**62** — e	**63** — b
64 — a	**65** — b	**66** — a	**67** — c	**68** — d	**69** — b
70 — a	**71** — a	**72** — a	**73** — b	**74** — c	**75** — d
76 — c	**77** — b	**78** — d	**79** — b	**80** — c	**81** — d
82 — a	**83** — c	**84** — d	**85** — b	**86** — b	**87** — b
88 — d	**89** — a	**90** — c	**91** — d	**92** — c	**93** — b
94 — a	**95** — c	**96** — d	**97** — c	**98** — a	**99** — c
100 — d	**101** — c	**102** — b	**103** — a	**104** — a	**105** — d
106 — a	**107** — c	**108** — d	**109** — c	**110** — d	**111** — c
112 — a	**113** — b	**114** — b	**115** — d	**116** — a	**117** — a
118 — d	**119** — b	**120** — c	**121** — b	**122** — c	**123** — a
124 — a	**125** — a	**126** — c	**127** — d	**128** — c	**129** — b
130 — c	**131** — d	**132** — c	**133** — b	**134** — d	**135** — 22
136 — 3 2	**137** — 12	**138** — 3 3.5	**139** — 7 **(в ключе 12)**	**140** — 9 9 **(в ключе 19)**	**141** — 5
142 — 4 3	**143** — 21	**144** — 11	**145** — 11	**146** — 7 5 **(в ключе 6 6)**	**147** — 11
148 — 5	**149** — 6 **(в ключе 12)**	**150** — 6	**151** — 15	**152** — 14 **(в ключе 4 2)**	**153** — 5
154 — 12	**155** — 10	**156** — 3 2	**157** — 10	**158** — 7	**159** — 16
160 — 6	**161** — 11	**162** — 12	**163** — 20	**164** — 5 1	**165** — b
166 — c	**167** — b	**168** — c	**169** — b	**170** — b	**171** — b
172 — b	**173** — c				

74. Ключ преподавателя — `random(70)-20`, отвечай так. Но строго говоря `random(n)` в Borland даёт числа от 0 до n−1, поэтому `random(70)-20` даёт [−20; 49]. Чтобы попасть ровно в [−20; 50], нужно `random(71)-20` (в вариантах такого нет).

75. Ключ — [−5,15], отвечай так. Точно: `random(20)` даёт 0…19, значит x ∈ [−5; 14]. Преподаватель считает границы «на глаз».

83. `pow(5,4)` = 625, `sqrt(625)` = 25.

104. Формулировка странная (перевод с казахского), но в ключе — «для понятности, можно и не писать»: текст в кавычках просто подписывает вывод, программа работает и без него.

107. `3/2` = 1 (целое деление), `3%2` = 1, 1·1 = 1.

121. `i++` печатает старое значение 10 (потом i = 11), `++i` сначала увеличивает до 12 и печатает 12.

137. Неопределённое поведение (x меняется дважды). Borland 6 даёт 12 — как в ключе. Запомни ответ, но так не программируй.

139. 10−3 = 7, 7·2 = 14, 14/2 = 7. В ключе 12 — ошибка.

140. Неопределённое поведение: x меняется дважды в одном выражении. Borland 6 считает `cout << a << b` справа налево и печатает «9 9»; современный C++17 напечатает «10 8». В ключе «19» — неверно в любом случае.

146. Неопределённое поведение (x меняется дважды в одном `cout`). Borland 6 вычисляет справа налево: сначала `x++` (даёт 5, x = 6), потом `++x` (x = 7) → «7 5». В C++17 будет «6 6» — как в ключе. На тесте пиши ответ по ключу (6 6), но в коде так не пиши.

149. 4·2 = 8, ++ → 9, 9−3 = 6. В ключе 12 — ошибка.

152. 14/5 = 2, 2·5 = 10, 14%5 = 4, 10 + 4 = 14 (выводится одно число). В ключе «4 2» — ошибка.

162. Неопределённое поведение. Borland 6 даёт 12 — как в ключе.

165. 7/2 считается в целых = 3, потом 3 превращается в 3.0 (выводится как 3).

167. Сдвиг вправо на 2 = целочисленное деление на 4: 17/4 = 4.

168. В `&&` левая часть ложна → деление не выполняется. В `||` левая часть истинна → `++b` не выполняется, b остаётся 0.

169. y/3*3 + y%3 восстанавливает y = 10 всегда. Запятая возвращает последнее значение 7. (Значение x здесь не печатаем — там неопределённое поведение.)

170. 110 & 100 = 100 (4); 110 | 100 = 110 (6); 110 ^ 100 = 010 (2); 6 && 4 = 1.

171. (int)2.7 = 2; x·2 = 5.4 → 5, 5 % 3 = 2; 2 + 2 = 4. ceil округляет вверх (−2.5 → −2), floor вниз (−2.5 → −3).

172. i++ отдаёт 5 (после этого i = 6), j = 10. Затем --i делает i = 5, j = 10 + 5 = 15. Здесь всё однозначно: каждое изменение i — в своём операторе.

Блок 3. Ветвления if, switch, goto и цикл for

174. Which word is used to write the selection (choice) operator? банк №18

a) switch
b) return
c) for
d) if
e) class

175. Which statement is used to exit a switch? банк №29

a) do
b) case
c) default
d) return
e) break

176. Which is the correct condition for checking that an array index is odd in C? банк №41

a) if (i%2=1)
b) if (i%2!=1)
c) if (i%2==1)
d) if (i%2==0)
e) No correct answer

177. In C, the correct condition for checking that an array index is even: банк №59

a) if (i%2==0)
b) if (a[i]%2==0)
c) if (i%2==1)
d) if (i%2=0)
e) No correct answer

178. In C. How many times does the loop repeat in the program below? банк №72

```
for (x=1; x <= 10; x=x+2)
```

a) 20
b) 5
c) 10
d) 15
e) No correct answer

179. In C. What is the value of k as a result of running the program? банк №76

```
k=0; for (x=1;x <=10; x++) if (x % 3==0) k++;
```

a) 3
b) 4
c) 5
d) 2
e) No correct answer

180. In C. What is the value of k as a result of running the program? банк №77

```
k=0; for (x=1;x <=10; x++) if (x % 5==0) k++;
```

a) 3
b) 4
c) 5
d) 2
e) No correct answer

181. In C. What is the value of k as a result of running the program? банк №78

```
k=0; for (x=1;x <=10; x++) if (x % 9==0) k++;
```

a) 4
b) 1
c) 3
d) 2
e) No correct answer

182. In C. The notation of the loop with a parameter: банк №81

a) for (initial value; final value; step)
b) do {loop body}while (condition)
c) switch () {}
d) while (condition) {loop body}
e) No correct answer

183. Which is the conditional operator? банк №110

a) if
b) for
c) while
d) switch
e) No correct answer

184. Which one is called the selection operator? банк №111

a) for
b) if
c) while
d) switch
e) No correct answer

185. What is a branching operator? банк №128

a) an operator that checks the fulfillment of only one condition

b) an operator that checks that conditions are not met

c) an operator responsible for the execution of any program in C

d) a division of an algorithm into several paths, branching depending on whether a certain condition is met or not

e) No correct answer

186. What values does i take in the statement `for(i=1;i<10;i+=2)` банк №140

a) 1 3 5 7 9

b) 2 4 6 8 10

c) 1 3 5 7 9 10

d) 1 2 3 4 5

e) No correct answer

187. What values does i take? банк №164

```
for (int i=100; i>0; i-=10)
```

a) 90,70,50,30,10

b) 100,90,80,70,60,50,40,30,20,10

c) 50,40,30,20,10

d) 100,80,60,40,20

e) No correct answer

188. What values does i take? банк №166

```
for (int i=100; i<=150; i+=10)
```

a) 100,110,120,130,140

b) 110,110,120,130,140,160

c) 110,120,130,140,150

d) 100,110,120,130,140,150

e) No correct answer

189. After the code below is executed, what is the final value of the variable a? банк №172

```
int a; for(a = 0; a < 10; a++) cout<<a;
```

a) 9

b) 0

c) 10

d) 1

e) No correct answer

190. If int n=3, what will the result be? банк №181

```
switch(n) {
case 2: cout << "aaa"; break;
case 3: cout << "bbb"; break;
default: cout << "vvv"; break; }
```

a) vvv
b) bbb
c) the code will not run
d) aaa
e) No correct answer

191. If int n=45, what will the result be? банк №182

```
switch(n) {
case 23: cout << "aaa";
case 45: cout << "bbb";
default: cout << "vvv";
break; }
```

a) bbbvvv
b) bbb
c) vvv
d) aaa
e) No correct answer

192. What is the output? 20 сек · B1·4

```
int x = 12;
if (x % 2 == 0 && x > 10)
    cout << x / 2;
else
    cout << x * 2;
```

Напиши, что выведет программа.

193. What is the output? 20 сек · B1·5

```
int x = 2;
switch (x) {
    case 1: cout << 5; break;
    case 2: cout << 7;
    case 3: cout << 9; break;
    default: cout << 0;
}
```

Напиши, что выведет программа.

194. What is the output? 20 сек · B2·4

```
int x = 8;
if (x > 5) {
    x += 2;
    if (x > 10)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

195. What is the output? 20 сек · B2·5

```
int x = 3, y = 2;
switch (x) {
    case 1: y += 5; break;
    case 2: y *= 2; break;
    case 3: y += 4;
    case 4: y *= 2; break;
}
cout << y;
```

Напиши, что выведет программа.

196. What is the output? 20 сек · B3·4

```
int x = 15;
if (x % 3 == 0)
    x /= 3;
else
    x += 3;
if (x < 10)
    x += 5;
cout << x;
```

Напиши, что выведет программа.

197. What is the output? 20 сек · B3·5

```
int x = 5;
for (int i = 1; i <= 4; i++)
    x += i;
cout << x;
```

Напиши, что выведет программа.

198. What is the output? 20 сек · B4·4

```
int x = 7;
if (x < 5 || x > 10)
    cout << "A";
else
    cout << "B";
```

Напиши, что выведет программа.

199. What is the output? 20 сек · B4·5

```
int s = 0;
for (int i = 1; i <= 6; i++)
    if (i % 2 == 0)
        s += i;
cout << s;
```

Напиши, что выведет программа.

200. What is the output? 20 сек · B5·4

```
int x = 9;
if (x % 2 == 0)
    cout << x / 2;
else
    cout << x + 2;
```

Напиши, что выведет программа.

201. What is the output? 20 сек · B5·5

```
int x = 2;
switch (x) {
    case 1: x += 10; break;
    case 2: x += 3;
    case 3: x *= 2; break;
    default: x = 0;
}
cout << x;
```

Напиши, что выведет программа.

202. What is the output? 20 сек · B6·4

```
int x = 14;
if (x % 2 == 0) {
    x /= 2;
    if (x > 5)
        x -= 2;
}
cout << x;
```

Напиши, что выведет программа.

203. What is the output? 20 сек · B6·5

```
int x = 4;
for (int i = 1; i <= 3; i++)
    x += i * 2;
cout << x;
```

Напиши, что выведет программа.

204. What is the output? 20 сек · B7·4

```
int x = 11;
if (x > 10)
    x -= 5;
else
    x += 5;
cout << x;
```

Напиши, что выведет программа.

205. What is the output? 20 сек · B7·5

```
int s = 0;
for (int i = 2; i <= 8; i += 2)
    s += i * i;
cout << s;
```

Напиши, что выведет программа.

206. What is the output? 20 сек · B8·4

```
int x = 6;
if (x > 3 && x < 10)
    cout << x * 2;
else
    cout << x / 2;
```

Напиши, что выведет программа.

207. What is the output? 20 сек · B8·5

```
int x = 3;
switch (x) {
    case 1: cout << 1; break;
    case 2: cout << 2; break;
    case 3: cout << 3;
    case 4: cout << 4; break;
    default: cout << 5;
}
```

Напиши, что выведет программа.

208. What is the output? 20 сек · B9·4

```
int x = 16;
if (x % 4 == 0)
    x /= 4;
if (x < 5)
    x += 10;
cout << x;
```

Напиши, что выведет программа.

209. What is the output? 20 сек · B9·5

```
int x = 1;
for (int i = 1; i <= 4; i++)
    x *= i;
cout << x;
```

Напиши, что выведет программа.

210. What is the output? 20 сек · B10·4

```
int x = 13;
if (x % 2 == 1 && x > 10)
    cout << x - 3;
else
    cout << x + 3;
```

Напиши, что выведет программа.

211. What is the output? 20 сек · B10·5

```
int s = 0;
for (int i = 1; i <= 5; i++)
    s += i * 2;
cout << s;
```

Напиши, что выведет программа.

212. What is the output? 20 сек · B11·4

```
int x = 5;
if (x > 3) {
    x += 4;
    if (x % 2 == 1)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

213. What is the output? 20 сек · B11·5

```
int s = 0;
for (int i = 1; i <= 7; i += 2)
    s += i;
cout << s;
```

Напиши, что выведет программа.

214. What is the output? 20 сек · B12·4

```
int x = 10;
if (x < 5)
    cout << 1;
else if (x < 10)
    cout << 2;
else
    cout << 3;
```

Напиши, что выведет программа.

215. What is the output? 20 сек · B12·5

```
int x = 2;
for (int i = 1; i <= 4; i++) {
    x += i;
}
cout << x;
```

Напиши, что выведет программа.

216. What is the output? 20 сек · B13·4

```
int x = 18;
if (x % 3 == 0) {
    x /= 3;
    if (x % 2 == 1)
        x += 5;
}
cout << x;
```

Напиши, что выведет программа.

217. What is the output? 20 сек · B13·5

```
int s = 0;
for (int i = 1; i <= 5; i++)
    if (i % 2 == 1)
        s += i;
cout << s;
```

Напиши, что выведет программа.

218. What is the output? 20 сек · B14·4

```
int x = 4;
if (x % 2 == 0)
    x *= 3;
else
    x += 3;
cout << x;
```

Напиши, что выведет программа.

219. What is the output? 20 сек · B14·5

```
int s = 0;
for (int i = 2; i <= 10; i += 2)
    s += i;
cout << s;
```

Напиши, что выведет программа.

220. What is the output? 20 сек · B15·4

```
int x = 20;
if (x > 10) {
    x -= 7;
    if (x % 2 == 1)
        x *= 2;
}
cout << x;
```

Напиши, что выведет программа.

221. What is the output? 20 сек · B15·5

```
int x = 1;
for (int i = 1; i <= 3; i++)
    x = x * 2 + i;
cout << x;
```

Напиши, что выведет программа.

222. What is printed? новый

```
int x = 10;
if (x = 0) cout << "zero";
else cout << "nonzero";
cout << x;
```

a) zero0
b) nonzero0
c) nonzero10
d) zero10

223. How many times is "Hi" printed? `for (int i = 0; i < 10; i += 3) cout << "Hi";` новый

a) 3
b) 4
c) 10
d) 5

224. Which of these is NOT allowed as a `case` label? новый

a) `case 'A':`
b) `case 5:`
c) `case x > 5:`
d) `case 2 + 3 * 4:`

225. What is the output? сложный

```
int s = 0;
for (int i = 1; i <= 4; i++)
    for (int j = i; j <= 4; j++)
        s++;
cout << s;
```

a) 16
b) 8
c) 10
d) 4

226. What is the output? сложный

```
int k = 0;
for (int i = 0; i < 5; i++) {
    switch (i % 3) {
        case 0: k += 1;
        case 1: k += 10; break;
        default: k += 100;
    }
}
cout << k;
```

a) 153
b) 142
c) 111
d) 131

227. What is the output? сложный

```
int a = 3, b = 7;
if (a > b)
    if (a > 0) cout << "X";
else
    cout << "Y";
cout << "Z";
```

a) YZ
b) XZ
c) Z
d) XYZ

228. What is the output? сложный

```
int i, s = 0;
for (i = 1; i < 20; i *= 2) ;
s = i;
for (i = 10; i > 0; i -= 4) s += i;
cout << s << " " << i;
```

a) 48 -2
b) 50 -2
c) 50 2
d) 34 0

229. Which statement is used to exit a `switch` block? новый

a) continue
b) exit
c) break
d) goto default

Ключи

174 — a	**175** — e	**176** — c	**177** — a	**178** — b	**179** — a
180 — d	**181** — b	**182** — a	**183** — a	**184** — d	**185** — d
186 — a	**187** — b	**188** — d	**189** — c (в ключе a)	**190** — b	**191** — a
192 — 6	**193** — 79 (в ключе 16)	**194** — 10	**195** — 12	**196** — 10 (в ключе 17)	**197** — 15 (в ключе 10)
198 — B	**199** — 12	**200** — 11	**201** — 10	**202** — 5	**203** — 16
204 — 6	**205** — 120	**206** — 12	**207** — 34	**208** — 14	**209** — 24
210 — 10	**211** — 30	**212** — 18	**213** — 16	**214** — 3	**215** — 12
216 — 6 (в ключе 11)	**217** — 9	**218** — 12	**219** — 30	**220** — 26	**221** — 19 (в ключе 14)
222 — b	**223** — b	**224** — c	**225** — c	**226** — b	**227** — c
228 — b	**229** — c				

189. Ошибка в ключе! Цикл печатает 0…9, но после последней итерации выполняется `a++` → a = 10, проверка `10 < 10` ложна, цикл заканчивается. Значит a = **10**. В ключе стоит 9 — на тесте выбирай по ситуации: если система проверяет по ключу, ответ «9».

191. Нет `break` после `case 45` — выполнение «проваливается» в `default`: печатается bbb, затем vvv.

193. В ключе «16» — сложили 7+9. Но `cout` ПЕЧАТАЕТ 7, потом (нет break) печатает 9 → на экране «79».

196. 15 % 3 == 0 → x = 5; 5 < 10 → x = 10. В ключе 17 — ошибка.

197. 5 + 1 + 2 + 3 + 4 = 15. В ключе 10 — ошибка.

216. 18/3 = 6, 6 — чётное, +5 не делаем → 6. В ключе 11 — ошибка.

221. x: 1 → 1·2+1 = 3 → 3·2+2 = 8 → 8·2+3 = 19. В ключе 14 — ошибка.

222. `x = 0` — присваивание, его значение 0 = ложь → ветка else. При этом x уже стал 0.

223. i = 0, 3, 6, 9 → 4 раза.

224. Метка — константное выражение. `2 + 3 * 4` — константа (14), а `x > 5` зависит от переменной.

225. 4 + 3 + 2 + 1 = 10.

226. i % 3 даёт 0, 1, 2, 0, 1. Для 0: +1 и провал в case 1: +10 (= 11). Для 1: +10. Для 2: default +100. Итого 11 + 10 + 100 + 11 + 10 = 142.

227. Висячий else относится к ближайшему (внутреннему) if. Внешнее условие 3 > 7 ложно, поэтому весь вложенный if пропускается. Печатается только Z.

228. Первый цикл (пустое тело из-за `;`): 1, 2, 4, 8, 16, 32 → i = 32. Второй: 10 + 6 + 2 = 18, i становится −2. s = 32 + 18 = 50.

Блок 4. Циклы while и do…while, break и continue, одномерные и двумерные массивы

230. What is an array? банк №2

a) a type that defines the size

b) a pointer that uses dynamic memory

c) a sequence of values of the same type under one name

d) a set of types that defines a reference

e) data located in cells

231. Which option correctly declares an array of 10 real numbers? банк №16

a) int a[1..10];
b) float a(10);
c) float a[10];
d) float a[1..10];
e) int a[10];

232. The correct declaration of an array a of 12 integers: банк №24

a) char a[12];
b) float a[10];
c) float a[12];
d) int a[12];
e) int a(12);

233. What does `float a[10];` mean? банк №28

a) declaring a 10x10 matrix a
b) declaring a sequence of integers from 1 to 10
c) declaring an array a of 10 real numbers
d) declaring a sequence of real numbers from 1 to 10
e) No correct answer

234. In C. How many times does the loop repeat in the program below? банк №73

```
x=2; while (x<10) {y=x*x; x=x+2;}
```

a) 5
b) 10
c) 4
d) 3
e) No correct answer

235. In C. How many times does the loop repeat in the program below? банк №74

```
x=7; while (x <=10) { cout<<x; x++; }
```

a) 7
b) 4
c) 6
d) 5
e) No correct answer

236. In C. How many times does the loop repeat in the program below? банк №75

```
x=7; while (x <10) { cout<<x; x++; }
```

a) 6
b) 4
c) 3
d) 5
e) No correct answer

237. In C. The notation of the pre-condition loop: банк №79

a) for (final value; initial value; step)
b) do {loop body}while (condition)
c) while (condition) {loop body}
d) for (initial value; final value; step)
e) No correct answer

238. In C. The notation of the post-condition loop: банк №80

a) for (final value; initial value; step)
b) while (condition) {loop body}
c) for (initial value; final value; step)
d) do {loop body}while (condition)
e) No correct answer

239. How many types of loop operators are there in C? банк №82

a) 2
b) 3
c) 4
d) 1
e) No correct answer

240. In C. An array is банк №84

a) a set of any elements
b) a set of elements of different types
c) a sequence of variables
d) a set of elements of the same type
e) No correct answer

241. In which loop operator do we exit the loop when the condition is true? банк №106

a) switch
b) do {} while(condition)
c) while (condition) { }
d) for
e) No correct answer

242. In which loop operator do we exit the loop when the condition is not true? банк №107

a) for
b) do {} while(condition)
c) while (condition) { }
d) switch
e) No correct answer

243. Array indexes start from which number: банк №137

a) 1
b) -1
c) 0
d) 2
e) No correct answer

244. What result does the program below give? банк №139

```
int a[3][3];
for(int i=0;i<3;i++)
for(int j=0;j<3;j++)
cin>>a[i][j];
int m=a[1][1],s=0;
for(int i=0;i<3;i++)
s+=a[i][i];
cout<<s;
```

a) the sum of all elements of the matrix
b) the sum of the even elements of the array
c) the sum of the odd elements of the array
d) the sum of the elements of the main diagonal of the matrix
e) No correct answer

245. What does the program below calculate? банк №159

```
s=0;
for(i=0; i<3; i++)
for(j=0; j<3; j++)
if (j<i) s+=a[i][j];
printf("\ns=%i",s);
getch();
```

a) the sum of the elements of the secondary diagonal of the given array
b) the sum of the elements located above the main diagonal of the given array
c) the sum of the elements of the main diagonal of the given array
d) the sum of the elements located below the main diagonal of the given array
e) No correct answer

246. What does the program fragment calculate? банк №167

```
for (i=0; i<=n; i++)
cin>>a[i];
s2=0;
for (i=0; i<=n; i++)
if (i %2== 0) s2=s2+a[i];
cout<<s2;
```

a) the sum of all elements
b) determines the number of all elements equal to 0
c) finds the sum of only the even-numbered elements
d) finds the sum of only the odd-numbered elements
e) No correct answer

247. What does the program fragment calculate? банк №168

```
for (i=0; i<=n; i++)
cin>>a[i];
s2=0;
for (i=0; i<=n; i++)
if (i %2!= 0) s2=s2+a[i];
cout<<s2;
```

a) the sum of all elements
b) finds the sum of only the odd-numbered elements
c) finds the sum of only the even-numbered elements
d) determines the number of all elements not equal to 0
e) No correct answer

248. Which of the operators below is NOT a loop operator of C++? банк №173

a) while
b) do while
c) for
d) repeat until
e) No correct answer

249. What else do we call an array that uses two indexes? банк №180

a) one-dimensional array
b) heterogeneous array
c) dynamic array
d) matrix
e) No correct answer

250. The number used to work with a single element of an array is called .... банк №183

a) matrix
b) arithmetic mean
c) type
d) index
e) No correct answer

251. Array elements are related by the fact that their ... are the same. банк №184

a) names
b) values
c) types
d) sizes
e) No correct answer

252. The process of arranging array elements in order is called .... банк №185

a) changing
b) sorting
c) searching
d) checking
e) No correct answer

253. What is the output? 20 сек · B1·6

```
int x = 3, s = 0;
while (x < 15) {
    s += x;
    x += 4;
}
cout << s;
```

Напиши, что выведет программа.

254. What is the output? 20 сек · B1·7

```
int x = 10;
do {
    x -= 3;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

255. What is the output? 20 сек · B2·6

```
int x = 64;
while (x > 4)
    x /= 2;
cout << x;
```

Напиши, что выведет программа.

256. What is the output? 20 сек · B2·7

```
int x = 2;
do {
    x *= 2;
} while (x < 10);
cout << x;
```

Напиши, что выведет программа.

257. What is the output? 20 сек · B3·6

```
int n = 2468;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

258. What is the output? 20 сек · B3·7

```
int n = 5;
int s = 0;
do {
    s += n;
    n -= 2;
} while (n > 0);
cout << s;
```

Напиши, что выведет программа.

259. What is the output? 20 сек · B4·6

```
int x = 1, s = 0;
while (x < 20) {
    s += x;
    x *= 2;
}
cout << s;
```

Напиши, что выведет программа.

260. What is the output? 20 сек · B4·7

```
int x = 10;
do {
    x /= 2;
} while (x > 1);
cout << x;
```

Напиши, что выведет программа.

261. What is the output? 20 сек · B5·6

```
int x = 3, s = 0;
while (x <= 12) {
    s += x;
    x += 3;
}
cout << s;
```

Напиши, что выведет программа.

262. What is the output? 20 сек · B5·7

```
int x = 1;
do {
    x = x * 2 + 1;
} while (x < 10);
cout << x;
```

Напиши, что выведет программа.

263. What is the output? 20 сек · B6·6

```
int x = 100;
while (x > 10)
    x /= 3;
cout << x;
```

Напиши, что выведет программа.

264. What is the output? 20 сек · B6·7

```
int x = 7;
do {
    x -= 2;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

265. What is the output? 20 сек · B7·6

```
int x = 2, s = 0;
while (x <= 10) {
    s += x;
    x += 2;
}
cout << s;
```

Напиши, что выведет программа.

266. What is the output? 20 сек · B7·7

```
int x = 20;
do {
    x -= 6;
} while (x > 5);
cout << x;
```

Напиши, что выведет программа.

267. What is the output? 20 сек · B8·6

```
int x = 1;
while (x < 30)
    x *= 3;
cout << x;
```

Напиши, что выведет программа.

268. What is the output? 20 сек · B8·7

```
int x = 3;
do {
    x += 4;
} while (x < 15);
cout << x;
```

Напиши, что выведет программа.

269. What is the output? 20 сек · B9·6

```
int n = 532;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

270. What is the output? 20 сек · B9·7

```
int x = 12;
do {
    x -= 5;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

271. What is the output? 20 сек · B10·6

```
int x = 50;
while (x > 5) {
    x -= 7;
}
cout << x;
```

Напиши, что выведет программа.

272. What is the output? 20 сек · B10·7

```
int x = 1;
do {
    x *= 3;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

273. What is the output? 20 сек · B11·6

```
int x = 81;
while (x > 3)
    x /= 3;
cout << x;
```

Напиши, что выведет программа.

274. What is the output? 20 сек · B11·7

```
int x = 2, s = 0;
do {
    s += x;
    x += 3;
} while (x < 12);
cout << s;
```

Напиши, что выведет программа.

275. What is the output? 20 сек · B12·6

```
int x = 5, s = 0;
while (x > 0) {
    s += x;
    x -= 2;
}
cout << s;
```

Напиши, что выведет программа.

276. What is the output? 20 сек · B12·7

```
int x = 5;
do {
    x *= 2;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

277. What is the output? 20 сек · B13·6

```
int n = 864;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
cout << s;
```

Напиши, что выведет программа.

278. What is the output? 20 сек · B13·7

```
int x = 9;
do {
    x -= 4;
} while (x > 0);
cout << x;
```

Напиши, что выведет программа.

279. What is the output? 20 сек · B14·6

```
int x = 2, s = 0;
while (x < 20) {
    s += x;
    x *= 2;
}
cout << s;
```

Напиши, что выведет программа.

280. What is the output? 20 сек · B14·7

```
int x = 15;
do {
    x /= 2;
} while (x > 1);
cout << x;
```

Напиши, что выведет программа.

281. What is the output? 20 сек · B15·6

```
int x = 3, s = 0;
while (x <= 15) {
    s += x;
    x += 3;
}
cout << s;
```

Напиши, что выведет программа.

282. What is the output? 20 сек · B15·7

```
int x = 4;
do {
    x += 5;
} while (x < 20);
cout << x;
```

Напиши, что выведет программа.

283. How many times does the body run? `int i = 10; while (i < 10) i++;` новый

a) 1
b) 0
c) 10
d) infinite

284. How many times does the body run? `int i = 10; do i++; while (i < 10);` новый

a) 0
b) 1
c) 10
d) infinite

285. What is the last valid index of `double t[12];`? новый

a) 12
b) 11
c) 13
d) 0

286. What is the output? сложный

```
int a[5] = {2, 4, 6};
int s = 0;
for (int i = 0; i < 5; i++) s += a[i] * i;
cout << s;
```

a) 12
b) 16
c) 28
d) garbage (uninitialized)

287. What is the output? сложный

```
int i = 0, s = 0;
while (i < 10) {
    i++;
    if (i % 3 == 0) continue;
    if (i == 8) break;
    s += i;
}
cout << s << " " << i;
```

a) 28 10
b) 19 8
c) 27 8
d) 19 10

288. What is the output? сложный

```
int m[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
int s = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        if (i + j == 2 || i == j) s += m[i][j];
cout << s;
```

a) 30
b) 25
c) 15
d) 20

289. What is the output? сложный

```
int a[6] = {6, 3, 8, 1, 9, 2};
int k = 0;
for (int i = 0; i < 5; i++)
    if (a[i] > a[i + 1]) {
        int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t;
        k++;
    }
cout << a[5] << " " << a[0] << " " << k;
```

a) 9 1 5
b) 9 3 4
c) 2 3 3
d) 9 3 3

290. Which loop always executes its body at least once? новый

a) for
b) while
c) do...while
d) none of them

Ключи

230 — c	**231** — c	**232** — d	**233** — c	**234** — c	**235** — b
236 — c	**237** — c	**238** — d	**239** — b	**240** — d	**241** — b
242 — c	**243** — c	**244** — d	**245** — d	**246** — c	**247** — b
248 — d	**249** — d	**250** — d	**251** — c	**252** — b	**253** — 21 **(в ключе 27)**
254 — -2	**255** — 4	**256** — 16	**257** — 20 **(в ключе 15)**	**258** — 9 **(в ключе 20)**	**259** — 31
260 — 1	**261** — 30	**262** — 15	**263** — 3	**264** — -1	**265** — 30
266 — 2	**267** — 81 **(в ключе 27)**	**268** — 15	**269** — 10	**270** — -3	**271** — 1
272 — 27	**273** — 3	**274** — 26 **(в ключе 20)**	**275** — 9	**276** — 20	**277** — 18
278 — -3	**279** — 30	**280** — 1	**281** — 45	**282** — 24	**283** — b
284 — b	**285** — b	**286** — b	**287** — b	**288** — b	**289** — d
290 — c					

241. Отвечай `do {} while(condition)` — так в ключе. Но в C++ ОБА цикла продолжают работу, пока условие истинно, и выходят, когда оно ложно. Выход «по истинному условию» — это `repeat…until` из Паскаля. Вопрос перенесён из Паскаля неудачно.

253. s = 3 + 7 + 11 = 21 (при x = 15 условие x < 15 ложно). В ключе 27 — ошибка.

257. Сумма цифр 2468 = 2+4+6+8 = 20. В ключе 15 — ошибка.

258. s = 5 + 3 + 1 = 9 (дальше n = −1, цикл закончился). В ключе 20 — ошибка.

267. 1 → 3 → 9 → 27 → 81: при x = 27 условие 27 < 30 ещё истинно, поэтому умножаем ещё раз. В ключе 27 — ошибка.

274. s = 2 + 5 + 8 + 11 = 26. В ключе 20 — ошибка.

286. Недостающие элементы равны 0: {2, 4, 6, 0, 0}. 2·0 + 4·1 + 6·2 = 16.

287. Складываем 1, 2, 4, 5, 7 (3 и 6 пропущены continue) = 19; при i = 8 выходим.

288. Главная: 1+5+9, побочная: 3+5+7. Центр 5 входит в обе, но прибавляется один раз (условие через ||): 15 + 15 − 5 = 25.

289. Это один проход пузырька. i=0: 6>3 → обмен (3 6 8 1 9 2); i=1: 6<8; i=2: 8>1 → обмен (3 6 1 8 9 2); i=3: 8<9; i=4: 9>2 → обмен (3 6 1 8 2 9). Три обмена, максимум 9 «всплыл» в конец, a[0] = 3.

Блок 5. Функции: объявление, параметры по значению и по ссылке, const, значения по умолчанию, inline, массивы, рекурсия

291. What is the purpose of the return (expression) operation? банк №7

a) there is no such operator in C
b) it is used when a function returns no value
c) its purpose is undefined; by law it must be at the end of the program
d) returns the value of the expression to the operator that called the function
e) it adds beauty to the program

292. What is a variable called that is known only inside a function and is defined within it? банк №9

a) known variable
b) unknown variable
c) recursive variable
d) local variable
e) global variable

293. What is a variable called that is declared outside any block or function? банк №11

a) known variable
b) base variable
c) global variable
d) local variable
e) unary variable

294. What is the void keyword used for in a function header? банк №12

a) to show that the function stores a value
b) to show that the function returns a value
c) the void keyword is not used in a function header
d) to show that the function returns no value
e) No correct answer

295. A defined sequence of operations and descriptions that performs some sequence of actions is called ....? банк №121

a) function
b) recursion
c) procedure
d) code
e) No correct answer

296. What are the parameters at the beginning of a function description called? банк №122

a) global
b) argument
c) formal
d) actual
e) No correct answer

297. What do we call a function that calls itself? банк №124

a) procedure
b) function
c) recursion
d) operator
e) No correct answer

298. What is the purpose of the return (expression) operation? банк №125

a) its purpose is undefined; by law it must be at the end of the program
b) there is no such operator in C
c) it is used when a function returns no value
d) returns the value of the expression to the operator that called the function
e) No correct answer

299. ..... helps the compiler determine the number and type of arguments passed to a function. банк №126

a) file prototype
b) local prototype
c) function prototype
d) function identifier
e) No correct answer

300. What is the void keyword used for in a function header? банк №127

a) to show that the function returns a value
b) to show that the function returns no value
c) to show that the function stores a value
d) the void keyword is not used in a function header
e) No correct answer

301. If `return ;` is written at the end of the program, what type does the function return? банк №138

a) float
b) void
c) int
d) char
e) No correct answer

302. What does the function below determine? банк №160

```
int m(int x, int y)
{ int t;
if (x<y) t=x;
else t=y;
return t;}
```

a) a function that determines the smaller of two real numbers
b) a function that determines the equality of two integers
c) a function that determines the smaller of two integers
d) a function that determines the larger of two integers
e) No correct answer

303. What does the function below determine? банк №161

```
int m(int x, int y)
{ if (x>y)
return (x);
else return (y);}
```

a) a function that determines the larger of two integers
b) a function that determines the smaller of two integers
c) a function that determines the smaller of two real numbers
d) a function that determines the equality of two integers
e) No correct answer

304. What is the value of the variable a after the function is called? банк №169

```
int inc2(int vv)
{
vv++;
return vv;
}
void main()
{int a,b=5;
a=inc2(b);
}
```

a) 5
b) 7
c) 6
d) 8
e) No correct answer

305. Which function header passes the argument by reference? новый

a) void f(int x)
b) void f(int &x)
c) void f(int *x)
d) void f(const int x)

306. Which declaration with default parameters is correct? новый

a) void f(int a = 1, int b);
b) void f(int a = 1, int b, int c = 3);
c) void f(int a, int b = 2, int c = 3);
d) void f(int = 1, b);

307. What is a function that calls itself called? новый

a) inline
b) overloaded
c) recursive
d) static

308. What is the output? сложный

```
int f(int n)
{
    static int s = 0;
    s += n;
    return s;
}
//---
f(1); f(2);
cout << f(3);
```

a) 3
b) 6
c) 0
d) 1

309. What is the output? сложный

```
void g(int a[], int n) { for (int i = 0; i < n; i++) a[i] *= 2; }
void h(int v) { v *= 2; }
//---
int a[3] = {1, 2, 3};
int v = 5;
g(a, 3); h(v); h(a[0]);
cout << a[0] + a[1] + a[2] << " " << v;
```

a) 6 5
b) 24 10
c) 12 5
d) 12 10

310. What is the output? сложный

```
int r(int n)
{
    if (n <= 0) return 0;
    return n + r(n - 2);
}
//---
cout << r(7) << " " << r(6);
```

a) 28 21
b) 16 12
c) 15 12
d) 16 6

311. What is the output? сложный

```
void p(int n)
{
    if (n == 0) return;
    p(n / 10);
    cout << n % 10 << "-";
}
//---
p(352);
```

a) 2-5-3-
b) 3-5-2-
c) 352-
d) 3-5-2

312. What is the output? сложный

```
int x = 10;
void m(int &x) { x++; ::x += 5; }
//---
int y = 1;
m(y);
m(x);
cout << x << " " << y;
```

a) 16 2
b) 21 2
c) 20 2
d) 16 7

Ключи

291 — d	**292** — d	**293** — c	**294** — d	**295** — a	**296** — c
297 — c	**298** — d	**299** — c	**300** — b	**301** — b	**302** — c
303 — a	**304** — c	**305** — b	**306** — c	**307** — c	**308** — b
309 — c	**310** — b	**311** — b	**312** — b		

305. `int *x` — это указатель (передача адреса), а ссылка обозначается `&` в объявлении.

308. static-переменная помнит значение между вызовами: 1, потом 3, потом 6.

309. Массив передаётся по адресу, поэтому g удваивает элементы (2 + 4 + 6 = 12). h получает копии, и ни v, ни a[0] не меняются.

310. r(7) = 7 + 5 + 3 + 1 = 16; r(6) = 6 + 4 + 2 = 12.

311. Сначала рекурсивно уходим до старшей цифры, а печатаем на «подъёме» — в прямом порядке цифр.

312. Первый вызов: y → 2, глобальный x → 15. Во втором вызове параметр-ссылка — это сам глобальный x: x++ → 16, затем ::x += 5 → 21.

Блок 6. Тип char и строки: массивы символов, string.h и класс string

313. Which function is used to determine the length of a string? банк №129

a) strlen
b) strcmp
c) strcat
d) strcpy
e) No correct answer

314. Which header file contains the prototype of the strlen() function? банк №131

a) <string.h>
b) <conio.h>
c) <iostream.h>
d) <stdlib.h>
e) No correct answer

315. string.h is a library of the following functions: банк №132

a) graphics functions
b) mathematical functions
c) basic input / output library
d) string functions
e) No correct answer

316. Which of the following functions reads a string from a file? банк №135

a) fputs()
b) fgetc()
c) fread()
d) fgets()
e) No correct answer

317. Which of the following functions writes a string to a file? банк №136

a) fgets()
b) fputs()
c) fwrite()
d) fread()
e) No correct answer

318. Which of the following functions writes data to a file? банк №142

a) fgets ()
b) fread ()
c) fscanf ()
d) fprintf ()
e) No correct answer

319. Which of the following functions reads data from a file? банк №143

a) fgets ()
b) fwrite ()
c) fscanf()
d) fprintf ()
e) No correct answer

320. What does the strcat(stroka1, stroka2) function do? банк №144

a) It compares the strings stroka1 and stroka2
b) It joins the strings stroka1 and stroka2 and assigns the result to the variable stroka1
c) It searches for the second given string inside the first string
d) It searches the first string for any character of the second string
e) No correct answer

321. What does the strcmp(stroka1,stroka2) function do? банк №145

a) it joins two strings
b) it is used to compare two strings
c) it searches the first string for any character of the second string
d) it searches for the second given string inside the first string
e) No correct answer

322. What is the result of this code? банк №146

```
main() { printf("%d\n",strcmp("Salem","Salem")); getch(); }
```

a) 7
b) 0
c) 2
d) 1
e) No correct answer

323. What does the strcpy(str1,str2) function do? банк №147

a) it is used to make a copy of a string; here the string in variable str2 is copied to variable str1
b) it searches for the second given string inside the first string
c) it is used to compare two strings
d) it joins two strings
e) No correct answer

324. What does the strstr(str1, str2) function do? банк №148

a) it searches for the second given string inside the first string
b) it joins two strings
c) it is used to compare two strings
d) it is used to make a copy of a string
e) No correct answer

325. What does the strrev(str) function do? банк №149

a) it is used to compare strings
b) it is used to delete a string
c) it is used to make a copy of a string
d) reverses all characters of the given string
e) No correct answer

326. The strncpy(S1,CS2,k) function: банк №150

a) copies the last k characters of string CS2 to S1
b) copies the first k characters of string CS2 to S1
c) deletes k characters from strings S1 and CS2
d) deletes the first k characters of string CS2 from S1
e) No correct answer

327. The strnset(S,ch,k) function: банк №151

a) repeats the character ch k times in string S
b) removes the characters ch and k from string S
c) deletes the character ch k times in string S
d) replaces the character ch in string S with character k
e) No correct answer

328. The strncat(S1,CS2,k) function: банк №152

a) appends k characters to strings CS2 and S1
b) appends the last k characters of string CS2 to string S1
c) repeats the character CS2 k times in string S1
d) appends the first k characters of string CS2 to string S1
e) No correct answer

329. The strlwr(S) function: банк №153

a) converts the characters of string S to Roman numerals
b) converts the characters of string S to lowercase letters
c) converts the characters of string S to Arabic numerals
d) converts the characters of string S to uppercase letters
e) No correct answer

330. The strupr(S) function: банк №154

a) converts the characters of string S to lowercase letters
b) converts the characters of string S to Arabic numerals
c) converts the characters of string S to uppercase letters
d) converts the characters of string S to Roman numerals
e) No correct answer

331. What is the strncmp(CS1,CS2,k) function intended for? банк №155

a) appends the first k characters of string CS1 to string CS2
b) compares only the first k characters of strings CS1 and CS2
c) compares only the last k characters of strings CS1 and CS2
d) appends the last k characters of string CS1 to string CS2
e) No correct answer

332. The stricmp(CS1,CS2) function: банк №156

a) it is used to compare two strings
b) takes into account the difference between uppercase and lowercase letter codes when comparing
c) it joins two strings
d) ignores the difference between uppercase and lowercase letter codes when comparing
e) No correct answer

333. The strchr(CS,ch) function: банк №157

a) scans string CS from left to right until the character ch is found
b) it joins two strings
c) it is used to compare two strings
d) scans string CS from the end to the beginning until the character ch is found
e) No correct answer

334. The strrchr(CS,ch) function: банк №158

a) scans string CS from left to right until the character ch is found
b) it is used to compare two strings
c) scans string CS from the end to the beginning until the character ch is found
d) it joins two strings
e) No correct answer

335. To close a file after you have finished working with it, the following function is used: банк №162

a) fclose()
b) fgets()
c) fexit()
d) fgetc()
e) No correct answer

336. Show the function for opening a file in C. банк №163

a) fclose()
b) fnew()
c) fexit()
d) fopen()
e) No correct answer

337. What is the result of the code below? банк №178

```
char s1[ 50 ] = "jack";
char s2[ 50 ] = "jill";
char s3[ 50 ] , *sptr;
cout << strlen( s1 ) + strlen( s2 ) << endl;
```

a) 7
b) 8
c) 10
d) 9
e) No correct answer

338. How many bytes does `char s[] = "Hello";` occupy? новый

a) 5
b) 6
c) 4
d) 10

339. Which is the correct way to compare two C-strings `a` and `b` for equality? новый

a) if (a == b)
b) if (a = b)
c) if (strcmp(a, b) == 0)
d) if (strcmp(a, b) == 1)

340. What does `cin >> name;` read from the input "John Smith"? новый

a) John Smith
b) John
c) Smith
d) nothing

341. What is the output? сложный

```
char s[] = "abcdef";
char *p = s + 2;
*p = 'X';
p += 2;
cout << s << " " << p << " " << strlen(p);
```

a) abcdef ef 2
b) abXdef ef 2
c) abXdef e 1
d) aXcdef def 3

342. What is the output? сложный

```
char s[20] = "data";
s[2] = '\0';
cout << s << " " << strlen(s) << " " << sizeof(s);
```

a) data 4 20
b) da 2 20
c) da 2 3
d) d 1 20

343. What is the output? сложный

```
char w[] = "kazakhstan";
int k = 0;
for (int i = 0; w[i]; i++)
    if (w[i] == 'a') k++;
    else w[i] = w[i] - 'a' + 'A';
cout << k << " " << w;
```

a) 3 KAZAKHSTAN
b) 4 KaZaKHSTaN
c) 3 KaZaKHSTaN
d) 3 kazakhstan

344. What is the output? сложный

```
char a[10] = "12", b[10] = "9";
cout << strcmp(a, b) << " " << (atoi(a) > atoi(b));
```

a) 1 1
b) -8 1
c) -1 0
d) 0 1

Ключи

313 — a	**314** — a	**315** — d	**316** — d	**317** — b	**318** — d
319 — c	**320** — b	**321** — b	**322** — b	**323** — a	**324** — a
325 — d	**326** — b	**327** — a	**328** — d	**329** — b	**330** — c
331 — b	**332** — d	**333** — a	**334** — c	**335** — a	**336** — d
337 — b	**338** — b	**339** — c	**340** — b	**341** — b	**342** — b
343 — c	**344** — b				

322. `strcmp` возвращает 0, если строки одинаковые. В казахском оригинале ключ ошибочно «1», в английском файле уже исправлено на 0.

338. 5 букв + завершающий '\0'.

340. Оператор `>>` читает до первого пробела.

341. p указывает на s[2], меняем его на 'X'. Потом p сдвигается на s[4]; вывод char* печатает строку с этого места до '\0': "ef".

342. '\0' в середине обрывает строку для всех строковых функций и для cout. sizeof — размер массива, он не меняется.

343. Буквы 'a' (их 3) остаются как есть, остальные маленькие буквы переводятся в большие формулой `c - 'a' + 'A'`.

344. Строки сравниваются посимвольно: '1' (49) < '9' (57), поэтому "12" < "9" и результат отрицательный (Borland возвращает разность кодов 49 − 57 = −8). Как числа 12 > 9.

Блок 7. Ключевые понятия и принципы объектно-ориентированного программирования

345. A class is best described as: новый

a) a variable that stores several values
b) a blueprint (template) that describes the data and behavior of objects
c) a function that returns an object
d) a library of functions

346. An object is: новый

a) a description of a class
b) an instance of a class
c) a method of a class
d) a keyword of C++

347. Hiding the internal data of a class and providing access only through methods is called: новый

a) inheritance
b) polymorphism
c) encapsulation
d) recursion

348. Creating a new class based on an existing class is called: новый

a) inheritance
b) encapsulation
c) abstraction
d) overloading

349. The ability of the same function call to behave differently for objects of different classes is called: новый

a) encapsulation
b) polymorphism
c) inheritance
d) abstraction

350. Showing only essential features of an object and hiding unnecessary details is called: новый

a) abstraction
b) polymorphism
c) inheritance
d) iteration

351. Which pair is an example of the "is-a" relationship (inheritance)? новый

a) Car — Engine
b) Poodle — Dog
c) Book — Page
d) Computer — Keyboard

352. In OOP, a function declared inside a class is called: новый

a) a field
b) an attribute
c) a method
d) an object

353. The set of current values of all fields of an object is its: новый

a) behavior
b) state
c) interface
d) method

354. Which kind of polymorphism is function overloading? новый

a) static (compile-time)
b) dynamic (run-time)
c) virtual
d) it is not polymorphism

355. What is the output? сложный

```
class A {
public:
    void show() { cout << "A"; }
};
class B : public A {
public:
    void show() { cout << "B"; }
};
//---
B b;
A *p = &b;
p->show();
b.show();
```

a) BB
b) AB
c) BA
d) AA

356. What is the output? сложный

```
class A {
public:
    virtual void show() { cout << "A"; }
};
class B : public A {
public:
    void show() { cout << "B"; }
};
//---
B b;
A *p = &b;
A a = b;
p->show();
a.show();
```

a) BB
b) BA
c) AB
d) AA

357. Which line causes a compilation error? сложный

```
class Counter {
    int value;                // line 1
public:
    void inc() { value++; }   // line 2
};
int main() {
    Counter c;                // line 3
    c.inc();                  // line 4
    c.value = 10;             // line 5
}
```

a) line 2
b) line 3
c) line 4
d) line 5

Ключи

345 — b **346** — b **347** — c **348** — a **349** — b **350** — a
351 — b **352** — c **353** — b **354** — a **355** — b **356** — b
357 — d

351. Пудель является собакой. Остальные пары — «имеет» (композиция).

355. Метод не виртуальный, поэтому через указатель типа A* вызывается A::show(). Напрямую у объекта b — B::show().

356. p указывает на объект B, функция виртуальная → B. A `A a = b;` создаёт **копию только части A** (срезка объекта), поэтому a — настоящий A.

357. По умолчанию члены класса — private. Изнутри класса (line 2) обращаться можно, снаружи (line 5) нельзя.

Блок 8. Классы и объекты в C++

358. By default, members of a `class` are: новый

a) public
b) private
c) protected
d) static

359. By default, members of a `struct` in C++ are: новый

a) public
b) private
c) protected
d) const

360. Which operator accesses a member through a pointer to an object? новый

a) .
b) ::
c) ->
d) &

361. What is `this` inside a member function? новый

a) the name of the class
b) a pointer to the object for which the method was called
c) a copy of the object
d) a static member

362. How is the method `area` of class `Circle` defined outside the class? новый

a) double Circle.area() { ... }
b) double area::Circle() { ... }
c) double Circle::area() { ... }
d) double Circle->area() { ... }

363. A static data member of a class is: новый

a) shared by all objects of the class
b) created separately for each object
c) accessible only through `this`
d) automatically public

364. Which keyword protects a member from outside access but allows access from derived classes? новый

a) private
b) public
c) protected
d) static

365. What is the output? сложный

```
class C {
public:
    static int n;
    int k;
    C() { n++; k = n * 10; }
};
int C::n = 0;
//---
C a, b;
C c;
cout << a.k + b.k + c.k << " " << C::n;
```

a) 30 3
b) 60 3
c) 90 3
d) 60 1

366. What is the output? сложный

```
class V {
public:
    int x;
    void set(int x) { x = x; }
    void set2(int x) { this->x = x; }
};
//---
V v;
v.x = 1;
v.set(5);
cout << v.x;
v.set2(7);
cout << v.x;
```

a) 57
b) 17
c) 55
d) 11

367. What is the output? сложный

```
class Box {
public:
    int v;
};
void f(Box b) { b.v = 100; }
void g(Box &b) { b.v += 5; }
void h(Box *b) { b->v *= 2; }
//---
Box x;
x.v = 1;
f(x); g(x); h(&x);
cout << x.v;
```

a) 100
b) 210
c) 12
d) 6

368. Which line does NOT compile? сложный

```
class T {
    int a;
public:
    int b;
    int getA() const { return a; }
    void setA(int v) const { a = v; }   // line X
};
```

a) int a;
b) int b;
c) int getA() const { return a; }
d) void setA(int v) const { a = v; }

Ключи

358 — b	**359** — a	**360** — c	**361** — b	**362** — c	**363** — a
364 — c	**365** — b	**366** — b	**367** — c	**368** — d	

365. a.k = 10, b.k = 20, c.k = 30 (k вычисляется в момент создания каждого объекта). Счётчик n общий, он равен 3.

366. В `set` параметр x присваивается сам себе, поле не меняется. В `set2` через `this->x` меняется поле.

367. f получает копию (оригинал не меняется), g — ссылку: 1 + 5 = 6, h — указатель: 6 · 2 = 12.

368. const-метод не может изменять поля объекта.

Блок 9. Интегрированная среда разработки Borland C++ Builder

369. What does IDE stand for? новый

a) Internal Debugging Engine
b) Integrated Development Environment
c) Interactive Design Editor
d) Integrated Data Exchange

370. Which window of C++ Builder shows the properties and events of the selected component? новый

a) Component Palette
b) Code Editor
c) Object Inspector
d) Project Manager

371. Where are ready-made components such as TButton and TEdit selected? новый

a) Component Palette
b) Object Inspector
c) Messages window
d) Watch list

372. Which file stores the description of a form (its components and their property values)? новый

a) Unit1.cpp
b) Unit1.h
c) Unit1.dfm
d) Project1.res

373. Which key compiles (if needed) and runs the program in C++ Builder? новый

a) F7
b) F8
c) F9
d) F12

374. Which key switches between the form and its code? новый

a) F9
b) F12
c) F5
d) Ctrl+F9

375. A program compiles and runs, but prints a wrong result. What kind of error is it? новый

a) syntax error
b) semantic error
c) logical error
d) linker error

376. Which debugging command executes the current line without entering called functions? новый

a) Trace Into (F7)
b) Step Over (F8)
c) Run to Cursor (F4)
d) Program Reset (Ctrl+F2)

377. What is a breakpoint? новый

a) an error that stops compilation
b) a marked line where program execution pauses in the debugger
c) the end of the main function
d) a `break` statement in a loop

378. Which library of visual components is used in Borland C++ Builder? новый

a) STL
b) MFC
c) VCL
d) Qt

379. Missing semicolon after a statement is an example of: новый

a) a syntax error
b) a logical error
c) a runtime error
d) a semantic error

380. Division by zero during program execution is: новый

a) a syntax error
b) a runtime error
c) a linker error
d) not an error

Ключи

369 – b	**370** – c	**371** – a	**372** – c	**373** – c	**374** – b
375 – c	**376** – b	**377** – b	**378** – c	**379** – a	**380** – b

Блок 10. Создание проекта. Работа с формами. Дизайнер форм

381. How do you create a new windowed (GUI) application in C++ Builder? новый

a) File → New → Console Wizard
b) File → New → Application
c) Project → Build
d) Component → New Component

382. Which file contains the C++ code of event handlers of Form1? новый

a) Unit1.dfm
b) Unit1.cpp
c) Project1.res
d) Project1.bpr

383. Which property of `TEdit` contains the text entered by the user? новый

a) Caption
b) Text
c) Lines
d) Value

384. Which property of `TLabel` holds the text displayed on it? новый

a) Caption
b) Text
c) Name
d) Items

385. Which function converts the text of an Edit into an integer? новый

a) IntToStr
b) StrToInt
c) atoi
d) FloatToStr

386. What does `Label1->Caption = Edit1->Text + Edit2->Text;` show if the user typed 2 and 3? новый

a) 5
b) 23
c) 2+3
d) an error

387. Which method opens a form so that the other forms of the application are blocked until it is closed? новый

a) Show()
b) ShowModal()
c) Open()
d) Visible = true

388. What must be set so that a button created at run time appears on the form? новый

a) its Owner
b) its Parent
c) its Name
d) its Tag

389. What do you add to Unit1.cpp to be able to use Form2 from Form1? новый

a) #include "Unit2.cpp"
b) #include "Unit2.h"
c) #include <Form2>
d) nothing

390. Which property value makes a component fill all free space of its container? новый

a) alTop
b) alNone
c) alClient
d) alLeft

391. What does the abbreviation GUI mean? новый

a) General User Input
b) Graphical User Interface
c) Global Unit Index
d) Graphical Unit Inspector

392. Which part of the form class is managed by the IDE (components and event handlers)? новый

a) private
b) public
c) __published
d) protected

Ключи

381 — b	382 — b	383 — b	384 — a	385 — b	386 — b
387 — b	388 — b	389 — b	390 — c	391 — b	392 — c

386. Text — строки; `+` для строк склеивает их.

Блок 11. Работа с формой: свойства, методы и события

393. `Width`, `Caption` and `Color` of a form are its: новый

a) methods
b) events
c) properties
d) handlers

394. `Show()`, `Close()` and `Hide()` of a form are its: новый

a) properties
b) methods
c) events
d) fields

395. `OnClick`, `OnCreate` and `OnKeyPress` are: новый

a) properties
b) methods
c) events
d) constructors

396. Which form event is the best place for initial settings when the program starts? новый

a) OnClick
b) OnPaint
c) OnCreate
d) OnClose

397. Which property makes a component visible but disabled (grayed out)? новый

a) Visible = false
b) Enabled = false
c) ReadOnly = false
d) Hide()

398. Which property value places the form in the center of the screen when it opens? новый

a) poDesigned
b) poScreenCenter
c) wsMaximized
d) alClient

399. What does the `Sender` parameter of an event handler contain? новый

a) the name of the form
b) a pointer to the object that generated the event
c) the key that was pressed
d) the mouse coordinates

400. In the handler `FormCloseQuery(TObject *Sender, bool &CanClose)`, how do you prevent the form from closing? новый

a) Close();
b) CanClose = false;
c) Sender = NULL;
d) return 0;

401. Which property of a TTimer sets the period between `OnTimer` events? новый

a) Period
b) Delay
c) Interval
d) Tick

402. In the `OnKeyPress(TObject *Sender, char &Key)` handler, what does `Key = 0;` do? новый

a) closes the program
b) cancels the entered character
c) presses the key again
d) clears the Edit

403. Which is a read-only property of a form? новый

a) Caption
b) Width
c) ComponentCount
d) Color

404. Which value of `BorderStyle` does NOT allow the user to resize the window? новый

a) bsSizeable
b) bsSingle
c) bsSizeToolWin
d) wsNormal

Ключи

393 — c	**394** — b	**395** — c	**396** — c	**397** — b	**398** — b
399 — b	**400** — b	**401** — c	**402** — b	**403** — c	**404** — b

Блок 12. Иерархия классов: наследование, переопределение, виртуальные функции

405. How do you declare class `Dog` publicly derived from class `Animal`? новый

a) class Dog extends Animal { };
b) class Dog :: Animal { };
c) class Dog : public Animal { };
d) class Dog inherits public Animal { };

406. Which members of a base class are NOT accessible in a derived class? новый

a) public
b) protected
c) private
d) static public

407. In which order are constructors called when an object of a derived class is created? новый

a) derived first, then base
b) base first, then derived
c) only the derived constructor is called
d) in random order

408. A class that contains at least one pure virtual function is called: новый

a) a friend class
b) a static class
c) an abstract class
d) a template class

409. How is a pure virtual function declared? новый

a) virtual void f() {}
b) void f() = 0;
c) virtual void f() = 0;
d) abstract void f();

410. Which class is the root of the VCL class hierarchy? новый

a) TComponent
b) TControl
c) TObject
d) TForm

411. Which VCL class introduces the `Owner` and `Name` of components? новый

a) TObject
b) TPersistent
c) TComponent
d) TGraphicControl

412. Which VCL class is the ancestor of controls that have their own window handle and can receive focus? новый

a) TGraphicControl
b) TWinControl
c) TPersistent
d) TCustomLabel

413. TLabel is derived (through TCustomLabel) from: новый

a) TWinControl
b) TGraphicControl
c) TButtonControl
d) TForm

414. With `private` inheritance, public members of the base class become in the derived class: новый

a) public
b) protected
c) private
d) inaccessible

415. What is the output? сложный

```
class P {
public:
    P() { cout << "P"; }
    ~P() { cout << "p"; }
};
class Q : public P {
public:
    Q() { cout << "Q"; }
    ~Q() { cout << "q"; }
};
//---
{
    Q a, b;
}
```

a) PQqpPQqp
b) PQPQqpqp
c) PQPQpqpq
d) QPQPqpqp

416. What is the output? сложный

```
class A {
public:
    virtual int f() { return 1; }
    int g() { return f() * 10; }
};
class B : public A {
public:
    int f() { return 2; }
};
//---
B b;
A a;
A *p = &b;
cout << a.g() << " " << p->g() << " " << b.g();
```

a) 10 10 10
b) 10 20 20
c) 10 10 20
d) 20 20 20

417. What is the output? сложный

```
class Base {
public:
    Base() { show(); }
    virtual void show() { cout << "Base "; }
};
class Der : public Base {
public:
    Der() { show(); }
    void show() { cout << "Der "; }
};
//---
Der d;
```

a) Der Der
b) Base Der
c) Base Base
d) Der Base

418. Which line causes a compilation error? сложный

```
class Shape { public: virtual double area() = 0; };
class Sq : public Shape { double a; public: Sq(double x) : a(x) {} double area() { return
a*a; } };
Shape *p = new Sq(2);    // line 1
Sq s(3);                 // line 2
Shape &r = s;            // line 3
Shape t;                 // line 4
```

a) line 1
b) line 2
c) line 3
d) line 4

Ключи

405 — c	**406** — c	**407** — b	**408** — c	**409** — c	**410** — c
411 — c	**412** — b	**413** — b	**414** — c	**415** — b	**416** — b
417 — b	**418** — d				

415. Объекты создаются по очереди (a, потом b), каждый: P, затем Q. Уничтожаются в обратном порядке создания (сначала b, потом a), каждый: q, затем p.

416. g() не виртуальный, но внутри него f() виртуальная, поэтому выбирается по реальному объекту: у a — 1, у b — 2.

417. Пока работает конструктор Base, объект ещё не стал Der, поэтому виртуальный вызов из конструктора базы попадает в Base::show(). Потом конструктор Der вызывает Der::show().

418. Объект абстрактного класса создать нельзя. Указатели и ссылки на абстрактный класс можно.

Блок 13. Пользовательский класс. Конструктор и деструктор

419. What is the name of a constructor of class `Book`? новый

a) constructor
b) Book
c) ~Book
d) init

420. What return type does a constructor have? новый

a) void
b) int
c) no return type at all
d) the class type

421. How many destructors can a class have? новый

a) any number (they can be overloaded)
b) two
c) exactly one
d) none

422. Which is the correct signature of a copy constructor of class `A`? новый

a) A(A other)
b) A(const A &other)
c) A copy(const A &other)
d) void A(const A &other)

423. When is the destructor of a local object called? новый

a) never
b) when `main` starts
c) when execution leaves the block where the object was declared
d) immediately after the constructor

424. A class has only the constructor `Point(int x, int y)`. Which declaration does NOT compile? новый

a) Point p(1, 2);
b) Point *q = new Point(0, 0);
c) Point r;
d) Point s = Point(3, 4);

425. Which members MUST be initialized in the constructor initializer list? новый

a) static members
b) const members and references
c) public members
d) all int members

426. What does `A a();` declare? новый

a) an object a created with the default constructor
b) a function a that returns an object of type A
c) an array of A
d) a compilation error

427. What is the output? сложный

```
class X {
    int v;
public:
    X(int a = 5) : v(a) { cout << v; }
    ~X() { cout << -v; }
};
//---
{
    X a, b(2);
    X *c = new X(7);
    delete c;
}
```

a) 527-5-2-7
b) 527-7-2-5
c) 52-2-57-7
d) 27-7-2

428. What is the output? сложный

```
class Cnt {
public:
    static int alive;
    Cnt() { alive++; }
    Cnt(const Cnt &) { alive++; }
    ~Cnt() { alive--; }
};
int Cnt::alive = 0;
void use(Cnt c) { cout << Cnt::alive; }
//---
Cnt a;
{
    Cnt b, c;
    use(a);
}
cout << Cnt::alive;
```

a) 31
b) 41
c) 33
d) 40

429. What is the output? сложный

```
class P {
    int *d;
public:
    P(int v) { d = new int(v); }
    P(const P &o) { d = new int(*o.d); }
    ~P() { delete d; }
    void set(int v) { *d = v; }
    int get() { return *d; }
};
//---
P a(10);
P b(a);
b.set(20);
cout << a.get() + b.get();
```

a) 40
b) 20
c) 30
d) program crashes

Ключи

419 — b	**420** — c	**421** — c	**422** — b	**423** — c	**424** — c
425 — b	**426** — b	**427** — b	**428** — b	**429** — c	

427. Создание: a (5), b (2), c (7). `delete c` → −7. Конец блока: сначала b (−2), потом a (−5).

428. Внутри use живут a, b, c и копия-параметр — 4 объекта. После блока остался только a — 1.

429. Конструктор копирования делает глубокую копию (новая ячейка памяти), поэтому изменение b не затрагивает a: 10 + 20 = 30.

Блок 14. Пользовательский класс: свойства

430. A method that returns the value of a private field is called: новый

a) a setter (mutator)
b) a getter (accessor)
c) a constructor
d) a destructor

431. How do you make a property read-only? новый

a) declare its field public
b) provide only a getter (or `read` without `write`)
c) provide only a setter
d) make the getter static

432. Where should the validation of a new property value be placed? новый

a) in the getter
b) in the setter
c) in the destructor
d) in `main`

433. Which C++ Builder keyword declares a property with `read` and `write` specifiers? новый

a) property
b) __property
c) __published
d) __fastcall

434. Properties declared in which section of a component are shown in the Object Inspector? новый

a) private
b) public
c) protected
d) __published

435. In `__property int Size = { read = FSize, write = SetSize };` what is called when you write `obj.Size = 5;`? новый

a) FSize is assigned directly
b) SetSize(5)
c) the getter
d) nothing, it is a compilation error

436. Which file usually contains the class declaration (interface)? новый

a) the header file (.h)
b) the implementation file (.cpp)
c) the form file (.dfm)
d) the project file (.bpr)

437. What is the purpose of `#ifndef RectH / #define RectH / #endif` in a header file? новый

a) to declare a constant
b) to prevent the header from being included twice
c) to speed up the program
d) to define a property

438. What is the output? сложный

```
class Box {
    int FV;
    void SetV(int v) { FV = (v > 10) ? 10 : v; }
public:
    Box() : FV(0) { }
    __property int V = { read = FV, write = SetV };
};
//---
Box b;
b.V = 7;
b.V = b.V + 5;
cout << b.V << " ";
b.V = 3;
b.V += b.V;
cout << b.V;
```

a) 12 6
b) 10 6
c) 10 10
d) 7 3

439. What is the output? сложный

```
class Acc {
    int bal;
public:
    Acc() : bal(100) { }
    int get() const { return bal; }
    bool take(int s) { if (s > bal) return false; bal -= s; return true; }
};
//---
Acc a;
int ok = 0;
for (int i = 1; i <= 5; i++)
    if (a.take(i * 15)) ok++;
cout << ok << " " << a.get();
```

a) 5 0
b) 3 10
c) 4 -50
d) 3 0

Ключи

430 — b **431** — b **432** — b **433** — b **434** — d **435** — b
436 — a **437** — b **438** — b **439** — b

438. 7 + 5 = 12, сеттер обрезает до 10. Потом 3 + 3 = 6 (`+=` читает через read и записывает через write).

439. Снимается 15, 30, 45 (итого 90, осталось 10); 60 и 75 больше остатка, отклонены.

Блок 15. Создание пользовательского компонента

440. Which class must a class inherit from (directly or indirectly) to become a component placed on the Component Palette? новый

a) TObject
b) TPersistent
c) TComponent
d) TForm

441. Which menu command starts the wizard for creating a new component? новый

a) File → New → Application
b) Component → New Component
c) Project → Add to Project
d) Tools → Options

442. Which function adds a component to a page of the Component Palette? новый

a) InstallComponent
b) AddToPalette
c) RegisterComponents
d) CreateForm

443. In which section must a property be declared to appear in the Object Inspector? новый

a) private
b) protected
c) public
d) __published

444. What is a component event in VCL? новый

a) a public method
b) a private field
c) a property of a method-pointer type (for example, TNotifyEvent)
d) a constructor

445. Which package is commonly used to install user components? новый

a) vcl.bpl
b) dclusr.bpk
c) rtl.lib
d) Project1.bpr

446. Which ancestor is best for a component that only draws itself and does not need keyboard focus? новый

a) TWinControl
b) TGraphicControl
c) TComponent
d) TForm

447. Why do you call `TEdit::KeyPress(Key)` inside an overridden `KeyPress` of a component derived from TEdit? новый

a) it is required by the compiler
b) to keep the standard behavior and the user's OnKeyPress handler
c) to register the component
d) to delete the component

448. Which code correctly fires an optional event `FOnChange` of a component? новый

a) FOnChange();
b) if (FOnChange) FOnChange(this);
c) OnChange = this;
d) FOnChange = NULL;

449. How is the namespace for the Register function named for the unit file `MyButton.cpp`? новый

a) MyButton
b) MYBUTTON
c) Mybutton
d) mybutton

Ключи

440 — c **441** — b **442** — c **443** — d **444** — c **445** — b
446 — b **447** — b **448** — b **449** — c

Финальный блок: смешанные сложные вопросы

450. What is the output? сложный

```
int f(int &a, int b)
{
    a += b;
    b += a;
    return a + b;
}
//---
int x = 2, y = 3;
int z = f(x, y);
cout << x << " " << y << " " << z;
```

a) 5 8 13
b) 5 3 13
c) 2 3 13
d) 5 3 10

451. What is the output? сложный

```
int a[6] = {1, 2, 3, 4, 5, 6};
int *p = a, s = 0;
while (p < a + 6) {
    s += *p;
    p += 2;
}
cout << s << " " << *(p - 2);
```

a) 21 6
b) 9 5
c) 9 6
d) 12 5

452. What is the output? сложный

```
class A {
public:
    int v;
    A(int x = 1) : v(x) { }
    A add(A o) { return A(v + o.v); }
};
//---
A p(5), q;
A r = p.add(q).add(p);
cout << r.v << " " << q.v;
```

a) 6 1
b) 11 1
c) 11 5
d) 10 1

453. What is the output? сложный

```
int t(int n)
{
    if (n < 3) return 1;
    return t(n - 1) + t(n - 3);
}
//---
cout << t(6);
```

a) 4
b) 6
c) 5
d) 8

454. What is the output? сложный

```
char s[] = "Programming";
int k = 0;
for (int i = 0; s[i]; i++)
    if (s[i] == s[i + 1]) k++;
cout << k << " " << s + 7 << " " << s[strlen(s) - 1];
```

a) 2 ming g
b) 1 ming g
c) 1 ing g
d) 1 ming n

455. What is the output? сложный

```
class B {
public:
    B() { cout << "B"; }
    virtual void f() { cout << "b"; }
    virtual ~B() { cout << "~B"; }
};
class D : public B {
public:
    D() { cout << "D"; }
    void f() { cout << "d"; }
    ~D() { cout << "~D"; }
};
void call(B obj) { obj.f(); }
void callRef(B &obj) { obj.f(); }
//---
D d;
call(d);
callRef(d);
cout << "|";
```

a) BDd~Bd|~D~B
b) BDb~Bd|~D~B
c) BDbd|~D~B
d) BDdd|~D~B

456. What is the output? сложный

```
int m[3][4];
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        m[i][j] = i * 4 + j;
int *p = &m[0][0];
cout << *(p + 6) << " " << m[2][1] << " " << sizeof(m) / sizeof(m[0]);
```

a) 6 9 4
b) 6 9 3
c) 9 6 3
d) 5 9 3

457. What is the output? сложный

```
int x = 5;
int y = x > 3 ? x < 10 ? 1 : 2 : 3;
int z = (x & 1) ? x << 1 : x >> 1;
cout << y << z;
```

a) 210
b) 110
c) 12
d) 310

458. What is the output? сложный

```
class Cnt {
    static int made;
    int id;
public:
    Cnt() : id(++made) { }
    Cnt(const Cnt &o) : id(o.id * 10) { ++made; }
    int getId() const { return id; }
    static int total() { return made; }
};
int Cnt::made = 0;
Cnt echo(Cnt c) { return c; }
//---
Cnt a, b;
Cnt c = b;
cout << a.getId() << b.getId() << c.getId() << " " << Cnt::total();
```

a) 123 3
b) 1220 3
c) 1220 2
d) 122 3

459. What is the output? сложный

```
int s = 0;
for (int i = 1; i <= 5; i++) {
    if (i == 2) continue;
    for (int j = i; j <= 5; j++) {
        if (j == 4) break;
        s += j;
    }
}
cout << s;
```

a) 10 **b)** 14
c) 9 **d)** 6

Ключи

450 — b **451** — b **452** — b **453** — b **454** — b **455** — b
456 — b **457** — b **458** — b **459** — b

450. a — ссылка на x: x = 2 + 3 = 5. b — копия y: b = 3 + 5 = 8, но y остаётся 3. Результат 5 + 8 = 13.

451. p проходит a[0], a[2], a[4]: 1 + 3 + 5 = 9. После цикла p = a + 6 (сразу за последним элементом), значит p − 2 указывает на a[4] = 5.

452. q создан с параметром по умолчанию: 1. p.add(q) = A(6), затем A(6).add(p) = A(11).

453. t(0) = t(1) = t(2) = 1; t(3) = t(2) + t(0) = 2; t(4) = t(3) + t(1) = 3; t(5) = t(4) + t(2) = 4; t(6) = t(5) + t(3) = 4 + 2 = 6.

454. Двойная буква одна — «mm». `s + 7` — указатель на 8-й символ: "ming". Последний символ — 'g'.

455. При передаче по значению создаётся копия типа B (срезка): вызывается B::f() → «b», копия уничтожается → «~B». По ссылке — настоящий D → «d». После `|` уничтожается d: ~D, ~B.

456. Матрица лежит в памяти строками подряд, поэтому `p + 6` — это m[1][2] = 6. m[2][1] = 2·4 + 1 = 9. sizeof(m)/sizeof(m[0]) — число строк, 3.

457. Вложенный тернарный: x > 3 и x < 10 → 1. x нечётный → x << 1 = 10.

458. a и b получают id 1 и 2. c — копия b: id = 20, счётчик made = 3. (Функция echo в этом коде не вызывается — не попадись!)

459. i=1: j = 1, 2, 3, на j = 4 break → 6. i=2 пропущен (continue). i=3: j = 3 → 3, потом break. i=4: сразу break. i=5: j = 5 → 5. Итого 6 + 3 + 5 = 14.

Как готовиться (план на 3 вечера)

1. **Вечер 1.** Прочитай карточки лекций 1–6 (ниже) и реши тест-блоки 1–6. Ошибся — открой лекцию и перечитай нужный пункт.
2. **Вечер 2.** Карточки 7–15, тест-блоки 7–15. Особое внимание: конструкторы/деструкторы (порядок), `virtual`, доступ при наследовании, свойства и события в C++ Builder.
3. **Вечер 3.** Реши «Итоговые задачи» и финальный блок сложных вопросов **на время**: 20 секунд на задачу «что выведет?», как в тесте преподавателя.

Что проверяет midterm и экзамен

По силлабусу: **Midterm 1** — темы 1–6, **Midterm 2** — темы 1–14, **экзамен** — темы 1–15. Итоговая оценка = экзамен × 0,4 + рейтинг допуска × 0,6. Тест по коду проходи «в уме», как компилятор: строка за строкой, записывая значения переменных.

Главные ловушки курса

Ловушка	Правильно
`7 / 2`	**3** (целое деление); `7.0 / 2` = 3.5
`pow(x, 1/3)`	1/3 = 0 → нужно `pow(x, 1.0/3)`
`if (x = 5)`	присваивание, всегда истина → `if (x == 5)`
`1 < x < 10`	всегда истина → `x > 1 && x < 10`
switch без `break`	выполнение идёт вниз до первого break (**печатает**, а не складывает: 7 и 9 → «79»)
`for (a = 0; a < 10; a++)`	после цикла a = **10**
`for (…);` / `while (…);`	`;` — пустое тело
`int a[5]`	индексы **0…4**, a[5] — выход за границы
`char s[5] = "Hello"`	нужно 6 байт (+ `'\0'`)
`s1 == s2` для `char[]`	сравнивает адреса → `strcmp(s1, s2) == 0`
`s = "text"` для `char[]`	ошибка → `strcpy(s, "text")`
параметр по значению	функция меняет **копию**; по ссылке `int &x` — оригинал
`x++` vs `++x`	старое значение / новое значение
`A a();`	объявление функции, а не объекта → `A a;`
конструкторы / деструкторы	база → наследник; разрушение в **обратном** порядке
метод без `virtual`	вызов по типу **указателя**; с `virtual` — по реальному объекту
private базы	недоступен наследнику никогда
`static` поле класса	нужно определить вне класса: `int A::n = 0;`
`Edit1->Text + Edit2->Text`	склейка строк → сначала `StrToInt`
компонент из `new`	без `Parent` его не видно

Ошибки в ключах преподавателя, найденные при проверке

Каждый ответ ниже проверен запуском в Borland C++ 6. В тестах эти вопросы помечены красным «(в ключе …)».

- **20-секундный тест**: в ключе **15** неверных ответов. В вариантах 1 (Q5, Q6), 3 (Q2–Q7 — почти весь вариант), 8 (Q2, Q6), 9 (Q3), 11 (Q7), 13 (Q4), 15 (Q5). Ещё 4 вопроса с `x++` / `++x` в одном выражении

(B2 Q2, B3 Q3, B6 Q3, B14 Q3) — неопределённое поведение: показан результат Borland.

- **Банк из 190 вопросов**: №172 — после `for(a = 0; a < 10; a++)` a = **10** (в ключе 9). Спорные: №52, №53 (границы `random`), №106 («выход по истинному условию» — так в C++ не бывает). В казахском оригинале №146 ключ «1», правильно **0** (в английском файле уже исправлено).

Итоговые задачи (на всё сразу)

Финальный блок: смешанные сложные вопросы